

EML-1 release, miniF2F: the corpus, the judgements, and what was thrown away

What this is

248 generated theorems, each with a complete Lean proof. Every one carries the reasoning behind each judgement it received, including the passes that disagreed, and the result of every mechanical check. The rows that did not make it are summarised here and listed in full in `eml_v1_rejected.jsonl`, because a corpus that shows only its survivors cannot be checked.

Canonical artifact. The Lean statements and certificates are the source of truth; this document is a review rendering.

Contents

1	What was thrown away	2
2	How a row got here	3
3	How much a verdict is worth	4
4	The corpus	5

1 What was thrown away

613 certified rows were considered; 248 were admitted. Missing the bar happens three ways, and they are not the same finding: a row both passes reject is a defect the pipeline caught, a row they split on is a fact about the judge, and a row both keep but neither calls strong is simply below the bar this release sets.

Outcome	Rows
admitted — strong from both passes	248
rejected by both passes	143
passes disagreed on keep/reject	126
kept by both passes but not strong in both	57
Lean proved it without a hypothesis it states	30
prose does not describe the elaborated goal	8
goal round-trip never returned a verdict	1

Among the 143 rows both passes rejected, the named failure modes were:

Failure	Rows	What it means
recall	60	solvable by recalling the parent
parallel	33	the parents are proved separately and joined at the end
decoration	22	the change is cosmetic; the parent's proof skeleton still closes it
universal_supplier	8	one parent proves the child on its own, so the other contributes nothing
constant_supplier	4	a parent is used only as a constant, not as an argument
weakening	4	the child is weaker than its parent
tractability_bound	3	the bound makes the statement true for reasons unrelated to the mathematics
parallel_crossover	2	the parents are proved separately and joined at the end
repeated_device	2	the same device as a sibling already kept from these parents
arithmetic_on_answers	2	
unlabelled	1	
conjunction	1	two unrelated claims stapled together
not_equivalent	1	

2 How a row got here

Each gate, and what kind of thing settles it. Lean decides the questions that have an answer; a model is asked only the questions that do not.

Check	By	Coverage	What it establishes
statement_type_check	Lean	248/248	The statement elaborates under Mathlib with <code>autoImplicit false</code> , so no free variable is silently invented.
proof_accepted	Lean	248/248	<code>take env</code> lean accepts the proof and it contains no <code>sorry</code> .
axiom_audit	Lean	248/248	<code>#print axioms</code> lists what the proof depends on; anything outside Lean's three standard axioms fails the row.
vacuity	Lean	248/248	The hypotheses are jointly satisfiable, so the theorem is not true merely by having no instances.
dead_hypotheses	Lean	248/248	Each hypothesis whose name the proof never writes is removed and the file recompiled; a proof that still closes proves the hypothesis was dead. Repeated to a fixpoint, because removing one can make another dead.
corpus_dedup	hash	248/248	The statement, alpha-normalised, is absent from every earlier campaign and from this one.
redundancy	Lean	235/235 applicable	Whether one parent already carries the child: for a crossover, that one parent proves it alone; for a mutation, that the parent gives it directly. Finding such a proof is decisive; failing to find one is not, and the inline probe runs without a prover so it reaches only what the tactic ladder can close.
hypothesis_preservation	parser	12/12 applicable	Silent mutations only. The parent's and child's binders are compared after alpha-normalisation. It cannot tell a hypothesis that was re-encoded from one that was dropped, so a finding is read, not obeyed.
comparator	Lean	248/248	An independent runner rebuilds the statement from the row alone, compares it against the submitted proof with <code>lean4export</code> , checks the axiom allowlist, and replays the proof through the Lean kernel. This is what separates <code>proof_checked</code> from <code>kernel_replayed</code> .
goal_roundtrip	model	248/248	Lean's elaborated goal goes to an informalizer that sees no prose, and a separate judge compares its output with the stated problem. Neither model sees the other's input.

3 How much a verdict is worth

The judge was run twice over all 1233 rows — same inputs, same order, no shared state — so that its disagreement with itself could be measured rather than assumed.

Measure	Agreement
keep or reject	984/1233 (79%)
exact quality grade	883/1233 (71%)
failure label, when both rejected	215/275 (78%)

pass 1 ↓ / pass 2 →	strong	acceptable	weak
strong	639	54	93
acceptable	50	22	36
weak	85	32	222

Admission requires the first cell: strong twice. That is why the release is 248 rows and not 613.

4 The corpus

4.1 001. coupled_difference_isLeast

The problem

Suppose f attains its least value a on S , and g attains its greatest value b on T . Show that the least value of $f(x) - g(y)$ on $S \times T$ is $a - b$.

Lean statement

```
theorem coupled_difference_isLeast
  (S T : Set ℝ) (f g : ℝ → ℝ) (a b : ℝ)
  (hf : IsLeast (Set.image f S) a)
  (hg : IsGreatest (Set.image g T) b) :
  IsLeast (Set.image2 (fun x y : ℝ => f x - g y) S T) (a - b)
```

Parent 1. Find the minimum value of $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ for $0 < x < \pi$. Show that it is 012.

Parent 1 in Lean

```
theorem aime_1983_p9
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 := by
```

What it contributes. Establish IsLeast for the image of the paired objective $(x,y) \mapsto f x - g y$, where the least bound from parent 0 and greatest bound from parent 1 jointly determine both the bound and its witness.

Parent 2. What is the maximum value of $\frac{(2^t - 3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent 2 in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS : S = {x : ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12 : ℝ) := by
```

What it contributes. Establish IsLeast for the image of the paired objective $(x,y) \mapsto f x - g y$, where the least bound from parent 0 and greatest bound from parent 1 jointly determine both the bound and its witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely uses both parent roles: the first least-value witness and lower bound, and the second greatest-value witness and upper bound, jointly establish attainment and the lower bound for the coupled difference objective. Neither extremum alone can derive the stated least-value result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses both parent conclusions: the first supplies a minimizing witness and lower bound, while the second supplies a maximizing witness and upper bound. These bounds combine specifically through subtraction to establish both attainment and minimality of the coupled image.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines both extremum conclusions: the first supplies the lower bound and witness for f , while the second supplies the upper bound and witness for g , yielding both attainment and the coupled inequality for $f x - g y$. Neither parent alone establishes this difference-objective least-value theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2364 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_difference_isLeast.extracted_1_1 (S T : Set ℝ) (f g : ℝ → ℝ) (a b : ℝ) (hf : IsLeast (f '' S) a)
(hg : IsGreatest (g '' T) b) : IsLeast (Set.image2 (fun x y => f x - g y) S T) (a - b)
```

Read back by a model that saw only the Lean. For sets S and T of real numbers, functions f and g from $\mathbb{R}$ to $\mathbb{R}$, and real numbers a and b , if a is the least element of the set $\{f(x) \mid x \in S\}$ and b is the greatest element of the set $\{g(y) \mid y \in T\}$, then $a - b$ is the least element of the set $\{f(x) - g(y) \mid x \in S \text{ and } y \in T\}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem coupled_difference_isLeast
  (S T : Set ℝ) (f g : ℝ → ℝ) (a b : ℝ)
  (hf : IsLeast (Set.image f S) a)
  (hg : IsGreatest (Set.image g T) b) :
  IsLeast (Set.image2 (fun x y : ℝ => f x - g y) S T) (a - b) := by
  rcases hf.1 with ⟨x, hx, hfx⟩
  rcases hg.1 with ⟨y, hy, hgy⟩
  have hattain : a - b ∈ Set.image2 (fun x y : ℝ => f x - g y) S T := by
    refine ⟨x, hx, y, hy, ?_⟩
    simp [hfx, hgy]
  have hbound : ∀ x ∈ S, ∀ y ∈ T, a - b ≤ f x - g y := by
    intro x hx y hy
    have hfx' : a ≤ f x := hf.2 ⟨x, hx, rfl⟩
    have hgy' : g y ≤ b := hg.2 ⟨y, hy, rfl⟩
    linarith
  constructor
  · exact hattain
  · intro z hz
    rcases hz with ⟨x, hx, y, hy, rfl⟩
    exact hbound x hx y hy
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p9__x__amc12b_2020_p22__xe__7194e753

Fusion mechanism. invariant_transplant Establish IsLeast for the image of the paired objective $(x,y) \mapsto f x - g y$, where the least bound from parent 0 and greatest bound from parent 1 jointly determine both the bound and its witness.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7194e753bfc7d7d3

4.2 002. quadratic_reciprocal_specialization

The problem

Suppose $x^2 - 5x - 4 \leq 10$. Show that $\lfloor \frac{9}{13} \rfloor \leq \frac{2}{12} + \frac{2}{10-x} + \frac{2}{x+4}$.

Lean statement

```
theorem quadratic_reciprocal_specialization (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :
  9 / 13 ≤ 2 / 12 + 2 / (10 - x) + 2 / (x + 4)
```

Parent 1. Show that for any three positive real numbers x , y , and z , $9/(x+y+z) \leq 2/(x+y) + 2/(y+z) + 2/(z+x)$.

Parent 1 in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Its positive-three-variable reciprocal inequality is reproduced as `h_parent1` and explicitly instantiated at $(x+3, 9-x, 1)$.

Parent 2. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent 2 in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

What it contributes. Its quadratic interval checkpoint is derived as `hinterval` and feeds all affine positivity hypotheses used by the reciprocal application.

Review

Every judgement this row received. Admission required `strong` from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines both parents: the quadratic hypothesis yields the interval bounds needed to prove positivity of the affine triple, and Parent 1's reciprocal inequality is then instantiated and normalized to the target.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The quadratic hypothesis is used to derive the interval bounds that certify positivity of the affine triple, and the reciprocal inequality is then obtained by applying parent 1 to that triple and normalizing. Both parents materially determine the proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The quadratic hypothesis is used to derive bounds on x , which certify positivity of the affine triple $(x+3, 9-x, 1)$; the reciprocal inequality is then instantiated on that triple and normalized into the conclusion. Both parents materially interact through this specialization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2515 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_reciprocal_specialization.extracted_1_1 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :
  9 / 13 ≤ 2 / 12 + 2 / (10 - x) + 2 / (x + 4)
```

Read back by a model that saw only the Lean. *For a real number x satisfying $x^2 - 5x - 4 \leq 10$, we have $9/13 \leq 2/12 + 2/(10 - x) + 2/(x + 4)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem quadratic_reciprocal_specialization (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :
  9 / 13 ≤ 2 / 12 + 2 / (10 - x) + 2 / (x + 4) := by
  have hinterval : x ∈ Set.Icc (-2 : ℝ) 7 := by
    constructor
    · by_contra hx
      have hx' : x < -2 := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      nlinarith
    · by_contra hx
      have hx' : 7 < x := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos (by linarith) (by linarith)
      nlinarith
  have hA : 0 < x + 3 := by
    linarith [hinterval.1]
  have hB : 0 < 9 - x := by
    linarith [hinterval.2]
  have hC : (0 : ℝ) < 1 := by
    linarith [hinterval.1, hinterval.2]
  have h_parent1 : ∀ a b c : ℝ, 0 < a ∧ 0 < b ∧ 0 < c →
    9 / (a + b + c) ≤ 2 / (a + b) + 2 / (b + c) + 2 / (c + a) := by
    intro a b c habc
    rcases habc with (ha, hb, hc)
    have hab : 0 < a + b := by linarith
    have hbc : 0 < b + c := by linarith
    have hca : 0 < c + a := by linarith
    have habc' : 0 < a + b + c := by linarith
    have hnum :
      0 ≤ 2 * (a + b + c) *
        ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b)) -
        9 * (a + b) * (b + c) * (c + a) := by
      have h1 : 0 ≤ ((a + b) - (b + c)) ^ 2 := sq_nonneg _
      have h2 : 0 ≤ ((b + c) - (c + a)) ^ 2 := sq_nonneg _
      have h3 : 0 ≤ ((c + a) - (a + b)) ^ 2 := sq_nonneg _
      have h1' : 0 ≤ (a + b) * ((a + b) - (b + c)) ^ 2 :=
        mul_nonneg (le_of_lt hab) h1
      have h2' : 0 ≤ (b + c) * ((b + c) - (c + a)) ^ 2 :=
        mul_nonneg (le_of_lt hbc) h2
      have h3' : 0 ≤ (c + a) * ((c + a) - (a + b)) ^ 2 :=
        mul_nonneg (le_of_lt hca) h3
      nlinarith
    have hden : 0 < (a + b + c) * (a + b) * (b + c) * (c + a) := by
      positivity
    have hleft :
      9 / (a + b + c) =
        (9 * (a + b) * (b + c) * (c + a)) /
        ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
```

```

field_simp [ne_of_gt habc', ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
have hright :
  2 / (a + b) + 2 / (b + c) + 2 / (c + a) =
    (2 * (a + b + c) *
      ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b))) /
      ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
field_simp [ne_of_gt habc', ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
rw [hleft, hright]
exact (div_le_div_iff_of_pos_right hden).2 (by nlinarith)
have hrec := h_parent1 (x + 3) (9 - x) 1 ⟨hA, hB, hC⟩
have hsum : (x + 3) + (9 - x) + 1 = (13 : ℝ) := by ring
have hab : (x + 3) + (9 - x) = (12 : ℝ) := by ring
have hbc : (9 - x) + 1 = 10 - x := by ring
have hca : (1 : ℝ) + (x + 3) = x + 4 := by ring
rw [hsum, hab, hbc, hca] at hrec
exact hrec

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__x__mathd_algebra_101__xe__7fee6821

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7fee68212b0d103c

4.3 003. weighted_energy_translated_square_min_unique

The problem

Suppose the four weighted-square equations for $n = 2, 4, 6, 8$ hold. If $a > 0$, show that the function $a(t - x)^2 + c$ on $[-6, 6]$ has minimum value c , attained uniquely at $t = x$.

Lean statement

```
theorem weighted_energy_translated_square_min_unique (x y z w a c : ℝ)
  (ha : 0 < a)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  IsLeast {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} c ∧
  ∀ t ∈ Set.Icc (-6 : ℝ) 6, (a * (t - x) ^ 2 + c = c ↔ t = x)
```

Parent 1. Show that no real numbers x, y, z, w satisfy the four weighted-square equations if $x \geq 7$.

Parent 1 in Lean

```
theorem weighted_square_infeasible_of_lower_bound (x y z w : ℝ)
  (hbound : 7 ≤ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  False
```

Parent 2. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent 2 in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The weighted-square equations are genuinely consumed to derive $x \in [-6, 6]$, which is necessary to attain the translated square's minimum on the interval. That checkpoint then feeds the minimum and uniqueness conclusion, so the child is a substantive pipeline rather than a parallel combination.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The weighted-energy hypotheses are used to derive the essential bound $x^2 \leq 36$, which places the minimizer $t=x$ inside the interval and enables the translated-square minimum. The square-minimum argument alone cannot establish attainment on the restricted domain.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The weighted-square hypotheses from Parent 1 derive $x^2 \leq 36$ and hence $x \in [-6, 6]$, which is essential to exhibit c as the minimum over the restricted interval. Parent 2's translated-square minimum structure then uses that parent-derived domain membership; without it, the minimum claim can fail.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2704 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem weighted_energy_translated_square_min_unique.extracted_1_1 (x y z w a c : ℝ) (ha : 0 < a)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  IsLeast {q | ∃ t ∈ Set.Icc (-6) 6, q = a * (t - x) ^ 2 + c} c ∧
  ∀ t ∈ Set.Icc (-6) 6, a * (t - x) ^ 2 + c = c ↔ t = x
```

Read back by a model that saw only the Lean. Let x, y, z, w, a , and c be real numbers with $a > 0$. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then c is the least element of the set of all values $a(t-x)^2+c$ as t ranges over the closed interval $[-6,6]$, and for every t in $[-6,6]$, $a(t-x)^2+c = c$ if and only if $t = x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem weighted_energy_translated_square_min_unique (x y z w a c : ℝ)
  (ha : 0 < a)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  IsLeast {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} c ∧
  ∀ t ∈ Set.Icc (-6 : ℝ) 6, (a * (t - x) ^ 2 + c = c ↔ t = x) := by
  norm_num at h₀ h₁ h₂ h₃
  have henergy : x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
    linarith
  have hx_sq : x ^ 2 ≤ 36 := by
    nlinarith [henergy, sq_nonneg y, sq_nonneg z, sq_nonneg w]
  have hinterval : x ∈ Set.Icc (-6 : ℝ) 6 := by
    constructor <|> nlinarith [sq_nonneg (x - 6), sq_nonneg (x + 6), hx_sq]
  have hmem : c ∈ {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} := by
    refine ⟨x, hinterval, ?_⟩
    ring
  have hlower : ∀ q ∈ {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c}, c ≤ q := by
    intro q hq
    rcases hq with ⟨t, ht, rfl⟩
    have hsq : 0 ≤ (t - x) ^ 2 := sq_nonneg (t - x)
    have hprod : 0 ≤ a * (t - x) ^ 2 := mul_nonneg (le_of_lt ha) hsq
    linarith
  have hleast : IsLeast {q : ℝ | ∃ t ∈ Set.Icc (-6 : ℝ) 6, q = a * (t - x) ^ 2 + c} c := by
    exact ⟨hmem, hlower⟩
  have hunique : ∀ t ∈ Set.Icc (-6 : ℝ) 6, (a * (t - x) ^ 2 + c = c ↔ t = x) := by
    intro t ht
    constructor
    · intro heq
      have hzero : a * (t - x) ^ 2 = 0 := by
        linarith
      have hsquare : (t - x) ^ 2 = 0 := by
```

```
    exact (mul_eq_zero.mp hzero).resolve_left (ne_of_gt ha)
  nlinarith
  · intro htx
    subst t
    ring
  exact ⟨hleast, hunique⟩
```

Operator. crossover / crossover_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__x__aime_1984_p15__xe.mh__483514d7__x__mathd_algebra_410__ms.xe__ac8bb441

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ac8bb4412ba21e23

4.4 004. entropy_mag_crossover

The problem

A geometric sequence begins with $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}$ and has a constant ratio. If a circuit has voltage $V = a_5 + i$ and impedance $Z = 2 - i$, find the current I satisfying $V = IZ$.

Lean statement

```
theorem entropy_mag_crossover
  (a : ℕ → ℚ)
  (d : ℚ)
  (v i z : ℂ)
  (hrec : ∀ n, a (n + 1) = a n * d)
  (h1 : a 0 = 27 / 125)
  (h2 : a 1 = 9 / 25)
  (h3 : a 2 = 3 / 5)
  (hcircuit : v = i * z)
  (hvoltage : v = (a 5 : ℂ) + Complex.I)
  (himpedance : z = 2 - Complex.I) :
  i = 41 / 45 + 43 / 45 * Complex.I
```

Parent 1. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent 1 in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h0 : ∀ n, a (n + 1) = a n * d)
  (h1 : a 0 = 27 / 125)
  (h2 : a 1 = 9 / 25)
  (h3 : a 2 = 3 / 5) :
  a 5 = 25 / 9 := by
```

What it contributes. The geometric-sequence value a_5 is cast into the complex voltage $V = (a_5 : \mathbb{C}) + i$; the circuit equation then determines a new current.

Parent 2. Complex numbers are often used when dealing with alternating current (AC) circuits. In the equation $V = IZ$, V is voltage, I is current, and Z is a value known as impedance. If $V = 1 + i$ and $Z = 2 - i$, find I . Show that it is $\frac{1}{5} + \frac{3}{5}i$.

Parent 2 in Lean

```
theorem mathd_algebra_313 (v i z : ℂ) (h0 : v = i * z) (h1 : v = 1 + Complex.I)
  (h2 : z = 2 - Complex.I) : i = 1 / 5 + 3 / 5 * Complex.I := by
```

What it contributes. The geometric-sequence value a_5 is cast into the complex voltage $V = (a_5 : \mathbb{C}) + i$; the circuit equation then determines a new current.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The recurrence is genuinely used to derive $a(5)=25/9$, which becomes the real part of the voltage and changes the complex current determined by the circuit equation. Neither parent can be discharged independently or recalled without performing this witness exchange.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The recurrence is genuinely needed to derive the real voltage component $a(5)=25/9$, which is then substituted into the circuit equation; that fused voltage determines the new complex current. Neither parent can be discharged independently and combined only at the end.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The recurrence is genuinely used to derive $a(5)=25/9$, which becomes the real component of the circuit voltage; that value changes the solved complex current. The circuit algebra then depends on this derived sequence output, so the parents are fused rather than discharged in parallel.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2450 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B indexes the listed initial terms from $a(0)$, so $a(5)$ is the sixth term, whereas Statement A's a_5 denotes the fifth term of the sequence. Consequently, B obtains $a(5)=25/9$ and current $41/45+(43/45)i$, while A gives $a_5=5/3$ and current $7/15+(11/15)i$.

Goal round-trip: what Lean elaborated

```
theorem entropy_mag_crossover.extracted_1_1 (a : ℕ → ℚ) (d : ℚ) (v i z : ℂ) (hrec : ∀ (n : ℕ), a (n + 1) = a n * d)
(h1 : a 0 = 27 / 125) (h2 : a 1 = 9 / 25) (h3 : a 2 = 3 / 5) (hcircuit : v = i * z)
(hvoltage : v = ↑(a 5) + Complex.I) (himpedance : z = 2 - Complex.I) : i = 41 / 45 + 43 / 45 * Complex.I
```

Read back by a model that saw only the Lean. Let a be a sequence of rational numbers, let d be a rational number, and let v , i , and z be complex numbers. Suppose that for every natural number n , $a(n + 1) = a(n)d$; $a(0) = 27/125$; $a(1) = 9/25$; $a(2) = 3/5$; $v = iz$; $v = a(5) + \text{the imaginary unit } I$, where $a(5)$ is regarded as a complex number; and $z = 2 - I$. Then $i = 41/45 + (43/45)I$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem entropy_mag_crossover
(a : ℕ → ℚ)
(d : ℚ)
(v i z : ℂ)
(hrec : ∀ n, a (n + 1) = a n * d)
(h1 : a 0 = 27 / 125)
(h2 : a 1 = 9 / 25)
(h3 : a 2 = 3 / 5)
(hcircuit : v = i * z)
(hvoltage : v = (a 5 : ℂ) + Complex.I)
(himpedance : z = 2 - Complex.I) :
i = 41 / 45 + 43 / 45 * Complex.I := by
have hd : d = 5 / 3 := by
have h := hrec 0
norm_num [h1, h2] at h
linarith
```

```

have h23 := hrec 2
norm_num [h3, hd] at h23
have h34 := hrec 3
norm_num [h23, hd] at h34
have h45 := hrec 4
norm_num [h34, hd] at h45
have ha5 : a 5 = 25 / 9 := by
  norm_num [h45]
have hFused : (25 / 9 : ℂ) + Complex.I = i * (2 - Complex.I) := by
  calc
    (25 / 9 : ℂ) + Complex.I = (a 5 : ℂ) + Complex.I := by norm_num [ha5]
    _ = v := by symm; exact hvoltage
    _ = i * z := hcircuit
    _ = i * (2 - Complex.I) := by rw [himpedance]
have hr := congrArg Complex.re hFused
have hm := congrArg Complex.im hFused
apply Complex.ext
· norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
  linarith
· norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
  linarith

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__x__mathd_algebra_313__xe__4b9136ba

Fusion mechanism. witness_exchange The geometric-sequence value $a\ 5$ is cast into the complex voltage $V = (a\ 5 : \mathbb{C}) + i$; the circuit equation then determines a new current.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b9136ba34508a07

4.5 005. latent_parameter_radical

The problem

Suppose $3 + \frac{1}{x} = \frac{7}{x}$ and $a = x^3$. Show that $(16\sqrt[3]{a^2})^{1/3} = 4$.

Lean statement

```
theorem latent_parameter_radical (x a : ℝ) (h₁ : 3 + 1 / x = 7 / x) (h₂ : a = x ^ 3) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4
```

Parent 1. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent 1 in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h₁ : 3 + 1 / x = 7 / x) : x = 2 := by
```

What it contributes. The solved witness $x = 2$ is substituted into $a = x^3$ to produce $a = 8$, the hypothesis consumed by the radical evaluation theorem.

Parent 2. If $a = 8$, what is the value of $(16\sqrt[3]{a^2})^{1/3}$? Show that it is 4.

Parent 2 in Lean

```
theorem mathd_algebra_114 (a : ℝ) (h₀ : a = 8) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 := by
```

What it contributes. The solved witness $x = 2$ is substituted into $a = x^3$ to produce $a = 8$, the hypothesis consumed by the radical evaluation theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines the parents: the reciprocal equation forces $x = 2$, which combines with $a = x^3$ to derive the radical theorem's required input $a = 8$. Neither parent conclusion is independently asserted or merely combined at the end.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The reciprocal equation is needed to derive $x = 2$, which is then used through $a = x^3$ to establish $a = 8$, the essential input for evaluating the radical expression. Neither parent can be discharged independently or combined only at the end.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The reciprocal equation is needed to force $x = 2$, which via $a = x^3$ supplies $a = 8$; that value is then required to evaluate the radical expression. Neither parent conclusion can be discharged independently of the other.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2571 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem latent_parameter_radical.extracted_1_1 (x a : ℝ) (h1 : 3 + 1 / x = 7 / x) (h2 : a = x ^ 3) :
  (16 * (a ^ 2) ^ (1 / 3)) ^ (1 / 3) = 4
```

Read back by a model that saw only the Lean. For real numbers x and a , if $3 + 1/x = 7/x$ and $a = x^3$, then $(16(a^2)^{(1/3)})^{(1/3)} = 4$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem latent_parameter_radical (x a : ℝ) (h1 : 3 + 1 / x = 7 / x) (h2 : a = x ^ 3) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 := by
  have hx : x ≠ 0 := by
    intro hx
    subst x
    norm_num at h1
  have hx2 : x = 2 := by
    field_simp [hx] at h1
    linarith
  have ha : a = 8 := by
    calc
      a = x ^ 3 := h2
      _ = 8 := by rw [hx2]; norm_num
  have hc : (64 : ℝ) ^ ((1 : ℝ) / 3) = 4 := by
    convert Real.pow_rpow_inv_natCast (x := (4 : ℝ)) (n := 3) (by norm_num) (by norm_num) using 1 <|> norm_num
  have hinner : (a ^ 2) ^ ((1 : ℝ) / 3) = 4 := by
    calc
      (a ^ 2) ^ ((1 : ℝ) / 3) = (64 : ℝ) ^ ((1 : ℝ) / 3) := by
        rw [ha]
        norm_num
      _ = 4 := hc
  calc
    (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = (16 * 4) ^ ((1 : ℝ) / 3) := by rw [hinner]
    _ = (64 : ℝ) ^ ((1 : ℝ) / 3) := by norm_num
    _ = 4 := hc
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__x__mathd_algebra_114__xe__e0f06d2f

Fusion mechanism. witness_exchange The solved witness $x = 2$ is substituted into $a = x^3$ to produce $a = 8$, the hypothesis consumed by the radical evaluation theorem.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e0f06d2fb129aad9

4.6 006. coupled_real_complex_unique

The problem

Show that the reciprocal condition $3 + 1/x = 7/x$ together with $V = I((x) - i)$, where $V = 1 + i$, determines a unique real parameter and complex current.

Lean statement

```
theorem coupled_real_complex_unique : ∃! p : ℝ × ℂ, 3 + 1 / p.1 = 7 / p.1 ∧ (1 + Complex.I : ℂ) = p.2 * ((p.1 : ℂ) - Complex.I)
```

Parent 1. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent 1 in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h₁ : 3 + 1 / x = 7 / x) : x = 2 := by
```

What it contributes. The root checkpoint $x = 2$ is substituted into the impedance definition, and the resulting nonzero complex factor is used to establish the unique current component of the coupled system.

Parent 2. Complex numbers are often used when dealing with alternating current (AC) circuits. In the equation $V = IZ$, V is voltage, I is current, and Z is a value known as impedance. If $V = 1 + i$ and $Z = 2 - i$, find I . Show that it is $\frac{1}{5} + \frac{3}{5}i$.

Parent 2 in Lean

```
theorem mathd_algebra_313 (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I)
(h₂ : z = 2 - Complex.I) : i = 1 / 5 + 3 / 5 * Complex.I := by
```

What it contributes. The root checkpoint $x = 2$ is substituted into the impedance definition, and the resulting nonzero complex factor is used to establish the unique current component of the coupled system.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The real reciprocal equation is used to force $x = 2$, which then specializes the complex impedance equation and is essential to proving uniqueness of the current. Neither parent can discharge the coupled uniqueness claim independently.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the real reciprocal equation to force $x = 2$, then substitutes that value into the complex impedance equation to uniquely determine the current. Neither parent alone establishes uniqueness of the coupled pair.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the reciprocal equation to force the real parameter $x = 2$, then substitutes that value into the complex impedance equation to establish uniqueness of the current. Neither parent can be discharged independently, and the cited siblings do not duplicate this same parent pair and coupling.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2446 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_real_complex_unique.extracted_1_1 :
  ∃! p, 3 + 1 / p.1 = 7 / p.1 ∧ 1 + Complex.I = p.2 * (↑p.1 - Complex.I)
```

Read back by a model that saw only the Lean. *There exists exactly one ordered pair (x, z) , where x is a real number and z is a complex number, such that $3 + 1/x = 7/x$ and $1 + i = z(x - i)$, with i denoting the imaginary unit.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem coupled_real_complex_unique : ∃! p : ℝ × ℂ, 3 + 1 / p.1 = 7 / p.1 ∧ (1 + Complex.I : ℂ) = p.2 * ((p.1 : ℂ) - Complex.I) := by
  let i₀ : ℂ := 1 / 5 + 3 / 5 * Complex.I
  have hroot : (3 : ℝ) + 1 / 2 = 7 / 2 := by
    norm_num
  have hcurrent : (1 + Complex.I : ℂ) = i₀ * ((2 : ℝ) - Complex.I) := by
    apply Complex.ext <|> norm_num [i₀, Complex.mul_re, Complex.mul_im]
  refine ⟨(2, i₀), ⟨hroot, hcurrent⟩, ?_⟩
  rintro ⟨x, i⟩ ⟨hxrec, hxc⟩
  change 3 + 1 / x = 7 / x at hxrec
  change (1 + Complex.I : ℂ) = i * ((x : ℂ) - Complex.I) at hxc
  have hxne : x ≠ 0 := by
    intro hx
    subst x
    norm_num at hxrec
  field_simp [hxne] at hxrec
  have hx : x = 2 := by
    linarith
  have hxc' := hxc
  rw [hx] at hxc'
  have hxr := congrArg Complex.re hxc'
  have hxi := congrArg Complex.im hxc'
  have hi : i = i₀ := by
    apply Complex.ext
    · norm_num [i₀, Complex.mul_re, Complex.mul_im] at hxr hxi
      linarith
    · norm_num [i₀, Complex.mul_re, Complex.mul_im] at hxr hxi
      linarith
  apply Prod.ext
  · exact hx
  · exact hi
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__x__mathd_algebra_313__xe__dbb2d393

Fusion mechanism. parameter_coupling The root checkpoint $x = 2$ is substituted into the impedance definition, and the resulting nonzero complex factor is used to establish the unique current component of the coupled system.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint dbb2d393cb6de0b2

4.7 007. quadratic_finset_offset_mod_residue

The problem

Let S be the finite set of positive natural numbers x satisfying $x^2 + 4x + 4 < 20$. If $2n \equiv 15 + |S| \pmod{47}$ for a natural number n , prove that $n \equiv 32 \pmod{47}$.

Lean statement

```
theorem quadratic_finset_offset_mod_residue (S : Finset ℕ) (n : ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) (h₁ : 2 * n ≡ 15 + S.card [MOD 47]) : n ≡ 32 [MOD 47] := by
```

Parent 1. For how many positive integer values of x is the sum $x^2 + 4x + 4$ less than 20? Show that it is 2.

Parent 1 in Lean

```
theorem mathd_algebra_405 (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) : S.card = 2 := by
```

What it contributes. The classification/count of the inequality-defined finite set fixes the additive offset applied to the parent's residue class modulo 47.

Parent 2. Solve the congruence $2n \equiv 15 \pmod{47}$, as a residue modulo 47. (Give an answer between 0 and 46.) Show that it is 31.

Parent 2 in Lean

```
theorem mathd_numbertheory_99 (n : ℕ) (h₀ : 2 * n ≡ 15 [MOD 47]) : n ≡ 31 [MOD 47] := by
```

What it contributes. The classification/count of the inequality-defined finite set fixes the additive offset applied to the parent's residue class modulo 47.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines the parents: Parent 1's quadratic finset characterization is used to derive $S.\text{card} = 2$, which changes Parent 2's modular premise to $2*n \equiv 17$ and yields the new residue $n \equiv 32$. Neither parent's conclusion alone proves the child's result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines Parent 1 into Parent 2: the quadratic finset proof establishes $S.\text{card} = 2$, which is consumed to transform the modular hypothesis into $2*n \equiv 17 \pmod{47}$ before deriving $n \equiv 32 \pmod{47}$. The cardinality is an active offset in the second derivation, not a parallel closing conjunction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: the quadratic finset classification supplies $S.\text{card} = 2$, which is substituted into the modular hypothesis to obtain $2*n \equiv 17$ before solving for $n \equiv 32$. The cardinality is an essential input to the second derivation, not a parallel closing conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1798 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_finset_offset_mod_residue.extracted_1_1 (S : Finset ℕ) (n : ℕ)
  (h₀ : ∀ (x : ℕ), x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) (h₁ : 2 * n ≡ 15 + S.card [MOD 47]) :
  n ≡ 32 [MOD 47]
```

Read back by a model that saw only the Lean. *Let S be a finite set of natural numbers and let n be a natural number. Suppose that for every natural number x , x belongs to S if and only if $0 < x$ and $x^2 + 4x + 4 < 20$. Suppose also that $2n$ is congruent modulo 47 to 15 plus the cardinality of S . Then n is congruent to 32 modulo 47.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem quadratic_finset_offset_mod_residue (S : Finset ℕ) (n : ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) (h₁ : 2 * n ≡ 15 +
  S.card [MOD 47]) : n ≡ 32 [MOD 47] := by
  have hS : S = {1, 2} := by
    ext x
    rw [h₀]
    constructor
    · rintro ⟨hx, hineq⟩
      have hxle : x ≤ 2 := by
        by_contra h
        have hx3 : 3 ≤ x := by omega
        have hsquare : 9 ≤ x ^ 2 := by
          simpa [pow_two] using Nat.mul_le_mul hx3 hx3
        omega
      interval_cases x <|> norm_num at hx ⊢
    · intro hx
      simp only [Finset.mem_insert, Finset.mem_singleton] at hx
      rcases hx with rfl | rfl <|> norm_num
  have hcard : S.card = 2 := by
    rw [hS]
    norm_num
  have hmod : 2 * n ≡ 17 [MOD 47] := by
    simp only [Nat.ModEq] at h₁ ⊢
    omega
  have hres : n ≡ 32 [MOD 47] := by
    simp only [Nat.ModEq] at hmod ⊢
    omega
  exact hres
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_405__x__mathd_numbertheory_99__xe__de242f87

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7e9d051f691873e2

4.8 008. incompatible_quadratic_extrema

The problem

Show that no real number x can both satisfy $x^2 - 6x + 13 = 4$ and minimize $x^2 - 14x + 3$ over all real numbers.

Lean statement

```
theorem incompatible_quadratic_extrema
  (x : ℝ)
  (h₁ : x ^ 2 - 6 * x + 13 = 4)
  (h₂ : IsMinOn (fun y : ℝ => y ^ 2 - 14 * y + 3) Set.univ x) :
  False
```

Parent 1. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent 1 in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x => x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. The level-attainment condition for $x^2 - 6x + 13$ supplies the obstruction $x=3$, which must eliminate global minimality for $x^2 - 14x + 3$ at the same input.

Parent 2. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent 2 in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x ^ 2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

What it contributes. The level-attainment condition for $x^2 - 6x + 13$ supplies the obstruction $x=3$, which must eliminate global minimality for $x^2 - 14x + 3$ at the same input.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The two parent results interact through an incompatibility: the first condition forces $x = 3$, while global minimality of the second quadratic requires comparison with its vertex $x = 7$, yielding a contradiction. Neither parent can be discharged independently and combined only at the end.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The first condition forces the shared input to be the first quadratic's vertex, $x = 3$. The second condition requires that input to globally minimize the other quadratic, but comparison with its vertex $x = 7$ yields a contradiction, so both parents interact through genuine incompatibility.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The first parent's minimum-level condition forces $x = 3$, while the second parent's global-minimizer condition requires the quadratic at x to be no larger than its value at 7; substituting $x = 3$ yields a contradiction. Both parent results interact through the shared input, producing a genuine incompatibility proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2674 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem incompatible_quadratic_extrema.extracted_1_1 (x : ℝ) (h1 : x ^ 2 - 6 * x + 13 = 4)
(h2 : IsMinOn (fun y => y ^ 2 - 14 * y + 3) Set.univ x) : False
```

Read back by a model that saw only the Lean. *Let x be a real number. Suppose that $x^2 - 6x + 13 = 4$ and that x is a global minimizer over all real numbers of the function $y \mapsto y^2 - 14y + 3$. Then a contradiction follows; that is, False.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem incompatible_quadratic_extrema
(x : ℝ)
(h1 : x ^ 2 - 6 * x + 13 = 4)
(h2 : IsMinOn (fun y : ℝ => y ^ 2 - 14 * y + 3) Set.univ x) :
False := by
have hx : x = 3 := by
  nlinarith [sq_nonneg (x - 3)]
have hglobal := h2
rw [isMinOn_univ_iff] at hglobal
have hbad := hglobal 7
rw [hx] at hbad
norm_num at hbad
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__x__mathd_algebra_113__xe__d052b38e

Fusion mechanism. obstruction_as_lemma The level-attainment condition for $x^2 - 6x + 13$ supplies the obstruction $x=3$, which must eliminate global minimality for $x^2 - 14x + 3$ at the same input.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d052b38e59cd7ec8

4.9 009. ternary_fibonacci_mod4_obstruction

The problem

Let $d \in \{1, 2\}$ and let a natural number have base-three representation $122d_3$. If (a_n) is the Fibonacci sequence, then the decoded number cannot have the same remainder modulo 4 as a_{100} , whose remainder is 3.

Lean statement

```
theorem ternary_fibonacci_mod4_obstruction
  (d x : ℕ)
  (a : ℕ → ℕ)
  (hd : d < 3)
  (hdpos : 0 < d)
  (h : Nat.digits 3 x = [d, 2, 2, 1])
  (h₀ : a 1 = 1)
  (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  x % 4 ≠ a 100 % 4
```

Parent 1. Let $d \in \{0, 1, 2\}$. If a natural number has base-three representation $122d_3$, show that its value is $d + 51$; in particular, when $d = 2$, its value is 53.

Parent 1 in Lean

```
theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

What it contributes. The ternary decoding checkpoint rewrites x modulo 4, while the Fibonacci checkpoint supplies the distinguished residue that excludes $d \neq 0$.

Parent 2. The Fibonacci sequence is the sequence 1, 1, 2, 3, 5, ... where each term is the sum of the previous two terms. What is the remainder when the 100th term of the sequence is divided by 4? Show that it is 3.

Parent 2 in Lean

```
theorem mathd_numbertheory_483 (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) : a 100 % 4 = 3 := by
```

What it contributes. The ternary decoding checkpoint rewrites x modulo 4, while the Fibonacci checkpoint supplies the distinguished residue that excludes $d \neq 0$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both results: the ternary decoding gives $x=d+51$ and $d \in \{1,2\}$, excluding $x \equiv 3 \pmod{4}$, while the Fibonacci recurrence gives $a(100) \equiv 3 \pmod{4}$. The obstruction then links these computed residues to prove the inequality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The ternary decoding derives $x = d + 51$ and, using $0 < d < 3$, proves $x \bmod 4 \neq 3$; the Fibonacci recurrence independently derives $a_{100} \bmod 4 = 3$, and these residues are combined to establish the obstruction. Neither parent can be discharged without affecting the final inequality.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The ternary decoding forces $x = d + 51$, and the positivity and bound on d then restrict x modulo 4 to 0 or 1. The Fibonacci recurrence independently forces a 100 modulo 4 to be 3, and these results interact to prove the obstruction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2535 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_fibonacci_mod4_obstruction.extracted_1_1 (d x : ℕ) (a : ℕ → ℕ) (hd : d < 3) (hdpos : 0 < d)
  (h : Nat.digits 3 x = [d, 2, 2, 1]) (h₀ : a 1 = 1) (h₁ : a 2 = 1) (h₂ : ∀ (n : ℕ), a (n + 2) = a (n + 1) + a n) :
  x % 4 ≠ a 100 % 4
```

Read back by a model that saw only the Lean. For natural numbers d and x and a function a from natural numbers to natural numbers, suppose that d is less than 3, d is positive, the base-3 digit expansion of x is exactly $[d, 2, 2, 1]$, $a(1) = 1$, $a(2) = 1$, and for every natural number n , $a(n + 2) = a(n + 1) + a(n)$. Then x modulo 4 is not equal to $a(100)$ modulo 4.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem ternary_fibonacci_mod4_obstruction
  (d x : ℕ)
  (a : ℕ → ℕ)
  (hd : d < 3)
  (hdpos : 0 < d)
  (h : Nat.digits 3 x = [d, 2, 2, 1])
  (h₀ : a 1 = 1)
  (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  x % 4 ≠ a 100 % 4 := by
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=
    (Nat.ofDigits_digits 3 x).symm
  have hvalue : x = d + 51 := by
    calc
      x = Nat.ofDigits 3 (Nat.digits 3 x) := hdecode
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [h]
      _ = d + 51 := by norm_num [Nat.ofDigits_cons]
  have ha : ∀ n, a n = Nat.fib n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      by_cases h0 : n = 0
      · subst n
        have hr := h₂ 0
        norm_num [h₀, h₁, Nat.fib_zero] at hr ⊢
        omega
      by_cases h1 : n = 1
      · subst n
        simp [h₀, Nat.fib_one]
      have hn : 2 ≤ n := by omega
```

```

have hr := h₂ (n - 2)
have hi2 := ih (n - 2) (by omega)
have hi1 := ih (n - 1) (by omega)
have hf := Nat.fib_add_two (n := n - 2)
rw [show n - 2 + 2 = n by omega, show n - 2 + 1 = n - 1 by omega] at hr hf
omega
have hFib : a 100 % 4 = 3 := by
  rw [ha 100]
  norm_num [Nat.fib]
have hd_cases : d = 1 ∨ d = 2 := by omega
have hterminal_obstruction : x % 4 ≠ 3 := by
  rcases hd_cases with rfl | rfl
  · norm_num [hvalue]
  · norm_num [hvalue]
have hobstruction : x % 4 ≠ a 100 % 4 := by
  intro hxa
  apply hterminal_obstruction
  rw [hFib] at hxa
  exact hxa
exact hobstruction

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me__0fd2ae3f__x__mathd_numbertheory_483__xe__b7ac3c8d

Fusion mechanism. obstruction_as_lemma The ternary decoding checkpoint rewrites x modulo 4, while the Fibonacci checkpoint supplies the distinguished residue that excludes $d \neq 0$.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint b7ac3c8dbd47f8d9

4.10 010. weighted_binomial_eq_scaled_complementary_iff

The problem

For positive integers n and k with $0 < k \leq n$, the weighted sum $\sum_{j=1}^n j \binom{n}{j}$ equals $n2^{n-1} \binom{n}{n-k}$ if and only if $k = n$.

Lean statement

```
theorem weighted_binomial_eq_scaled_complementary_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (hkn : k ≤ n) :
  (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
    (n * 2 ^ (n - 1)) * Nat.choose n (n - k) ↔ k = n
```

Parent 1. Show that for positive integers n and k , if $k \leq n$, then $\sum_{k=1}^n (k * C_n^k) = n * 2^{n-1}$.

Parent 1 in Lean

```
theorem numbertheory_sumkmlncqnmul2pownm1 (n : ℕ) (hn : 0 < n) :
  (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
```

What it contributes. Use parent 1 to rewrite the global sum into a nonzero normalization factor, then cancel that factor so parent 4 classifies the remaining complementary coefficient.

Parent 2. For integers $n \geq 2$ and $0 < k \leq n$, classify all k such that the complementary binomial coefficient $\binom{n}{n-k}$ equals 1.

Parent 2 in Lean

```
theorem complementary_choose_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose n (n - k) = 1 ↔ k = n
```

What it contributes. Use parent 1 to rewrite the global sum into a nonzero normalization factor, then cancel that factor so parent 4 classifies the remaining complementary coefficient.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines parent 1's weighted-sum identity into parent 2's complementary-binomial characterization: positivity permits cancellation of the normalization factor, leaving exactly the condition that the complementary coefficient is 1. Both parent results are essential to the coupled iff.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: parent 1 rewrites the weighted sum to the nonzero normalization factor, whose cancellation reduces the equality to the complementary binomial coefficient being 1; parent 2 then characterizes that condition as $k=n$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The equality first uses parent 1 to identify the weighted binomial sum with the nonzero factor $n2^{n-1}$, then cancels that factor and uses parent 2 to characterize when the complementary binomial coefficient is 1. Neither parent alone determines the child's iff.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2661 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B uses $\binom{n-k}{1}$ instead of Statement A's $\binom{n}{n-k}$, changing the conclusion.

Goal round-trip: what Lean elaborated

```
theorem weighted_binomial_eq_scaled_complementary_iff.extracted_1_1 (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (hkn : k ≤ n) :
  ∑ j ∈ Finset.Icc 1 n, j * n.choose j = n * 2 ^ (n - 1) * n.choose (n - k) ↔ k = n
```

Read back by a model that saw only the Lean. For natural numbers n and k , if n is positive, k is positive, and $k \leq n$, then the sum from $j = 1$ to n of j times the binomial coefficient $\binom{n}{j}$ equals $(n \cdot 2^{n-1} \cdot \binom{n-k}{1})$ if and only if $k = n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators

theorem weighted_binomial_eq_scaled_complementary_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (hkn : k ≤ n) :
  (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
    (n * 2 ^ (n - 1)) * Nat.choose n (n - k) ↔ k = n := by
  have hset : Finset.range (n + 1) = insert 0 (Finset.Icc 1 n) := by
    rw [Finset.range_eq_Ico]
    rw [← Finset.insert_Ico_add_one_left_eq_Ico (a := 0) (b := n + 1) (by omega)]
    simp [Finset.Ico_add_one_right_eq_Icc]
  have hsum :
    (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) = n * 2 ^ (n - 1) := by
    calc
      (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
        ∑ j ∈ insert 0 (Finset.Icc 1 n), j * Nat.choose n j := by
          rw [Finset.sum_insert (by simp)]
          simp
      _ = ∑ j ∈ Finset.range (n + 1), j * Nat.choose n j := by
          rw [hset]
      _ = n * 2 ^ (n - 1) := Nat.sum_range_mul_choose n
  have hnorm : 0 < n * 2 ^ (n - 1) := by
    positivity
  have hcomp : Nat.choose n (n - k) = 1 ↔ k = n := by
    have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
      Nat.choose_symm hkn
    have hone : Nat.choose n k = 1 ↔ k = 0 ∨ n = k :=
      Nat.choose_eq_one_iff
  constructor
  · intro h
    have hcases : k = 0 ∨ n = k := hone.mp (hsymm.symm.trans h)
    rcases hcases with hk0 | hnk
    · omega
    · exact hnk.symm
  · intro h
    subst k
    simp
  have hbridge :
    (∑ j ∈ Finset.Icc 1 n, j * Nat.choose n j) =
      (n * 2 ^ (n - 1)) * Nat.choose n (n - k) ↔
      Nat.choose n (n - k) = 1 := by
    constructor
    · intro h
      have hmul :
```

```

      (n * 2 ^ (n - 1)) * 1 =
      (n * 2 ^ (n - 1)) * Nat.choose n (n - k) := by
calc
  (n * 2 ^ (n - 1)) * 1 = n * 2 ^ (n - 1) := by simp
  _ =  $\sum j \in \text{Finset.Icc } 1 \ n, j * \text{Nat.choose } n \ j := \text{hsum.symm}$ 
  _ = (n * 2 ^ (n - 1)) * Nat.choose n (n - k) := h
have hone : 1 = Nat.choose n (n - k) :=
  Nat.mul_left_cancel hnorm hmul
exact hone.symm
· intro h
  rw [hsum, h]
  simp
have hcoupled :
  ( $\sum j \in \text{Finset.Icc } 1 \ n, j * \text{Nat.choose } n \ j$ ) =
    (n * 2 ^ (n - 1)) * Nat.choose n (n - k)  $\leftrightarrow$  k = n :=
  hbridge.trans hcomp
exact hcoupled

```

Operator. crossover / crossover_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_summulnckeqnmul2pownm1__x__numbertheory_nckeqnm1ckpnm1ckm1__ms.me.xe__e5dfcd7a

Fusion mechanism. parameter_coupling Use parent 1 to rewrite the global sum into a nonzero normalization factor, then cancel that factor so parent 4 classifies the remaining complementary coefficient.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e5dfcd7afa017abc

4.11 011. bounded_rational_recurrence_transfer

The problem

Let t and u be sequences of rational numbers satisfying $t_1 = u_1 = 20$, $t_2 = u_2 = 21$, and, for every integer $n \geq 3$, $t_n = \frac{5t_{n-1}+1}{25t_{n-2}}$ and $u_n = \frac{5u_{n-1}+1}{25u_{n-2}}$. If the denominator of t_{2020} plus its numerator equals 626, prove that $t_n = u_n$ for every $1 \leq n \leq 2020$, and that the denominator of u_{2020} plus its numerator also equals 626.

Lean statement

```
theorem bounded_rational_recurrence_transfer (t u : ℕ → ℚ)
  (ht₀ : t 1 = 20) (ht₁ : t 2 = 21)
  (ht₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2)))
  (hu₀ : u 1 = 20) (hu₁ : u 2 = 21)
  (hu₂ : ∀ n ≥ 3, u n = (5 * u (n - 1) + 1) / (25 * u (n - 2)))
  (hInv : ↑(t 2020).den + (t 2020).num = 626) :
  (∀ n, 1 ≤ n → n ≤ 2020 → t n = u n) ∧
  ↑(u 2020).den + (u 2020).num = 626
```

Parent. Define a sequence recursively by $t_1 = 20$, $t_2 = 21$, and $t_n = \frac{5t_{n-1}+1}{25t_{n-2}}$ for all $n \geq 3$. Then t_{2020} can be expressed as $\frac{p}{q}$, where p and q are relatively prime positive integers. Find $p + q$. Show that it is 626.

Parent in Lean

```
theorem aimeII_2020_p6 (t : ℕ → ℚ) (h₀ : t 1 = 20) (h₁ : t 2 = 21)
  (h₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2))) :
  ↑(t 2020).den + (t 2020).num = 626 := by
```

What it contributes. Supplies the rational recurrence, initial values, bounded horizon 2020, and endpoint numerator-denominator invariant transferred here to a second admissible sequence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive bounded uniqueness theorem: strong induction proves $t_n = u_n$ for every index from the shared initial data and recurrence, then transfers the endpoint invariant to u . The parent's explicit five-step cycle proof does not supply this sequence-comparison argument, and memorizing it offers little help.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new bounded uniqueness law by strong induction, showing $t_n = u_n$ from shared initial values and recurrence, then transfers the endpoint invariant. The parent's five-cycle computation does not supply this sequence-to-sequence uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new bounded uniqueness law by strong induction, comparing both sequences at each predecessor and predecessor-two recurrence step. The endpoint transfer then uses this newly established equality, so the parent's five-cycle computation is not recalled or embedded.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1735 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_rational_recurrence_transfer.extracted_1_1 (t u : ℕ → ℚ) (ht₀ : t 1 = 20) (ht₁ : t 2 = 21)
  (ht₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2))) (hu₀ : u 1 = 20) (hu₁ : u 2 = 21)
  (hu₂ : ∀ n ≥ 3, u n = (5 * u (n - 1) + 1) / (25 * u (n - 2))) (hInv : ↑(t 2020).den + (t 2020).num = 626) :
  (∀ (n : ℕ), 1 ≤ n → n ≤ 2020 → t n = u n) ∧ ↑(u 2020).den + (u 2020).num = 626
```

Read back by a model that saw only the Lean. Let t and u be functions from the natural numbers to the rational numbers. Suppose $t(1)=20$, $t(2)=21$, and for every natural number n with $n \geq 3$, $t(n)=(5t(n-1)+1)/(25t(n-2))$, where the subtractions are natural-number subtractions. Suppose likewise that $u(1)=20$, $u(2)=21$, and for every natural number n with $n \geq 3$, $u(n)=(5u(n-1)+1)/(25u(n-2))$. Suppose also that the natural-number denominator of $t(2020)$, coerced to a rational number, plus the numerator of $t(2020)$, equals 626. Then, for every natural number n with $1 \leq n \leq 2020$, $t(n)=u(n)$, and the denominator of $u(2020)$, coerced to a rational number, plus the numerator of $u(2020)$, equals 626.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_rational_recurrence_transfer (t u : ℕ → ℚ)
  (ht₀ : t 1 = 20) (ht₁ : t 2 = 21)
  (ht₂ : ∀ n ≥ 3, t n = (5 * t (n - 1) + 1) / (25 * t (n - 2)))
  (hu₀ : u 1 = 20) (hu₁ : u 2 = 21)
  (hu₂ : ∀ n ≥ 3, u n = (5 * u (n - 1) + 1) / (25 * u (n - 2)))
  (hInv : ↑(t 2020).den + (t 2020).num = 626) :
  (∀ n, 1 ≤ n → n ≤ 2020 → t n = u n) ∧
  ↑(u 2020).den + (u 2020).num = 626 := by
  have huniq : ∀ n : ℕ, 1 ≤ n → n ≤ 2020 → t n = u n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn hN
      by_cases h1 : n = 1
      · subst n
        exact ht₀.trans hu₀.symm
      by_cases h2 : n = 2
      · subst n
        exact ht₁.trans hu₁.symm
      have hn3 : 3 ≤ n := by omega
      have hp1 : t (n - 1) = u (n - 1) :=
        ih (n - 1) (by omega) (by omega) (by omega)
      have hp2 : t (n - 2) = u (n - 2) :=
        ih (n - 2) (by omega) (by omega) (by omega)
      have ht := ht₂ n (by omega)
      have hu := hu₂ n (by omega)
      calc
        t n = (5 * t (n - 1) + 1) / (25 * t (n - 2)) := ht
        _ = (5 * u (n - 1) + 1) / (25 * u (n - 2)) := by rw [hp1, hp2]
        _ = u n := hu.symm
  constructor
  · exact huniq
  · rw [← huniq 2020 (by norm_num) (by norm_num)]
    exact hInv
```


Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aimeII_2020_p6__me__ce035b37

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 15ad3ed09766e4d3

4.12 012. parameterized_radical_root_product

The problem

For $0 \leq t \leq 1$, let the distinct real roots of $x^2 + 18x + 30 + t = 2\sqrt{x^2 + 18x + 45}$ be collected. Prove that their product is $28 + t - 2\sqrt{16 - t}$.

Lean statement

```
theorem parameterized_radical_root_product (t : ℝ) (f : ℝ → ℝ)
  (ht₁ : t ≤ 1)
  (h₀ : ∀ x, f x = x ^ 2 + (18 * x + 30 + t) - 2 * Real.sqrt (x ^ 2 + (18 * x + 45)))
  (h₁ : Fintype (f ⁻¹' {0})) :
  (∏ x ∈ (f ⁻¹' {0}).toFinset, x) = 28 + t - 2 * Real.sqrt (16 - t) := by
```

Parent. What is the product of the $\mathbb{R}$ roots of the equation $x^2 + 18x + 30 = 2\sqrt{x^2 + 18x + 45}$? Show that it is 020.

Parent in Lean

```
theorem aime_1983_p3 (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = x ^ 2 + (18 * x + 30) - 2 * Real.sqrt (x ^ 2 + (18 * x + 45)))
  (h₁ : Fintype (f ⁻¹' {0})) : (∏ x ∈ (f ⁻¹' {0}).toFinset, x) = 20 := by
```

What it contributes. Retains the parent’s radical substitution, nonnegativity checkpoint, exact root-set characterization, finite-set product normalization, and root-product computation, while replacing the fixed coefficients by the bounded parameter t and adding explicit extraneous-branch exclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The parameter t genuinely changes the radical equation and forces a new quadratic-branch analysis, including admissibility of the negative branch and a parameter-dependent root product. The parent’s fixed constant argument does not directly establish the child’s symbolic formula.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The parameter t remains symbolic and changes the admissible quadratic branch, requiring new domain and positivity arguments before deriving the roots and their product. The parent’s fixed- $t=0$ proof does not directly establish this parameterized theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The parameter t changes the quadratic branch structure: proving the admissible branch requires using $t \leq 1$ to show $\sqrt{16-t} > 1$ and exclude the extraneous solution. The resulting symbolic root product is not obtainable by simply recalling the parent’s $t=0$ proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2699 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_radical_root_product.extracted_1_1 (t : ℝ) (f : ℝ → ℝ) (ht₁ : t ≤ 1)
  (h₀ : ∀ (x : ℝ), f x = x ^ 2 + (18 * x + 30 + t) - 2 * √(x ^ 2 + (18 * x + 45))) (h₁ : Fintype ↑(f ⁻¹' {0})) :
  ∏ x ∈ (f ⁻¹' {0}).toFinset, x = 28 + t - 2 * √(16 - t)
```

Read back by a model that saw only the Lean. Let t be a real number with $t \leq 1$, and let $f: \mathbb{R} \rightarrow \mathbb{R}$ satisfy, for every real x , $f(x) = x^2 + (18x + 30 + t) - 2\sqrt{x^2 + (18x + 45)}$. Assume that the set of real roots of f , namely $f^{-1}(\{0\})$, is finite. Then the product of all real numbers x such that $f(x) = 0$ is $28 + t - 2\sqrt{16 - t}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem parameterized_radical_root_product (t : ℝ) (f : ℝ → ℝ)
  (ht₁ : t ≤ 1)
  (h₀ : ∀ x, f x = x ^ 2 + (18 * x + 30 + t) - 2 * Real.sqrt (x ^ 2 + (18 * x + 45)))
  (h₁ : Fintype (f ⁻¹' {0})) :
  (∏ x ∈ (f ⁻¹' {0}).toFinset, x) = 28 + t - 2 * Real.sqrt (16 - t) := by
  letI := h₁
  let r : ℝ := Real.sqrt (16 - t)
  let q : ℝ := Real.sqrt (36 + (1 + r) ^ 2)
  have hrt : 0 ≤ 16 - t := by linarith
  have hr : 0 ≤ r := by
    dsimp [r]
    exact Real.sqrt_nonneg _
  have hr2 : r ^ 2 = 16 - t := by
    dsimp [r]
    nlinarith [Real.sq_sqrt hrt]
  have hro : 1 < r := by
    nlinarith [hr2]
  have hqarg : 0 ≤ (36 : ℝ) + (1 + r) ^ 2 := by
    nlinarith [sq_nonneg (1 + r)]
  have hq : q ^ 2 = 36 + (1 + r) ^ 2 := by
    dsimp [q]
    nlinarith [Real.sq_sqrt hqarg]
  have hchar : ∀ x : ℝ, f x = 0 ↔ x = -9 - q ∨ x = -9 + q := by
    intro x
    constructor
    · intro hx
      have he := h₀ x
      rw [hx] at he
      let z : ℝ := x ^ 2 + (18 * x + 45)
      have hez : 0 = z - 15 + t - 2 * Real.sqrt z := by
        dsimp [z]
        nlinarith [he]
      have hz : 0 ≤ z := by
        nlinarith [hez, Real.sqrt_nonneg z]
      have hv : (Real.sqrt z) ^ 2 - 2 * Real.sqrt z + t - 15 = 0 := by
        nlinarith [hez, Real.sq_sqrt hz]
      have hbranches :
        (Real.sqrt z - (1 + r)) * (Real.sqrt z - (1 - r)) = 0 := by
          nlinarith [hv, hr2]
      have hvalid : Real.sqrt z = 1 + r := by
        rcases mul_eq_zero.mp hbranches with hplus | hminus
        · linarith
        · exfalso
          nlinarith [Real.sqrt_nonneg z, hro]
      have hzs : z = (1 + r) ^ 2 := by
```

```

    nlinarith [Real.sq_sqrt hz, hvalid]
have hfactor : (x + 9 - q) * (x + 9 + q) = 0 := by
  dsimp [z] at hzs
  nlinarith [hq, hzs]
rcases mul_eq_zero.mp hfactor with hleft | hright
· right
  linarith
· left
  linarith
· intro hx
  have hy : 0 ≤ 1 + r := by linarith
  have hsq : Real.sqrt ((1 + r) ^ 2) = 1 + r := by
    have hs := Real.sq_sqrt (sq_nonneg (1 + r))
    nlinarith [Real.sqrt_nonneg ((1 + r) ^ 2)]
  rcases hx with rfl | rfl
  · have harg : (-9 - q) ^ 2 + (18 * (-9 - q) + 45) = (1 + r) ^ 2 := by
      nlinarith [hq]
      rw [h₀, harg, hsq]
      nlinarith [hq, hr2]
  · have harg : (-9 + q) ^ 2 + (18 * (-9 + q) + 45) = (1 + r) ^ 2 := by
      nlinarith [hq]
      rw [h₀, harg, hsq]
      nlinarith [hq, hr2]
have hset : f ⁻¹' {0} = ({-9 - q, -9 + q} : Set ℝ) := by
  ext x
  simp only [Set.mem_preimage, Set.mem_singleton_iff, Set.mem_insert_iff,
    Set.mem_singleton_iff]
  exact hchar x
have hfin : (f ⁻¹' {0}).toFinset = ({-9 - q, -9 + q} : Finset ℝ) := by
  ext x
  simp [hset]
rw [hfin]
rw [Finset.prod_insert]
· rw [Finset.prod_singleton]
  change (-9 - q) * (-9 + q) = 28 + t - 2 * r
  nlinarith [hq, hr2]
· intro h
  simp only [Finset.mem_singleton] at h
  nlinarith [hq]

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p3__me__c0629d00

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3925eaedea382347

4.13 013. aime_1984_p15_x_squared_scaled

The problem

Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = r$, where $r \geq 0$. Show that $x^2 = \frac{11025}{1024} r$.

Lean statement

```
theorem aime_1984_p15_x_squared_scaled (x y z w r : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = r)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = r)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = r)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = r) :
  x ^ 2 = (11025 / 1024 : ℝ) * r
```

Parent. Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = 1$. What exact value must x^2 equal?

Parent in Lean

```
theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

What it contributes. Retains the parent's four weighted equations and normalization/elimination method, while replacing the fixed right-hand side 1 with the shared nonnegative parameter r .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the common right-hand side, requiring the solver to derive the symbolic scaling relation $x^2 = (11025/1024)r$ rather than the parent's fixed numerical value. The proof uses the same linear structure but the parent's conclusion cannot simply be recalled, and no hypothesis is redundant.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the common right-hand side and proves the resulting scaling law for x^2 , rather than merely decorating the fixed-value conclusion. The parameter r remains mathematically essential and is used in the derived equation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the shared right-hand side and derives the scaled value of x^2 , requiring the coefficient relation to remain symbolic in r . It is not a wrapper or decoration, and the parent's fixed-value conclusion cannot be recalled verbatim.

Check

statement_type_check

By

lean

Result

type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2724 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x_squared_scaled.extracted_1_1 (x y z w r : ℝ) (hr : 0 ≤ r)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = r)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = r)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = r)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = r) :
  x ^ 2 = 11025 / 1024 * r
```

Read back by a model that saw only the Lean. For real numbers x, y, z, w , and r with $r \geq 0$, if $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = r$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = r$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = r$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = r$, then $x^2 = (11025/1024)r$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x_squared_scaled (x y z w r : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = r)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = r)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = r)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = r) :
  x ^ 2 = (11025 / 1024 : ℝ) * r := by
  norm_num at h₀ h₁ h₂ h₃
  have hlinear : 1024 * x ^ 2 = 11025 * r := by
    linarith [h₀, h₁, h₂, h₃]
  have hscale : x ^ 2 = (11025 / 1024 : ℝ) * r := by
    linarith [hlinear]
  exact hscale
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh.ms.me__bebfb24

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint a5628f834d130201

4.14 014. aime_1984_p15_x

The problem

Suppose x, y, z, w are real numbers satisfying the four weighted-square equations above, and $x \geq 0$. Determine x .

Lean statement

```
theorem aime_1984_p15_x (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
  (hx : 0 ≤ x) : x = 105 / 32 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  nlinarith [hx_sq]
```

Parent. Determine $x^2 + y^2 + z^2 + w^2$ if $\frac{x^2}{2^2-1} + \frac{y^2}{2^2-3^2} + \frac{z^2}{2^2-5^2} + \frac{w^2}{2^2-7^2} = 1$ $\frac{x^2}{4^2-1} + \frac{y^2}{4^2-3^2} + \frac{z^2}{4^2-5^2} + \frac{w^2}{4^2-7^2} = 1$ $\frac{x^2}{6^2-1} + \frac{y^2}{6^2-3^2} + \frac{z^2}{6^2-5^2} + \frac{w^2}{6^2-7^2} = 1$ $\frac{x^2}{8^2-1} + \frac{y^2}{8^2-3^2} + \frac{z^2}{8^2-5^2} + \frac{w^2}{8^2-7^2} = 1$ Show that it is 036.

Parent in Lean

```
theorem aime_1984_p15 (x y z w : ℝ)
  (h₀ :
    x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
    w ^ 2 / (2 ^ 2 - 7 ^ 2) =
    1)
  (h₁ :
    x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
    w ^ 2 / (4 ^ 2 - 7 ^ 2) =
    1)
  (h₂ :
    x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
    w ^ 2 / (6 ^ 2 - 7 ^ 2) =
    1)
  (h₃ :
    x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
    w ^ 2 / (8 ^ 2 - 7 ^ 2) =
    1) :
  x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
```

What it contributes. Retains the parent's full four-equation weighted-square system and its normalization/elimination checkpoint, while specializing with $x \geq 0$ and identifying the coordinate rather than the total energy.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from the total squared norm to a specific coordinate and uses the new non-negativity hypothesis to select the positive root. The parent's proof only derives the aggregate sum and does not determine x .

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from the total sum of squares to a uniquely determined coordinate value. It must first isolate x^2 from the four equations and then use nonnegativity to select the positive square root, reasoning not supplied by the parent's concluding equality.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from determining the total sum of squares to determining the unique nonnegative value of x . The parent's linear elimination only yields the aggregate result; the child additionally derives x^2 and uses nonnegativity with nonlinear reasoning, so recalling the parent proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2696 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x.extracted_1_1 (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
(hx : 0 ≤ x) : x = 105 / 32
```

Read back by a model that saw only the Lean. Let x, y, z , and w be real numbers satisfying $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. If x is nonnegative, then $x = 105/32$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
(hx : 0 ≤ x) : x = 105 / 32 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  nlinarith [hx_sq]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1984_p15__me__3f82f5b1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3f82f5b1b213b0d1

4.15 015. squared_coordinate_classification

The problem

The four quadratic normalization identities uniquely determine the squared coordinates: $x^2 = 11025/1024$, $y^2 = 10395/1024$, $z^2 = 9009/1024$, and $w^2 = 6435/1024$.

Lean statement

```
theorem squared_coordinate_classification (x y z w : ℝ)
(h₀ :
  x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
  w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ :
  x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
  w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ :
  x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
  w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ :
  x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
  w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x ^ 2 = (11025 : ℝ) / 1024 ∧
y ^ 2 = (10395 : ℝ) / 1024 ∧
z ^ 2 = (9009 : ℝ) / 1024 ∧
w ^ 2 = (6435 : ℝ) / 1024
```

Parent. Determine $x^2 + y^2 + z^2 + w^2$ if $\frac{x^2}{2^2-1} + \frac{y^2}{2^2-3^2} + \frac{z^2}{2^2-5^2} + \frac{w^2}{2^2-7^2} = 1$
 $\frac{y^2}{4^2-3^2} + \frac{z^2}{4^2-5^2} + \frac{w^2}{4^2-7^2} = 1$
 $\frac{x^2}{6^2-1} + \frac{y^2}{6^2-3^2} + \frac{z^2}{6^2-5^2} + \frac{w^2}{6^2-7^2} = 1$
 $\frac{x^2}{8^2-1} + \frac{y^2}{8^2-3^2} + \frac{z^2}{8^2-5^2} + \frac{w^2}{8^2-7^2} = 1$ Show that it is 036.

Parent in Lean

```
theorem aime_1984_p15 (x y z w : ℝ)
(h₀ :
  x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
  w ^ 2 / (2 ^ 2 - 7 ^ 2) =
1)
(h₁ :
  x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
  w ^ 2 / (4 ^ 2 - 7 ^ 2) =
1)
(h₂ :
  x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
  w ^ 2 / (6 ^ 2 - 7 ^ 2) =
1)
(h₃ :
  x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
  w ^ 2 / (8 ^ 2 - 7 ^ 2) =
1) :
x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
```

What it contributes. Retains the original variables and all four quadratic normalization identities, while replacing the aggregate-energy target with the uniquely determined squared-coordinate vector.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child replaces the parent's single total-energy consequence with a full classification of all four squared coordinates. Although the same normalization and linear-arithmetic method applies, the parent proof does not supply these individual values, so this is a substantive change of shape.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the conclusion from the total squared-energy sum to a full classification of each squared coordinate. Proving four individual values requires solving the underlying four-equation linear system; the parent's final linarith step for the aggregate identity does not supply those coefficients.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a single total-energy sum to a full classification of all four squared coordinates. Proving each coordinate requires solving the entire linear system, which is not supplied by the parent's direct total-sum 'linarith' proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1820 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem squared_coordinate_classification.extracted_1_1 (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024
```

Read back by a model that saw only the Lean. Let $x, y, z,$ and w be real numbers. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x^2 = 11025/1024$, $y^2 = 10395/1024$, $z^2 = 9009/1024$, and $w^2 = 6435/1024$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem squared_coordinate_classification (x y z w : ℝ)
(h₀ :
  x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) +
  w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ :
  x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) +
  w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ :
  x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) +
  w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ :
  x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) +
  w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x ^ 2 = (11025 : ℝ) / 1024 ∧
y ^ 2 = (10395 : ℝ) / 1024 ∧
z ^ 2 = (9009 : ℝ) / 1024 ∧
```

```

w ^ 2 = (6435 : ℝ) / 1024 := by
norm_num at h₀ h₁ h₂ h₃
have hclass :
  x ^ 2 = (11025 : ℝ) / 1024 ∧
  y ^ 2 = (10395 : ℝ) / 1024 ∧
  z ^ 2 = (9009 : ℝ) / 1024 ∧
  w ^ 2 = (6435 : ℝ) / 1024 := by
constructor
· linarith [h₀, h₁, h₂, h₃]
constructor
· linarith [h₀, h₁, h₂, h₃]
constructor
· linarith [h₀, h₁, h₂, h₃]
· linarith [h₀, h₁, h₂, h₃]
exact hclass

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1984_p15__me__8fb70bef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b1915c90a2b9f7a

4.16 016. cubic_affine_evaluation_injective

The problem

For complex numbers a , b , and c , if the affine coefficient $a^2 - (b + c)a + cb$ is nonzero, then the induced affine evaluation map is injective.

Lean statement

```
theorem cubic_affine_evaluation_injective (a b c : ℂ)
  (hs : a ^ 2 - (b + c) * a + c * b ≠ 0) :
  Function.Injective
    (fun x : ℂ =>
      -((a ^ 2 - (b + c) * a + c * b) * x) +
      (a ^ 2 - (b + c) * a + c * b) * a)
```

Parent. For complex numbers a, b, c, d, e , if $(a - d)(a - c)(a - b) = -((a^2 - (b + c)a + cb)d) + (a^2 - (b + c)a + cb)a$ and $(a - e)(a - c)(a - b) = -((a^2 - (b + c)a + cb)e) + (a^2 - (b + c)a + cb)a$, with $a - c \neq 0$, then the two evaluations are equal if and only if $d = e$ or $a = b$.

Parent in Lean

```
theorem cubic_factored_evaluation_collision (a b c d e : ℂ)
  (hfacd : (a - d) * (a - c) * (a - b) =
    -((a ^ 2 - (b + c) * a + c * b) * d) +
    (a ^ 2 - (b + c) * a + c * b) * a)
  (hfcae : (a - e) * (a - c) * (a - b) =
    -((a ^ 2 - (b + c) * a + c * b) * e) +
    (a ^ 2 - (b + c) * a + c * b) * a)
  (hnd : a - c ≠ 0) :
  (-((a ^ 2 - (b + c) * a + c * b) * d) +
    (a ^ 2 - (b + c) * a + c * b) * a =
    -((a ^ 2 - (b + c) * a + c * b) * e) +
    (a ^ 2 - (b + c) * a + c * b) * a) ↔
  d = e ∨ a = b
```

What it contributes. Reuses the parent's complex affine evaluation expression while promoting its pairwise collision analysis to a function-level injectivity theorem with an explicit nonzero-slope hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the collision disjunction into injectivity of an affine map and strengthens nondegeneracy to a nonzero slope. Its proof requires direct cancellation of that symbolic coefficient, rather than reusing the parent's two-point transport argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the collision characterization into injectivity of an affine map and proves it by deriving a nonzero slope and cancelling it. This is not a recalled application of the parent's disjunction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the two-point collision equivalence into injectivity of the affine evaluation map and proves it by explicit nonzero-slope cancellation. This is not a recall wrapper around the parent's disjunctive factorization argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1858 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_affine_evaluation_injective.extracted_1_1 (a b c : ℂ) (hs : a ^ 2 - (b + c) * a + c * b ≠ 0) :
  Function.Injective fun x => -((a ^ 2 - (b + c) * a + c * b) * x) + (a ^ 2 - (b + c) * a + c * b) * a
```

Read back by a model that saw only the Lean. For complex numbers a , b , and c , if $a^2 - (b + c)a + cb \neq 0$, then the function from $\mathbb{C}$ to $\mathbb{C}$ defined by $x \mapsto -(a^2 - (b + c)a + cb)x + (a^2 - (b + c)a + cb)a$ is injective.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem cubic_affine_evaluation_injective (a b c : ℂ)
  (hs : a ^ 2 - (b + c) * a + c * b ≠ 0) :
  Function.Injective
    (fun x : ℂ =>
      -((a ^ 2 - (b + c) * a + c * b) * x) +
      (a ^ 2 - (b + c) * a + c * b) * a) := by
  have hslope : -(a ^ 2 - (b + c) * a + c * b) ≠ 0 := by
    exact neg_ne_zero.mpr hs
  intro x y hxy
  have hdiff :
    (-(a ^ 2 - (b + c) * a + c * b)) * x =
    (-(a ^ 2 - (b + c) * a + c * b)) * y := by
    linear_combination hxy
  exact mul_left_cancel₀ hslope hdiff
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta__me.mh.me__7de9810f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7a676a66f66e1f02

4.17 017. cubic_zero_locus_exact

The problem

Let a, b , and c be pairwise distinct complex numbers. Show that the zeros of $(x - a)(x - b)(x - c)$ are exactly a, b , and c , with each zero different from the other two.

Lean statement

```
theorem cubic_zero_locus_exact (a b c : ℂ) (hab : a ≠ b) (hac : a ≠ c) (hbc : b ≠ c) :
  ∀ x : ℂ, (x - a) * (x - b) * (x - c) = 0 ↔
    ((x = a ∧ x ≠ b ∧ x ≠ c) ∨
     (x = b ∧ x ≠ a ∧ x ≠ c) ∨
     (x = c ∧ x ≠ a ∧ x ≠ b))
```

Parent. Show that for any complex numbers a, b, c, d , $(a-d)(a-c)(a-b) = -(((a^2 - (b+c)a) + cb)d) + (a^2 - (b+c)a + cb)a$.

Parent in Lean

```
theorem algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta (a b c d : ℂ) :
  (a - d) * (a - c) * (a - b) =
  -((a ^ 2 - (b + c) * a + c * b) * d) + (a ^ 2 - (b + c) * a + c * b) * a := by
```

What it contributes. Reuses the parent cubic factorization surface, but changes the conclusion to an exact zero-locus classification with converse and distinctness exclusions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes an algebraic identity into an exact zero-locus characterization under pairwise-distinctness hypotheses. Its proof requires factoring a zero product, case-splitting over complex factors, and deriving uniqueness inequalities, none of which follows from recalling the parent's ring proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves an exact zero-locus characterization using pairwise distinctness and case analysis on vanishing factors. The parent's purely algebraic ring identity supplies none of this reasoning, so recalling it does not substantially help.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine zero-locus characterization requiring factor-by-factor case analysis and use of pairwise distinctness hypotheses. The parent is only a ring identity and provides none of this reasoning, so recalling its proof does not substantially help.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2729 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_zero_locus_exact.extracted_1_1 : ∀ (a b c : ℂ),
  a ≠ b →
  a ≠ c →
  b ≠ c →
  ∀ (x : ℂ),
    (x - a) * (x - b) * (x - c) = 0 ↔
    x = a ∧ x ≠ b ∧ x ≠ c ∨ x = b ∧ x ≠ a ∧ x ≠ c ∨ x = c ∧ x ≠ a ∧ x ≠ b
```

Read back by a model that saw only the Lean. For all complex numbers a , b , and c such that $a \neq b$, $a \neq c$, and $b \neq c$, and for every complex number x , $(x - a)(x - b)(x - c) = 0$ if and only if either $x = a$ and $x \neq b$ and $x \neq c$, or $x = b$ and $x \neq a$ and $x \neq c$, or $x = c$ and $x \neq a$ and $x \neq b$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem cubic_zero_locus_exact (a b c : ℂ) (hab : a ≠ b) (hac : a ≠ c) (hbc : b ≠ c) :
  ∀ x : ℂ, (x - a) * (x - b) * (x - c) = 0 ↔
    ((x = a ∧ x ≠ b ∧ x ≠ c) ∨
     (x = b ∧ x ≠ a ∧ x ≠ c) ∨
     (x = c ∧ x ≠ a ∧ x ≠ b)) := by
  intro x
  constructor
  · intro hx
    have hfactor : ((x - a) * (x - b)) * (x - c) = 0 := by
      simpa [mul_assoc] using hx
    rcases mul_eq_zero.mp hfactor with h_ab | h_c
    · rcases mul_eq_zero.mp h_ab with h_a | h_b
      · left
        have hxa : x = a := sub_eq_zero.mp h_a
        refine ⟨hxa, ?_, ?_⟩
        · intro hxb
          exact hab (hxa.symm.trans hxb)
        · intro hxc
          exact hac (hxa.symm.trans hxc)
      · right
        left
        have hxb : x = b := sub_eq_zero.mp h_b
        refine ⟨hxb, ?_, ?_⟩
        · intro hxa
          exact hab (hxa.symm.trans hxb)
        · intro hxc
          exact hbc (hxb.symm.trans hxc)
    · right
      right
      have hxc : x = c := sub_eq_zero.mp h_c
      refine ⟨hxc, ?_, ?_⟩
      · intro hxa
        exact hac (hxa.symm.trans hxc)
      · intro hxb
        exact hbc (hxb.symm.trans hxc)
  · intro hx
    rcases hx with hxa | hxb | hxc
    · rcases hxa with ⟨rfl, _, _⟩
      simp
    · rcases hxb with ⟨rfl, _, _⟩
      simp
    · rcases hxc with ⟨rfl, _, _⟩
      simp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta__me__6f44f522

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6f44f522c277cf76

4.18 018. ternary_coupled_reciprocal_strict

The problem

Let $d \in \{0, 1, 2\}$, and suppose a natural number has base-three representation $122d_3$. If positive reals a, b, c have sum equal to that number and are not all equal, show that $\frac{9}{d+51} < \frac{2}{a+b} + \frac{2}{b+c} + \frac{2}{c+a}$.

Lean statement

```
theorem ternary_coupled_reciprocal_strict
  (d n : ℕ) (a b c : ℝ)
  (hdigits : Nat.digits 3 n = [d, 2, 2, 1])
  (hpos : 0 < a ∧ 0 < b ∧ 0 < c)
  (hne : ¬ (a = b ∧ b = c))
  (htotal : a + b + c = (n : ℝ)) :
  9 / ((d : ℝ) + 51) < 2 / (a + b) + 2 / (b + c) + 2 / (c + a)
```

Parent. Show that for any three positive real numbers x, y , and z , $9/(x + y + z) \leq 2/(x + y) + 2/(y + z) + 2/(z + x)$.

Parent in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (ha : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Supplies the reciprocal-inequality clearing strategy, positivity checkpoints, common-denominator rewrites, and weighted squared-difference certificate; the child strengthens it to strictness and couples the total to the decoded parameter.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine strict strengthening: the non-equality hypothesis requires proving a positive squared-difference contribution, not merely replaying the parent's non-strict certificate. The digit coupling also forces the parameterized denominator through a new base-3 decoding argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the parent's non-strict inequality: the non-equality hypothesis is used to make one squared-difference term strictly positive, yielding a strict numerator certificate before clearing denominators. The digit decoding and total constraint also require additional reasoning, and the parent's proof alone cannot establish the strict conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the parent inequality to a strict one and must use the non-equality hypothesis to prove a squared-difference term is positive. The digit decoding and total constraint also replace the parent's symbolic denominator with a parameterized expression, so recalling the parent's non-strict proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2543 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_coupled_reciprocal_strict.extracted_1_1 (d n : ℕ) (a b c : ℝ) (hdigits : Nat.digits 3 n = [d, 2, 2, 1])
  (hpos : 0 < a ∧ 0 < b ∧ 0 < c) (hne : ¬(a = b ∧ b = c)) (htotal : a + b + c = ↑n) :
  9 / (↑d + 51) < 2 / (a + b) + 2 / (b + c) + 2 / (c + a)
```

Read back by a model that saw only the Lean. *Let d and n be natural numbers, and let a , b , and c be real numbers. Suppose that the base-3 digit list of n is exactly $[d, 2, 2, 1]$, that a , b , and c are all positive, that it is not the case that $a = b$ and $b = c$, and that $a + b + c = n$. Then $9/(d + 51) < 2/(a + b) + 2/(b + c) + 2/(c + a)$, where d and n are regarded as real numbers in the fractions.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem ternary_coupled_reciprocal_strict
  (d n : ℕ) (a b c : ℝ)
  (hdigits : Nat.digits 3 n = [d, 2, 2, 1])
  (hpos : 0 < a ∧ 0 < b ∧ 0 < c)
  (hne : ¬(a = b ∧ b = c))
  (htotal : a + b + c = (n : ℝ)) :
  9 / ((d : ℝ) + 51) < 2 / (a + b) + 2 / (b + c) + 2 / (c + a) := by
  rcases hpos with ⟨ha, hb, hc⟩
  have hvalue : n = d + 51 := by
    have hdecode : n = Nat.ofDigits 3 (Nat.digits 3 n) :=
      (Nat.ofDigits_digits 3 n).symm
    calc
      n = Nat.ofDigits 3 (Nat.digits 3 n) := hdecode
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [hdigits]
      _ = d + 51 := by norm_num [Nat.ofDigits]
  have hsum : a + b + c = (d : ℝ) + 51 := by
    calc
      a + b + c = (n : ℝ) := htotal
      _ = (d : ℝ) + 51 := by rw [hvalue]; norm_num
  have hab : 0 < a + b := by linarith
  have hbc : 0 < b + c := by linarith
  have hca : 0 < c + a := by linarith
  have habc : 0 < a + b + c := by linarith
  have h1 : 0 ≤ ((a + b) - (b + c)) ^ 2 := sq_nonneg _
  have h2 : 0 ≤ ((b + c) - (c + a)) ^ 2 := sq_nonneg _
  have h3 : 0 ≤ ((c + a) - (a + b)) ^ 2 := sq_nonneg _
  have h1' : 0 ≤ (a + b) * ((a + b) - (b + c)) ^ 2 :=
    mul_nonneg (le_of_lt hab) h1
  have h2' : 0 ≤ (b + c) * ((b + c) - (c + a)) ^ 2 :=
    mul_nonneg (le_of_lt hbc) h2
  have h3' : 0 ≤ (c + a) * ((c + a) - (a + b)) ^ 2 :=
    mul_nonneg (le_of_lt hca) h3
  have hneq : a ≠ b ∨ b ≠ c := by
    by_cases hab' : a = b
    · right
      intro hbc'
      exact hne ⟨hab', hbc'⟩
    · exact Or.inl hab'
  have hstrictnum :
    0 < 2 * (a + b + c) *
      ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b)) -
      9 * (a + b) * (b + c) * (c + a) := by
    rcases hneq with hab' | hbc'
    · have hdiff : (b + c) - (c + a) ≠ 0 := by
```

```

    intro hz
    apply hab'
    linarith
have hs : 0 < ((b + c) - (c + a)) ^ 2 := by
  have hm : 0 < ((b + c) - (c + a)) * ((b + c) - (c + a)) :=
    mul_self_pos.mpr hdiff
  simp [pow_two] using hm
have hs' : 0 < (b + c) * ((b + c) - (c + a)) ^ 2 :=
  mul_pos hbc hs
nlinarith
· have hdiff : (c + a) - (a + b) ≠ 0 := by
  intro hz
  apply hbc'
  linarith
have hs : 0 < ((c + a) - (a + b)) ^ 2 := by
  have hm : 0 < ((c + a) - (a + b)) * ((c + a) - (a + b)) :=
    mul_self_pos.mpr hdiff
  simp [pow_two] using hm
have hs' : 0 < (c + a) * ((c + a) - (a + b)) ^ 2 :=
  mul_pos hca hs
nlinarith
have hden : 0 < (a + b + c) * (a + b) * (b + c) * (c + a) := by
  positivity
have hleft :
  9 / (a + b + c) =
    (9 * (a + b) * (b + c) * (c + a)) /
      ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
    field_simp [ne_of_gt habc, ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
have hright :
  2 / (a + b) + 2 / (b + c) + 2 / (c + a) =
    (2 * (a + b + c) *
      ((a + b) * (b + c) + (b + c) * (c + a) + (c + a) * (a + b))) /
      ((a + b + c) * (a + b) * (b + c) * (c + a)) := by
    field_simp [ne_of_gt habc, ne_of_gt hab, ne_of_gt hbc, ne_of_gt hca] <|> ring
rw [← hsum, hleft, hright]
exact (div_lt_div_iff_of_pos_right hden).2 (by nlinarith [hstrictnum])

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__me__7dc1d005

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 7dc1d00539fc0f42

4.19 019. scaled_reciprocal_amgm

The problem

Let λ and x be positive real numbers. Show that $2 - \sqrt{2\lambda} \geq 2 - x - \frac{\lambda}{2x}$, with equality exactly when $x = \frac{\sqrt{2\lambda}}{2}$.

Lean statement

```
theorem scaled_reciprocal_amgm :
  ∀ (lam x : ℝ), lam > 0 → x > 0 →
    (2 - Real.sqrt (2 * lam) ≥ 2 - x - lam / (2 * x)) ∧
    ((2 - Real.sqrt (2 * lam) = 2 - x - lam / (2 * x)) ↔
     x = Real.sqrt (2 * lam) / 2) := by
```

Parent. Let x be a positive real number. Show that $2 - \sqrt{2} \geq 2 - x - \frac{1}{2x}$.

Parent in Lean

```
theorem algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x :
  ∀ x > 0, 2 - Real.sqrt 2 ≥ 2 - x - 1 / (2 * x) := by
```

What it contributes. Generalizes the parent's reciprocal AM-GM denominator-clearing strategy from fixed scale 1 to the parameterized scale λ , while adding an equality characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the scale and adds a sharp iff equality characterization, requiring symbolic square expansion and a separate equality argument. A memorized proof of the fixed-scale parent does not supply the generalized equality locus.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal inequality by a positive scale and adds a nontrivial equality characterization. Its proof must establish both directions of the equality case using the symbolic square identity, so recalling the parent's fixed-constant inequality is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the constant by introducing positive λ and additionally proves the sharp equality locus $x = \sqrt{(2\lambda)}/2$. Its proof requires symbolic square-root algebra and an equality characterization not supplied by the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2735 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem scaled_reciprocal_amgm.extracted_1_1 : ∀ (lam x : ℝ),
  lam > 0 →
  x > 0 →
  2 - √(2 * lam) ≥ 2 - x - lam / (2 * x) ∧ (2 - √(2 * lam) = 2 - x - lam / (2 * x) ↔ x = √(2 * lam) / 2)
```

Read back by a model that saw only the Lean. For all real numbers λ and x , if $\lambda > 0$ and $x > 0$, then $2 - \sqrt{(2\lambda)} \geq 2 - x - \lambda/(2x)$, and equality holds if and only if $x = \sqrt{(2\lambda)}/2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem scaled_reciprocal_amgm :
  ∀ (lam x : ℝ), lam > 0 → x > 0 →
    (2 - Real.sqrt (2 * lam) ≥ 2 - x - lam / (2 * x)) ∧
    ((2 - Real.sqrt (2 * lam) = 2 - x - lam / (2 * x)) ↔
      x = Real.sqrt (2 * lam) / 2) := by
  intro lam x hlam hx
  have hs : (Real.sqrt (2 * lam)) ^ 2 = 2 * lam := by
    have hnonneg : 0 ≤ 2 * lam := by positivity
    simpa using Real.sq_sqrt hnonneg
  have hmul :
    2 * x * Real.sqrt (2 * lam) ≤ 2 * x * x + lam := by
    have hsq : 0 ≤ (2 * x - Real.sqrt (2 * lam)) ^ 2 :=
      sq_nonneg (2 * x - Real.sqrt (2 * lam))
    nlinarith [hsq, hs]
  have hden : 0 < 2 * x := by positivity
  have hdiv :
    Real.sqrt (2 * lam) ≤ (2 * x * x + lam) / (2 * x) := by
    apply (le_div_iff₀ hden).2
    nlinarith [hmul]
  have hid :
    x + lam / (2 * x) = (2 * x * x + lam) / (2 * x) := by
    field_simp [ne_of_gt hx] <|> ring
  have hroot : Real.sqrt (2 * lam) ≤ x + lam / (2 * x) := by
    rw [hid]
    exact hdiv
  constructor
  · linarith
  · constructor
    · intro hEq
      have hsum : x + lam / (2 * x) = Real.sqrt (2 * lam) := by
        linarith [hEq]
      have hfrac :
        (2 * x * x + lam) / (2 * x) = Real.sqrt (2 * lam) := by
        rw [← hid]
        exact hsum
      have hb := (div_eq_iff (ne_of_gt hden)).mp hfrac
      have hbalance :
        2 * x * x + lam = 2 * x * Real.sqrt (2 * lam) := by
        nlinarith [hb]
      have hz : (2 * x - Real.sqrt (2 * lam)) ^ 2 = 0 := by
        nlinarith [sq_nonneg (2 * x - Real.sqrt (2 * lam)), hbalance, hs]
      nlinarith [hz]
    · intro hxeq
      subst x
      have hr : 0 < Real.sqrt (2 * lam) := by positivity
      field_simp [ne_of_gt hr] <|> nlinarith [hs]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x__me__2a3e888a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2a3e888a0ab283e4

4.20 020. bilinear_iff_square_eq_neg_one

The problem

For complex numbers a , b , and c satisfying $a + b = 2c$, prove that $ac + bc = -2$ if and only if $c^2 = -1$.

Lean statement

```
theorem bilinear_iff_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) : a * c + b * c = -2 ↔ c * c = -1
```

Parent. Given that $a + b = 2c$ and $c^2 = -1$ for complex numbers a , b , and c , show that $ac + bc = -2$.

Parent in Lean

```
theorem bilinear_of_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) : a * c + b * c = -2 := by
```

What it contributes. Reuses the parent’s distributive factorization and sum substitution checkpoints, while strengthening the conclusion to an equivalence and explicitly deriving the square condition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the one-way implication into a genuine iff: besides the parent direction, it must recover $c * c = -1$ by cancelling the nonzero scalar 2 from the bilinear equality. The shared factorization is useful, but the reverse algebraic characterization is new mathematics rather than a trivial wrapper.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse: under the sum constraint, the bilinear value determines $c*c$ via invertibility of 2. The parent supplies only the forward implication from the square constraint, so memorizing its proof does not solve the iff.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The reverse implication is a genuine new obligation: from the bilinear value it must recover $c * c = -1$ by using the factorization and cancellation by the nonzero scalar 2. The parent only proves the forward consequence and does not supply this characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1754 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bilinear_iff_square_eq_neg_one.extracted_1_1 (a b c : ℂ) (hsum : a + b = 2 * c) :  
  a * c + b * c = -2 ↔ c * c = -1
```

Read back by a model that saw only the Lean. For complex numbers a , b , and c satisfying $a + b = 2c$, the equation $ac + bc = -2$ holds if and only if $c^2 = -1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bilinear_iff_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) : a * c + b * c = -2 ↔ c * c = -1 := by
  have hfactor : a * c + b * c = 2 * (c * c) := by
    calc
      a * c + b * c = (a + b) * c := by ring
      _ = (2 * c) * c := by rw [hsum]
      _ = 2 * (c * c) := by ring
  constructor
  · intro hbilinear
    calc
      c * c = (1 / 2 : ℂ) * (2 * (c * c)) := by ring
      _ = (1 / 2 : ℂ) * (a * c + b * c) := by rw [hfactor]
      _ = -1 := by rw [hbilinear]; norm_num
  · intro hsq
    calc
      a * c + b * c = 2 * (c * c) := hfactor
      _ = -2 := by rw [hsq]; norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. algebra_manipexpr_apbeq2cceqiacpbceqm2__me.me__dbede63d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, finderprint ef375b1eca9c3081

4.21 021. bilinear_of_square_eq_neg_one

The problem

Given that $a + b = 2c$ and $c^2 = -1$ for complex numbers a , b , and c , show that $ac + bc = -2$.

Lean statement

```
theorem bilinear_of_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) : a * c + b * c = -2 := by
```

Parent. Given that $a + b = 2c$ and $c = \text{Im}(1)$, show that $ac + bc = -2$.

Parent in Lean

```
theorem algebra_manipexpr_apbeq2cceqiacpbceqm2 (a b c : ℂ) (h₀ : a + b = 2 * c)
(h₁ : c = Complex.I) : a * c + b * c = -2 := by
```

What it contributes. Preserves the parent sum relation and bilinear factoring checkpoint while replacing $c = \text{Complex.I}$ with the explicit square-root-of-negative-one hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the concrete hypothesis $c = \text{Complex.I}$ with the strictly more general algebraic condition $c * c = -1$. The bilinear rearrangement is shared, but the final simplification must use the new square-root condition directly rather than the parent's imaginary-unit lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the concrete witness $c = \text{Complex.I}$ with the algebraic condition $c * c = -1$, so the final step must use a new hypothesis rather than Complex.I_mul_I . The bilinear rearrangement is shared, but the parent's proof does not directly supply the generalized square-root argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the hypothesis from the specific value $c = \text{Complex.I}$ to the algebraic condition $c^2 = -1$, so the conclusion must be derived from a general square-root-of-negative-one property rather than the concrete imaginary-unit lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1740 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bilinear_of_square_eq_neg_one.extracted_1_1 (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) :
a * c + b * c = -2
```

Read back by a model that saw only the Lean. For complex numbers a , b , and c , if $a + b = 2c$ and $c^2 = -1$, then $ac + bc = -2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bilinear_of_square_eq_neg_one (a b c : ℂ) (hsum : a + b = 2 * c) (hsq : c * c = -1) : a * c + b * c = -2 := by
  have hbilinear : a * c + b * c = (a + b) * c := by
    ring
  have hsum_mul : (a + b) * c = (2 * c) * c := by
    rw [hsum]
  have hscaled : (2 * c) * c = 2 * (c * c) := by
    ring
  calc
    a * c + b * c = (a + b) * c := hbilinear
    _ = (2 * c) * c := hsum_mul
    _ = 2 * (c * c) := hscaled
    _ = -2 := by rw [hsq]; norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_manipexpr_apbeq2cceqiacpbceqm2__me__1fd261c9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f6782ae7e80f9e5b

4.22 022. bounded_subtractive_balance_quadratic_negative_between_roots

The problem

Let $p < q$ be distinct primes between 4 and 18, and suppose their product minus their sum is t . Prove that $z^2 - (p + q - 2)z + t + 1 < 0$ for every integer z strictly between $p - 1$ and $q - 1$.

Lean statement

```
theorem bounded_subtractive_balance_quadratic_negative_between_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (p : ℤ) - 1 < z →
      z < (q : ℤ) - 1 →
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 < 0
```

Parent. Let $p < q$ be distinct primes between 4 and 18, and suppose their product minus their sum is t . Prove that $z^2 - (p + q - 2)z + t + 1 > 0$ for every integer $z > q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_positive_right_of_larger_root :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (q : ℤ) - 1 < z →
      0 < z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1
```

What it contributes. Reuses the parent’s integer balance conversion and factorization checkpoint, then changes the sign analysis from outside the larger root to opposite factor signs inside the roots.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from positivity outside the larger root to strict negativity between both roots, requiring a new opposite-sign argument and the lemma ‘mul_neg_of_pos_of_neg’. Although it reuses the factorization derivation as instructed, the parent proof does not supply this interval-sign reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the domain from points right of the larger root to points strictly between both roots, requiring an opposite-sign argument and strict negativity via multiplication. The parent’s positive-factor proof does not establish this conclusion by recall alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from positivity to strict negativity between the two roots, requiring a new sign argument using one positive and one negative factor and ‘mul_neg_of_pos_of_neg’. Recalling the parent’s proof does not establish this conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2464 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_balance_quadratic_negative_between_roots.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
  p * q - (p + q) = t → ∀ (z : ℤ), ↑p - 1 < z → z < ↑q - 1 → z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1 < 0
```

Read back by a model that saw only the Lean. For every natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $p < q$, and $p \cdot q - (p + q) = t$ (with subtraction in the natural numbers), then for every integer z satisfying $p - 1 < z < q - 1$, we have $z^2 - (p + q - 2)z + t + 1 < 0$, where the natural numbers p , q , and t are viewed as integers in this expression.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_balance_quadratic_negative_between_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (p : ℤ) - 1 < z →
      z < (q : ℤ) - 1 →
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 < 0 := by
  intro t p q hdom hbal z hpz hqz
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne, hpq⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hprod : (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) < 0 := by
    have hleft : 0 < z - ((p : ℤ) - 1) := by
      linarith
    have hright : z - ((q : ℤ) - 1) < 0 := by
      linarith
    exact mul_neg_of_pos_of_neg hleft hright
  calc
    z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 =
      (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := by
        calc
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 =
            z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + ((p : ℤ) - 1) * ((q : ℤ) - 1) := by
              nlinarith [hfactor]
          _ = (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := by ring
    _ < 0 := hprod
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh.me__0bc05ceb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-

gerprint 0bc05ceb87eb7b15

4.23 023. amc12_2000_p6_prime_products

The problem

Which natural numbers are products of two distinct primes between 4 and 18? Show that they are exactly 35, 55, 65, 77, 85, 91, 119, 143, 187, and 221.

Lean statement

```
theorem amc12_2000_p6_prime_products :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = n) ↔
      n = 35 ∨ n = 55 ∨ n = 65 ∨ n = 77 ∨ n = 85 ∨ n = 91 ∨ n = 119 ∨ n = 143 ∨ n = 187 ∨ n = 221
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces five existence/nonexistence claims about $p*q-(p+q)$ with a full biconditional characterizing all products of two distinct bounded primes. Its forward finite enumeration and ten explicit reverse witnesses require reasoning not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from ruling out specific product-minus-sum values to a full biconditional characterization of all products of two distinct bounded primes. Its proof requires a two-variable finite enumeration and ten explicit witness constructions, which the parent's one-variable contradiction proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new biconditional characterizing all products of two distinct primes in the interval, including a converse with ten explicit witnesses. The parent's contradiction from the single product-minus-sum equation does not supply this enumeration or converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2404 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12_2000_p6_prime_products.extracted_1_1 : ∀ (n : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = n) ↔
    n = 35 ∨ n = 55 ∨ n = 65 ∨ n = 77 ∨ n = 85 ∨ n = 91 ∨ n = 119 ∨ n = 143 ∨ n = 187 ∨ n = 221
```

Read back by a model that saw only the Lean. *For every natural number n , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that p times q equals n , if and only if n is one of 35, 55, 65, 77, 85, 91, 119, 143, 187, or 221.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem amc12_2000_p6_prime_products :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = n) ↔
      n = 35 ∨ n = 55 ∨ n = 65 ∨ n = 77 ∨ n = 85 ∨ n = 91 ∨ n = 119 ∨ n = 143 ∨ n = 187 ∨ n = 221 := by
  intro n
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
    interval_cases p <|> interval_cases q <|>
      simp_all (config := {decide := true})
  · intro h
    rcases h with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
    · exact ⟨5, 7, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨13, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__me__2dc5d985

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2dc5d98568c75d91

4.24 024. bounded_prime_pair_power_residue

The problem

Let $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that there is a unique ordered pair of distinct primes $p < q$ in $[4, 18]$ satisfying $pq - (p + q) = 119$, and that for this pair $A^{6q+1} - B^{6q+1} \equiv 3 \pmod{7}$.

Lean statement

```
theorem bounded_prime_pair_power_residue (A B : Z)
  (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) :
  ∃ p q : N,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧
    p ≠ q ∧ p < q ∧ p * q - (p + q) = 119 ∧
    (∀ r s : N,
      Nat.Prime r ∧ Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧
      r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
      r = p ∧ s = q) ∧
    A ^ (6 * q + 1) - B ^ (6 * q + 1) ≡ 3 [ZMOD 7]
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : N, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Supplied the bounded prime-pair domain and the explicit witness $(11,13)$ satisfying the value 119; its bounded-check style is reused for uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses the bounded prime-pair classification to fix $q=13$, then proves a genuinely new modular exponentiation claim: $A^{79} - B^{79} \equiv 3 \pmod{7}$ from the variable congruence hypotheses. Recalling the parent supplies the witness and interval classification but does not solve the residue argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires constructing and uniquely classifying the ordered pair $(11,13)$, then using the larger coordinate to prove a separate modular exponentiation law from hypotheses on A and B. The parent's contradiction-and-interval proof does not supply either the uniqueness argument or the residue computation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires proving uniqueness of the bounded ordered prime pair and deriving a modular power-residue law from arbitrary congruence hypotheses. The parent only eliminates the first impossible candidate by bounded case analysis and supplies neither classification nor modular exponent reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2491 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_pair_power_residue.extracted_1_1 (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) :
  ∃ p q,
    Nat.Prime p ∧
    Nat.Prime q ∧
    4 ≤ p ∧
    p ≤ 18 ∧
    4 ≤ q ∧
    q ≤ 18 ∧
    p ≠ q ∧
    p < q ∧
    p * q - (p + q) = 119 ∧
    (∀ (r s : ℕ),
      Nat.Prime r ∧
      Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧ r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
      r = p ∧ s = q) ∧
    A ^ (6 * q + 1) - B ^ (6 * q + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For integers A and B , if A is congruent to 1 modulo 7 and B is congruent to 5 modulo 7, then there exist natural numbers p and q such that p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $p < q$, and $pq - (p + q) = 119$; moreover, these p and q are unique among all natural numbers r and s satisfying the same primality, bounds, distinctness, ordering, and equation conditions, in the sense that any such r and s satisfy $r = p$ and $s = q$; and $A^{(6q+1)} - B^{(6q+1)}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_prime_pair_power_residue (A B : ℤ)
  (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) :
  ∃ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧
    p ≠ q ∧ p < q ∧ p * q - (p + q) = 119 ∧
    (∀ r s : ℕ,
      Nat.Prime r ∧ Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧
      r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
      r = p ∧ s = q) ∧
    A ^ (6 * q + 1) - B ^ (6 * q + 1) ≡ 3 [ZMOD 7] := by
  have hpair :
    Nat.Prime 11 ∧ Nat.Prime 13 ∧ 4 ≤ 11 ∧ 11 ≤ 18 ∧ 4 ≤ 13 ∧ 13 ≤ 18 ∧
    11 ≠ 13 ∧ 11 < 13 ∧ 11 * 13 - (11 + 13) = 119 := by
```

```

norm_num
have huniq : ∀ r s : ℕ,
  Nat.Prime r ∧ Nat.Prime s ∧ 4 ≤ r ∧ r ≤ 18 ∧ 4 ≤ s ∧ s ≤ 18 ∧
  r ≠ s ∧ r < s ∧ r * s - (r + s) = 119 →
  r = 11 ∧ s = 13 := by
  intro r s h
  rcases h with ⟨hr, hs, hr4, hr18, hs4, hs18, hrs, hlt, heq⟩
  interval_cases r <|> interval_cases s <|> norm_num at *
  refine ⟨11, 13, hpair.1, hpair.2.1, hpair.2.2.1, hpair.2.2.2.1,
    hpair.2.2.2.2.1, hpair.2.2.2.2.2.1, hpair.2.2.2.2.2.2.1,
    hpair.2.2.2.2.2.2.2.1, hpair.2.2.2.2.2.2.2.2, ?_, ?_⟩
  · intro r s hs
    exact huniq r s hs
  · change A ^ 79 - B ^ 79 ≡ 3 [ZMOD 7]
    have hdiff : A ^ 79 - B ^ 79 ≡ (1 : ℤ) ^ 79 - 5 ^ 79 [ZMOD 7] := by
      exact (hA.pow 79).sub (hB.pow 79)
    have hnum : (1 : ℤ) ^ 79 - 5 ^ 79 ≡ 3 [ZMOD 7] := by
      norm_num [Int.ModEq]
    exact hdiff.trans hnum

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__me__4312632b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4312632b71c131d7

4.25 025. bounded_prime_product_minus_sum_characterization

The problem

For every nonnegative integer t , characterize when t can be obtained by subtracting the sum of two distinct primes between 4 and 18 from their product. Show that this happens exactly when $(p-1)(q-1) = t+1$ for such primes.

Lean statement

```
theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? **(A)** 22 **(B)** 60 **(C)** 119 **(D)** 180 **(E)** 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Retains the parent's bounded distinct-prime domain and product-minus-sum expression, while replacing its fixed target list with a parameterized factorization characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes five finite attainability checks into an iff characterization for every t , requiring symbolic handling of natural-number subtraction and the factorization identity $(p-1)(q-1)=pq-p-q+1$. The parent's bounded interval enumeration cannot supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child parameterizes the target value and proves a two-way factorization equivalence, requiring algebraic manipulation in both directions rather than recalling the parent's finite contradiction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces an arbitrary parameter and proves a genuine two-way algebraic characterization via $(p-1)(q-1) = t+1$. The parent's finite contradiction and interval case split do not supply this general equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2401 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_product_minus_sum_characterization.extracted_1.1 : ∀ (t : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = t) ↔
  ∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ (p - 1) * (q - 1) = t + 1
```

Read back by a model that saw only the Lean. *For every natural number t , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $p \cdot q - (p + q) = t$, if and only if there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $(p - 1)(q - 1) = t + 1$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1) := by
  intro t
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, ht⟩
    have hle : p + q ≤ p * q := by
      nlinarith
    have ht_z : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast ht
    rw [Nat.cast_sub hle] at ht_z
    norm_num [Nat.cast_mul, Nat.cast_add] at ht_z
    have hfac_z : (((p - 1) * (q - 1)) : ℕ) : ℤ = (t : ℤ) + 1 := by
      norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
        Nat.cast_sub (by omega : 1 ≤ q)]
      nlinarith [ht_z]
    have hfactor : (p - 1) * (q - 1) = t + 1 := by
      exact_mod_cast hfac_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
    have hle : p + q ≤ p * q := by
      nlinarith
    have hfac_z : (((p - 1) * (q - 1)) : ℕ) : ℤ = (t : ℤ) + 1 := by
      exact_mod_cast hfactor
    norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
      Nat.cast_sub (by omega : 1 ≤ q)] at hfac_z
    have ht_z : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      rw [Nat.cast_sub hle]
      norm_num [Nat.cast_mul, Nat.cast_add]
      nlinarith [hfac_z]
    have hprod : p * q - (p + q) = t := by
      exact_mod_cast ht_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hprod⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__me__c3538557

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c35385572f20d6e9

4.26 026. arbitrary_parameter_shifted_roots

The problem

Let p and q be distinct natural numbers with $p, q \geq 2$, and let $t = pq - (p + q)$. Show that $(p - 1)(q - 1) = t + 1$, and that the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$ are exactly p and q .

Lean statement

```
theorem arbitrary_parameter_shifted_roots :
  ∀ p q t : ℕ,
    2 ≤ p → 2 ≤ q → p ≠ q →
    p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

Parent. Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that $(p - 1)(q - 1) = t + 1$, and that p and q are exactly the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$.

Parent in Lean

```
theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

What it contributes. Generalized the parent's prime-and-bounded setup to arbitrary natural parameters at least 2, while retaining its algebraic factorization and integer-root checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely removes the primality and finite bounds, requiring a new general argument that $p, q \geq 2$ implies $p + q \leq pq$; the resulting algebraic identity and exact integer-root characterization then hold for arbitrary distinct natural parameters. This is the planned bounded generalization and is not recoverable by simply recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from bounded distinct primes to arbitrary distinct naturals at least 2. Its proof replaces finite interval checking with the symbolic inequality $(p-2)(q-2) \geq 0$, so recalling the parent does not supply the needed argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from bounded distinct primes to arbitrary distinct natural parameters. The parent's finite interval-case argument no longer applies; the child requires a general nonnegativity argument and symbolic algebra to establish the factorization and root characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2543 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem arbitrary_parameter_shifted_roots.extracted_1_1 : ∀ (p q t : ℕ),
  2 ≤ p →
  2 ≤ q →
  p ≠ q →
  p * q - (p + q) = t →
  (p - 1) * (q - 1) = t + 1 ∧ ∀ (x : ℤ), x ^ 2 - (↑p + ↑q) * x + (↑t + ↑p + ↑q) = 0 ↔ x = ↑p ∨ x = ↑q
```

Read back by a model that saw only the Lean. For all natural numbers p , q , and t , if $2 \leq p$, $2 \leq q$, $p \neq q$, and $p \cdot q - (p + q) = t$, then $(p - 1)(q - 1) = t + 1$, and for every integer x , $x^2 - (p + q)x + (t + p + q) = 0$ if and only if $x = p$ or $x = q$, where p , q , and t are regarded as integers in the latter equation.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem arbitrary_parameter_shifted_roots :
  ∀ p q t : ℕ,
    2 ≤ p → 2 ≤ q → p ≠ q →
    p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
  intro p q t hp hq hne hrel
  have hp_repr : p = (p - 1) + 1 := by
    omega
  have hq_repr : q = (q - 1) + 1 := by
    omega
  have hsum_le : p + q ≤ p * q := by
    have hnonneg : 0 ≤ (p - 2) * (q - 2) := Nat.zero_le _
    nlinarith
  have hprod : p * q = t + (p + q) := by
    exact (Nat.sub_eq_iff_eq_add hsum_le).mp hrel
  have hfac : (p - 1) * (q - 1) = t + 1 := by
    nlinarith [hprod, hp_repr, hq_repr]
  have hrelZ : (p : ℤ) * q = (t : ℤ) + p + q := by
    have hprodZ : (p : ℤ) * q = (t : ℤ) + ((p + q : ℕ) : ℤ) := by
      exact_mod_cast hprod
    simpa [Nat.cast_add, add_assoc] using hprodZ
  have hpoly : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
    (x - p) * (x - q) := by
    intro x
    nlinarith [hrelZ]
  have hroots : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
    x = p ∨ x = q := by
    intro x
    constructor
    · intro hx
```

```

have hzero : (x - p) * (x - q) = 0 := by
  rw [← hpoly x]
  exact hx
rcases mul_eq_zero.mp hzero with hzero | hzero
· left
  omega
· right
  omega
· intro hx
  rcases hx with rfl | rfl
  · simp [hpoly]
  · simp [hpoly]
exact ⟨hfacs, hroots⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.me__11c1e1ee

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 11c1e1eef1d5f3e9

4.27 027. bounded_threshold_mutation

The problem

Let a sequence begin with 4, 7, and let each later term be the units digit of the sum of the two preceding terms. If S_n is the sum of the first n terms, show that 2 is the least n such that $S_n > 10$ and $n \leq 3$.

Lean statement

```
theorem bounded_threshold_mutation
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10 ∧ n ≤ 3}) :
  IsLeast Sₙ 2
```

Parent. Consider the sequence of numbers: 4, 7, 1, 8, 9, 7, 6, ... For $n > 2$, the n -th term of the sequence is the units digit of the sum of the two previous terms. Let S_n denote the sum of the first n terms of this sequence. Show that the smallest value of n for which $S_n > 10,000$ is 1999.

Parent in Lean

```
theorem amc12a_2002_p21
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10000}) :
  IsLeast Sₙ 1999 := by
```

What it contributes. Used the recurrence and finite partial-sum derivation pattern, adapting the threshold and replacing the unbounded minimum with a bounded IsLeast classification.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a valid bounded least-element characterization with a different threshold and an essential finite case analysis for indices below 2. Its proof derives $u\ 2$ and the partial sums explicitly, then excludes earlier candidates, so recalling the parent's 1999-threshold monotonicity argument does not solve it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded-threshold characterization: it computes the new local values, establishes membership at 2, and excludes earlier indices by finite case analysis. The parent's large-sum monotonicity argument and IsLeast result do not provide this proof.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2713 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_threshold_mutation.extracted_1_1 (u S : ℕ → ℕ) (h₀ : u 0 = 4) (h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) (h₃ : ∀ (n : ℕ), S n = ∑ k ∈ Finset.range n, u k) (Sn : Set ℕ)
(h₄ : Sn = {n | S n > 10 ∧ n ≤ 3}) : IsLeast Sn 2
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers. Assume $u(0)=4$, $u(1)=7$, and for every natural number n with $n \geq 2$, $u(n)=(u(n-2)+u(n-1))$ modulo 10. Assume also that for every natural number n , $S(n)=\sum_{k \in \{0,1,\dots,n-1\}} u(k)$. Let S_n be a set of natural numbers, and assume $S_n=\{n \in \mathbb{N} \mid S(n) > 10 \text{ and } n \leq 3\}$. Then 2 is the least element of S_n ; that is, $2 \in S_n$ and every element of S_n is greater than or equal to 2.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_threshold_mutation
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sn : Set ℕ)
  (h₄ : Sn = {n | S n > 10 ∧ n ≤ 3}) :
  IsLeast Sn 2 := by
  have hu2 : u 2 = 1 := by
    have h := h₂ 2 (by omega)
    norm_num [h₀, h₁] at h ⊢
    exact h
  have hS0 : S 0 = 0 := by
    rw [h₃]
    simp
  have hS1 : S 1 = 4 := by
    rw [h₃]
    simp [h₀]
  have hS2 : S 2 = 11 := by
    rw [h₃]
    norm_num [Finset.sum_range_succ, h₀, h₁, hu2]
  constructor
  · rw [h₄]
    change S 2 > 10 ∧ 2 ≤ 3
    constructor <|> omega
  · intro n hn
    rw [h₄] at hn
    change S n > 10 ∧ n ≤ 3 at hn
    rcases hn with ⟨hSn, hn3⟩
    interval_cases n <|> simp_all [hS0, hS1, hS2]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2002_p21__me__a407401f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a407401f0770c6be

4.28 028. least_threshold_residue_from_period_block

The problem

Let a sequence have period p and block sum b . If every partial sum before $kp + r$ is at most T , the sum at $kp + r$ exceeds T , and that sum equals b^{2q+1} for some $b \equiv 2 \pmod{3}$, prove that $kp + r$ is the least index whose partial sum exceeds T and is congruent to 2 modulo 3.

Lean statement

```
theorem least_threshold_residue_from_period_block
  (u : ℕ → ℕ)
  (p b T k r q : ℕ)
  (hp : 0 < p)

  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)
  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T)
  (hbridge : S (k * p + r) = b ^ (2 * q + 1))
  (hb : b % 3 = 2) :
  IsLeast {n | S n > T ∧ S n % 3 = 2} (k * p + r)
```

Parent. Let a sequence satisfy the same units-digit recurrence, and suppose it has period p with one-period sum b . If every partial block sum before $kp + r$ is at most T , while the sum at $kp + r$ exceeds T , prove that $kp + r$ is the least index whose partial sum exceeds T .

Parent in Lean

```
theorem least_threshold_from_period_block
  (u : ℕ → ℕ)

  (p b T k r : ℕ)
  (hp : 0 < p)
  (hperiod : ∀ n, u (n + p) = u n)
  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)

  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
  IsLeast {n | S n > T} (k * p + r) := by
```

What it contributes. Supplied the period-block decomposition, candidate threshold checkpoint, quotient/remainder decomposition, and earlier-index bound used to establish leastness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new residue argument: it must prove that an odd power of a number congruent to 2 modulo 3 is itself congruent to 2 modulo 3, then establish leastness in the intersected threshold-and-residue set. The parent supplies only threshold leastness and cannot certify the new conjunct.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target to leastness within a residue-refined threshold set and must prove the candidate's residue via an independent odd-power modular induction. This reasoning is absent from the parent's proof, while the threshold-minimality portion is appropriately reused.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target to leastness within a residue-refined threshold set and must prove the candidate's residue from an odd-power congruence via a new induction. Although the threshold-minimality argument is reused, the parent cannot establish membership in the refined set.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2778 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem least_threshold_residue_from_period_block.extracted_1_1 (u S : ℕ → ℕ) (p b T k r q : ℕ) (hp : 0 < p)
  (hperiod : ∀ (n : ℕ), u (n + p) = u n) (hblock : ∀ (j s : ℕ), S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)
  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ (j s : ℕ), j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T)
  (hbridge : S (k * p + r) = b ^ (2 * q + 1)) (hb : b % 3 = 2) :
  IsLeast {n | S n > T ∧ S n % 3 = 2} (k * p + r)
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers, and let p, b, T, k, r , and q be natural numbers. Assume that p is positive; for every natural number n , $u(n + p) = u(n)$; for all natural numbers j and s , $S(jp + s) = jb + \sum_{i \in \text{Finset.range}(s)} u(i)$; $kb + \sum_{i \in \text{Finset.range}(r)} u(i) > T$; for all natural numbers j and s , if $jp + s < kp + r$, then $jb + \sum_{i \in \text{Finset.range}(s)} u(i) \leq T$; $S(kp + r) = b^{(2q + 1)}$; and $b \equiv 2 \pmod{3}$. Then $kp + r$ is the least natural number n such that $S(n) > T$ and $S(n) \equiv 2 \pmod{3}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem least_threshold_residue_from_period_block
  (u S : ℕ → ℕ)
  (p b T k r q : ℕ)
  (hp : 0 < p)

  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)
  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T)
  (hbridge : S (k * p + r) = b ^ (2 * q + 1))
  (hb : b % 3 = 2) :
  IsLeast {n | S n > T ∧ S n % 3 = 2} (k * p + r) := by
  have hodd : ∀ m : ℕ, b ^ (2 * m + 1) % 3 = 2 := by
    intro m
    induction m with
    | zero => simp [hb]
    | succ m ih =>
      rw [show 2 * Nat.succ m + 1 = (2 * m + 1) + 2 by omega, pow_add]
      have hb2 : b ^ 2 % 3 = 1 := by
        simp [pow_two, Nat.mul_mod, hb]
      simp [Nat.mul_mod, ih, hb2]
  have hthreshold : S (k * p + r) > T := by
```

```

    rw [hblock]
    exact hcross
have hresidue : S (k * p + r) % 3 = 2 := by
    rw [hbridge]
    exact hodd q
constructor
· exact (hthreshold, hresidue)
· intro n hn
  change S n > T ∧ S n % 3 = 2 at hn
  by_contra hnot
  have hnlt : n < k * p + r := Nat.lt_of_not_ge hnot
  let j := n / p
  let s := n % p
  have hslt : s < p := Nat.mod_lt n hp
  have hdecomp : j * p + s = n := by
    dsimp [j, s]
    simpa [Nat.mul_comm] using Nat.div_add_mod n p
  have harg : j * p + s < k * p + r := by
    simpa [hdecomp] using hnlt
  have hsum : j * b + ∑ i ∈ Finset.range s, u i ≤ T := hbound j s harg
  have hSn : S n = j * b + ∑ i ∈ Finset.range s, u i := by
    rw [← hdecomp]
    exact hblock j s
  have hle : S n ≤ T := by
    rw [hSn]
    exact hsum
  omega

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2002_p21__mh.me__0c83ce1f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5bb41ad5ca1bc9d3

4.29 029. bounded_recurrence_window_unique

The problem

Let N be a nonnegative integer, let p and q be real numbers, and let a and b be real sequences such that, for every nonnegative integer n , $a_{n+2} - a_{n+1} = a_{n+1} - a_n$ and $b_{n+2} - b_{n+1} = b_{n+1} - b_n$. If $a_1 = b_1 = p$ and $a_2 = b_2 = 9$, prove that $a_n = b_n$ for every integer n with $1 \leq n \leq N$.

Lean statement

```
theorem bounded_recurrence_window_unique (N : ℕ) (p q : ℝ) (a b : ℕ → ℝ)
  (ha : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (hb : ∀ n, b (n + 2) - b (n + 1) = b (n + 1) - b n)
  (h1 : a 1 = p) (h2 : a 2 = 9)

  (hb1 : b 1 = p) (hb2 : b 2 = 9) :
  ∀ n, 1 ≤ n → n ≤ N → a n = b n
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the 2010th term of this sequence? Show that it is 8041.

Parent in Lean

```
theorem amc12a_2010_p10 (p q : ℝ) (a : ℕ → ℝ) (h0 : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h1 : a 1 = p) (h2 : a 2 = 9) (h3 : a 3 = 3 * p - q) (h4 : a 4 = 3 * p + q) : a 2010 = 8041 := by
```

What it contributes. Retains the parent recurrence, domain, fixed initial values, and auxiliary evaluations while replacing the fixed endpoint conclusion with bounded recurrence uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different uniqueness theorem: two sequences satisfying the same second-difference recurrence and matching at indices 1 and 2 agree throughout any bounded window. Its strong-induction argument is independent of the parent's parameter-solving and endpoint evaluation, and the extra hypotheses on a are unused rather than carrying over the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine uniqueness theorem for two recurrence sequences via strong induction, using matching initial values and recurrence propagation. This is a different proof obligation and route from the parent's fixed-endpoint arithmetic-progression evaluation, so recalling the parent does not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine two-sequence uniqueness theorem by strong induction from matching initial values and equal second differences. This reasoning does not rely on the parent's derivation of $p = 5$ or its fixed endpoint evaluation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed h_3 , h_4

Check	By	Result
corpus_dedup	hash	unique against 1694 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_recurrence_window_unique.extracted_1_1 : ∀ (N : ℕ) (p : ℝ) (a b : ℕ → ℝ),
  (∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) →
  (∀ (n : ℕ), b (n + 2) - b (n + 1) = b (n + 1) - b n) →
  a 1 = p → a 2 = 9 → b 1 = p → b 2 = 9 → ∀ (n : ℕ), 1 ≤ n → n ≤ N → a n = b n
```

Read back by a model that saw only the Lean. For every natural number N , every real number p , and every pair of functions $a, b: \mathbb{N} \rightarrow \mathbb{R}$, if for every natural number n , $a(n+2) - a(n+1) = a(n+1) - a(n)$, and for every natural number n , $b(n+2) - b(n+1) = b(n+1) - b(n)$, and $a(1) = p$, $a(2) = 9$, $b(1) = p$, and $b(2) = 9$, then for every natural number n such that $1 \leq n$ and $n \leq N$, we have $a(n) = b(n)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_recurrence_window_unique (N : ℕ) (p q : ℝ) (a b : ℕ → ℝ)
  (h₀a : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₀b : ∀ n, b (n + 2) - b (n + 1) = b (n + 1) - b n)
  (h₁ : a 1 = p) (h₂ : a 2 = 9)

  (hb₁ : b 1 = p) (hb₂ : b 2 = 9) :
  ∀ n, 1 ≤ n → n ≤ N → a n = b n := by
  intro n hn₁ hnN
  revert hn₁ hnN
  induction n using Nat.strong_induction_on with
  | h n ih =>
    intro hn₁ hnN
    by_cases h_one : n = 1
    · subst n
      exact h₁.trans hb₁.symm
    by_cases h_two : n = 2
    · subst n
      exact h₂.trans hb₂.symm
    have hn₃ : 3 ≤ n := by omega
    have hprev₁ : a (n - 1) = b (n - 1) := by
      apply ih (n - 1)
      · omega
      · omega
      · omega
    have hprev₂ : a (n - 2) = b (n - 2) := by
      apply ih (n - 2)
      · omega
      · omega
      · omega
    have ha := h₀a (n - 2)
    have hb := h₀b (n - 2)
    have hidx₁ : n - 2 + 2 = n := by omega
    have hidx₂ : n - 2 + 1 = n - 1 := by omega
    rw [hidx₁, hidx₂] at ha hb
    linarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2010_p10__me__8f91749d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7045cbe3a1ce5b8d

4.30 030. bounded_interval_factorization

The problem

For integers $0 \leq m \leq n$, show that $\prod_{k=m}^n (2^{2^k} + 3^{2^k}) = 3^{2^{n+1}} - 2^{2^{n+1}}$.

Lean statement

```
theorem bounded_interval_factorization :
  ∀ m n : ℕ, m ≤ n →
    (∏ k ∈ Finset.range (n + 1),
      if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
      ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m)) =
      (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
```

Parent. For every nonnegative integer n , show that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Parent in Lean

```
theorem power_sum_product_telescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

What it contributes. Generalizes the parent's initial-segment telescoping induction to a bounded lower-index interval, retaining its exponent-doubling and ring-factorization checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the lower endpoint and proves a bounded interval factorization, requiring a new induction with a boundary case for $m = n + 1$. The parent's initial-range telescoping proof does not directly supply this interval handling, and no hypotheses are redundant.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the lower endpoint and proves a bounded-interval factorization, requiring a new induction split on whether the lower bound has entered the range. The parent's initial-range telescoping proof does not supply this shifted-boundary reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely shifts the product start from 0 to an arbitrary m and must handle the new boundary case $m = n + 1$, where the product is empty. The parent's induction only establishes the initial-range identity and does not supply this interval factorization argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2412 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_interval_factorization.extracted_1_1 : ∀ (m n : ℕ),
  m ≤ n →
  (∏ k ∈ Finset.range (n + 1), if m ≤ k then 2 ^ 2 ^ k + 3 ^ 2 ^ k else 1) * (3 ^ 2 ^ m - 2 ^ 2 ^ m) =
  3 ^ 2 ^ (n + 1) - 2 ^ 2 ^ (n + 1)
```

Read back by a model that saw only the Lean. For all natural numbers m and n , if $m \leq n$, then the product over all natural numbers k in the range from 0 through n of the factor $2^{(2^k)} + 3^{(2^k)}$ when $m \leq k$ and the factor 1 otherwise, multiplied by $3^{(2^m)} - 2^{(2^m)}$, equals $3^{(2^{(n+1)})} - 2^{(2^{(n+1)})}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem bounded_interval_factorization :
  ∀ m n : ℕ, m ≤ n →
    (∏ k ∈ Finset.range (n + 1),
      if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
      ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m)) =
      (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
    intro m n hmn
    induction n generalizing m with
    | zero =>
      have hm : m = 0 := Nat.eq_zero_of_le_zero hmn
      subst m
      norm_num
    | succ n ih =>
      by_cases hmn' : m ≤ n
      · rw [Finset.prod_range_succ]
        rw [if_pos (Nat.le_succ_of_le hmn')]
        have hi := ih m hmn'
        have hpow : (2 : ℕ) ^ (n + 1 + 1) = (2 : ℕ) ^ (n + 1) * 2 := by
          rw [pow_succ]
        have h3 : (3 : ℤ) ^ (2 ^ (n + 1 + 1)) = ((3 : ℤ) ^ (2 ^ (n + 1))) ^ 2 := by
          rw [hpow, pow_mul]
        have h2 : (2 : ℤ) ^ (2 ^ (n + 1 + 1)) = ((2 : ℤ) ^ (2 ^ (n + 1))) ^ 2 := by
          rw [hpow, pow_mul]
        calc
          ((∏ k ∈ Finset.range (n + 1),
            if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
            ((2 : ℤ) ^ (2 ^ (n + 1)) + (3 : ℤ) ^ (2 ^ (n + 1)))) *
            ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m)) =
          ((∏ k ∈ Finset.range (n + 1),
            if m ≤ k then ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k)) else 1) *
            ((3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m))) *
            ((2 : ℤ) ^ (2 ^ (n + 1)) + (3 : ℤ) ^ (2 ^ (n + 1))) := by ring
          = ((3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1))) *
            ((2 : ℤ) ^ (2 ^ (n + 1)) + (3 : ℤ) ^ (2 ^ (n + 1))) := by rw [hi]
          = (3 : ℤ) ^ (2 ^ (n + 1 + 1)) - (2 : ℤ) ^ (2 ^ (n + 1 + 1)) := by
            rw [h3, h2]
            ring
          = (3 : ℤ) ^ (2 ^ (Nat.succ n + 1)) - (2 : ℤ) ^ (2 ^ (Nat.succ n + 1)) := by
            simp [Nat.succ_eq_add_one, Nat.add_assoc]
      · have hm : m = n + 1 := by omega
        subst m
        rw [Finset.prod_range_succ]
        have hprod :
```

```

      (∏ k ∈ Finset.range (n + 1),
        if n + 1 ≤ k then ((2 : Z) ^ (2 ^ k) + (3 : Z) ^ (2 ^ k)) else 1) = 1 := by
    apply Finset.prod_eq_one
    intro k hk
    have hk' : k ≤ n := by simpa using (Finset.mem_range.mp hk)
    simp [show ¬ n + 1 ≤ k by omega]
    rw [hprod]
    simp only [if_pos (le_refl (n + 1)), one_mul]
    have hpow : (2 : N) ^ (n + 1 + 1) = (2 : N) ^ (n + 1) * 2 := by
      rw [pow_succ]
    have h3 : (3 : Z) ^ (2 ^ (n + 1 + 1)) = ((3 : Z) ^ (2 ^ (n + 1))) ^ 2 := by
      rw [hpow, pow_mul]
    have h2 : (2 : Z) ^ (2 ^ (n + 1 + 1)) = ((2 : Z) ^ (2 ^ (n + 1))) ^ 2 := by
      rw [hpow, pow_mul]
    rw [h3, h2]
    ring

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__2706e7a7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2706e7a74c656432

4.31 031. dyadic_product_difference_scaled_bounded

The problem

For all natural numbers m and n with $n \leq 8$, prove $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Lean statement

```
theorem dyadic_product_difference_scaled_bounded :
  ∀ m n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
      ((3 : ℤ) ^ m - (2 : ℤ) ^ m) =
      (3 : ℤ) ^ (m * 2 ^ n) - (2 : ℤ) ^ (m * 2 ^ n)
```

Parent. For every integer n with $0 \leq n \leq 8$, prove that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Parent in Lean

```
theorem dyadic_product_difference_bounded :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

What it contributes. Supplied the bounded induction structure, product-range expansion, and exponent-doubling checkpoint; the child introduces the scale parameter m and the necessary factor $3^m - 2^m$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the exponent scale with an essential natural number m and carries the factor $(3^m - 2^m)$ through the induction. Its successor step requires transporting $m \cdot 2^{(n+1)}$ to $(m \cdot 2^n) \cdot 2$ and applying the difference-of-squares argument, so recalling the parent does not directly solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the identity to every natural scale m and introduces the factor $(3^m - 2^m)$, so the parent's fixed-scale theorem cannot solve it. The induction must preserve this symbolic scale through the exponent-doubling factorization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the exponent scale by an arbitrary natural number m . The induction and difference-of-squares argument must now establish symbolic exponent identities involving m , so recalling the parent's fixed-scale theorem does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2627 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dyadic_product_difference_scaled_bounded.extracted_1_1 : ∀ (m n : ℕ),
  n ≤ 8 →
  (∏ k ∈ Finset.range n, (2 ^ (m * 2 ^ k) + 3 ^ (m * 2 ^ k))) * (3 ^ m - 2 ^ m) =
  3 ^ (m * 2 ^ n) - 2 ^ (m * 2 ^ n)
```

Read back by a model that saw only the Lean. *For all natural numbers m and n , if n is at most 8, then the product over all natural numbers k less than n of (2 raised to the power $m \cdot 2^k$ plus 3 raised to the power $m \cdot 2^k$), multiplied by $(3^m - 2^m)$, equals 3 raised to the power $m \cdot 2^n$ minus 2 raised to the power $m \cdot 2^n$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem dyadic_product_difference_scaled_bounded :
  ∀ m n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
      ((3 : ℤ) ^ m - (2 : ℤ) ^ m) =
      (3 : ℤ) ^ (m * 2 ^ n) - (2 : ℤ) ^ (m * 2 ^ n) := by
  intro m n hn
  induction n with
  | zero =>
    simp
  | succ n ih =>
    have ih' := ih (by omega)
    have h3 :
      (3 : ℤ) ^ (m * 2 ^ (Nat.succ n)) =
      ((3 : ℤ) ^ (m * 2 ^ n)) ^ 2 := by
      rw [show m * 2 ^ (Nat.succ n) = (m * 2 ^ n) * 2 by
        simp [pow_succ, Nat.mul_assoc, Nat.mul_comm, Nat.mul_left_comm]]
      rw [pow_mul]
    have h2 :
      (2 : ℤ) ^ (m * 2 ^ (Nat.succ n)) =
      ((2 : ℤ) ^ (m * 2 ^ n)) ^ 2 := by
      rw [show m * 2 ^ (Nat.succ n) = (m * 2 ^ n) * 2 by
        simp [pow_succ, Nat.mul_assoc, Nat.mul_comm, Nat.mul_left_comm]]
      rw [pow_mul]
    rw [Finset.prod_range_succ]
    calc
      ((∏ k ∈ Finset.range n,
        ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
        ((2 : ℤ) ^ (m * 2 ^ n) + (3 : ℤ) ^ (m * 2 ^ n))) *
        ((3 : ℤ) ^ m - (2 : ℤ) ^ m) =
      ((∏ k ∈ Finset.range n,
        ((2 : ℤ) ^ (m * 2 ^ k) + (3 : ℤ) ^ (m * 2 ^ k))) *
        ((3 : ℤ) ^ m - (2 : ℤ) ^ m)) *
        ((2 : ℤ) ^ (m * 2 ^ n) + (3 : ℤ) ^ (m * 2 ^ n)) := by ring
      = ((3 : ℤ) ^ (m * 2 ^ n) - (2 : ℤ) ^ (m * 2 ^ n)) *
        ((2 : ℤ) ^ (m * 2 ^ n) + (3 : ℤ) ^ (m * 2 ^ n)) := by rw [ih']
      = (3 : ℤ) ^ (m * 2 ^ (Nat.succ n)) -
        (2 : ℤ) ^ (m * 2 ^ (Nat.succ n)) := by
        rw [h3, h2]
      ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me.me__820eb7ea

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 820eb7eabf23c1db

4.32 032. parameterized_dyadic_difference_of_squares

The problem

For integers x, y and every nonnegative integer n , prove $\prod_{k=0}^{n-1} (x^{2^k} + y^{2^k}) (x-y) = x^{2^n} - y^{2^n}$.

Lean statement

```
theorem parameterized_dyadic_difference_of_squares :
  ∀ (x y : ℤ) (n : ℕ),
    (∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) * (x - y) =
      x ^ (2 ^ n) - y ^ (2 ^ n) := by
```

Parent. For every nonnegative integer n , show that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Parent in Lean

```
theorem power_sum_product_telescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

What it contributes. Generalizes the parent's dyadic finite-product induction and exponent-doubling checkpoints from fixed bases 2 and 3 to arbitrary integer bases, adding the necessary initial-difference factor.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child parameterizes both bases and makes the initial difference factor explicit, requiring a genuinely symbolic difference-of-squares induction rather than replaying the fixed-base proof. Its proof must establish the generalized algebraic invariant for arbitrary integers x and y .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both fixed bases and makes the initial difference factor explicit, proving a universal difference-of-squares product law over arbitrary integers. Although the induction structure is shared, the parent's fixed-coefficient result does not itself establish the symbolic two-variable identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both bases and makes the initial difference factor explicit, requiring the general difference-of-squares identity for arbitrary integers rather than the parent's fixed coefficients. Although the induction skeleton is shared, the parent does not prove the child by specialization or recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2445 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_dyadic_difference_of_squares.extracted_1_1 : ∀ (x y : ℤ) (n : ℕ),
  (∏ k ∈ Finset.range n, (x ^ 2 ^ k + y ^ 2 ^ k)) * (x - y) = x ^ 2 ^ n - y ^ 2 ^ n
```

Read back by a model that saw only the Lean. *For all integers x and y and every natural number n , the product over $k = 0, 1, \dots, n - 1$ of $(x^{2^k} + y^{2^k})$, multiplied by $(x - y)$, equals $x^{2^n} - y^{2^n}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem parameterized_dyadic_difference_of_squares :
  ∀ (x y : ℤ) (n : ℕ),
    (∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) * (x - y) =
      x ^ (2 ^ n) - y ^ (2 ^ n) := by
  intro x y n
  induction n with
  | zero =>
    norm_num
  | succ n ih =>
    rw [Finset.prod_range_succ]
    have hpow : (2 : ℕ) ^ (n + 1) = (2 : ℕ) ^ n * 2 := by
      simp [pow_succ]
    have hx : x ^ (2 ^ (n + 1)) = (x ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    have hy : y ^ (2 ^ (n + 1)) = (y ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    calc
      (∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) *
        (x ^ (2 ^ n) + y ^ (2 ^ n)) * (x - y) =
        ((∏ k ∈ Finset.range n, (x ^ (2 ^ k) + y ^ (2 ^ k))) * (x - y)) *
          (x ^ (2 ^ n) + y ^ (2 ^ n)) := by ring
      = (x ^ (2 ^ n) - y ^ (2 ^ n)) *
        (x ^ (2 ^ n) + y ^ (2 ^ n)) := by rw [ih]
      = x ^ (2 ^ (n + 1)) - y ^ (2 ^ (n + 1)) := by
        rw [hx, hy]
      ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__e421c102

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e421c1028fbd8f18

4.33 033. dyadic_subinterval_endpoint_factorization

The problem

For natural numbers $a \leq b$, prove that $\left[\left(\prod_{k=a}^b \left(2^{2^k} + 3^{2^k} \right) \right) \left(3^{2^a} - 2^{2^a} \right) \right] = 3^{2^{b+1}} - 2^{2^{b+1}}$.

Lean statement

```
theorem dyadic_subinterval_endpoint_factorization :
  ∀ a b : ℕ, a ≤ b →
    (∏ k ∈ Finset.Icc a b,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
      ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) =
      (3 : ℤ) ^ (2 ^ (b + 1)) - (2 : ℤ) ^ (2 ^ (b + 1))
```

Parent. For every integer n with $0 \leq n \leq 8$, prove that $\left[\prod_{k=1}^n \left(2^{2^{k-1}} + 3^{2^{k-1}} \right) \right] \left(3^{2^n} - 2^{2^n} \right)$.

Parent in Lean

```
theorem dyadic_product_difference_interval :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.Icc 1 n,
      ((2 : ℤ) ^ (2 ^ (k - 1)) + (3 : ℤ) ^ (2 ^ (k - 1)))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n)
```

What it contributes. Supplied the dyadic product structure and exponent-doubling factorization checkpoint; the child generalizes it to an arbitrary lower endpoint and consumes that endpoint factor in the induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from the fixed interval starting at 1 to every nonempty bounded interval $[a, b]$. Its proof requires a new induction over the upper endpoint and an endpoint factorization argument; the parent's finite interval_cases proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the identity to arbitrary nonempty bounded subintervals and proves it by induction, using a symbolic endpoint factorization absent from the parent's bounded finite check. Recalling the parent's proof does not establish the child's unrestricted interval law.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from the parent's bounded initial interval to arbitrary nonempty subintervals and requires induction plus endpoint factorization. Its proof does not reuse the parent's finite case split or conclusion as a recalled lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2636 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dyadic_subinterval_endpoint_factorization.extracted_1_1 : ∀ (a b : ℕ),
  a ≤ b →
  (∏ k ∈ Finset.Icc a b, (2 ^ 2 ^ k + 3 ^ 2 ^ k)) * (3 ^ 2 ^ a - 2 ^ 2 ^ a) =
  3 ^ 2 ^ (b + 1) - 2 ^ 2 ^ (b + 1)
```

Read back by a model that saw only the Lean. *For all natural numbers a and b with $a \leq b$, the product over all natural numbers k in the interval $[a, b]$ of $(2^{2^k} + 3^{2^k})$, multiplied by $(3^{2^a} - 2^{2^a})$, equals $3^{2^{b+1}} - 2^{2^{b+1}}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem dyadic_subinterval_endpoint_factorization :
  ∀ a b : ℕ, a ≤ b →
    (∏ k ∈ Finset.Icc a b,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
      ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) =
      (3 : ℤ) ^ (2 ^ (b + 1)) - (2 : ℤ) ^ (2 ^ (b + 1)) := by
  have hfac : ∀ n : ℕ,
    ((2 : ℤ) ^ (2 ^ n) + (3 : ℤ) ^ (2 ^ n)) *
      ((3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n)) =
      (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
    intro n
    rw [show 2 ^ (n + 1) = 2 ^ n + 2 ^ n by
      rw [pow_succ]
      ring]
    rw [pow_add, pow_add]
    ring
  intro a b hab
  induction b generalizing a with
  | zero =>
    have ha : a = 0 := by omega
    subst a
    norm_num
  | succ b ih =>
    by_cases h : a ≤ b
    · have hprev := ih a h
      have hprod :
        (∏ k ∈ Finset.Icc a (b + 1),
          ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
          (∏ k ∈ Finset.Icc a b,
            ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
            ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1)))) := by
          rw [Finset.prod_Icc_succ_top (by omega)]
        calc
          (∏ k ∈ Finset.Icc a (b + 1),
            ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
            ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) =
            ((∏ k ∈ Finset.Icc a b,
              ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
              ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1)))) *
              ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a)) := by rw [hprod]
          = ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1))) *
            ((∏ k ∈ Finset.Icc a b,
              ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
              ((3 : ℤ) ^ (2 ^ a) - (2 : ℤ) ^ (2 ^ a))) := by ring
```

```

    _ = ((2 : ℤ) ^ (2 ^ (b + 1)) + (3 : ℤ) ^ (2 ^ (b + 1))) *
      ((3 : ℤ) ^ (2 ^ (b + 1)) - (2 : ℤ) ^ (2 ^ (b + 1))) := by rw [hprev]
    _ = (3 : ℤ) ^ (2 ^ (b + 1 + 1)) - (2 : ℤ) ^ (2 ^ (b + 1 + 1)) := by simpa [Nat.add_assoc] using hfac (b + 1)
  · have ha : a = b + 1 := by omega
    subst a
    simpa using hfac (b + 1)

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me.ms.me__a0719f65

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a0719f6522070116

4.34 034. dyadic_product_difference_bounded

The problem

For every integer n with $0 \leq n \leq 8$, prove that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Lean statement

```
theorem dyadic_product_difference_bounded :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

Parent. Show that the following equality holds: $(2+3)(2^2+3^2)(2^4+3^4)(2^8+3^8)(2^{16}+3^{16})(2^{32}+3^{32})(2^{64}+3^{64}) = 3^{128} - 2^{128}$.

Parent in Lean

```
theorem amc12a_2021_p9 :
  (∏ k ∈ Finset.range (7 : ℕ),
    ((2 : ℕ) ^ (2 ^ k) + (3 : ℕ) ^ (2 ^ (k : ℕ)))) =
    (3 : ℕ) ^ (128 : ℕ) - (2 : ℕ) ^ (128 : ℕ) := by
```

What it contributes. Retains the parent's dyadic product and difference-of-powers structure while promoting the fixed seven-stage identity to a bounded parameterized law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the fixed seven-factor computation with a parameterized product identity and proves it by induction, using the factorization step uniformly across stages. Memorizing the parent's norm_num proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uniformly parameterizes the product length and proves the product-to-difference-of-powers identity by induction, using the doubling exponent law and an algebraic factorization. This reasoning is not supplied by the parent's fixed seven-term 'norm_num' computation, so recalling the parent does not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed seven-factor identity to a variable stage and requires induction plus symbolic power and ring reasoning. The parent's direct normalization proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2595 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dyadic_product_difference_bounded.extracted_1_1 : ∀ n ≤ 8,
  ∏ k ∈ Finset.range n, (2 ^ 2 ^ k + 3 ^ 2 ^ k) = 3 ^ 2 ^ n - 2 ^ 2 ^ n
```

Read back by a model that saw only the Lean. *For every natural number n with $n \leq 8$, the product over all natural numbers k in the range $0 \leq k < n$ of $(2^{2^k} + 3^{2^k})$ equals $3^{2^n} - 2^{2^n}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem dyadic_product_difference_bounded :
  ∀ n : ℕ, n ≤ 8 →
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
  intro n hn
  induction n with
  | zero =>
    simp
  | succ n ih =>
    have ih' := ih (by omega)
    rw [Finset.prod_range_succ, ih']
    have h3 :
      (3 : ℤ) ^ (2 ^ (n + 1)) = ((3 : ℤ) ^ (2 ^ n)) ^ 2 := by
      calc
        (3 : ℤ) ^ (2 ^ (n + 1)) = (3 : ℤ) ^ (2 ^ n * 2) := by
          rw [show 2 ^ (n + 1) = 2 ^ n * 2 by simp [pow_succ]]
        _ = ((3 : ℤ) ^ (2 ^ n)) ^ 2 := by rw [pow_mul]
    have h2 :
      (2 : ℤ) ^ (2 ^ (n + 1)) = ((2 : ℤ) ^ (2 ^ n)) ^ 2 := by
      calc
        (2 : ℤ) ^ (2 ^ (n + 1)) = (2 : ℤ) ^ (2 ^ n * 2) := by
          rw [show 2 ^ (n + 1) = 2 ^ n * 2 by simp [pow_succ]]
        _ = ((2 : ℤ) ^ (2 ^ n)) ^ 2 := by rw [pow_mul]
    rw [h3, h2]
  ring
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__6082dd30

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6082dd3056b8dc69

4.35 035. power_sum_product_telescopes

The problem

For every nonnegative integer n , show that $\prod_{k=0}^{n-1} (2^{2^k} + 3^{2^k}) = 3^{2^n} - 2^{2^n}$.

Lean statement

```
theorem power_sum_product_telescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
```

Parent. Show that the following equality holds: $(2+3)(2^2+3^2)(2^4+3^4)(2^8+3^8)(2^{16}+3^{16})(2^{32}+3^{32})(2^{64}+3^{64}) = 3^{128} - 2^{128}$.

Parent in Lean

```
theorem amc12a_2021_p9 :
  (∏ k ∈ Finset.range (7 : ℕ),
    ((2 : ℕ) ^ (2 ^ k : ℕ) + (3 : ℕ) ^ (2 ^ (k : ℕ)))) =
    (3 : ℕ) ^ (128 : ℕ) - (2 : ℕ) ^ (128 : ℕ) := by
```

What it contributes. Generalizes the parent’s seven-factor power-difference identity to an arbitrary bounded product length while retaining its telescoping algebra.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed seven-factor identity to every natural index and changes the codomain to integers. Its induction and algebraic telescoping argument cannot be obtained by recalling the parent’s ‘norm_num’ proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor computation with a genuine induction over arbitrary n , requiring the telescoping factorization at each successor step. This proof cannot be obtained by recalling the parent’s norm_num computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor natural-number identity with a universal integer-valued telescoping theorem. Its induction and algebraic power-doubling argument are not supplied by the parent’s one-shot norm_num proof, so recalling the parent does not substantially solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2388 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_sum_product_teslescopes.extracted_1_1 : ∀ (n : ℕ),
  ∏ k ∈ Finset.range n, (2 ^ 2 ^ k + 3 ^ 2 ^ k) = 3 ^ 2 ^ n - 2 ^ 2 ^ n
```

Read back by a model that saw only the Lean. For every natural number n , the product over all natural numbers k in the range $0 \leq k < n$ of $(2^{2^k} + 3^{2^k})$ equals $3^{2^n} - 2^{2^n}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem power_sum_product_teslescopes :
  ∀ n : ℕ,
    (∏ k ∈ Finset.range n,
      ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
      (3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n) := by
  intro n
  induction n with
  | zero =>
    norm_num
  | succ n ih =>
    rw [Finset.prod_range_succ]
    have hpow : (2 : ℕ) ^ (n + 1) = (2 : ℕ) ^ n * 2 := by
      simp [pow_succ]
    have h3 : (3 : ℤ) ^ (2 ^ (n + 1)) = ((3 : ℤ) ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    have h2 : (2 : ℤ) ^ (2 ^ (n + 1)) = ((2 : ℤ) ^ (2 ^ n)) ^ 2 := by
      rw [hpow, pow_mul]
    calc
      (∏ k ∈ Finset.range n,
        ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) *
        ((2 : ℤ) ^ (2 ^ n) + (3 : ℤ) ^ (2 ^ n)) =
        ((3 : ℤ) ^ (2 ^ n) - (2 : ℤ) ^ (2 ^ n)) *
        ((2 : ℤ) ^ (2 ^ n) + (3 : ℤ) ^ (2 ^ n)) := by
          rw [ih]
      _ = (3 : ℤ) ^ (2 ^ (n + 1)) - (2 : ℤ) ^ (2 ^ (n + 1)) := by
          rw [h3, h2]
      _ =
        ring
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2021_p9__me__e0c02869

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e0c028695321139b

4.36 036. bounded_radix_geometric_product

The problem

For every natural number q with $q = 2$ or $q = 3$, all integers a and b , and every natural number n , there exists a sequence of natural numbers $e(k)$ with $e(0) = 1$ and $e(k + 1) = qe(k)$ for every k , such that
$$(b - a) \prod_{k=0}^{n-1} \left(\sum_{j=0}^{q-1} b^{(q-1-j)e(k)} a^{je(k)} \right) = b^{e(n)} - a^{e(n)}.$$

Lean statement

```
theorem bounded_radix_geometric_product :
  ∀ (q : ℕ), q = 2 ∨ q = 3 → ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = q * e k) ∧
    (∀ k ∈ Finset.range n,
      (∑ j ∈ Finset.range q,
        b ^ ((q - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
      b ^ e n - a ^ e n
```

Parent. For every pair of integers a and b and every natural number n , there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $(b - a) \prod_{k=0}^{n-1} (b^{e_k} + a^{e_k}) = b^{e_n} - a^{e_n}$.

Parent in Lean

```
theorem symbolic_doubling_power_product :
  ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∀ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) =
      b ^ e n - a ^ e n
```

What it contributes. Supplies the exponent-recurrence and product-telescoping induction pattern; the proof extends the binary factorization to the $q=3$ cubic factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the telescoping identity from doubling to radix 2 or 3, requiring separate geometric-factor expansions and inductive factorization for the cubic case. The parent's direct 'pow_two_pow_sub_pow_two_pow' application does not supply the new finite-sum factorization or bounded-radix case split.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the telescoping product to $q=3$, requiring a new cubic geometric-factor identity and an induction argument; the parent only supplies the $q=2$ square case. The bounded q split is used to construct distinct radix-specific exponent sequences.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The $q = 3$ branch requires a new cubic geometric-factor identity and an induction over the product, which the parent's doubling-specific lemma does not provide. The bounded q case split therefore demands genuinely different reasoning rather than recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1827 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_radix_geometric_product.extracted_1.1 : ∀ (q : ℕ),
  q = 2 ∨ q = 3 →
  ∀ (a b : ℤ) (n : ℕ),
    ∃ e,
      e 0 = 1 ∧
      (∀ (k : ℕ), e (k + 1) = q * e k) ∧
      (∏ k ∈ Finset.range n, ∑ j ∈ Finset.range q, b ^ ((q - 1 - j) * e k) * a ^ (j * e k)) * (b - a) =
        b ^ e n - a ^ e n
```

Read back by a model that saw only the Lean. For every natural number q , if $q = 2$ or $q = 3$, then for every pair of integers a and b and every natural number n , there exists a sequence e of natural numbers such that $e(0) = 1$, $e(k + 1) = q \cdot e(k)$ for every natural number k , and $(\prod_{k=0}^{n-1} \sum_{j=0}^{q-1} b^{(q-1-j) \cdot e(k)} \cdot a^{j \cdot e(k)}) \cdot (b-a) = b^{e(n)} - a^{e(n)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators
open BigOperators Real Nat Topology Rat

theorem bounded_radix_geometric_product :
  ∀ (q : ℕ), q = 2 ∨ q = 3 → ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = q * e k) ∧
    (∏ k ∈ Finset.range n,
      (∑ j ∈ Finset.range q,
        b ^ ((q - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
      b ^ e n - a ^ e n := by
  intro q hq a b n
  rcases hq with rfl | rfl
  · let e : ℕ → ℕ := fun k => 2 ^ k
    refine ⟨e, ?_, ?_, ?_⟩
    · simp [e]
    · intro k
      simp [e, pow_succ, Nat.mul_comm]
    · induction n with
      | zero => simp [e]
      | succ n ih =>
        have he : e (n + 1) = 2 * e n := by
          simp [e, pow_succ, Nat.mul_comm]
        have hb2 : b ^ (2 * e n) = (b ^ e n) ^ 2 := by
          rw [show 2 * e n = e n * 2 by omega, pow_mul]
        have ha2 : a ^ (2 * e n) = (a ^ e n) ^ 2 := by
          rw [show 2 * e n = e n * 2 by omega, pow_mul]
        have hfactor :
          (b ^ e n - a ^ e n) *
            (∑ j ∈ Finset.range 2,
              b ^ ((2 - 1 - j) * e n) * a ^ (j * e n)) =
              b ^ (2 * e n) - a ^ (2 * e n) := by
          rw [hb2, ha2]
```



```

norm_num [Finset.sum_range_succ]
ring
calc
  (Π k ∈ Finset.range (Nat.succ n),
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
    ((Π k ∈ Finset.range n,
      (Σ j ∈ Finset.range 2,
        b ^ ((2 - 1 - j) * e k) * a ^ (j * e k))) *
      (Σ j ∈ Finset.range 2,
        b ^ ((2 - 1 - j) * e n) * a ^ (j * e n))) * (b - a) := by
      rw [Finset.prod_range_succ]
  - = ((Π k ∈ Finset.range n,
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e k) * a ^ (j * e k))) * (b - a)) *
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e n) * a ^ (j * e n)) := by ring
  - = (b ^ e n - a ^ e n) *
    (Σ j ∈ Finset.range 2,
      b ^ ((2 - 1 - j) * e n) * a ^ (j * e n)) := by rw [ih]
  - = b ^ (2 * e n) - a ^ (2 * e n) := hfactor
  - = b ^ e (Nat.succ n) - a ^ e (Nat.succ n) := by rw [he]
· let e : ℕ → ℕ := fun k => 3 ^ k
refine (e, ?_, ?_, ?_)
· simp [e]
· intro k
  simp [e, pow_succ, Nat.mul_comm]
· induction n with
| zero => simp [e]
| succ n ih =>
  have he : e (n + 1) = 3 * e n := by
    simp [e, pow_succ, Nat.mul_comm]
  have hb3 : b ^ (3 * e n) = (b ^ e n) ^ 3 := by
    rw [show 3 * e n = e n * 3 by omega, pow_mul]
  have ha3 : a ^ (3 * e n) = (a ^ e n) ^ 3 := by
    rw [show 3 * e n = e n * 3 by omega, pow_mul]
  have hfactor :
    (b ^ e n - a ^ e n) *
      (Σ j ∈ Finset.range 3,
        b ^ ((3 - 1 - j) * e n) * a ^ (j * e n)) =
      b ^ (3 * e n) - a ^ (3 * e n) := by
    rw [hb3, ha3]
  norm_num [Finset.sum_range_succ]
  ring
calc
  (Π k ∈ Finset.range (Nat.succ n),
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e k) * a ^ (j * e k))) * (b - a) =
    ((Π k ∈ Finset.range n,
      (Σ j ∈ Finset.range 3,
        b ^ ((3 - 1 - j) * e k) * a ^ (j * e k))) *
      (Σ j ∈ Finset.range 3,
        b ^ ((3 - 1 - j) * e n) * a ^ (j * e n))) * (b - a) := by
      rw [Finset.prod_range_succ]
  - = ((Π k ∈ Finset.range n,
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e k) * a ^ (j * e k))) * (b - a)) *
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e n) * a ^ (j * e n)) := by ring
  - = (b ^ e n - a ^ e n) *
    (Σ j ∈ Finset.range 3,
      b ^ ((3 - 1 - j) * e n) * a ^ (j * e n)) := by rw [ih]
  - = b ^ (3 * e n) - a ^ (3 * e n) := hfactor
  - = b ^ e (Nat.succ n) - a ^ e (Nat.succ n) := by rw [he]

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12a_2021_p9__mh.me.me__3967729c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 59bc3742bbb4ce64

4.37 037. symbolic_doubling_power_product

The problem

For every pair of integers a and b and every natural number n , there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $(b - a) \prod_{k=0}^{n-1} (b^{e_k} + a^{e_k}) = b^{e_n} - a^{e_n}$.

Lean statement

```
theorem symbolic_doubling_power_product :
  ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) =
      b ^ e n - a ^ e n
```

Parent. For every natural number n , show that there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $\prod_{k=0}^{n-1} (2^{e_k} + 3^{e_k}) = 3^{e_n} - 2^{e_n}$.

Parent in Lean

```
theorem bounded_power_sum_product :
  ∀ n : ℕ, ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, ((2 : ℤ) ^ e k + (3 : ℤ) ^ e k)) =
      (3 : ℤ) ^ e n - (2 : ℤ) ^ e n
```

What it contributes. Generalizes the parent's fixed bases 2 and 3 to arbitrary integer bases a and b , retaining the doubling exponent sequence while exposing the symbolic difference-of-squares step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both bases and requires the doubling-exponent factorization for arbitrary integers a and b ; the parent only establishes the fixed-base case. Although the witness sequence is reused, the parent theorem cannot directly prove the symbolic identity.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the bases and introduces the symbolic factor ' $(b - a)$ ', requiring a general power-difference telescoping lemma over $\mathbb{Z}$. The parent's specialized induction for bases 2 and 3 does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the telescoping identity from fixed bases 2 and 3 to arbitrary integer bases a and b , introducing the factor $(b - a)$. Its proof requires a general power-difference lemma rather than recalling the parent's fixed-base induction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1820 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem symbolic_doubling_power_product.extracted_1.1 : ∀ (a b : ℤ) (n : ℕ),
  ∃ e,
    e 0 = 1 ∧
    (∀ (k : ℕ), e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) = b ^ e n - a ^ e n
```

Read back by a model that saw only the Lean. For every pair of integers a and b and every natural number n , there exists a function e from the natural numbers to the natural numbers such that $e(0) = 1$, for every natural number k we have $e(k + 1) = 2e(k)$, and $(\prod_{k=0}^{n-1} (b^{e(k)} + a^{e(k)}))(b - a) = b^{e(n)} - a^{e(n)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem symbolic_doubling_power_product :
  ∀ (a b : ℤ) (n : ℕ), ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, (b ^ e k + a ^ e k)) * (b - a) =
      b ^ e n - a ^ e n := by
  intro a b n
  let e : ℕ → ℕ := fun k => 2 ^ k
  refine ⟨e, ?_, ?_, ?_⟩
  · simp [e]
  · intro k
    simp [e, pow_succ, Nat.mul_comm]
  · simpa [e] using
    (pow_two_pow_sub_pow_two_pow (R := ℤ) (x := b) (y := a) n).symm
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2021_p9__mh.me__ddd6340a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 014c3b0057680c73

4.38 038. natural_iterate_period

The problem

Let f be a real-valued function with a positive period p . Show that every natural multiple of p is also a period of f .

Lean statement

```
theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
```

Parent. Let a be a positive real number and f be a real function such that $\forall x \in \mathbb{R}, f(x+a) = \frac{1}{2} + \sqrt{f(x) - f(x)^2}$. Show that there exists a positive real number b such that $\forall x \in \mathbb{R}, f(x+b) = f(x)$.

Parent in Lean

```
theorem imo_1968_p5_1 (a : ℝ) (f : ℝ → ℝ) (ha : 0 < a)
  (h1 : ∀ x, f (x + a) = 1 / 2 + Real.sqrt (f x - f x ^ 2)) :
  ∃ b > 0, ∀ x, f (x + b) = f x := by
```

What it contributes. The parent supplies the positive-period checkpoint: its existential conclusion provides a suitable p and $hperiod$ for this reusable natural-iterate law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a new induction argument to lift a one-step period to every natural multiple, including the arithmetic normalization of successive translates. This reasoning is not supplied by the parent's proof, which only derives a single period from the recurrence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an existential period conclusion into a natural-indexed translation law and requires induction to propagate the period across arbitrary multiples. The parent's analytic argument only derives one period and does not supply this iterative reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a reusable natural-number iterate law from an arbitrary period, requiring induction and transport of the casted successor. The parent's proof only derives one specific positive period and supplies no iteration argument, so recalling it does not solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2487 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem natural_iterate_period.extracted_1_1 : ∀ (f : ℝ → ℝ) (p : ℝ),
  0 < p → (∀ (x : ℝ), f (x + p) = f x) → ∀ (n : ℕ) (x : ℝ), f (x + ↑n * p) = f x
```

Read back by a model that saw only the Lean. *For every function f from the real numbers to the real numbers and every real number p , if p is positive and $f(x + p) = f(x)$ for every real number x , then for every natural number n and every real number x , $f(x + n p) = f(x)$, where n is regarded as a real number.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
  intro n
  induction n with
  | zero =>
    intro x
    simp
  | succ n ih =>
    intro x
    have hstep : f ((x + (n : ℝ) * p) + p) = f (x + (n : ℝ) * p) :=
      hperiod (x + (n : ℝ) * p)
    have hprev : f (x + (n : ℝ) * p) = f x := ih x
    calc
      f (x + (Nat.succ n : ℝ) * p) = f ((x + (n : ℝ) * p) + p) := by
        congr 1
      norm_num [Nat.cast_succ]
      ring
      _ = f (x + (n : ℝ) * p) := hstep
      _ = f x := hprev
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me__04dc59b8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c91ed953a9e4530c

4.39 039. coupled_configuration_impossible

The problem

Let $p < q < r$ be integers greater than 1 such that $(p-1)(q-1)(r-1)$ divides $pqr-1$. If r is one of the two possible classified values 8 or 15, show that $r \neq 10^n - (-1)^n$ for every natural number n .

Lean statement

```
theorem coupled_configuration_impossible (p q r : ℤ) (n : ℕ)

(hclassified : r = 8 ∨ r = 15)
(hcouple : r = (10 : ℤ) ^ n - (-1 : ℤ) ^ n) : False
```

Parent. Let p, q, r be integers with $1 < p < q < r$. Show that if $(p-1) * (q-1) * (r-1)$ divides $p * q * r - 1$, then $(p, q, r) = (2, 4, 8)$ or $(p, q, r) = (3, 5, 15)$.

Parent in Lean

```
theorem imo_1992_p1 (p q r : ℤ) (h₀ : 1 < p ∧ p < q ∧ q < r)
(h₁ : (p - 1) * (q - 1) * (r - 1) ∣ p * q * r - 1) :
(p, q, r) = (2, 4, 8) ∨ (p, q, r) = (3, 5, 15) := by
```

What it contributes. Supplies the classification checkpoint that the possible third coordinate is 8 or 15, represented here by `hclassified`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child uses the certified classification $r = 8$ or 15 but introduces genuinely new induction proving 11 divides $10^n - (-1)^n$, then derives contradictions for both classified values. A memorized proof of the parent would not supply this divisibility argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the parent-derived classification of r but requires a new induction proving $11 \mid 10^n - (-1)^n$, then derives a contradiction for both classified values. Recalling the parent does not supply this divisibility argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not solvable by recalling the parent's classification proof: it independently proves that 11 divides $10^n - (-1)^n$ by induction, then contradicts the classified values 8 and 15 . This is a smaller executable child using the available classification checkpoint and introduces genuinely different mathematics.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2738 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_configuration_impossible.extracted_1_1 (p q r : ℤ) (n : ℕ) (h₀ : 1 < p ∧ p < q ∧ q < r)
(h₁ : (p - 1) * (q - 1) * (r - 1) ∣ p * q * r - 1) (hclassified : r = 8 ∨ r = 15) (hcouple : r = 10^n - (-1)^n) :
False
```

Read back by a model that saw only the Lean. *Let p, q , and r be integers and let n be a natural number. Suppose that $1 < p < q < r$, that $(p-1)(q-1)(r-1)$ divides $pqr-1$, that r is either 8 or 15, and that $r = 10^n - (-1)^n$. Then a contradiction follows; equivalently, False holds.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem coupled_configuration_impossible (p q r : ℤ) (n : ℕ)

  (hclassified : r = 8 ∨ r = 15)
  (hcouple : r = (10 : ℤ) ^ n - (-1 : ℤ) ^ n) : False := by
have hpow : ∀ m : ℕ, (11 : ℤ) ∣ (10 : ℤ) ^ m - (-1 : ℤ) ^ m := by
  intro m
  induction m with
  | zero => norm_num
  | succ m ih =>
    rcases ih with ⟨k, hk⟩
    refine ⟨10 * k + (-1 : ℤ) ^ m, ?_⟩
    simp only [pow_succ]
    calc
      10 ^ m * 10 - (-1) ^ m * (-1) =
        10 * (10 ^ m - (-1) ^ m) + 11 * (-1) ^ m := by ring
      _ = 10 * (11 * k) + 11 * (-1) ^ m := by rw [hk]
      _ = 11 * (10 * k + (-1) ^ m) := by ring
have hdiv : (11 : ℤ) ∣ r := by
  rw [hcouple]
  exact hpow n
rcases hclassified with h8 | h15
· rw [h8] at hdiv
  norm_num at hdiv
· rw [h15] at hdiv
  norm_num at hdiv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. imo_1992_p1__me__0afbec7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 4f2b9338b2a23037

4.40 040. bounded_residue_power_mod11

The problem

If an integer base is congruent to 10 modulo 11, then every natural power of that base is congruent to the corresponding signed power of -1 modulo 11.

Lean statement

```
theorem bounded_residue_power_mod11 (b : ℤ) (h : Int.ModEq 11 b 10) : ∀ n : ℕ, Int.ModEq 11 (b ^ n) ((-1 : ℤ) ^ n)
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonmn1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Supplied the signed-power invariant and modulus/base residue pattern; the child parameterizes the base and proves the modular congruence directly.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base: from an arbitrary residue-class hypothesis $b \equiv 10 \pmod{11}$, it proves all powers $b^n \equiv (-1)^n$ by induction and multiplicativity of `Int.ModEq`. The parent's fixed-base divisibility lemma does not supply this argument for arbitrary b .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and changes divisibility into a modular-congruence statement. Its proof must transfer $b \equiv 10 \equiv -1 \pmod{11}$ and propagate that relation through multiplication and induction, which the parent's single fixed-base divisibility lemma does not provide.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed base 10 result to every integer base congruent to 10 modulo 11. Its induction uses modular-equivalence transitivity and multiplicativity, reasoning not supplied by the parent's direct divisibility lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1874 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_residue_power_mod11.extracted_1_1 : ∀ (b : ℤ),  
  b ≡ 10 [ZMOD 11] → ∀ (n : ℕ), b ^ n ≡ (-1) ^ n [ZMOD 11]
```

Read back by a model that saw only the Lean. *For every integer b , if b is congruent to 10 modulo 11, then for every natural number n , b raised to the n th power is congruent to (-1) raised to the n th power modulo 11.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_residue_power_mod11 (b : ℤ) (h : Int.ModEq 11 b 10) : ∀ n : ℕ, Int.ModEq 11 (b ^ n) ((-1 : ℤ) ^ n) := by  
  intro n  
  induction n with  
  | zero =>  
    simp [Int.ModEq]  
  | succ n ih =>  
    have hten : Int.ModEq 11 (10 : ℤ) (-1 : ℤ) := by  
      norm_num [Int.ModEq]  
    have hstep : Int.ModEq 11 b (-1 : ℤ) := h.trans hten  
    have hmul : Int.ModEq 11 (b ^ n * b) ((-1 : ℤ) ^ n * (-1 : ℤ)) := ih.mul hstep  
    simp [pow_succ] using hmul
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__0077ec87

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 35afb0214da0a4fd

4.41 041. bounded_mod11_product

The problem

For any finite list of natural numbers, the difference between the product of the corresponding powers of 10 and the product of the corresponding powers of -1 is divisible by 11.

Lean statement

```
theorem bounded_mod11_product (xs : List ℕ) :
  (11 : ℤ) ∣ (xs.map (fun n => (10 : ℤ) ^ n)).prod -
  (xs.map (fun n => (-1 : ℤ) ^ n)).prod
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonmn1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Supplies the modulo-11 divisibility checkpoint for the aggregate exponent via the standard difference-of-powers argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine aggregate generalization: it requires induction to identify each product with a power indexed by the list sum, then applies the modulo-11 law. Recalling the parent only supplies the final divisibility step and does not solve the new product-to-power reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new induction proving that both list products collapse to powers indexed by the sum of exponents, then combines these identities with the single-exponent divisibility law. Recalling the parent only supplies the final divisibility step and cannot solve the aggregate statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires an induction to identify each finite product with a power at the aggregate exponent, then applies the parent law. This aggregate normalization is new reasoning beyond recalling the parent's single-exponent divisibility proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2707 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	MISMATCH: Statement B uses a natural-number interpretation for the difference (and potentially for -1), whereas Statement A uses integer arithmetic; for example, with $xs=[1]$, A gives $10-(-1)=11$, but B's natural interpretation gives $10-0=10$, which is not divisible by 11.

Goal round-trip: what Lean elaborated

```
theorem bounded_mod11_product.extracted_1_1 (xs : List ℕ) :
  11 ∣ (List.map (fun n => 10 ^ n) xs).prod - (List.map (fun n => (-1) ^ n) xs).prod
```

Read back by a model that saw only the Lean. *For every list xs of natural numbers, 11 divides the natural-number difference between the product of 10^n over all n in xs and the product of $(-1)^n$ over all n in xs .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_mod11_product (xs : List ℕ) :
  (11 : ℤ) ∣ (xs.map (fun n => (10 : ℤ) ^ n)).prod -
    (xs.map (fun n => (-1 : ℤ) ^ n)).prod := by
  have hagg :
    (xs.map (fun n => (10 : ℤ) ^ n)).prod = (10 : ℤ) ^ xs.sum ∧
    (xs.map (fun n => (-1 : ℤ) ^ n)).prod = (-1 : ℤ) ^ xs.sum := by
    induction xs with
    | nil => simp
    | cons a xs ih =>
      constructor
      · simpa [ih.1, pow_add]
      · simpa [ih.2, pow_add]
  have hdiv : (11 : ℤ) ∣ (10 : ℤ) ^ xs.sum - (-1 : ℤ) ^ xs.sum := by
    simpa using (sub_dvd_pow_sub_pow (10 : ℤ) (-1 : ℤ) xs.sum)
  rw [hagg.1, hagg.2]
  exact hdiv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__16570d78

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 16570d7852a5e928

4.42 042. alternating_power_divisibility_iff

The problem

For an integer a , prove that 11 divides $a^n - (-1)^n$ for every natural number n if and only if a is congruent to -1 modulo 11.

Lean statement

```
theorem alternating_power_divisibility_iff (a : ℤ) : (∀ n : ℕ, (11 : ℤ) ∣ a ^ n - (-1 : ℤ) ^ n) ↔ a ≡ (-1 : ℤ) [ZMOD 11]
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonm1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Generalizes the parent's alternating-power divisibility argument from the fixed base 10 to a parameter a , consuming the congruence-derived divisor through the power-difference lemma.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fixed divisibility claim into an exact iff characterization over arbitrary integers. It requires both extracting the congruence from the $n = 1$ case and proving the universal law from an arbitrary congruence, so recalling the parent does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes all integers satisfying the divisibility law modulo 11. Its reverse direction uses the power-difference divisibility lemma parametrically, while the forward direction extracts the congruence from the exponent-one case; recalling the parent's fixed-base proof does not solve this equivalence.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the task from proving one fixed-base divisibility law to characterizing exactly which integers satisfy the law for every exponent. It requires a new reverse-direction argument via the $n = 1$ case and modular equivalence, so recalling the parent's single application of the power-difference lemma is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2759 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem alternating_power_divisibility_iff.extracted_1_1 (a : Z) :  
  (∀ (n : N), 11 ∣ a ^ n - (-1) ^ n) ↔ a ≡ -1 [ZMOD 11]
```

Read back by a model that saw only the Lean. For an integer a , 11 divides $a^n - (-1)^n$ for every natural number n if and only if a is congruent to -1 modulo 11.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem alternating_power_divisibility_iff (a : Z) : (∀ n : N, (11 : Z) ∣ a ^ n - (-1 : Z) ^ n) ↔ a ≡ (-1 : Z) [ZMOD 11] := by  
  constructor  
  · intro h  
    have h1 : (11 : Z) ∣ a - (-1 : Z) := by  
      simpa using h 1  
    have hneg : (11 : Z) ∣ (-1 : Z) - a := by  
      simpa using (dvd_neg.mpr h1)  
    rw [Int.modEq_iff_dvd]  
    exact hneg  
  · intro h  
    have hdiff : (11 : Z) ∣ a - (-1 : Z) := by  
      exact h.symm.dvd  
    intro n  
    have hexp : a - (-1 : Z) ∣ a ^ n - (-1 : Z) ^ n := by  
      exact sub_dvd_pow_sub_pow a (-1 : Z) n  
    exact dvd_trans hdiff hexp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__6e895ca9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 6e895ca941e9e1a4

4.43 043. parameterized_11_dvd_pow_sub_pow

The problem

Let b be an integer such that $11 \mid b^2 - 1$ but $11 \nmid b - 1$. Prove that $11 \mid b^n - (-1)^n$ for every natural number n .

Lean statement

```
theorem parameterized_11_dvd_pow_sub_pow (b : ℤ) (hquad : (11 : ℤ) ∣ b ^ 2 - 1) (hne : ¬ (11 : ℤ) ∣ b - 1) : ∀ n : ℕ, (11 : ℤ) ∣ b ^ n - (-1 : ℤ) ^ n
```

Parent. Show that for any natural number n , we have $11 \mid 10^n - (-1)^n$.

Parent in Lean

```
theorem induction_11div10tonmn1ton (n : ℕ) : 11 ∣ 10 ^ n - (-1 : ℤ) ^ n := by
```

What it contributes. Reuses the parent's difference-of-powers exponent law while replacing the fixed base 10 with a parameter whose residue is classified from the quadratic hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child requires genuinely new reasoning: factoring the quadratic, using primality of 11 and the nondivisibility hypothesis to classify b modulo 11, then transporting that result to all powers. The parent is a direct application of `sub_dvd_pow_sub_pow` and does not supply this classification argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and requires new reasoning: factoring the quadratic, using primality of 11 and the nondivisibility hypothesis to classify b modulo 11, then transferring that divisibility to all powers. The parent's direct fixed-base application cannot solve this classification step.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and must derive the residue classification from the quadratic divisibility and nondivisibility hypotheses before applying the power-difference lemma. The parent's direct proof for base 10 does not supply that classification or the required divisibility transfer.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2736 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_11_dvd_pow_sub_pow.extracted_1_1 : ∀ (b : ℤ),  
  11 ∣ b ^ 2 - 1 → ¬11 ∣ b - 1 → ∀ (n : ℕ), 11 ∣ b ^ n - (-1) ^ n
```

Read back by a model that saw only the Lean. For every integer b , if 11 divides $b^2 - 1$ and 11 does not divide $b - 1$, then for every natural number n , 11 divides $b^n - (-1)^n$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem parameterized_11_dvd_pow_sub_pow (b : ℤ) (hquad : (11 : ℤ) ∣ b ^ 2 - 1) (hne : ¬ (11 : ℤ) ∣ b - 1) : ∀ n : ℕ, (11 : ℤ) ∣ b ^ n -  
  (-1 : ℤ) ^ n := by  
  have hfactor : b ^ 2 - 1 = (b - 1) * (b + 1) := by  
    ring  
  have hprod : (11 : ℤ) ∣ (b - 1) * (b + 1) := by  
    rw [← hfactor]  
    exact hquad  
  have hp11 : Prime (11 : ℤ) := by  
    norm_num  
  have hcases : (11 : ℤ) ∣ b - 1 ∨ (11 : ℤ) ∣ b + 1 :=  
    (hp11.dvd_mul).mp hprod  
  have hplus : (11 : ℤ) ∣ b + 1 := by  
    cases hcases with  
    | inl hleft => exact False.elim (hne hleft)  
    | inr hright => exact hright  
  have hbase : (11 : ℤ) ∣ b - (-1 : ℤ) := by  
    simpa using hplus  
  intro n  
  have hpow : b - (-1 : ℤ) ∣ b ^ n - (-1 : ℤ) ^ n := by  
    exact sub_dvd_pow_sub_pow b (-1 : ℤ) n  
  exact dvd_trans hbase hpow
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_11div10tonmn1ton__me__a4706044

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a4706044089b5e7d

4.44 044. odd_power_geometric_factorization

The problem

For natural numbers b , n , and q with $b = 3q + 2$, prove that $(b^{2n+1} + 1) = (b + 1) \sum_{k=0}^{2n} (-1)^k b^{2n-k}$ and $b + 1 = 3(q + 1)$, with these equalities interpreted over the integers.

Lean statement

```
theorem odd_power_geometric_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ((b : ℤ) ^ (2 * n + 1) + 1 = ((b : ℤ) + 1) * ∑ k ∈ Finset.range (2 * n + 1), (-1 : ℤ) ^ k * (b : ℤ) ^ (2 * n - k)) ∧ (b : ℤ) + 1 = 3 * (q + 1)
```

Parent. For natural numbers b , n , and q , if $b = 3q + 2$, prove that there exists a natural number s such that $b^{2n+1} + 1 = 3(q + 1)s$.

Parent in Lean

```
theorem odd_power_plus_one_parameterized_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ∃ s : ℕ, b ^ (2 * n + 1) + 1 = 3 * (q + 1) * s
```

What it contributes. Uses the parent decomposition $b = 3q + 2$ and its checkpoint $b + 1 = 3(q + 1)$, while replacing the existential divisibility witness with a symbolic geometric-factorization identity.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces existential divisibility with an explicit alternating geometric-factorization identity over $\mathbb{Z}$. Proving this requires a new induction on the exponent and sign analysis; the parent's divisibility lemma and final witness construction do not supply that reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces existential divisibility with an explicit integer geometric-factorization identity, proved by induction over the exponent, while separately deriving the factor relation from the decomposition hypothesis. This requires genuinely new reasoning and cannot be solved by recalling the parent's divisibility proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces existential divisibility with an explicit integer geometric-factorization identity, whose proof requires a new induction and ring algebra absent from the parent. The appended base equality is shared, but the substantive factorization is not recoverable by recalling the parent's divisibility argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2066 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_geometric_factorization.extracted_1_1 (b n q : ℕ) (hdecomp : b = 3 * q + 2) :
  ↑b ^ (2 * n + 1) + 1 = (↑b + 1) * ∑ k ∈ Finset.range (2 * n + 1), (-1) ^ k * ↑b ^ (2 * n - k) ∧
  ↑b + 1 = 3 * (↑q + 1)
```

Read back by a model that saw only the Lean. For natural numbers b , n , and q , if $b = 3q + 2$, then both of the following hold: (1) the integer b raised to the natural-number exponent $2n + 1$, plus 1, equals $(b + 1)$ times the sum over k in $\text{Finset.range } (2n + 1)$ of $(-1)^k$ times b raised to the natural-number exponent $2n - k$; and (2) $b + 1 = 3(q + 1)$, where b and q are regarded as integers via the natural-number embedding.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem odd_power_geometric_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ((b : ℤ) ^ (2 * n + 1) + 1 = ((b : ℤ) + 1) * ∑ k ∈
  ↪ Finset.range (2 * n + 1), (-1 : ℤ) ^ k * (b : ℤ) ^ (2 * n - k)) ∧ (b : ℤ) + 1 = 3 * (q + 1) := by
  have hgeom : ∀ m : ℕ, ((b : ℤ) + 1) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1)) = (b : ℤ) ^ m - (-1 : ℤ) ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      simp [Finset.sum_range_succ, Nat.succ_sub_succ_eq_sub] at ih ⊢
      have hs :
        (∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - k)) =
          (b : ℤ) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1)) := by
            rw [Finset.mul_sum]
            apply Finset.sum_congr rfl
            intro k hk
            have hkm : k < m := Finset.mem_range.mp hk
            have he : m - k = (m - (k + 1)) + 1 := by omega
            rw [he, pow_succ]
            ring
      calc
      ((b : ℤ) + 1) *
        ((∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - k)) + (-1 : ℤ) ^ m) =
        ((b : ℤ) + 1) *
          ((b : ℤ) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1)) + (-1 : ℤ) ^ m) := by rw [hs]
      _ = (b : ℤ) * (((b : ℤ) + 1) * ∑ k ∈ Finset.range m, (-1 : ℤ) ^ k * (b : ℤ) ^ (m - (k + 1))) + ((b : ℤ) + 1) * (-1 : ℤ) ^ m :=
        ↪ by ring
      _ = (b : ℤ) * ((b : ℤ) ^ m - (-1 : ℤ) ^ m) + ((b : ℤ) + 1) * (-1 : ℤ) ^ m := by rw [ih]
      _ = (b : ℤ) ^ (m + 1) - (-1 : ℤ) ^ (m + 1) := by
        rw [pow_succ, pow_succ]
        ring
      have hsign : (-1 : ℤ) ^ (2 * n + 1) = -1 := by
        induction n with
        | zero => norm_num
        | succ n ih =>
          rw [show 2 * (Nat.succ n) + 1 = (2 * n + 1) + 2 by omega, pow_add]
          norm_num [ih]
      have hfactor : (b : ℤ) ^ (2 * n + 1) + 1 = ((b : ℤ) + 1) * ∑ k ∈ Finset.range (2 * n + 1), (-1 : ℤ) ^ k * (b : ℤ) ^ (2 * n - k) := by
        simpa [Nat.add_sub_add_right, hsign] using (hgeom (2 * n + 1)).symm
      have hbase : (b : ℤ) + 1 = 3 * (q + 1) := by
        omega
      exact ⟨hfactor, hbase⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me.me.me__3f8b13d5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3f8b13d5913ed6df

4.45 045. residue_power_minus_one_iff_odd

The problem

For every natural base b and exponent m , if $b \equiv 2 \pmod{3}$, then 3 divides $b^m + 1$ over the integers if and only if m is odd.

Lean statement

```
theorem residue_power_minus_one_iff_odd (b m : ℕ) (hmod : b % 3 = 2) : (3 : ℤ) ∣ (b : ℤ) ^ m + 1 ↔ Odd m
```

Parent. For every natural bound B , base b , and exponent parameter n , if $b \leq B$ and $b \equiv 2 \pmod{3}$, then 3 divides $b^{(2n+1)} + 1$ over the integers.

Parent in Lean

```
theorem bounded_residue_odd_power (B b n : ℕ) (hb : b ≤ B) (hmod : b % 3 = 2) : 3 ∣ (b : ℤ) ^ (2 * n + 1) + 1
```

What it contributes. Supplies the residue decomposition, divisibility of $b+1$ by 3, and the odd-power sum factorization used in the odd direction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child upgrades the parent's one-way odd-exponent divisibility result to an iff characterization. Its converse requires a new modular argument showing even powers are congruent to 1 modulo 3, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's one-way odd-exponent result into an exact iff characterization. Its reverse direction requires a new even/odd case split and modular induction, so recalling the parent cannot solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine iff characterization: besides the parent's odd-exponent direction, it establishes that even exponents cannot yield the congruence, using parity decomposition and an inductive residue argument. This converse is not supplied by recalling the parent's proof, and the hypothesis is materially used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1770 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem residue_power_minus_one_iff_odd.extracted_1_1 (b m : ℕ) (hmod : b % 3 = 2) : 3 ∣ ↑b ^ m + 1 ↔ Odd m
```

Read back by a model that saw only the Lean. For natural numbers b and m , if b modulo 3 equals 2, then 3 divides the integer $b^m + 1$ if and only if m is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem residue_power_minus_one_iff_odd (b m : ℕ) (hmod : b % 3 = 2) : (3 : ℤ) ∣ (b : ℤ) ^ m + 1 ↔ Odd m := by
  have hdecomp : b = 3 * (b / 3) + 2 := by
    omega
  have hdecompZ : (b : ℤ) = 3 * (↑(b / 3) : ℤ) + 2 := by
    exact_mod_cast hdecomp
  have hbase : (3 : ℤ) ∣ (b : ℤ) + 1 := by
    refine ⟨(↑(b / 3) : ℤ) + 1, ?_⟩
    rw [hdecompZ]
    ring
  have hbsq : b ^ 2 % 3 = 1 := by
    norm_num [pow_two, Nat.mul_mod, hmod]
  have h_even_mod : ∀ k : ℕ, b ^ (2 * k) % 3 = 1 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [Nat.mul_succ, Nat.pow_add, Nat.mul_mod, ih, hbsq]
  constructor
  · intro hd
    have hdNat : 3 ∣ b ^ m + 1 := by
      exact_mod_cast hd
    rcases Nat.even_or_odd m with he | ho
    · rcases he with (k, hk)
      have hzero : (b ^ m + 1) % 3 = 0 := Nat.dvd_iff_mod_eq_zero.mp hdNat
      have hkm : k + k = 2 * k := by omega
      rw [hk, hkm, Nat.add_mod, h_even_mod] at hzero
      norm_num at hzero
    · exact ho
  · intro ho
    have hpow : (b : ℤ) + 1 ∣ (b : ℤ) ^ m + (1 : ℤ) ^ m := by
      exact Odd.add_dvd_pow_add_pow (b : ℤ) 1 ho
    have htarget : (b : ℤ) + 1 ∣ (b : ℤ) ^ m + 1 := by
      simpa using hpow
    rcases hbase with ⟨c, hc⟩
    rcases htarget with ⟨d, hd⟩
    refine ⟨c * d, ?_⟩
    rw [hd, hc]
    ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.me__bd3f2cd6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 469f52043806a3be

4.46 046. power_mod_three_if

The problem

Let b and k be natural numbers with $b \equiv 2 \pmod{3}$. Show that the remainder of b^k upon division by 3 is 1 when k is even and 2 when k is odd.

Lean statement

```
theorem power_mod_three_if (b k : ℕ) (hb : b % 3 = 2) : b ^ k % 3 = if k % 2 = 0 then 1 else 2
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Reuses the parent's explicit base-class condition modulo 3 and its odd-power residue mechanism, generalized here through exponent induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from odd exponents and divisibility to an exact two-branch residue characterization for every exponent. Its induction tracks parity and multiplication modulo 3, requiring reasoning not supplied by the parent's divisibility argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from odd exponents of the form $2n+1$ to every exponent and characterizes both residue branches modulo 3. Its induction and parity case split supply genuinely new reasoning beyond the parent's odd-power divisibility argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from odd exponents to every natural exponent and gives the complete parity-dependent residue formula modulo 3. Its induction and parity case split require reasoning not supplied by the parent's divisibility argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2757 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_mod_three_if.extracted_1_1 (b k : ℕ) (hb : b % 3 = 2) : b ^ k % 3 = if k % 2 = 0 then 1 else 2
```

Read back by a model that saw only the Lean. For natural numbers b and k , if b leaves remainder 2 upon division by 3, then b raised to the k -th power leaves remainder 1 modulo 3 when k is even, and remainder 2 modulo 3 when k is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_mod_three_if (b k : ℕ) (hb : b % 3 = 2) : b ^ k % 3 = if k % 2 = 0 then 1 else 2 := by
  induction k with
  | zero => simp
  | succ k ih =>
    rw [pow_succ, Nat.mul_mod]
    by_cases hk : k % 2 = 0
    · have hk1 : (Nat.succ k) % 2 = 1 := by omega
      rw [ih]
      simp [hk, hk1, hb]
    · have hk1 : k % 2 = 1 := by omega
      have hk2 : (Nat.succ k) % 2 = 0 := by omega
      rw [ih]
      simp [hk, hk1, hk2, hb]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.me__dd5e8f57

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint dd5e8f579d17e3eb

4.47 047. divisibility_iff_odd_exponent_mod_three

The problem

Let b and k be natural numbers with $b \equiv 2 \pmod{3}$. Prove that $3 \mid (b^k + 1)$ if and only if k is odd, including the case $k = 0$.

Lean statement

```
theorem divisibility_iff_odd_exponent_mod_three (b k : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ k + 1 ↔ Odd k
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Supplies the base divisibility $3 \mid b + 1$ and odd-power sum-divisibility argument; the child adds the reverse parity implication and the explicit $k=0$ case.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine converse: divisibility forces oddness by handling zero and proving all even exponents give residue 1 modulo 3. Recalling the parent only supplies the odd-to-divisibility direction, not the parity argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes all exponents: beyond reusing the parent's odd-power argument, it proves the converse by deriving the modular behavior of even powers and handling the zero exponent separately. This reverse direction is new mathematics and cannot be obtained by recalling the parent theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-way divisibility lemma into a full iff characterization. Besides reusing the parent mechanism for odd exponents as required, it introduces genuinely new parity decomposition and modular-periodicity reasoning to exclude even exponents, including the $k = 0$ case.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2724 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem divisibility_iff_odd_exponent_mod_three.extracted_1_1 (b k : ℕ) (hb : b % 3 = 2) :  
  3 ∣ b ^ k + 1 ↔ Odd k
```

Read back by a model that saw only the Lean. For natural numbers b and k satisfying b modulo 3 equals 2, 3 divides $b^k + 1$ if and only if k is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem divisibility_iff_odd_exponent_mod_three (b k : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ k + 1 ↔ Odd k := by
  have hzero : (¬ 3 ∣ b ^ 0 + 1) ∧ ¬ Odd 0 := by
    simp [Nat.dvd_iff_mod_eq_zero]
  have h_two : ∀ n : ℕ, 2 ^ (2 * n) % 3 = 1 := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      rw [show 2 * (n + 1) = 2 * n + 2 by omega, pow_add]
      rw [Nat.mul_mod, ih]
      norm_num
  have h_even_mod : ∀ n : ℕ, b ^ (2 * n) % 3 = 1 := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      rw [show 2 * (n + 1) = 2 * n + 2 by omega, pow_add]
      rw [Nat.mul_mod, Nat.pow_mod, Nat.pow_mod]
      simp [h_two n, hb, Nat.pow_mod]
  constructor
  · intro hd
    by_cases hk0 : k = 0
    · subst k
      exact False.elim (hzero.1 (by simp using hd))
    · rcases Nat.even_or_odd k with he | ho
      · rcases he with ⟨n, hn⟩
        have hm : (b ^ k + 1) % 3 = 0 := Nat.dvd_iff_mod_eq_zero.mp hd
        have hn2 : k = 2 * n := by omega
        rw [hn2] at hm
        rw [Nat.add_mod, h_even_mod n] at hm
        omega
      · exact ho
  · intro hk
    by_cases hk0 : k = 0
    · subst k
      exact False.elim (hzero.2 hk)
    · rcases hk with ⟨n, hn⟩
      rw [hn]
      have hbase : 3 ∣ b + 1 := by
        have hdecomp : b % 3 + 3 * (b / 3) = b := Nat.mod_add_div b 3
        rw [hb] at hdecomp
        use b / 3 + 1
        omega
      have hodd : Odd (2 * n + 1) := by
        refine ⟨n, ?_⟩
        omega
      have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 ^ (2 * n + 1) := by
        exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
      have hpow2 : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
        simp using hpow
      have hfinal : 3 ∣ b ^ (2 * n + 1) + 1 := dvd_trans hbase hpow2
      exact hfinal
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me.me__e1a43802

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e1a438021c39f610

4.48 048. odd_power_plus_one_parameterized_factorization

The problem

For natural numbers b , n , and q , if $b = 3q + 2$, prove that there exists a natural number s such that $b^{2n+1} + 1 = 3(q + 1)s$.

Lean statement

```
theorem odd_power_plus_one_parameterized_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ∃ s : ℕ, b ^ (2 * n + 1) + 1 = 3 * (q + 1)
  ↪ * s
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Reuses the parent's odd-exponent power-sum divisibility checkpoint, while strengthening the conclusion by retaining the symbolic factor $q+1$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the quotient factor: it derives $b + 1 = 3(q + 1)$, constructs the odd-power cofactor, and transports that cofactor into an explicit three-factor equality. This is not recoverable by merely recalling the parent's divisibility proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the factorization: it derives the exact identity $b+1=3(q+1)$ and carries the symbolic cofactor through to an explicit three-factor equality. The parent only supplies divisibility by 3 and does not provide the stronger quotient-sensitive conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the decomposition and proves the stronger exact factorization with the non-trivial factor $q+1$. A memorized proof of mere divisibility by 3 does not supply this quotient construction or the rewritten three-factor equality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2057 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_plus_one_parameterized_factorization.extracted_1_1 (b n q : ℕ) (hdecomp : b = 3 * q + 2) :  
  ∃ s, b ^ (2 * n + 1) + 1 = 3 * (q + 1) * s
```

Read back by a model that saw only the Lean. For natural numbers b , n , and q , if $b = 3q + 2$, then there exists a natural number s such that $b^{(2n+1)} + 1 = 3(q+1)s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_power_plus_one_parameterized_factorization (b n q : ℕ) (hdecomp : b = 3 * q + 2) : ∃ s : ℕ, b ^ (2 * n + 1) + 1 = 3 * (q + 1)
  <= * s := by
  have hbase : b + 1 = 3 * (q + 1) := by
    omega
  have hodd : Odd (2 * n + 1) := by
    refine ⟨n, ?_⟩
    omega
  have hpow : b + 1 | b ^ (2 * n + 1) + 1 := by
    have h := Odd.nat_add_dvd_pow_add_pow b 1 hodd
    simpa using h
  rcases hpow with ⟨s, hs⟩
  refine ⟨s, ?_⟩
  calc
    b ^ (2 * n + 1) + 1 = (b + 1) * s := hs
    _ = 3 * (q + 1) * s := by rw [hbase]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.me__f9d6e83f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f9d6e83f43f91234

4.49 049. odd_exponent_divisibility_of_base_mod_three

The problem

Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Lean statement

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

Parent. For a natural number n , show that $3 \mid (2^{2n+1} + 1)$.

Parent in Lean

```
theorem induction_divisibility_3div2tooddp1 (n : ℕ) : 3 ∣ 2 ^ (2 * n + 1) + 1 := by
```

What it contributes. Generalizes the parent's odd-exponent divisibility checkpoint from base 2 to an arbitrary base residue class modulo 3, replacing induction with the reusable odd-power divisibility template.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed base 2 to an arbitrary natural base congruent to 2 modulo 3. Its proof requires deriving $3 \mid b+1$ from the modular hypothesis and applying an odd-exponent sum-of-powers divisibility lemma, which is not supplied by the parent's induction argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed base from 2 to every natural number congruent to 2 modulo 3. Its proof uses modular decomposition and the odd-exponent factorization theorem, rather than recalling the parent's induction argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed base 2 to arbitrary natural bases congruent to 2 modulo 3. Its proof requires deriving divisibility of $b+1$ and applying an odd-power divisibility lemma, so the parent's induction proof cannot simply be recalled.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2050 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_exponent_divisibility_of_base_mod_three.extracted_1_1 (b n : ℕ) (hb : b % 3 = 2) :
  3 ∣ b ^ (2 * n + 1) + 1
```

Read back by a model that saw only the Lean. For natural numbers b and n , if the remainder when b is divided by 3 is 2, then 3 divides b raised to the power $2n + 1$ plus 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1 := by
  have hbase : 3 ∣ b + 1 := by
    have hdecomp : b % 3 + 3 * (b / 3) = b := Nat.mod_add_div b 3
    rw [hb] at hdecomp
    use b / 3 + 1
    omega
  have hodd : Odd (2 * n + 1) := by
    refine ⟨n, ?_⟩
    omega
  have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 ^ (2 * n + 1) := by
    exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
  have hpow2 : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
    simpa using hpow
  exact dvd_trans hbase hpow2
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me__46cb0a18

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 46cb0a1847bea7fd

4.50 050. odd_power_divisibility_iff_base_congruent_two

The problem

For natural numbers b and n , prove that $3 \mid b^{2n+1} + 1$ if and only if b leaves remainder 2 upon division by 3.

Lean statement

```
theorem odd_power_divisibility_iff_base_congruent_two (b n : ℕ) : 3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2
```

Parent. For a natural number n , show that $3 \mid (2^{2n+1} + 1)$.

Parent in Lean

```
theorem induction_divisibility_3div2tooddp1 (n : ℕ) : 3 ∣ 2 ^ (2 * n + 1) + 1 := by
```

What it contributes. Generalizes the parent's odd-exponent divisibility mechanism from the fixed base 2 to an arbitrary base, while adding a converse residue classification.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fixed-base divisibility fact into an iff characterization, requiring a new forward residue analysis to exclude bases congruent to 0 or 1 modulo 3. Its reverse direction also generalizes odd-power divisibility to an arbitrary base, so recalling the parent does not substantially solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed divisibility fact into an iff characterization for arbitrary bases. Its forward direction requires modular residue elimination, and its reverse direction uses a general odd-power divisibility argument that the parent's induction proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine converse residue characterization for arbitrary bases: the forward direction excludes residues 0 and 1 modulo 3, while the reverse direction uses odd-power divisibility from $3 \mid b + 1$. The parent only handles the fixed base 2 by induction and supplies neither the residue analysis nor the converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2472 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_divisibility_iff_base_congruent_two.extracted_1_1 (b n : ℕ) :  
  3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2
```

Read back by a model that saw only the Lean. *For natural numbers b and n , 3 divides b raised to the power $2n + 1$ plus 1 if and only if the remainder when b is divided by 3 is 2.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_power_divisibility_iff_base_congruent_two (b n : ℕ) : 3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2 := by
  have hepos : 0 < 2 * n + 1 := by omega
  constructor
  · intro hd
    have hzero : (b ^ (2 * n + 1) + 1) % 3 = 0 :=
      Nat.dvd_iff_mod_eq_zero.mp hd
    have hlt : b % 3 < 3 := Nat.mod_lt b (by norm_num)
    have hcases : b % 3 = 0 ∨ b % 3 = 1 ∨ b % 3 = 2 := by omega
    rcases hcases with hres | hres | hres
    · have hbmod : Nat.ModEq 3 b 0 := by
        simp [Nat.ModEq] using hres
      have hp := hbmod.pow (2 * n + 1)
      have hpm0 : b ^ (2 * n + 1) % 3 = 0 := by
        simp [Nat.ModEq, hepos] using hp
      rw [Nat.add_mod, hpm0] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 3 b 1 := by
        simp [Nat.ModEq] using hres
      have hp := hbmod.pow (2 * n + 1)
      have hpm1 : b ^ (2 * n + 1) % 3 = 1 := by
        simp [Nat.ModEq] using hp
      rw [Nat.add_mod, hpm1] at hzero
      norm_num at hzero
    · exact hres
  · intro hres
    have hbase : 3 ∣ b + 1 := by
      refine ⟨b / 3 + 1, ?_⟩
      have hdecomp := Nat.mod_add_div b 3
      omega
    have hodd : Odd (2 * n + 1) := by
      exact ⟨n, by omega⟩
    have hpow : b + 1 ∣ b ^ (2 * n + 1) + 1 ^ (2 * n + 1) := by
      exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
    have hpow' : b + 1 ∣ b ^ (2 * n + 1) + 1 := by
      simp using hpow
    exact dvd_trans hbase hpow'
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me__acf5ec57

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint acf5ec572f5a01a5

4.51 051. pow_mod_three_period_two

The problem

For every natural base b with $b \equiv 2 \pmod{3}$ and every natural number n , shifting the exponent by two preserves its residue modulo 3.

Lean statement

```
theorem pow_mod_three_period_two (b n : ℕ) (hb : b % 3 = 2) : Nat.ModEq 3 (b ^ (n + 2)) (b ^ n) := by
```

Parent. For every natural number n , shifting the exponent of 2 by two preserves its residue modulo 3.

Parent in Lean

```
theorem pow_two_mod_three_period_two (n : ℕ) : Nat.ModEq 3 (2 ^ (n + 2)) (2 ^ n) := by
```

What it contributes. Supplied the exponent-shift and modular-product strategy, generalized from the fixed base 2 to an arbitrary residue-class base b .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base: it must derive that every b with $b \% 3 = 2$ has $b^2 \% 3 = 1$, then use that residue law for arbitrary powers. The parent's fixed-base calculation does not supply this argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: proving period two for every b with $b \% 3 = 2$ requires using the hypothesis to establish $b^2 \equiv 1 \pmod{3}$. This reasoning is not supplied by the fixed-base parent proof and is not a recall or decoration variant.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the period-two congruence from the fixed base 2 to every base satisfying $b \equiv 2 \pmod{3}$. Its proof requires transporting congruence through powers and establishing $b^2 \equiv 1$, rather than recalling the parent's fixed-base multiplication argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1769 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem pow_mod_three_period_two.extracted_1_1 (b n : ℕ) (hb : b % 3 = 2) : b ^ (n + 2) ≡ b ^ n [MOD 3]
```

Read back by a model that saw only the Lean. For natural numbers b and n , if b has remainder 2 upon division by 3,

then b raised to the power $n + 2$ is congruent modulo 3 to b raised to the power n .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem pow_mod_three_period_two (b n : ℕ) (hb : b % 3 = 2) : Nat.ModEq 3 (b ^ (n + 2)) (b ^ n) := by
  have hbmod : Nat.ModEq 3 b 2 := hb
  have hsq : Nat.ModEq 3 (b ^ 2) (2 ^ 2) := hbmod.pow 2
  have hbase : b ^ 2 % 3 = 1 := by
    simpa [Nat.ModEq] using hsq
  change b ^ (n + 2) % 3 = b ^ n % 3
  rw [pow_add, Nat.mul_mod, hbase]
  simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__mh.me__7de29617

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint c7cdcab0799d9197

4.52 052. quadratic_strict_sublevel_unique_minimizer

The problem

For every positive radius r , there exists a unique real number x such that $x^2 - 5x - 4 < r^2 - \frac{41}{4}$ and, for every real number y satisfying $y^2 - 5y - 4 < r^2 - \frac{41}{4}$, we have $x^2 - 5x - 4 \leq y^2 - 5y - 4$.

Lean statement

```
theorem quadratic_strict_sublevel_unique_minimizer (r : ℝ) (hr : 0 < r) : ∃! x : ℝ, x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ∧ ∀ y : ℝ, y ^ 2 - 5 * y - 4 < r ^ 2 - 41 / 4 → x ^ 2 - 5 * x - 4 ≤ y ^ 2 - 5 * y - 4
```

Parent. For every positive radius and every real number x , the strict quadratic sublevel condition $x^2 - 5x - 4 < r^2 - 41/4$ holds exactly when x lies in the centered open interval $(5/2 - r, 5/2 + r)$.

Parent in Lean

```
theorem quadratic_strict_sublevel_radius (r x : ℝ) (hr : 0 < r) : x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔ x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r)
```

What it contributes. The parent's positive-radius centered-interval characterization supplies the geometric feasibility checkpoint for the vertex; the child changes the target to unique minimum attainment.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine existence-and-uniqueness statement, requiring a new completed-square argument to establish that the vertex minimizes the quadratic over the strict sublevel set. The parent's interval characterization does not supply the minimization or uniqueness proof, and the child does not recall or embed the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a genuine unique-minimizer statement and proves uniqueness via completing the square and nonnegativity, with feasibility requiring the positive-radius hypothesis. It does not recall or reuse the parent's interval-equivalence argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine unique-minimizer statement, requiring quadratic completion, nonnegativity, feasibility of the vertex, and an equality argument; the parent only characterizes the strict sublevel interval. Although it does not use the interval characterization as specified, it is not recall or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1784 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_strict_sublevel_unique_minimizer.extracted_1.1 (r : ℝ) (hr : 0 < r) :
  ∃! x,
    x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ∧
    ∀ (y : ℝ), y ^ 2 - 5 * y - 4 < r ^ 2 - 41 / 4 → x ^ 2 - 5 * x - 4 ≤ y ^ 2 - 5 * y - 4
```

Read back by a model that saw only the Lean. *For every real number r with $r > 0$, there exists exactly one real number x such that $x^2 - 5x - 4 < r^2 - 41/4$, and such that for every real number y satisfying $y^2 - 5y - 4 < r^2 - 41/4$, one has $x^2 - 5x - 4 \leq y^2 - 5y - 4$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem quadratic_strict_sublevel_unique_minimizer (r : ℝ) (hr : 0 < r) :
  ∃! x : ℝ,
    x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ∧
    ∀ y : ℝ,
      y ^ 2 - 5 * y - 4 < r ^ 2 - 41 / 4 →
      x ^ 2 - 5 * x - 4 ≤ y ^ 2 - 5 * y - 4 := by
  have hcompleted_vertex :
    (5 / 2 : ℝ) ^ 2 - 5 * (5 / 2) - 4 = -41 / 4 := by
    ring
  have hvertex_feasible :
    (5 / 2 : ℝ) ^ 2 - 5 * (5 / 2) - 4 < r ^ 2 - 41 / 4 := by
    have hr2 : 0 < r ^ 2 := sq_pos_of_pos hr
    linarith [hcompleted_vertex]
  have hcompleted : ∀ y : ℝ,
    y ^ 2 - 5 * y - 4 = (y - 5 / 2) ^ 2 - 41 / 4 := by
    intro y
    ring
  refine (5 / 2, ⟨hvertex_feasible, ?_⟩, ?_)
  · intro y hy
    have hnonneg : 0 ≤ (y - 5 / 2) ^ 2 := sq_nonneg (y - 5 / 2)
    rw [hcompleted y, hcompleted_vertex]
    linarith
  · intro z hz
    rcases hz with ⟨hzfeasible, hzmin⟩
    have hmin_at_vertex := hzmin (5 / 2) hvertex_feasible
    rw [hcompleted z, hcompleted_vertex] at hmin_at_vertex
    have hnonneg : 0 ≤ (z - 5 / 2) ^ 2 := sq_nonneg (z - 5 / 2)
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_101__me.mh.me__e8655936

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e19e254c4bf2c1e0

4.53 053. circle_x_projection_noncentral

The problem

Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$ at a point whose y -coordinate is not 3? Show that they are exactly the numbers in $(-9, 1)$.

Lean statement

```
theorem circle_x_projection_noncentral (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0 ∧ y ≠ 3) ↔
  (-9 < x ∧ x < 1)
```

Parent. Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$? Show that they are exactly the numbers in $[-9, 1]$.

Parent in Lean

```
theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1)
```

What it contributes. Reuses the parent circle equation and completed-square algebra, while strengthening the projection bounds through the hypothesis $y \neq 3$ and an explicit positive-square boundary obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes the circle projection by excluding the central point, changing the endpoint bounds from non-strict to strict. Its proof requires strict positivity of $(y-3)^2$ and of the square root, which the parent proof does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The added condition $y \neq 3$ changes the projection from a closed interval to its interior. The proof genuinely needs strict square positivity and a strictly positive square-root witness, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes the projection to noncentral points, forcing strict inequalities and a strictly positive square-root witness. The parent only establishes the closed interval and does not supply the endpoint exclusion or noncentrality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2629 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_x_projection_noncentral.extracted_1_1 (x : ℝ) :  
  (∃ y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0 ∧ y ≠ 3) ↔ -9 < x ∧ x < 1
```

Read back by a model that saw only the Lean. For every real number x , there exists a real number y such that $x^2 + 8x + y^2 - 6y = 0$ and $y \neq 3$ if and only if $-9 < x$ and $x < 1$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem circle_x_projection_noncentral (x : ℝ) :  
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0 ∧ y ≠ 3) ↔  
    (-9 < x ∧ x < 1) := by  
  constructor  
  · rintro ⟨y, hy, hne⟩  
    have hsq : 0 < (y - 3) ^ 2 := sq_pos_of_ne_zero (sub_ne_zero.mpr hne)  
    have hquad : (x + 4) ^ 2 < 25 := by  
      nlinarith [hy, hsq]  
    constructor  
    · by_contra h  
      have hx : x ≤ -9 := le_of_not_gt h  
      have hp : 0 ≤ (x + 9) * (x - 1) := by  
        apply mul_nonneg_of_nonpos_of_nonpos  
        · linarith  
        · linarith  
      nlinarith [hp, hquad]  
    · by_contra h  
      have hx : 1 ≤ x := le_of_not_gt h  
      have hp : 0 ≤ (x + 9) * (x - 1) := by  
        apply mul_nonneg  
        · linarith  
        · linarith  
      nlinarith [hp, hquad]  
  · rintro ⟨hx₁, hx₂⟩  
    have hprod : 0 < (x + 9) * (1 - x) := by  
      apply mul_pos  
      · linarith  
      · linarith  
    have hd : 0 < 25 - (x + 4) ^ 2 := by  
      nlinarith [hprod]  
    have hspos : 0 < Real.sqrt (25 - (x + 4) ^ 2) := Real.sqrt_pos.2 hd  
    refine ⟨3 + Real.sqrt (25 - (x + 4) ^ 2), ?_, ?_⟩  
    · have hs : (Real.sqrt (25 - (x + 4) ^ 2)) ^ 2 = 25 - (x + 4) ^ 2 := by  
      exact Real.sq_sqrt (le_of_lt hd)  
      nlinarith [hs]  
    · nlinarith [hspos]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.me__b2ecfb10

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b2ecfb10dcb7e680

4.54 054. quadratic_y_solvable_parameterized

The problem

For real numbers x , b , and t , the equation $x^2 + 8x + y^2 - 2by = t$ has a real solution exactly when $x^2 + 8x - b^2 \leq t$.

Lean statement

```
theorem quadratic_y_solvable_parameterized (x b t : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 2 * b * y = t) ↔
  x ^ 2 + 8 * x - b ^ 2 ≤ t
```

Parent. For real numbers x and t , the equation $x^2 + 8x + y^2 - 6y = t$ determines a unique real value of y exactly when $x^2 + 8x - 9 = t$.

Parent in Lean

```
theorem circle_unique_y_coordinate_level (x t : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t
```

What it contributes. Uses the parent's completed-square structure and real-parameter level equation, while changing the final obligation from uniqueness to solvability.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces uniqueness at the completed-square level with a genuine real-solvability characterization, parameterizes the linear coefficient by b , and requires a square-root construction for the reverse direction. This is not recoverable by recalling the parent's reflection-and-uniqueness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes unique solvability at the completed-square minimum into unrestricted real solvability, parameterizes the linear coefficient by b , and requires a square-root witness plus nonnegativity reasoning. The parent's reflection and uniqueness argument does not supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes unique solvability at the fixed completed-square level into a parameterized existence characterization for arbitrary b and t . Its reverse direction requires constructing a square-root witness and proving the radicand is nonnegative, reasoning not supplied by the parent's reflection-and-uniqueness argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2673 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_y_solvable_parameterized.extracted_1_1 (x b t : ℝ) :  
  (∃ y, x ^ 2 + 8 * x + y ^ 2 - 2 * b * y = t) ↔ x ^ 2 + 8 * x - b ^ 2 ≤ t
```

Read back by a model that saw only the Lean. For all real numbers x , b , and t , there exists a real number y such that $x^2 + 8x + y^2 - 2by = t$ if and only if $x^2 + 8x - b^2 \leq t$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_y_solvable_parameterized (x b t : ℝ) :  
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 2 * b * y = t) ↔  
    x ^ 2 + 8 * x - b ^ 2 ≤ t := by  
  constructor  
  · rintro ⟨y, hy⟩  
    have hnonneg : 0 ≤ (y - b) ^ 2 := sq_nonneg (y - b)  
    nlinarith [hy, hnonneg]  
  · intro hlevel  
    have hradicand : 0 ≤ t - (x ^ 2 + 8 * x - b ^ 2) := by  
      nlinarith  
    have hsqrt : (Real.sqrt (t - (x ^ 2 + 8 * x - b ^ 2))) ^ 2 =  
      t - (x ^ 2 + 8 * x - b ^ 2) := by  
      exact Real.sq_sqrt hradicand  
    refine ⟨b + Real.sqrt (t - (x ^ 2 + 8 * x - b ^ 2)), ?_⟩  
    nlinarith [hsqrt]
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.mh.me.me__8589c85d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 8589c85d6ee74ec5

4.55 055. circle_unique_y_coordinate_level

The problem

For real numbers x and t , the equation $x^2 + 8x + y^2 - 6y = t$ determines a unique real value of y exactly when $x^2 + 8x - 9 = t$.

Lean statement

```
theorem circle_unique_y_coordinate_level (x t : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t
```

Parent. For which real values of x does the equation $x^2 + 8x + y^2 - 6y = 0$ determine a unique real value of y ? Show that these values are $x = -9$ and $x = 1$.

Parent in Lean

```
theorem circle_unique_y_coordinate (x : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1
```

What it contributes. Reuses the parent's reflection argument and unique-y checkpoint, generalized from level 0 to an arbitrary real parameter t .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the level and replaces the parent's fixed-root characterization with the completed-square condition $x^2 + 8x - 9 = t$. The parameter t is essential, and the proof must establish the new equivalence rather than merely reuse the parent's factorization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The level parameter t changes the conclusion from two isolated x -values to an exact completed-square condition, and the reverse direction must derive uniqueness from $(y-3)^2 = 0$ under the new level equation. This is a genuine parameterized generalization rather than a wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the level t and proves the unique-y condition via the new completed-square equation $x^2 + 8x - 9 = t$. Although it reuses the reflection idea, the fixed-level parent does not directly supply the generalized characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2654 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_unique_y_coordinate_level.extracted_1_1 (x t : ℝ) :  
  (∃! y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t
```

Read back by a model that saw only the Lean. For real numbers x and t , there exists exactly one real number y satisfying $x^2 + 8x + y^2 - 6y = t$ if and only if $x^2 + 8x - 9 = t$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem circle_unique_y_coordinate_level (x t : ℝ) :  
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = t) ↔ x ^ 2 + 8 * x - 9 = t := by  
  constructor  
  · rintro ⟨y, hy, huniq⟩  
    have hy_reflected : x ^ 2 + 8 * x + (6 - y) ^ 2 - 6 * (6 - y) = t := by  
      nlinarith [hy]  
    have heq : 6 - y = y := huniq (6 - y) hy_reflected  
    have hy3 : y = 3 := by  
      linarith  
    nlinarith [hy, hy3]  
  · intro hlevel  
    refine ⟨3, ?_, ?_⟩  
    · nlinarith [hlevel]  
    · intro y hy  
      have hcompleted : (y - 3) ^ 2 = 0 := by  
        nlinarith [hy, hlevel]  
      nlinarith [hcompleted]
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.mh.me__d1798a1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d1798a1d31fc74f9

4.56 056. circle_x_projection

The problem

Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$? Show that they are exactly the numbers in $[-9, 1]$.

Lean statement

```
theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1)
```

Parent. Find the radius of the circle with equation $x^2 + 8x + y^2 - 6y = 0$. Show that it is 5.

Parent in Lean

```
theorem mathd_algebra_107
  (x y : ℝ)
  (h₀ : x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) :
  ∃ a b r : ℝ, 0 < r ∧ (x + a) ^ 2 + (y + b) ^ 2 = r ^ 2 ∧ r = 5 := by
```

What it contributes. Uses the parent's completed-square radius-five identity, extending it to an exact existential characterization of the x -coordinate projection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an existential circle-membership witness into an exact characterization of the entire x -coordinate projection, requiring both interval necessity and a square-root construction for sufficiency. The parent's fixed-center/radius witness proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential circle witness into an exact characterization of the x -coordinate projection. Its necessity argument bounds a quadratic and its sufficiency constructs a square-root witness, neither of which is supplied by the parent's existential parameter choice.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's existential circle-witness claim into an exact characterization of all possible x -coordinates. Its proof requires completing the square, bounding a quadratic, and constructing a witness with a square root, so recalling the parent's short substitution proof is not substantially helpful.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2614 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_x_projection.extracted_1_1 (x : ℝ) : (∃ y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ -9 ≤ x ∧ x ≤ 1
```

Read back by a model that saw only the Lean. For every real number x , there exists a real number y such that $x^2 + 8x + y^2 - 6y = 0$ if and only if $-9 \leq x$ and $x \leq 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
    (-9 ≤ x ∧ x ≤ 1) := by
  constructor
  · rintro ⟨y, hy⟩
    have hsq : 0 ≤ (y - 3) ^ 2 := sq_nonneg (y - 3)
    have hquad : (x + 4) ^ 2 ≤ 25 := by
      nlinarith [hy, hsq]
    constructor
    · by_contra h
      have hx : x < -9 := lt_of_not_ge h
      have hp : 0 < (x + 9) * (x - 1) := by
        apply mul_pos_of_neg_of_neg
        · linarith
        · linarith
      nlinarith [hp, hquad]
    · by_contra h
      have hx : x > 1 := lt_of_not_ge h
      have hp : 0 < (x + 9) * (x - 1) := by
        apply mul_pos
        · linarith
        · linarith
      nlinarith [hp, hquad]
  · intro hx
    have hprod : 0 ≤ (x + 9) * (1 - x) := by
      apply mul_nonneg
      · linarith
      · linarith
    have hd : 0 ≤ 25 - (x + 4) ^ 2 := by
      nlinarith [hprod]
    refine ⟨3 + Real.sqrt (25 - (x + 4) ^ 2), ?_⟩
    have hs : (Real.sqrt (25 - (x + 4) ^ 2)) ^ 2 = 25 - (x + 4) ^ 2 := by
      exact Real.sq_sqrt hd
    nlinarith [hs]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_107__me__68485970

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 68485970de78008b

4.57 057. scaled_shifted_square_range

The problem

For real numbers c, a, b with $c > 0$, let $f(x) = c(x - a)^2 + b$. For every real y , there exists a real x such that $f(x) = y$ if and only if $b \leq y$.

Lean statement

```
theorem scaled_shifted_square_range
  (c a b : ℝ)
  (hc : 0 < c)
  (f : ℝ → ℝ)
  (hf : f = fun x => c * (x - a)^2 + b) :
  ∀ y : ℝ, (∃ x : ℝ, f x = y) ↔ b ≤ y
```

Parent. Let $a \in \mathbb{R}$ and $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x) = (x - a)^2$ for all $x \in \mathbb{R}$. Prove that for every $y \in \mathbb{R}$, y is a global minimizer of f if and only if $y = a$.

Parent in Lean

```
theorem shifted_square_unique_minimizer
  (a : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => (x - a)^2) :
  ∀ y : ℝ, IsMinOn f Set.univ y ↔ y = a
```

What it contributes. Preserved the shifted-square minimization checkpoint: the square is minimized at its center and is nonnegative, then extended it to a scaled and translated attained-range characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes minimization into a full range characterization and introduces positive scaling and vertical translation. Its reverse direction requires constructing a preimage using division and `Real.sqrt`, mathematics absent from the parent's minimizer proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from identifying the unique minimizer to characterizing the entire range, while introducing symbolic positive scaling and vertical translation. Its proof requires constructing a square-root witness and handling division by c , reasoning not supplied by the parent's minimizer proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the range characterization to a positively scaled and vertically translated square. Proving the converse requires constructing a square-root witness and using positivity of the scale, reasoning absent from the parent's minimizer proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1755 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem scaled_shifted_square_range.extracted_1_1 : ∀ (c a b : ℝ),
  0 < c → ∀ (f : ℝ → ℝ), (f = fun x => c * (x - a) ^ 2 + b) → ∀ (y : ℝ), (∃ x, f x = y) ↔ b ≤ y
```

Read back by a model that saw only the Lean. For all real numbers c , a , and b , if c is positive, then for every function f from the real numbers to the real numbers such that $f(x) = c(x - a)^2 + b$ for all real x , and for every real number y , there exists a real number x with $f(x) = y$ if and only if $b \leq y$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem scaled_shifted_square_range
  (c a b : ℝ)
  (hc : 0 < c)
  (f : ℝ → ℝ)
  (hf : f = fun x => c * (x - a)^2 + b) :
  ∀ y : ℝ, (∃ x : ℝ, f x = y) ↔ b ≤ y := by
  intro y
  rw [hf]
  constructor
  · rintro ⟨x, hx⟩
    have hs : 0 ≤ (x - a)^2 := sq_nonneg (x - a)
    nlinarith
  · intro hy
    have hnonneg : 0 ≤ y - b := sub_nonneg.mpr hy
    have hq : 0 ≤ (y - b) / c := div_nonneg hnonneg (le_of_lt hc)
    have hsqrt : (Real.sqrt ((y - b) / c))^2 = (y - b) / c := Real.sq_sqrt hq
    have hdiv : c * ((y - b) / c) = y - b := by
      field_simp [ne_of_gt hc]
    refine ⟨a + Real.sqrt ((y - b) / c), ?_⟩
    nlinarith [hsqrt, hdiv]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__me.me__a47a2f74

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-gerprint 93d83fb32a180755

4.58 058. shifted_square_unique_minimizer

The problem

Let $a \in \mathbb{R}$ and $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x) = (x - a)^2$ for all $x \in \mathbb{R}$. Prove that for every $y \in \mathbb{R}$, y is a global minimizer of f if and only if $y = a$.

Lean statement

```
theorem shifted_square_unique_minimizer
  (a : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => (x - a)^2) :
  ∀ y : ℝ, IsMinOn f Set.univ y ↔ y = a
```

Parent. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

What it contributes. Generalizes the parent’s completion-of-the-square global-minimum argument from fixed vertex 7 to a bounded real parameter and strengthens minimum existence to an iff uniqueness characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child parameterizes the vertex and changes a single minimum claim into a characterization of all minimizers, requiring both uniqueness and the converse minimization argument. The boundedness hypotheses are unused, but the core proof demands reasoning beyond the parent’s fixed-instance minimum.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed minimizer to an arbitrary real shift and proves the converse uniqueness direction, requiring evaluation at the candidate minimizer and a separate case split. This is not recoverable by simply recalling the parent’s proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the vertex and strengthens the conclusion to a full unique-minimizer characterization. Its reverse direction requires an equality-case argument using evaluation at the parameter, which is not supplied by the parent’s one-sided minimum proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed ha_lower, ha_upper
corpus_dedup	hash	unique against 1742 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_square_unique_minimizer.extracted_1_1 : ∀ (a : ℝ) (f : ℝ → ℝ),
  (f = fun x => (x - a) ^ 2) → ∀ (y : ℝ), IsMinOn f Set.univ y ↔ y = a
```

Read back by a model that saw only the Lean. *For every real number a and every function f from $\mathbb{R}$ to $\mathbb{R}$, if $f(x) = (x - a)^2$ for all real x , then for every real number y , y is a global minimizer of f on $\mathbb{R}$ if and only if $y = a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem shifted_square_unique_minimizer
  (a : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => (x - a)^2) :
  ∀ y : ℝ, IsMinOn f Set.univ y ↔ y = a := by
  intro y
  rw [hf, isMinOn_univ_iff]
  constructor
  · intro hy
    have hya : (y - a)^2 ≤ (a - a)^2 := hy a
    nlinarith [sq_nonneg (y - a)]
  · intro hya
    subst y
    intro x
    nlinarith [sq_nonneg (x - a)]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_113__me__a9461eb6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7743b44fb2139a32

4.59 059. radical_difference_gcd_candidate_impossible

The problem

Let b be a positive natural number and let k be divisible by both 2 and 3. If the positive integer y is the common-multiple radical difference and its greatest common divisor with 40 is greater than y , show that this is impossible.

Lean statement

```
theorem radical_difference_gcd_candidate_impossible
  (b k y : ℕ)

  (hy : (y : ℝ) = Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3))
  (hidentity : Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hpos : 0 < (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hg : Nat.gcd 40 y > y) :
  False
```

Parent. For natural numbers b, k with $1 \leq b$, $1 \leq k \leq 600$, and k divisible by both 2 and 3, show that $\sqrt{b^k} - \sqrt[3]{b^k} = b^{k/2} - b^{k/3}$, where the quotients are natural-number quotients.

Parent in Lean

```
theorem bounded_common_multiple_radical_identity (b k : ℕ) (h2 : 2 ∣ k) (h3 : 3 ∣ k) :
  Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
```

What it contributes. Supplied the common-multiple radical identity checkpoint, used as `hidentity` to rewrite the candidate before the gcd obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child does not require reproving or recalling the parent identity: it uses that identity only as a hypothesis, then derives positivity of the natural candidate and applies the independent fact that a positive natural number's gcd with 40 cannot exceed it. The proof therefore introduces genuinely different gcd reasoning, although the radical setup overlaps an already-kept family.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the certified radical identity only to transfer positivity to the natural-number candidate, then applies the independent fact that a gcd is at most its right argument and derives a contradiction. This gcd-bound argument is not supplied by the parent's radical decomposition proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not proved by replaying the parent: it assumes the radical identity and instead combines positivity with the independent fact that $\gcd(40, y) \leq y$ for positive y . All hypotheses contribute to deriving the contradiction.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2536 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem radical_difference_gcd_candidate_impossible.extracted_1_1 (b k y : ℕ) (hb : 0 < b) (h2 : 2 ∣ k) (h3 : 3 ∣ k)
(hy : ↑y = √(↑b ^ k) - (↑b ^ k) ^ (1 / 3)) (hidentity : √(↑b ^ k) - (↑b ^ k) ^ (1 / 3) = ↑b ^ (k / 2) - ↑b ^ (k / 3))
(hpos : 0 < ↑b ^ (k / 2) - ↑b ^ (k / 3)) (hg : Nat.gcd 40 y > y) : False
```

Read back by a model that saw only the Lean. For natural numbers b, k , and y , assume that b is positive, k is divisible by 2 and by 3, y (viewed as a real number) equals the square root of b^k minus $(b^k)^{1/3}$, and this difference also equals $b^{k/2}$ minus $b^{k/3}$. Assume further that $b^{k/2}$ minus $b^{k/3}$ is positive and that the greatest common divisor of 40 and y is greater than y . Then a contradiction follows; that is, False.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem radical_difference_gcd_candidate_impossible
  (b k y : ℕ)

  (hy : (y : ℝ) = Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3))
  (hidentity : Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hpos : 0 < (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3))
  (hg : Nat.gcd 40 y > y) :
  False := by
  have hcandidate : (y : ℝ) = (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
    calc
      (y : ℝ) = Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) := hy
      _ = (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := hidentity
  have hypos_real : (0 : ℝ) < y := by
    linarith [hpos, hcandidate]
  have hypos : 0 < y := by
    exact_mod_cast hypos_real
  have hgcd : Nat.gcd 40 y ≤ y := by
    exact Nat.gcd_le_right 40 hypos
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_208__me.me.me__0b049e9b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 164222055c3632b8

4.60 060. bounded_common_multiple_radical_identity

The problem

For natural numbers b, k with $1 \leq b$, $1 \leq k \leq 600$, and k divisible by both 2 and 3, show that $\sqrt{b^k} - \sqrt[3]{b^k} = b^{k/2} - b^{k/3}$, where the quotients are natural-number quotients.

Lean statement

```
theorem bounded_common_multiple_radical_identity (b k : ℕ) (h2 : 2 ∣ k) (h3 : 3 ∣ k) :
  Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
  (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
```

Parent. For a natural number b with $1 \leq b \leq 100$, show that $\sqrt{b^6} - \sqrt[3]{b^6} = b^3 - b^2$.

Parent in Lean

```
theorem bounded_radical_identity (b : ℕ) (hb₁ : 1 ≤ b) :
  Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) =
  (b : ℝ) ^ 3 - (b : ℝ) ^ 2 := by
```

What it contributes. Reuses the parent's positive perfect-power checkpoints for the square root and cube-root, generalized from exponent 6 to a bounded exponent divisible by both 2 and 3.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the sixth-power identity to every exponent divisible by both 2 and 3, requiring new quotient and divisibility reasoning to derive the square-root and cube-root exponents. The parent's fixed-exponent proof cannot simply be recalled to establish this parameterized result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from 6 to an arbitrary common multiple of 2 and 3. Its proof must extract divisibility witnesses, establish the quotient identities, and apply generalized square-root and cube-root reasoning, so recalling the parent proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed sixth-power identity to an arbitrary exponent divisible by both 2 and 3. Its proof introduces divisibility witnesses, derives quotient identities with arithmetic reasoning, and applies the radical arguments parametrically rather than recalling the parent's fixed-exponent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2514 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_common_multiple_radical_identity.extracted_1_1 (b k : ℕ) (hb : 1 ≤ b) (hk : 1 ≤ k) (hk_upper : k ≤ 600)
  (h2 : 2 ∣ k) (h3 : 3 ∣ k) : √(↑b ^ k) - (↑b ^ k) ^ ((1 : ℝ) / 3) = ↑b ^ (k / 2) - ↑b ^ (k / 3)
```

Read back by a model that saw only the Lean. For natural numbers b and k , if $1 \leq b$, $1 \leq k$, $k \leq 600$, 2 divides k , and 3 divides k , then the square root of the real number b^k minus the real one-third power of b^k equals $b^{k/2}$ minus $b^{k/3}$, where the exponents $k/2$ and $k/3$ are natural-number quotients.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_common_multiple_radical_identity (b k : ℕ) (h2 : 2 ∣ k) (h3 : 3 ∣ k) :
  Real.sqrt ((b : ℝ) ^ k) - ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) =
    (b : ℝ) ^ (k / 2) - (b : ℝ) ^ (k / 3) := by
  rcases h2 with ⟨a, ha⟩
  rcases h3 with ⟨c, hc⟩
  have hka : k / 2 = a := by
    omega
  have hkc : k / 3 = c := by
    omega
  have hsqrt : Real.sqrt ((b : ℝ) ^ k) = (b : ℝ) ^ a := by
    calc
      Real.sqrt ((b : ℝ) ^ k) = Real.sqrt ((b : ℝ) ^ (2 * a)) := by rw [ha]
      _ = Real.sqrt (((b : ℝ) ^ a) ^ 2) := by
        congr 1
        simp [Nat.mul_comm] using (pow_mul (b : ℝ) a 2)
      _ = |(b : ℝ) ^ a| := by rw [Real.sqrt_sq_eq_abs]
      _ = (b : ℝ) ^ a := by rw [abs_of_nonneg] <|> positivity
  have hcuberoot : ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) = (b : ℝ) ^ c := by
    calc
      ((b : ℝ) ^ k) ^ ((1 : ℝ) / 3) = (((b : ℝ) ^ c) ^ 3) ^ ((1 : ℝ) / 3) := by
        congr 1
        rw [hc]
        simp [Nat.mul_comm] using (pow_mul (b : ℝ) c 3)
      _ = (b : ℝ) ^ c := by
        have hroot := Real.pow_rpow_inv_natCast (x := (b : ℝ) ^ c) (n := 3)
        (by positivity) (by norm_num)
        convert hroot using 1 <|> norm_num
      _ = (b : ℝ) ^ c := by
        rw [hsqrt, hcuberoot, hka, hkc]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_208__me.me__9013b35e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 3a92415ad8f828ba

4.61 061. factorization_identity

The problem

Show that $(\sqrt{1,000,000} - \sqrt[3]{1,000,000})(\sqrt{1,000,000} + \sqrt[3]{1,000,000}) = 990,000$.

Lean statement

```
theorem factorization_identity : (Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3)) * (Real.sqrt 1000000 + 1000000 ^ ((1 : ℝ) / 3)) = 990000
```

Parent. What is the value of $\sqrt{1,000,000} - \sqrt[3]{1,000,000}$? Show that it is 900.

Parent in Lean

```
theorem mathd_algebra_208 : Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3) = 900 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires independently evaluating both the square root and real cube-root power, then proving a new product identity. The parent's difference equation alone does not determine the needed sum factor.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves separate square-root and real cube-root evaluations, then establishes a new difference-of-squares factorization. The parent's single evaluated difference does not supply the individual values or the product obligation, so recalling its proof does not substantially solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves separate evaluations of the square root and real cube-root power, then establishes a new product identity; the parent's difference equation alone does not supply this factorization proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1713 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem factorization_identity.extracted_1_1 :
  (√1000000 - 1000000 ^ (1 / 3)) * (√1000000 + 1000000 ^ (1 / 3)) = 990000
```

Read back by a model that saw only the Lean. *The product of the difference and sum of the square root of 1,000,000 and 1,000,000 raised to the power 1/3 equals 990,000.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem factorization_identity : (Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3)) * (Real.sqrt 1000000 + 1000000 ^ ((1 : ℝ) / 3)) = 990000
↪ := by
  have hsqrt : Real.sqrt (1000000 : ℝ) = 1000 := by
    norm_num
  have hpow : (1000000 : ℝ) ^ ((1 : ℝ) / 3) = 100 := by
    norm_num [Real.rpow_def_of_pos]
  rw [hsqrt, hpow]
  norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_208__me__dab0e446

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint dab0e446d87a110c

4.62 062. mathd_algebra_214_bounded_height_fiber

The problem

For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, and for any real number h , the equality $f(x) = h$ holds exactly when $(x - 2)^2 = 4(h - 3)$.

Lean statement

```
theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

Parent. For a quadratic real function attaining an extremum at 2 with $f 2 = 3$ and $f 4 = 4$, the equality $f x = 3$ holds exactly when $x = 2$.

Parent in Lean

```
theorem mathd_algebra_214_extremum_eq_three
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x : ℝ, f x = 3 ↔ x = 2
```

What it contributes. Supplied the extremum-based derivation of $b = -4a$, the parameter recovery $a = 1/4$ and $c = 4$, and the completed-square normalization used before solving the generalized height fiber.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely characterizes every level set of the quadratic, requiring derivation of the explicit vertex-centered identity and algebraic equivalence for arbitrary h . This is not recoverable by merely recalling the parent's fixed-level equivalence, and h is materially used.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed level 3 to an arbitrary real height and classifies every fiber via $(x-2)^2=4(h-3)$. Proving this requires deriving the full completed-square form and comparing with an arbitrary h , not merely reusing the parent's equality-at-the-vertex conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child classifies every level set of the quadratic via the squared displacement from the vertex, requiring new algebra beyond the parent's single-level equality claim. The arbitrary real height h makes it stronger than the planned bounded version, not a recall or decorative wrapper.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2082 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_214_bounded_height_fiber.extracted_1_1 : ∀ (a b c : ℝ) (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = a * x ^ 2 + b * x + c) →
    IsExtrOn f Set.univ 2 → f 2 = 3 → f 4 = 4 → ∀ (h x : ℝ), f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

Read back by a model that saw only the Lean. For all real numbers a , b , and c and every function f from the real numbers to the real numbers, if for every real number x we have $f(x) = ax^2 + bx + c$, if 2 is an extremum of f on the set of all real numbers, if $f(2) = 3$, and if $f(4) = 4$, then for every pair of real numbers h and x , $f(x) = h$ if and only if $(x - 2)^2 = 4(h - 3)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3) := by
  have hb : b = -4 * a := by
    rcases h₁ with hmin | hmax
    · have h1 : f 2 ≤ f 1 := by simpa using hmin (Set.mem_univ (1 : ℝ))
      have h3' : f 2 ≤ f 3 := by simpa using hmin (Set.mem_univ (3 : ℝ))
      rw [h₀ 1, h₂] at h1
      rw [h₀ 3, h₂] at h3'
      have e2 := h₀ 2
      have e4 := h₀ 4
      rw [h₂] at e2
      rw [h₃] at e4
      have ha : 0 < a := by
        by_contra h
        have ha' : a = 0 := by nlinarith
        nlinarith [h1, h3', e2, e4]
      have ha_ne : a ≠ 0 := ne_of_gt ha
      let x : ℝ := 2 - (4 * a + b) / (2 * a)
      have hx : f 2 ≤ f x := by simpa using hmin (Set.mem_univ x)
      rw [h₀ x, h₂] at hx
      have hpoly :
        a * x ^ 2 + b * x + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
          -(4 * a + b) ^ 2 / (4 * a) := by
        dsimp [x]
        field_simp [ha_ne]
        ring
      have hdiff : 0 ≤ -(4 * a + b) ^ 2 / (4 * a) := by
        nlinarith [hx, e2, hpoly]
```

```

have hprod :  $-(4 * a + b) ^ 2 / (4 * a) * (4 * a) = -(4 * a + b) ^ 2 :=$  by
  field_simp [ha_ne]
have hd :  $(4 * a + b) ^ 2 = 0 :=$  by
  have hp := mul_nonneg hdiff (le_of_lt (by nlinarith :  $0 < 4 * a$ ))
    nlinarith [hprod]
nlinarith
• have h1 :  $f \ 1 \leq f \ 2 :=$  by simp using hmax (Set.mem_univ (1 :  $\mathbb{R}$ ))
have h3' :  $f \ 3 \leq f \ 2 :=$  by simp using hmax (Set.mem_univ (3 :  $\mathbb{R}$ ))
rw [h₀ 1, h₂] at h1
rw [h₀ 3, h₂] at h3'
have e2 := h₀ 2
have e4 := h₀ 4
rw [h₂] at e2
rw [h₃] at e4
have ha :  $a < 0 :=$  by
  by_contra h
  have ha' :  $a = 0 :=$  by nlinarith
  nlinarith [h1, h3', e2, e4]
have ha_ne :  $a \neq 0 :=$  ne_of_lt ha
let x :  $\mathbb{R} := 2 - (4 * a + b) / (2 * a)$ 
have hx :  $f \ x \leq f \ 2 :=$  by simp using hmax (Set.mem_univ x)
rw [h₀ x, h₂] at hx
have hpoly :
   $a * x ^ 2 + b * x + c - (a * (2 : \mathbb{R}) ^ 2 + b * 2 + c) =$ 
   $-(4 * a + b) ^ 2 / (4 * a) :=$  by
  dsimp [x]
  field_simp [ha_ne]
  ring
have hdiff :  $-(4 * a + b) ^ 2 / (4 * a) \leq 0 :=$  by
  nlinarith [hx, e2, hpoly]
have hprod :  $-(4 * a + b) ^ 2 / (4 * a) * (4 * a) = -(4 * a + b) ^ 2 :=$  by
  field_simp [ha_ne]
have hd :  $(4 * a + b) ^ 2 = 0 :=$  by
  have hp := mul_nonneg_of_nonpos_of_nonpos hdiff (le_of_lt (by nlinarith :  $4 * a < 0$ ))
    nlinarith [hprod]
nlinarith
have ha :  $a = (1 : \mathbb{R}) / 4 :=$  by
  have e2 := h₀ 2
  have e4 := h₀ 4
  rw [h₂] at e2
  rw [h₃] at e4
  nlinarith [e2, e4, hb]
have hc :  $c = 4 :=$  by
  have e2 := h₀ 2
  rw [h₂] at e2
  nlinarith [e2, ha, hb]
have hform :  $\forall x : \mathbb{R}, f \ x = (x - 2) ^ 2 / 4 + 3 :=$  by
  intro x
  rw [h₀ x, hb, ha, hc]
  ring
intro x
rw [hform x]
constructor
• intro hx
  nlinarith
• intro hx
  nlinarith

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_214__me.mh.me__5125f923

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c64bc860433ce35c

4.63 063. quadratic_bounded_inverse

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x) = 3 + (x - 2)^2/4$ for every real x . For every real h with $3 \leq h \leq 4$, there exists a unique $x \in [2, 4]$ such that $f(x) = h$.

Lean statement

```
theorem quadratic_bounded_inverse
  (f : ℝ → ℝ)
  (hf : ∀ x : ℝ, f x = 3 + (x - 2) ^ 2 / 4) :
  ∀ h : ℝ, 3 ≤ h → h ≤ 4 →
    ∃! x : ℝ, x ∈ Set.Icc 2 4 ∧ f x = h
```

Parent. For every real x , the quadratic function satisfies $f(x) = 3 + (x - 2)^2/4$.

Parent in Lean

```
theorem quadratic_completed_square
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x : ℝ, f x = 3 + (x - 2) ^ 2 / 4
```

What it contributes. Supplies the completed-square law used for both witness construction and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to a bounded range-and-inverse characterization: it constructs the unique right-branch preimage using square roots and proves uniqueness from the interval restriction. This is genuinely different reasoning from the parent's completed-square identity and is not a forced wrapper or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a bounded range-and-inverse theorem, requiring construction via square roots and proving uniqueness on the right branch. The parent's completed-square derivation does not supply this existence-and-injectivity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying a quadratic to proving bounded range coverage and uniqueness of a preimage. Its proof requires constructing the inverse with a square root and proving branch uniqueness, mathematics not supplied by the parent's completed-square argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2091 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_bounded_inverse.extracted_1_1 : ∀ (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = 3 + (x - 2) ^ 2 / 4) → ∀ (h : ℝ), 3 ≤ h → h ≤ 4 → ∃! x, x ∈ Set.Icc 2 4 ∧ f x = h
```

Read back by a model that saw only the Lean. *For every function $f: \mathbb{R} \rightarrow \mathbb{R}$ satisfying $f(x) = 3 + (x - 2)^2/4$ for every real number x , and for every real number h with $3 \leq h \leq 4$, there exists exactly one real number x such that $2 \leq x \leq 4$ and $f(x) = h$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_bounded_inverse
  (f : ℝ → ℝ)
  (hf : ∀ x : ℝ, f x = 3 + (x - 2) ^ 2 / 4) :
  ∀ h : ℝ, 3 ≤ h → h ≤ 4 →
  ∃! x : ℝ, x ∈ Set.Icc 2 4 ∧ f x = h := by
  intro h hh3 hh4
  have hnonneg : 0 ≤ h - 3 := by
    linarith
  let y : ℝ := Real.sqrt (h - 3)
  have hy0 : 0 ≤ y := by
    dsimp [y]
    exact Real.sqrt_nonneg _
  have hy_sq : y ^ 2 = h - 3 := by
    dsimp [y]
    exact Real.sq_sqrt hnonneg
  have hy1 : y ≤ 1 := by
    nlinarith [hy_sq]
  let x : ℝ := 2 + 2 * y
  have hxmem : x ∈ Set.Icc 2 4 := by
    constructor
    · dsimp [x]
      nlinarith
    · dsimp [x]
      nlinarith
  have hxeq : f x = h := by
    rw [hf x]
    dsimp [x]
    nlinarith [hy_sq]
  refine ⟨x, ⟨hxmem, hxeq⟩, ?_⟩
  intro z hz
  have hzmem : z ∈ Set.Icc 2 4 := hz.1
  have hzeq : f z = h := hz.2
  have hzsquare : (z - 2) ^ 2 = 4 * (h - 3) := by
    rw [hf z] at hzeq
    nlinarith [hzeq]
  have hzleft : 0 ≤ z - 2 := by
    exact sub_nonneg.mpr hzmem.1
  dsimp [x]
  nlinarith [hsquare, hy_sq]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_214__mh.me__20a18ab2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20a18ab2df106de0

4.64 064. bounded_geometric_term_law

The problem

A geometric sequence begins with $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}$. Determine its common ratio and show that, for every index n with $0 \leq n \leq 5$, the n th term is $\frac{27}{125} \left(\frac{5}{3}\right)^n$.

Lean statement

```
theorem bounded_geometric_term_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  d = 5 / 3 ∧ ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 25 / 9 := by
```

What it contributes. Retains the parent sequence, recurrence, and initial values; generalizes the parent's fixed sixth-term computation into a bounded quantified law, with the ratio determination as an explicit checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child replaces the isolated computation of a_5 with a bounded universal formula for every $n \leq 5$. Its induction over n is genuinely new reasoning, and recalling the parent's sequential calculation does not directly establish the law.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the isolated computation of a_5 with a bounded law for every $n \leq 5$, proved by induction from the initial terms. Although determining d reuses the parent's initial algebra, the induction argument and omission of the parent's a_2 hypothesis require new reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the isolated sixth-term computation with a bounded law for every term through index 5. Proving that law requires induction and is not obtained by recalling the parent's finite chain of calculations.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2673 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_geometric_term_law.extracted_1_1 (a : ℕ → ℚ) (d : ℚ) (h₀ : ∀ (n : ℕ), a (n + 1) = a n * d)
(h₁ : a 0 = 27 / 125) (h₂ : a 1 = 9 / 25) : d = 5 / 3 ∧ ∀ n ≤ 5, a n = 27 / 125 * (5 / 3) ^ n
```

Read back by a model that saw only the Lean. *Let a be a function from the natural numbers to the rational numbers, and let d be a rational number. Suppose that for every natural number n , $a(n + 1) = a(n) \cdot d$, that $a(0) = 27/125$, and that $a(1) = 9/25$. Then $d = 5/3$, and for every natural number n with $n \leq 5$, $a(n) = (27/125) \cdot (5/3)^n$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_geometric_term_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  d = 5 / 3 ∧ ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
  have hd : d = 5 / 3 := by
    have h := h₀ 0
    norm_num [h₁, h₂] at h
    linarith
  have hlaw : ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
    intro n hn
    induction n with
    | zero =>
      norm_num [h₁]
    | succ n ih =>
      have hs := h₀ n
      rw [hd] at hs
      rw [ih (by omega)] at hs
      simpa [pow_succ, mul_assoc, mul_left_comm, mul_comm] using hs
  exact ⟨hd, hlaw⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__me__360cecbc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ebb5b4e6ea5a5c76

4.65 065. bounded_recurrence_transfer

The problem

Let a and b be rational sequences and let d be rational. If for every natural number n , $a(n+1) = a(n)d$ and $b(n+1) = b(n)d$, and $a(0) = b(0)$, prove that $a(n) = b(n)$ for every natural number $n \leq 5$.

Lean statement

```
theorem bounded_recurrence_transfer
  (a b : ℕ → ℚ)
  (d : ℚ)
  (ha : ∀ n, a (n + 1) = a n * d)
  (hb : ∀ n, b (n + 1) = b n * d)
  (h₀ : a 0 = b 0) :
  ∀ n, n ≤ 5 → a n = b n
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{25}{9}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 25 / 9 := by
```

What it contributes. Preserves the parent's rational multiplicative recurrence setting while changing the proof role from one-sequence endpoint evaluation to bounded uniqueness for two sequences.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a fixed endpoint computation with a genuine transfer theorem for two arbitrary rational sequences. Its induction proves equality is preserved by a shared recurrence, reasoning not supplied by the parent's ratio computation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a fixed endpoint computation with a transfer theorem for two arbitrary rational recurrences. Its induction propagates equality from the shared initial value, requiring reasoning absent from the parent's coefficient calculation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single fixed endpoint computation with a transfer theorem for two arbitrary rational recurrences. Its induction argument establishes equality through the bound from a shared initial value and recurrence, which is not supplied by the parent's ratio computation.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2046 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_recurrence_transfer.extracted_1_1 : ∀ (a b : ℕ → ℚ) (d : ℚ),
  (∀ (n : ℕ), a (n + 1) = a n * d) → (∀ (n : ℕ), b (n + 1) = b n * d) → a 0 = b 0 → ∀ n ≤ 5, a n = b n
```

Read back by a model that saw only the Lean. *For all functions a and b from the natural numbers to the rational numbers and every rational number d , if for every natural number n , $a(n+1) = a(n) \cdot d$, if for every natural number n , $b(n+1) = b(n) \cdot d$, and if $a(0) = b(0)$, then for every natural number n with $n \leq 5$, $a(n) = b(n)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_recurrence_transfer
  (a b : ℕ → ℚ)
  (d : ℚ)
  (ha : ∀ n, a (n + 1) = a n * d)
  (hb : ∀ n, b (n + 1) = b n * d)
  (h₀ : a 0 = b 0) :
  ∀ n, n ≤ 5 → a n = b n := by
  intro n
  induction n with
  | zero =>
    intro _
    exact h₀
  | succ n ih =>
    intro hn
    have hprev : a n = b n := ih (by omega)
    have ha_step : a (Nat.succ n) = a n * d := by
      simpa [Nat.succ_eq_add_one] using ha n
    have hb_step : b (Nat.succ n) = b n * d := by
      simpa [Nat.succ_eq_add_one] using hb n
    calc
      a (Nat.succ n) = a n * d := ha_step
      _ = b n * d := by rw [hprev]
      _ = b (Nat.succ n) := by symm; exact hb_step
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__me__40e78e5c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 40e78e5c41e17eb3

4.66 066. bounded_geometric_law

The problem

Prove that if (a_n) is a sequence of rational numbers and d is rational such that $a_{n+1} = a_n d$ for every natural number n , with $a_0 = \frac{27}{125}$ and $a_1 = \frac{9}{25}$, then for every natural number $n \leq 5$, $a_n = \frac{27}{125} \left(\frac{5}{3}\right)^n$.

Lean statement

```
theorem bounded_geometric_law
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 1 / 25 := by
```

What it contributes. Retains the parent recurrence and rational initial data, using its ratio checkpoint $d = 5 / 3$ while generalizing the terminal computation to a bounded quantified law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a single terminal value with a bounded closed-form law and requires induction over the index. Although the ratio derivation is shared with the parent, the parent's finite unrolling does not supply the universal recurrence argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's single terminal computation with a bounded closed-form law and proves it by induction over the index. The parent's finite unfolding argument does not supply this universal statement directly, and the hypotheses are substantively used to determine the ratio.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's single terminal evaluation with a bounded closed-form law and requires induction over the index, while deriving the ratio from the initial two values. This is not recall of the parent's forward three-step calculation and is mathematically distinct from the kept recurrence-transfer child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2040 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_geometricLaw.extracted_1_1 : ∀ (a : ℕ → ℚ) (d : ℚ),
  (∀ (n : ℕ), a (n + 1) = a n * d) → a 0 = 27 / 125 → a 1 = 9 / 25 → ∀ n ≤ 5, a n = 27 / 125 * (5 / 3) ^ n
```

Read back by a model that saw only the Lean. For every function a from the natural numbers to the rational numbers and every rational number d , if for every natural number n we have $a(n + 1) = a(n) \cdot d$, if $a(0) = 27/125$, and if $a(1) = 9/25$, then for every natural number n with $n \leq 5$, $a(n) = (27/125) \cdot (5/3)^n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_geometricLaw
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n : ℕ, n ≤ 5 → a n = (27 / 125) * (5 / 3) ^ n := by
  have hd : d = 5 / 3 := by
    have h := h₀ 0
    norm_num [h₁, h₂] at h
    linarith
  intro n hn
  induction n with
  | zero =>
    norm_num [h₁]
  | succ n ih =>
    have hnle : n ≤ 5 := Nat.le_of_succ_le hn
    rw [h₀ n, ih hnle, hd]
    simp [pow_succ]
    ring
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__me__8bb838c0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 46495cc8b496d3a7

4.67 067. reciprocal_equation_unique_solution

The problem

Let a, b be real numbers with $a \neq 0$. The equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $b \neq 1$, and every such solution equals $\frac{b-1}{a}$.

Lean statement

```
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
```

Parent. Let a, b be positive real numbers. If $a + \frac{1}{x} = \frac{b}{x}$, prove that $x = \frac{b-1}{a}$; in particular, when $a = 3$ and $b = 7$, $x = 2$.

Parent in Lean

```
theorem reciprocal_parameter_uniqueness (a b x : ℝ) (ha : 0 < a) (h : a + 1 / x = b / x) : x = (b - 1) / a ∧ ((a = 3 ∧ b = 7) → x = 2)
  ↪ := by
```

What it contributes. Retains the parent's reciprocal-equation algebra and denominator-clearing argument, while changing the target to an existence-and-uniqueness characterization for arbitrary nonzero a .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the parent's conditional solution formula into an existence-and-uniqueness characterization, including proving that no nonzero solution exists when $b=1$. This requires new reasoning beyond recalling the parent's derivation, while retaining the parameterized coefficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially changes the claim to an existence-and-uniqueness characterization and weakens the coefficient hypothesis from positivity to nonzeroness. Proving existence, nonzero witness validity, and uniqueness requires reasoning not supplied by the parent's direct solution formula.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine existence-and-uniqueness characterization: for arbitrary nonzero a , a nonzero solution exists exactly when $b \neq 1$, and uniqueness must be proved. The parent only derives a formula from an already-given solution under $a > 0$, so its proof does not supply the child's converse or witness construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2453 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_equation_unique_solution.extracted_1_1 (a b : ℝ) (ha : a ≠ 0) :  
  (b ≠ 1 ↔ ∃! x, x ≠ 0 ∧ a + 1 / x = b / x) ∧ (b ≠ 1 → ∀ (x : ℝ), x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a)
```

Read back by a model that saw only the Lean. For real numbers a and b , assuming a is nonzero, b is not equal to 1 if and only if there exists exactly one nonzero real number x satisfying $a + 1/x = b/x$; moreover, if b is not equal to 1, then every nonzero real number x satisfying $a + 1/x = b/x$ is equal to $(b - 1)/a$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :  
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧  
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by  
  have hcharacterization : b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x := by  
    constructor  
    · intro hb  
      have hnum : b - 1 ≠ 0 := sub_ne_zero.mpr hb  
      have hx0 : (b - 1) / a ≠ 0 := div_ne_zero hnum ha  
      have heq : a + 1 / ((b - 1) / a) = b / ((b - 1) / a) := by  
        field_simp [hx0, ha]  
      ring  
      refine ⟨(b - 1) / a, ⟨hx0, heq⟩, ?_⟩  
      intro y hy  
      rcases hy with ⟨hy0, hyeq⟩  
      have hyeq' := hyeq  
      field_simp [hy0] at hyeq'  
      field_simp [ha]  
      nlinarith [hyeq']  
    · intro hex hb  
      rcases hex with ⟨x, ⟨hx0, hxeq⟩, huniq⟩  
      have hxeq' := hxeq  
      field_simp [hx0] at hxeq'  
      rw [hb] at hxeq'  
      have hax : a * x = 0 := by  
        nlinarith [hxeq']  
      have haz : a = 0 := (mul_eq_zero.mpr hax).resolve_right hx0  
      exact ha haz  
  have hformula : b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a := by  
    intro hb x hx  
    rcases hx with ⟨hx0, hxeq⟩  
    have hxeq' := hxeq  
    field_simp [hx0] at hxeq'  
    field_simp [ha]  
    nlinarith [hxeq']  
  exact ⟨hcharacterization, hformula⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_251__me.me__3c0a1ee0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3c0a1ee08cd89089

4.68 068. coupled_geometric_reciprocal_parameters

The problem

A geometric sequence begins with nonzero real terms p, q and has common ratio d . If a, b satisfy $a + \frac{1}{d} = \frac{b}{d}$, prove that $b = 1 + \frac{q}{p}a$.

Lean statement

```
theorem coupled_geometric_reciprocal_parameters (a b d p q : ℝ) (u : ℕ → ℝ) (hp : p ≠ 0) (hq : q ≠ 0) (h0 : u 0 = p) (h1 : u 1 = q) (hrec : ∀ n : ℕ, u (n + 1) = d * u n) (h : a + 1 / d = b / d) : b = 1 + (q / p) * a
```

Parent. A geometric sequence begins with 3, 5 and has common ratio d . If positive real numbers a, b satisfy $a + \frac{1}{d} = \frac{b}{d}$, prove that $b = 1 + \frac{5a}{3}$.

Parent in Lean

```
theorem coupled_geometric_reciprocal_coefficients (a b d : ℝ) (u : ℕ → ℝ) (h0 : u 0 = 3) (h1 : u 1 = 5) (hrec : ∀ n : ℕ, u (n + 1) = d * u n) (h : a + 1 / d = b / d) : b = 1 + 5 * a / 3
```

What it contributes. Generalizes the parent's first-step ratio recovery and reciprocal-equation simplification from fixed initial terms 3 and 5 to symbolic nonzero parameters p and q .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the initial geometric terms: it must derive the symbolic ratio $d = q/p$ and establish $d \neq 0$ before transforming the reciprocal equation. This proof cannot be obtained by recalling the parent's fixed 3-to-5 calculation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed values 3 and 5 are replaced by symbolic nonzero parameters, requiring derivation of $d = q/p$ and separately establishing $d \neq 0$ before transforming the reciprocal equation. This is not recoverable by recalling the parent's specialized proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the initial geometric terms and must derive the symbolic ratio $d = q/p$, establish $d \neq 0$, and solve the reciprocal equation algebraically. The parent's fixed-value normalization does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2465 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_geometric_reciprocal_parameters.extracted_1_1 (a b d p q : ℝ) (u : ℕ → ℝ) (hp : p ≠ 0) (hq : q ≠ 0)
  (h0 : u 0 = p) (h1 : u 1 = q) (hrec : ∀ (n : ℕ), u (n + 1) = d * u n) (h : a + 1 / d = b / d) :
  b = 1 + q / p * a
```

Read back by a model that saw only the Lean. For real numbers $a, b, d, p,$ and $q,$ and a sequence u of real numbers indexed by the natural numbers, suppose that p is nonzero, q is nonzero, $u(0) = p, u(1) = q,$ and for every natural number $n,$ $u(n + 1) = d \cdot u(n).$ If $a + 1/d = b/d,$ then $b = 1 + (q/p) \cdot a.$

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem coupled_geometric_reciprocal_parameters (a b d p q : ℝ) (u : ℕ → ℝ) (hp : p ≠ 0) (hq : q ≠ 0) (h0 : u 0 = p) (h1 : u 1 = q) (hrec
↪ : ∀ n : ℕ, u (n + 1) = d * u n) (h : a + 1 / d = b / d) : b = 1 + (q / p) * a := by
  have hstep : u 1 = d * u 0 := by
    simp using hrec 0
  have hratio : d = q / p := by
    apply (eq_div_iff hp).2
    rw [← h1, ← h0]
    exact hstep.symm
  have hd : d ≠ 0 := by
    intro hd0
    have hzero : u 1 = 0 := by
      calc
        u 1 = d * u 0 := hstep
        _ = 0 := by rw [hd0]; simp
    apply hq
    rw [← h1]
    exact hzero
  have hcoeff : b = 1 + a * d := by
    field_simp [hd] at h
    linarith
  calc
    b = 1 + a * d := hcoeff
    _ = 1 + (q / p) * a := by rw [hratio]; ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_251__me__80e0f764__x__mathd_algebra_234__xe.me__0801ad48

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0801ad48a199c288

4.69 069. unique_reciprocal_solution

The problem

For real numbers a, b, c with $a \neq 0$ and $c - b \neq 0$, the equation $a + \frac{b}{x} = \frac{c}{x}$ has exactly one nonzero real solution. Show that this solution is $x = \frac{c-b}{a}$.

Lean statement

```
theorem unique_reciprocal_solution (a b c : ℝ) (ha : a ≠ 0) (hcb : c - b ≠ 0) :
  (∃! x : ℝ, x ≠ 0 ∧ a + b / x = c / x) ∧
  ∀ x : ℝ, x ≠ 0 → a + b / x = c / x → x = (c - b) / a
```

Parent. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h1 : 3 + 1 / x = 7 / x) : x = 2 := by
```

What it contributes. Retains the parent's reciprocal-equation strategy of establishing a nonzero denominator, clearing denominators, and finishing by linear arithmetic, while parameterizing all constants and adding existence and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the reciprocal equation and proves a symbolic nonzero candidate, existence, uniqueness, and the universal solution formula. Its denominator clearing and use of the essential hypotheses require reasoning beyond recalling the parent's isolated numerical proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the isolated numerical equation to arbitrary real coefficients and proves both existence and uniqueness of the nonzero solution, including the symbolic formula. Its candidate nonzeroness and uniqueness derivation are substantive obligations not supplied by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal equation and proves both existence and uniqueness of the symbolic nonzero solution. It requires candidate nonzeroness, verification, denominator clearing, and a general algebraic uniqueness argument beyond recalling the parent's isolated calculation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2569 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_reciprocal_solution.extracted_1_1 (a b c : ℝ) (ha : a ≠ 0) (hcb : c - b ≠ 0) :  
  (∃! x, x ≠ 0 ∧ a + b / x = c / x) ∧ ∀ (x : ℝ), x ≠ 0 → a + b / x = c / x → x = (c - b) / a
```

Read back by a model that saw only the Lean. For real numbers a , b , and c , if a is nonzero and $c - b$ is nonzero, then there exists exactly one nonzero real number x such that $a + b/x = c/x$; moreover, every real number x satisfying $x \neq 0$ and $a + b/x = c/x$ is equal to $(c - b)/a$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem unique_reciprocal_solution (a b c : ℝ) (ha : a ≠ 0) (hcb : c - b ≠ 0) :  
  (∃! x : ℝ, x ≠ 0 ∧ a + b / x = c / x) ∧  
    ∀ x : ℝ, x ≠ 0 → a + b / x = c / x → x = (c - b) / a := by  
  have hcandidate : (c - b) / a ≠ 0 := div_ne_zero hcb ha  
  have hverify : a + b / ((c - b) / a) = c / ((c - b) / a) := by  
    field_simp [ha, hcb] <=> ring  
  constructor  
  · refine ⟨(c - b) / a, ⟨hcandidate, hverify⟩, ?_⟩  
    intro y hy  
    rcases hy with ⟨hy0, hyEq⟩  
    have hcleared := hyEq  
    field_simp [hy0] at hcleared  
    have hmul : a * y = c - b := by  
      linarith  
    apply (eq_div_iff ha).2  
    nlinarith [hmul]  
  · intro x hx hEq  
    have hcleared := hEq  
    field_simp [hx] at hcleared  
    have hmul : a * x = c - b := by  
      linarith  
    apply (eq_div_iff ha).2  
    nlinarith [hmul]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__me__84df140b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 84df140bbd03007e

4.70 070. bounded_quadratic_parameter_characterization

The problem

For all real numbers b and c with $1 \leq b \leq 5$, the equation $2x^2 + bx + c = 0$ has a real solution if and only if $c \leq \frac{b^2}{8}$.

Lean statement

```
theorem bounded_quadratic_parameter_characterization :
  ∀ b c : ℝ, 1 ≤ b → b ≤ 5 →
    ((∃ x : ℝ, 2 * x ^ 2 + b * x + c = 0) ↔ c ≤ b ^ 2 / 8) := by
```

Parent. Show that $2x^2 + 5x + c = 0$ has at least one real solution if and only if $c \leq \frac{25}{8}$.

Parent in Lean

```
theorem quadratic_parameter_characterization
  (S : Set ℝ)
  (hS : S = {c : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  ∀ c : ℝ, c ∈ S ↔ c ≤ (25 / 8 : ℝ) := by
```

What it contributes. Supplied the quadratic solvability pattern, sharp-square obstruction, and explicit square-root witness; the fixed coefficient 5 was parameterized to b over a bounded positive range.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the linear coefficient: the threshold becomes the symbolic discriminant bound $c \leq b^2/8$, and the proof constructs a root using $\sqrt{(b^2 - 8c)}$. This requires carrying b through the completed-square and root-construction arguments rather than merely reusing the fixed-coefficient conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the linear coefficient and proves the symbolic discriminant threshold for arbitrary b , requiring the completed-square argument to survive symbolically rather than merely instantiating the parent at $b = 5$. The bounds on b are unused but do not undermine the substantive generalization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the linear coefficient and proves the symbolic discriminant threshold using a variable completed square and square root construction. The parent's fixed-coefficient proof does not directly supply this generalized argument, and the bounds are harmlessly redundant rather than the source of the result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1747 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_quadratic_parameter_characterization.extracted_1_1 : ∀ (b c : ℝ),
  1 ≤ b → b ≤ 5 → ((∃ x, 2 * x ^ 2 + b * x + c = 0) ↔ c ≤ b ^ 2 / 8)
```

Read back by a model that saw only the Lean. *For all real numbers b and c , if $1 \leq b$ and $b \leq 5$, then there exists a real number x satisfying $2x^2 + bx + c = 0$ if and only if $c \leq b^2/8$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_quadratic_parameter_characterization :
  ∀ b c : ℝ, 1 ≤ b → b ≤ 5 →
    ((∃ x : ℝ, 2 * x ^ 2 + b * x + c = 0) ↔ c ≤ b ^ 2 / 8) := by
  intro b c hb_lower hb_upper
  have hb_pos : 0 < b := by
    linarith
  constructor
  · rintro ⟨x, hx⟩
    have hsquare : 0 ≤ (4 * x + b) ^ 2 := by
      exact sq_nonneg (4 * x + b)
    nlinarith
  · intro hc
    have hdisc : 0 ≤ b ^ 2 - 8 * c := by
      nlinarith
    have hsqrt : (Real.sqrt (b ^ 2 - 8 * c)) ^ 2 = b ^ 2 - 8 * c := by
      exact Real.sq_sqrt hdisc
    refine ⟨(-b + Real.sqrt (b ^ 2 - 8 * c)) / 4, ?_⟩
    nlinarith [hsqrt]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__me__37417bdf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint df675ab28c6ab34d

4.71 071. quadratic_unique_root_iff

The problem

For real numbers a, b, c with $a > 0$, prove that $ax^2 + bx + c = 0$ has exactly one real solution if and only if $c = \frac{b^2}{4a}$.

Lean statement

```
theorem quadratic_unique_root_iff (a b c : ℝ) (ha : 0 < a) :
  (∃! x : ℝ, a * x ^ 2 + b * x + c = 0) ↔ c = b ^ 2 / (4 * a)
```

Parent. For a real number a with $0 < a \leq 10$, what is the largest number c such that $ax^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_real_root_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | ∃ x : ℝ, a * x ^ 2 + 5 * x + c = 0}) : IsGreatest S (25 / (4 * a))
```

What it contributes. Generalizes the parent's positive-leading-coefficient quadratic and reuses its completed-square/vertex argument, while changing the conclusion to a unique-root characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the conclusion from a greatest-element characterization to an iff characterization of unique real roots, while also parameterizing the fixed linear coefficient 5 as arbitrary b . Its proof requires a distinct symmetry/uniqueness argument and is not recoverable by recalling the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the extremal-value claim into a unique-real-root characterization. Its proof requires a new symmetry argument showing any unique root must be the vertex, plus a separate uniqueness proof from square nonnegativity; the parent's greatest-element proof does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine change of shape: it characterizes the discriminant-zero case by existence and uniqueness of a real root. Its proof uses the reflected second root and proves uniqueness algebraically, rather than recalling the parent's IsGreatest argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2606 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_unique_root_iff.extracted_1_1 (a b c : ℝ) (ha : 0 < a) :  
  (∃! x, a * x ^ 2 + b * x + c = 0) ↔ c = b ^ 2 / (4 * a)
```

Read back by a model that saw only the Lean. For real numbers a , b , and c with $a > 0$, the quadratic equation $ax^2 + bx + c = 0$ has exactly one real solution if and only if $c = b^2/(4a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_unique_root_iff (a b c : ℝ) (ha : 0 < a) :  
  (∃! x : ℝ, a * x ^ 2 + b * x + c = 0) ↔ c = b ^ 2 / (4 * a) := by
  have hane : a ≠ 0 := ne_of_gt ha
  constructor
  · rintro ⟨x, hx, huniq⟩
    have hreflect : a * (-b / a - x) ^ 2 + b * (-b / a - x) + c = 0 := by
      have hid : a * (-b / a - x) ^ 2 + b * (-b / a - x) + c =  
        a * x ^ 2 + b * x + c := by
          field_simp [hane]
          ring
      rw [hid]
      exact hx
    have heq : -b / a - x = x := huniq (-b / a - x) hreflect
    have hvertex : 2 * a * x + b = 0 := by
      field_simp [hane] at heq
      nlinarith [heq]
    have hb : b = -2 * a * x := by
      linarith [hvertex]
    rw [hb] at hx
    apply (eq_div_iff (by nlinarith [ha])).2
    rw [hb]
    nlinarith [hx]
  · intro hc
    refine ⟨-b / (2 * a), ?_, ?_⟩
    · rw [hc]
      field_simp [hane]
      ring
    · intro y hy
      rw [hc] at hy
      have hs : (2 * a * y + b) ^ 2 = 0 := by
        calc
          (2 * a * y + b) ^ 2 =  
            4 * a * (a * y ^ 2 + b * y + b ^ 2 / (4 * a)) := by
              field_simp [hane]
              ring
          _ = 0 := by rw [hy]; ring
      have hlin : 2 * a * y + b = 0 := by
        nlinarith [hs]
      apply (eq_div_iff (by nlinarith [ha])).2
      nlinarith [hlin]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__me__fce4aeba

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fce4aeba45e34408

4.72 072. quadratic_parameterized_extremum

The problem

For positive real numbers a and b , let S be the set of constants c for which $ax^2 + bx + c = 0$ has a real solution. Show that S has greatest element $\frac{b^2}{4a}$, and that at this value the equation has the unique root $-\frac{b}{2a}$.

Lean statement

```
theorem quadratic_parameterized_extremum (a b : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | ∃ x : ℝ, a * x ^ 2 + b * x + c = 0}) :
  ↪ IsGreatest S (b ^ 2 / (4 * a)) ∧ ∀ x : ℝ, a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0 ↔ x = -b / (2 * a)
```

Parent. For a positive real number a , the values of c for which $ax^2 + 5x + c = 0$ has a real solution are exactly those satisfying $25 - 4ac \geq 0$. Show that their greatest value is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))
```

What it contributes. Supplied the discriminant/completed-square extremality pattern; the mutation replaces the fixed coefficient 5 by the positive parameter b and adds root uniqueness at the extremal constant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the fixed constant 25 into b^2 and changes the feasible-set description to existence of a real quadratic root, then proves the additional uniqueness characterization at the extremal constant. Its proof requires completing the square and deriving the zero-square condition, which is not supplied by recalling the parent's direct inequality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both coefficients and changes the set characterization from a linear discriminant inequality to existence of a real quadratic root. Its proof requires completing-square reasoning, constructing a witness, and proving uniqueness of the extremal root, none of which is supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the linear coefficient and changes the feasible-set definition from a discriminant inequality to existence of a real root. Its proof requires constructing a root, completing the square, and proving uniqueness via a zero-square argument, none of which is supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2624 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_parameterized_extremum.extracted_1_1 (a b : ℝ) (ha : 0 < a) (hb : 0 < b) (S : Set ℝ)
(hS : S = {c | ∃ x, a * x ^ 2 + b * x + c = 0}) :
IsGreatest S (b ^ 2 / (4 * a)) ∧ ∀ (x : ℝ), a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0 ↔ x = -b / (2 * a)
```

Read back by a model that saw only the Lean. For real numbers a and b with $0 < a$ and $0 < b$, and for a set S of real numbers satisfying $S = \{c \in \mathbb{R} \mid \text{there exists } x \in \mathbb{R} \text{ such that } ax^2 + bx + c = 0\}$, the number $b^2/(4a)$ is the greatest element of S , and for every real number x , $ax^2 + bx + b^2/(4a) = 0$ if and only if $x = -b/(2a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem quadratic_parameterized_extremum (a b : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | ∃ x : ℝ, a * x ^ 2 + b * x + c = 0}) :
  IsGreatest S (b ^ 2 / (4 * a)) ∧ ∀ x : ℝ, a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0 ↔ x = -b / (2 * a) := by
  rw [hS]
  constructor
  · constructor
    · change ∃ x : ℝ, a * x ^ 2 + b * x + b ^ 2 / (4 * a) = 0
      refine ⟨-b / (2 * a), ?_⟩
      field_simp [ha.ne']
      ring
    · intro c hc
      change ∃ x : ℝ, a * x ^ 2 + b * x + c = 0 at hc
      rcases hc with ⟨x, hx⟩
      have hpos : 0 < 4 * a := by
        nlinarith [ha]
      apply (le_div_iff hpos).2
      nlinarith [sq_nonneg (2 * a * x + b), hx]
  · intro x
    constructor
    · intro hx
      have hclean : 4 * a * (a * x ^ 2 + b * x) + b ^ 2 = 0 := by
        calc
          4 * a * (a * x ^ 2 + b * x) + b ^ 2 = 4 * a * (a * x ^ 2 + b * x + b ^ 2 / (4 * a)) := by
            field_simp [ha.ne']
          _ = 0 := by rw [hx]; ring
      have hsq : (2 * a * x + b) ^ 2 = 0 := by
        nlinarith [hclean]
      have hzero : 2 * a * x + b = 0 := by
        nlinarith [hsq]
      apply (eq_div_iff (by nlinarith [ha] : (2 * a) ≠ 0)).2
      nlinarith [hzero]
    · intro hx
      rw [hx]
      field_simp [ha.ne']
      ring
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_28__me.ms.me__1f5d0186

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b1ffb242803e785a

4.73 073. quadratic_parameter_characterization

The problem

Show that $2x^2 + 5x + c = 0$ has at least one real solution if and only if $c \leq \frac{25}{8}$.

Lean statement

```
theorem quadratic_parameter_characterization
  (S : Set ℝ)
  (hS : S = {c : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  ∀ c : ℝ, c ∈ S ↔ c ≤ (25 / 8 : ℝ) := by
```

Parent. What is the largest number c such that $2x^2 + 5x + c = 0$ has at least one real solution? Express your answer as a common fraction. Show that it is $\frac{25}{8}$.

Parent in Lean

```
theorem mathd_algebra_28
  (S : Set ℝ)
  (hS : S = {c | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25/8 : ℝ) := by
```

What it contributes. Supplies the sharp bound $25/8$ and the completed-square inequality used in the necessity direction; the same bound is used to establish discriminant nonnegativity for the converse.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child characterizes the entire parameter range, requiring a constructive real-root argument via the discriminant and square root for every $c \leq 25/8$. The parent only proves that $25/8$ is the greatest attainable value and does not establish this converse.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the greatest-element claim into a full range characterization, requiring the new converse that every $c \leq 25/8$ admits a real root. Its explicit square-root construction is not supplied by the parent's upper-bound argument, so recalling the parent does not solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's greatest-element claim into an exact characterization of S , requiring a new converse construction of a real root using the discriminant and square root. The parent proof only establishes the upper bound and one extremal witness, so recalling it does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1726 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_parameter_characterization.extracted_1_1 : ∀ (S : Set ℝ),
  S = {c | ∃ x, 2 * x ^ 2 + 5 * x + c = 0} → ∀ (c : ℝ), c ∈ S ↔ c ≤ 25 / 8
```

Read back by a model that saw only the Lean. *For every set S of real numbers, if S is exactly the set of real numbers c for which there exists a real number x satisfying $2x^2 + 5x + c = 0$, then for every real number c , c belongs to S if and only if $c \leq 25/8$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_parameter_characterization
  (S : Set ℝ)
  (hS : S = {c : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  ∀ c : ℝ, c ∈ S ↔ c ≤ (25 / 8 : ℝ) := by
  rw [hS]
  intro c
  simp only [Set.mem_setOf_eq]
  have sharp_bound : ∀ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0 → c ≤ (25 / 8 : ℝ) := by
    intro x hx
    nlinarith [sq_nonneg (4 * x + 5)]
  have root_construction : c ≤ (25 / 8 : ℝ) → ∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0 := by
    intro hc
    have hdisc : 0 ≤ 25 - 8 * c := by
      nlinarith
    have hsqrt : (Real.sqrt (25 - 8 * c)) ^ 2 = 25 - 8 * c := by
      exact Real.sq_sqrt hdisc
    refine ((-5 + Real.sqrt (25 - 8 * c)) / 4, ?_)
    nlinarith [hsqrt]
  constructor
  · rintro ⟨x, hx⟩
    exact sharp_bound x hx
  · intro hc
    exact root_construction hc
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_28__me__770a1b59

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b17ecd8fe38642b9

4.74 074. ap_quadratic_unique_double_root

The problem

The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. Prove that the equation $a_4x^2 + 5x + \frac{25}{68} = 0$ has exactly one real solution.

Lean statement

```
theorem ap_quadratic_unique_double_root
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∃! x : ℝ, a 4 * x ^ 2 + 5 * x + 25 / 68 = 0
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the greatest real number c for which $a_4x^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{68}$.

Parent in Lean

```
theorem coupled_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hS : S = {c | ∃ x : ℝ, a 4 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a 4)) : ℝ := by
```

What it contributes. Uses the parent's AP recurrence derivation to identify $p = 5$, $q = 2$, and the terminal coefficient $a_4 = 17$; replaces the parent's extremal-set conclusion with unique double-root characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a greatest-set extremum to an explicit existence-and-uniqueness statement for the double root. Uniqueness requires a separate square-nonnegativity argument, so the parent's proof does not directly solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a greatest-element characterization of attainable constants to uniqueness of the quadratic root at the extremal coefficient. After deriving the AP coefficient, it requires a distinct global uniqueness argument via completing the square; the parent's greatest-set proof does not directly supply this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the extremum/set-membership conclusion into an explicit unique-root theorem. Proving uniqueness via the completed-square argument is new reasoning not supplied by the parent's greatest-element proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2642 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ap_quadratic_unique_double_root.extracted_1.1 (p q : ℝ) (a : ℕ → ℝ)
  (h₀ : ∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) : ∃! x, a 4 * x ^ 2 + 5 * x + 25 / 68 = 0
```

Read back by a model that saw only the Lean. *Let p and q be real numbers, and let a be a sequence of real numbers indexed by the natural numbers. Suppose that for every natural number n , $a(n+2) - a(n+1) = a(n+1) - a(n)$, that $a(1) = p$, that $a(2) = 9$, that $a(3) = 3p - q$, and that $a(4) = 3p + q$. Then there exists exactly one real number x such that $a(4)x^2 + 5x + 25/68 = 0$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem ap_quadratic_unique_double_root
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∃! x : ℝ, a 4 * x ^ 2 + 5 * x + 25 / 68 = 0 := by
  have hA := h₀ 1
  have hB := h₀ 2
  norm_num at hA hB
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith
  have hq : q = 2 := by
    linarith [hA, hB, hp]
  have hcoef : a 4 = 17 := by
    rw [h₄, hp, hq]
    norm_num
  refine ⟨-5 / 34, ?_, ?_⟩
  · rw [hcoef]
    norm_num
  · intro y hy
    rw [hcoef] at hy
    nlinarith [sq_nonneg (34 * y + 5)]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__x__amc12a_2010_p10__xe.me__06a52718

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 06a5271847415a28

4.75 075. bounded_ap_quadratic

The problem

The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. If $k \in \{4, 5\}$, what is the greatest real number c for which $a_k x^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{4a_k}$.

Lean statement

```
theorem bounded_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (k : ℕ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hk : 4 ≤ k ∧ k ≤ 5)
  (hS : S = {c | ∃ x : ℝ, a k * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a k)) : ℝ
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the greatest real number c for which $a_4 x^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{68}$.

Parent in Lean

```
theorem coupled_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hS : S = {c | ∃ x : ℝ, a 4 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a 4)) : ℝ := by
```

What it contributes. Uses the parent's arithmetic-progression deductions and quadratic-extremum structure, extended to the bounded adjacent index $k \in \{4, 5\}$ with a new recurrence-derived fifth-term obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The bounded index forces a genuine case split between $k=4$ and $k=5$. The $k=5$ branch requires deriving $a(5)=21$ from the arithmetic-progression recurrence and proving a new quadratic extremal case, so recalling the parent proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the coefficient index from fixed a_4 to bounded a_k and must handle the new case $k=5$, derive $a_5=21$, and split on the index. The parent proof does not supply this case analysis or the additional recurrence computation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounded index introduces a genuine new case, $k=5$, requiring derivation of $a(5)=21$ and a separate extremal argument; recalling the parent only handles the $k=4$ branch. This matches the planned bounded generalization and is not merely decorative.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2599 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_ap_quadratic.extracted_1.1 (S : Set ℝ) (p q : ℝ) (a : ℕ → ℝ) (k : ℕ)
  (h₀ : ∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) (hk : 4 ≤ k ∧ k ≤ 5) (hS : S = {c | ∃ x, a k * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a k))
```

Read back by a model that saw only the Lean. Let S be a subset of the real numbers, let p and q be real numbers, let a be a sequence of real numbers indexed by the natural numbers, and let k be a natural number. Assume that for every natural number n , $a(n+2) - a(n+1) = a(n+1) - a(n)$; that $a(1) = p$; that $a(2) = 9$; that $a(3) = 3p - q$; that $a(4) = 3p + q$; that $4 \leq k$ and $k \leq 5$; and that S is exactly the set of real numbers c for which there exists a real number x satisfying $a(k)x^2 + 5x + c = 0$. Then $25/(4a(k))$ is the greatest element of S .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem bounded_ap_quadratic
  (S : Set ℝ)
  (p q : ℝ)
  (a : ℕ → ℝ)
  (k : ℕ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hk : 4 ≤ k ∧ k ≤ 5)
  (hS : S = {c | ∃ x : ℝ, a k * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25 / (4 * a k) : ℝ) := by
  have hA := h₀ 1
  have hB := h₀ 2
  have hC := h₀ 3
  norm_num at hA hB hC
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  rw [h₄, h₃] at hC
  have hp : p = 5 := by
    linarith [hA, hB]
  have hq : q = 2 := by
    linarith [hA, hB, hp]
  have hcoef4 : a 4 = 17 := by
    rw [h₄, hp, hq]
```

```

norm_num
have hcoef5 : a 5 = 21 := by
  rw [hp, hq] at hC
  linarith [hC, hcoef4]
have hk_cases : k = 4 ∨ k = 5 := by
  omega
rcases hk_cases with rfl | rfl
· have hpos : 0 < a 4 := by
  rw [hcoef4]
  norm_num
have hquad_bound : ∀ x c : ℝ,
  a 4 * x ^ 2 + 5 * x + c = 0 → c ≤ 25 / (4 * a 4) := by
  intro x c hx
  rw [hcoef4] at hx ⊢
  nlinarith [sq_nonneg (34 * x + 5), hpos]
rw [hS]
constructor
· refine ⟨-5 / (2 * a 4), ?_⟩
  rw [hcoef4]
  norm_num
· intro c hc
  rcases hc with ⟨x, hx⟩
  exact hquad_bound x c hx
· have hpos : 0 < a 5 := by
  rw [hcoef5]
  norm_num
have hquad_bound : ∀ x c : ℝ,
  a 5 * x ^ 2 + 5 * x + c = 0 → c ≤ 25 / (4 * a 5) := by
  intro x c hx
  rw [hcoef5] at hx ⊢
  nlinarith [sq_nonneg (42 * x + 5), hpos]
rw [hS]
constructor
· refine ⟨-5 / (2 * a 5), ?_⟩
  rw [hcoef5]
  norm_num
· intro c hc
  rcases hc with ⟨x, hx⟩
  exact hquad_bound x c hx

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__x__amc12a_2010_p10__xe.me__1f523ee9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1f523ee96233b1a2

4.76 076. classify_terminal_index_bounded

The problem

The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. If $4 \leq N$, show that among the terms with indices from 1 through N , the value 17 occurs exactly at index 4.

Lean statement

```
theorem classify_terminal_index_bounded
  (p q : ℝ)
  (a : ℕ → ℝ)
  (N : ℕ)

  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∀ n : ℕ, n ∈ Finset.Icc 1 N → (a n = 17 ↔ n = 4)
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. Determine exactly which index among 1, 2, 3, 4 has value 17.

Parent in Lean

```
theorem classify_terminal_index
  (p q : ℝ)
  (a : ℕ → ℝ)
  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∀ n : ℕ, n ∈ Finset.Icc 1 4 → (a n = 17 ↔ n = 4)
```

What it contributes. Supplied the initial arithmetic-progression hypotheses and the endpoint equations used to solve $p=5$ and $q=2$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the conclusion from indices through 4 to an arbitrary endpoint N . Its proof must derive the full arithmetic-sequence formula and solve the resulting equation, rather than recalling the parent's finite case split.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the classification to arbitrary N and must derive the full affine formula for a by induction. This reasoning is not supplied by the parent's finite interval case split, so recalling the parent would not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the conclusion to an arbitrary bounded endpoint N and must derive the entire arithmetic progression by induction, rather than relying on the parent's finite four-case enumeration. The added N is used substantively, and the proof is not recoverable by simply recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2643 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem classify_terminal_index_bounded.extracted_1.1 : ∀ (p q : ℝ) (a : ℕ → ℝ) (N : ℕ),
  4 ≤ N →
  (∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) →
  a 1 = p → a 2 = 9 → a 3 = 3 * p - q → a 4 = 3 * p + q → ∀ n ∈ Finset.Icc 1 N, a n = 17 ↔ n = 4
```

Read back by a model that saw only the Lean. For all real numbers p and q , all functions a from the natural numbers to the real numbers, and all natural numbers N , if $4 \leq N$, if for every natural number n we have $a(n+2) - a(n+1) = a(n+1) - a(n)$, and if $a(1) = p$, $a(2) = 9$, $a(3) = 3p - q$, and $a(4) = 3p + q$, then for every natural number n with $1 \leq n \leq N$, $a(n) = 17$ if and only if $n = 4$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem classify_terminal_index_bounded
  (p q : ℝ)
  (a : ℕ → ℝ)
  (N : ℕ)

  (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p)
  (h₂ : a 2 = 9)
  (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) :
  ∀ n : ℕ, n ∈ Finset.Icc 1 N → (a n = 17 ↔ n = 4) := by
  have hA := h₀ 1
  have hB := h₀ 2
  norm_num at hA hB
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith [hA, hB]
  have hq : q = 2 := by
    linarith [hA, hB, hp]
  have hlaw : ∀ n : ℕ, a (n + 1) = 5 + 4 * (n : ℝ) := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      by_cases hn : n = 0
      · subst n
        rw [h₁, hp]
        norm_num
      · obtain ⟨k, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hn
        by_cases hk : k = 0
        · subst k
          norm_num [h₂]
        · obtain ⟨j, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hk
          have hprev : a (j + 1) = 5 + 4 * (j : ℝ) := ih j (by omega)
          have hcurr : a (j + 2) = 5 + 4 * ((j + 1) : ℝ) := by
            simpa [Nat.add_assoc] using ih (j + 1) (by omega)
          have hr : a (j + 3) - a (j + 2) = a (j + 2) - a (j + 1) := by
```

```

      simpa [Nat.add_assoc] using h₀ (j + 1)
    push_cast at hcurr ⊢
      linarith [hr, hprev, hcurr]
intro n hn
have hn' : 1 ≤ n ∧ n ≤ N := by
  simpa [Finset.mem_Icc] using hn
have hn0 : n ≠ 0 := by omega
obtain ⟨k, rfl⟩ := Nat.exists_eq_succ_of_ne_zero hn0
have hk := hlaw k
constructor
· intro heq
  have hkval : (k : ℝ) = 3 := by
    linarith [heq, hk]
  have hkN : k = 3 := by
    exact_mod_cast hkval
  exact congrArg Nat.succ hkN
· intro heq
  have hk3 : k = 3 := by omega
  subst k
  have h := hlaw 3
  norm_num at h ⊢
  exact h

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_28__x__amc12a_2010_p10__xe.mh.me__87e1cfc6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint c69284efcceb600e

4.77 077. bounded_abs_interval

The problem

Prove that for real numbers a and c satisfying $0 < c \leq 3$, the inequality $\frac{1}{5}|9 + ca| < 1$ holds if and only if a lies in the open interval $\left(-\frac{14}{c}, -\frac{4}{c}\right)$.

Lean statement

```
theorem bounded_abs_interval (a c : ℝ) (hc : 0 < c ∧ c ≤ 3) :
  (1 / 5 * abs (9 + c * a) < 1) ↔ a ∈ Set.Ioo (-14 / c) (-4 / c)
```

Parent. Solve for a : $\frac{1}{5}|9 + 2a| < 1$. Express your answer in interval notation. Show that it is $(-7, -2)$.

Parent in Lean

```
theorem mathd_algebra_327
  (a : ℝ) :
  (1 / 5 * abs (9 + 2 * a) < 1) ↔ a ∈ Set.Ioo (-7:ℝ) (-2) := by
```

What it contributes. Generalizes the parent's absolute-value-to-open-interval proof by replacing the fixed coefficient 2 with a bounded positive parameter c and using positive-denominator interval scaling in both directions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The coefficient 2 is genuinely replaced by a symbolic positive c , and proving the scaled interval requires division by c and explicit positivity reasoning absent from the parent proof. The upper bound $c \leq 3$ is unnecessary but does not make the child a recall or decoration mutation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the coefficient of a and must use the new positivity hypothesis on c to divide inequalities and derive the scaled interval. This is not recoverable by simply recalling the parent's fixed-coefficient proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the coefficient and must use the new positivity hypothesis to divide inequalities by c and derive scaled interval endpoints. The parent's fixed-coefficient linear arithmetic does not directly supply this symbolic division argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2076 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_abs_interval.extracted_1_1 (a c : ℝ) (hc : 0 < c ∧ c ≤ 3) :  
  1 / 5 * |9 + c * a| < 1 ↔ a ∈ Set.Ioo (-14 / c) (-4 / c)
```

Read back by a model that saw only the Lean. For real numbers a and c satisfying $0 < c$ and $c \leq 3$, one has $(1/5)$ times the absolute value of $9 + ca$ is less than 1 if and only if a lies in the open interval $(-14/c, -4/c)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_abs_interval (a c : ℝ) (hc : 0 < c ∧ c ≤ 3) :  
  (1 / 5 * abs (9 + c * a) < 1) ↔ a ∈ Set.Ioo (-14 / c) (-4 / c) := by  
  constructor  
  · intro h  
    have habs : abs (9 + c * a) < 5 := by  
      nlinarith  
    rw [abs_lt] at habs  
    have hlow : -14 / c < a := by  
      apply (div_lt_iff₀ hc.1).2  
      nlinarith [habs.1]  
    have hupp : a < -4 / c := by  
      apply (lt_div_iff₀ hc.1).2  
      nlinarith [habs.2]  
    exact ⟨hlow, hupp⟩  
  · intro h  
    have hlow : -14 < c * a := by  
      have h' := (div_lt_iff₀ hc.1).1 h.1  
      nlinarith [h']  
    have hupp : c * a < -4 := by  
      have h' := (lt_div_iff₀ hc.1).1 h.2  
      nlinarith [h']  
    have hinterval : -5 < 9 + c * a ∧ 9 + c * a < 5 := by  
      constructor <|> nlinarith  
    rw [← abs_lt] at hinterval  
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_327__me__c47e671f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c47e671f35455c30

4.78 078. unique_positive_quadratic_above_threshold

The problem

For every natural cutoff c with $9 < c \leq 10$, there exists exactly one positive natural number x such that $x^2 + 4x + 4 < c$.

Lean statement

```
theorem unique_positive_quadratic_above_threshold (c : ℕ) (hc : c ≤ 10) (hcut : 9 < c) : ∃! x : ℕ, 0 < x ∧ x ^ 2 + 4 * x + 4 < c
```

Parent. For every natural number $c \leq 10$, let S be the finite set of positive natural numbers x such that $x^2 + 4x + 4 < c$. Show that S has cardinality 0 if $c \leq 9$ and cardinality 1 otherwise.

Parent in Lean

```
theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S → 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :  
  S.card = if c ≤ 9 then 0 else 1
```

What it contributes. Reuses the parent's bounded cutoff and positive quadratic predicate, while replacing its cardinality conclusion with an existence-and-uniqueness characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the cardinality conclusion into an explicit uniqueness statement and must prove that every positive witness equals 1. Although it shares the cutoff arithmetic with the parent, the parent's finite-set cardinality proof does not directly supply this $\exists!$ characterization.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the cardinality conclusion into a genuine uniqueness statement and proves that any positive witness must satisfy $y \leq 1$, then $y = 1$. Although the bound $c = 10$ and inequality argument parallel the parent, the parent's finset-cardinality proof does not directly establish this existential uniqueness claim.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a genuine change of shape from a cardinality equality to an existence-and-uniqueness characterization. Its proof must establish a witness and prove arbitrary positive candidates equal it, rather than merely recalling the parent's Finset-cardinality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1819 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_positive_quadratic_above_threshold.extracted_1_1 (c : ℕ) (hc : c ≤ 10) (hcut : 9 < c) :  
  ∃! x, 0 < x ∧ x ^ 2 + 4 * x + 4 < c
```

Read back by a model that saw only the Lean. For a natural number c satisfying $c \leq 10$ and $9 < c$, there exists exactly one natural number x such that $0 < x$ and $x^2 + 4x + 4 < c$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem unique_positive_quadratic_above_threshold (c : ℕ) (hc : c ≤ 10) (hcut : 9 < c) : ∃! x : ℕ, 0 < x ∧ x ^ 2 + 4 * x + 4 < c := by  
  have hc10 : c = 10 := by omega  
  have h_one : 0 < 1 ∧ 1 ^ 2 + 4 * 1 + 4 < c := by  
    norm_num [hc10]  
  refine ⟨1, h_one, ?_⟩  
  intro y hy  
  have hypos : 0 < y := hy.1  
  have hyineq : y ^ 2 + 4 * y + 4 < c := hy.2  
  have hyle : y ≤ 1 := by  
    by_contra h  
    have hy2 : 2 ≤ y := by omega  
    have hysq : 4 ≤ y ^ 2 := by  
      simpa [pow_two] using Nat.mul_le_mul hy2 hy2  
    omega  
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_405__me.me__9834fd26

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a3852ffd0f770e73

4.79 079. bounded_quadratic_cutoff_cardinality

The problem

For every natural number $c \leq 10$, let S be the finite set of positive natural numbers x such that $x^2 + 4x + 4 < c$. Show that S has cardinality 0 if $c \leq 9$ and cardinality 1 otherwise.

Lean statement

```
theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
  S.card = if c ≤ 9 then 0 else 1
```

Parent. For how many positive integer values of x is the sum $x^2 + 4x + 4$ less than 20? Show that it is 2.

Parent in Lean

```
theorem mathd_algebra_405 (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < 20) :
  S.card = 2 := by
```

What it contributes. Reused the parent's membership-biconditional encoding and finite-set extensional classification strategy, generalized to a bounded cutoff with a threshold case split.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the cutoff and classifies two threshold regimes, requiring separate arguments for emptiness when $c \leq 9$ and the singleton case $c = 10$. The parent's fixed $c = 20$ proof does not supply this bounded case analysis or its cardinality classification.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the cutoff and classifies both threshold cases, requiring a new case split and cardinality analysis for $c \leq 9$ versus $c = 10$. The parent's fixed-cutoff proof at 20 does not supply this reasoning or conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the cutoff and classifies two threshold regimes, requiring new reasoning for $c \leq 9$ and $c = 10$. The parent's fixed-cutoff proof does not supply this case analysis or the resulting cardinality classification.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1803 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_quadratic_cutoff_cardinality.extracted_1_1 (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ)
(h₀ : ∀ (x : ℕ), x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) : S.card = if c ≤ 9 then 0 else 1
```

Read back by a model that saw only the Lean. *Let c be a natural number with $c \leq 10$, and let S be a finite set of natural numbers such that, for every natural number x , x belongs to S if and only if $0 < x$ and $x^2 + 4x + 4 < c$. Then the cardinality of S equals 0 if $c \leq 9$, and equals 1 otherwise.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
S.card = if c ≤ 9 then 0 else 1 := by
  by_cases hcut : c ≤ 9
  · have hS : S = ∅ := by
      ext x
      rw [h₀]
      constructor
      · rintro ⟨hx, hineq⟩
        have hsquare : 1 ≤ x ^ 2 := by
          simp [pow_two] using Nat.mul_le_mul hx hx
        omega
      · intro hx
        simp at hx
    rw [hS]
    simp [hcut]
  · have hc10 : c = 10 := by omega
    have hS : S = {1} := by
      ext x
      rw [h₀]
      constructor
      · rintro ⟨hx, hineq⟩
        have hxle : x ≤ 1 := by
          by_contra h
          have hx2 : 2 ≤ x := by omega
          have hsquare : 4 ≤ x ^ 2 := by
            simp [pow_two] using Nat.mul_le_mul hx2 hx2
          omega
        have hxone : x = 1 := by omega
        simp [hxone]
      · intro hx
        have hxone : x = 1 := by simp using hx
        subst x
        norm_num [hc10]
    rw [hS]
    simp [hcut]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_405__me__71e06a20

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e507191f044f7129

4.80 080. translated_square_bounded_min_unique

The problem

For real numbers a, b, c, l, u with $a > 0$ and $l \leq b \leq u$, the function $x \mapsto a(x - b)^2 + c$ has least value c on $[l, u]$, attained uniquely at $x = b$.

Lean statement

```
theorem translated_square_bounded_min_unique (a b c l u : ℝ) (ha : 0 < a) (hl : l ≤ b) (hu : b ≤ u) : IsLeast {y : ℝ | ∃ x ∈ Set.Icc l u,
  ↪ y = a * (x - b) ^ 2 + c} c ∧ ∀ x ∈ Set.Icc l u, (a * (x - b) ^ 2 + c = c ↔ x = b)
```

Parent. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$ with $a > 0$. If $f(x) = a(x - b)^2 + c$, what is the minimum possible value of $f(x)$? Show that it is c .

Parent in Lean

```
theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↪ a * (x - b) ^ 2 + c) : IsLeast
  ↪ (Set.range f) c
```

What it contributes. Reuses the parent's completed-square lower-bound argument and positive-coefficient reasoning, while consuming interval hypotheses to prove a restricted minimum and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child restricts the range to a closed interval and additionally proves that equality at the minimum uniquely characterizes the vertex. The uniqueness direction requires using positivity of a and vanishing of the square, which is not supplied by the parent's `IsLeast` proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness characterization: equality at the minimum requires proving the square vanishes using positivity of a , while also restricting the range to an interval containing the vertex. The parent's least-value result does not supply this uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine unique-minimizer characterization: equality at the minimum requires proving that the positive coefficient forces the square to vanish, then deriving $x = b$. The interval hypotheses are also used materially to establish that the vertex lies in the bounded domain, so recalling the parent's least-range proof does not supply the full argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1783 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_bounded_min_unique.extracted_1_1 (a b c l u : ℝ) (ha : 0 < a) (hl : l ≤ b) (hu : b ≤ u) :  
  IsLeast {y | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c} c ∧  
  ∀ x ∈ Set.Icc l u, a * (x - b) ^ 2 + c = c ↔ x = b
```

Read back by a model that saw only the Lean. For real numbers a, b, c, l , and u , if $a > 0$, $l \leq b$, and $b \leq u$, then c is the least element of the set of all real numbers y for which there exists an $x \in [l, u]$ such that $y = a(x - b)^2 + c$; moreover, for every $x \in [l, u]$, $a(x - b)^2 + c = c$ if and only if $x = b$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_bounded_min_unique (a b c l u : ℝ) (ha : 0 < a) (hl : l ≤ b) (hu : b ≤ u) : IsLeast {y : ℝ | ∃ x ∈ Set.Icc l u,  
  ↪ y = a * (x - b) ^ 2 + c} c ∧ ∀ x ∈ Set.Icc l u, (a * (x - b) ^ 2 + c = c ↔ x = b) := by
  have hvertex : b ∈ Set.Icc l u := ⟨hl, hu⟩
  have hmem : c ∈ {y : ℝ | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c} := by
    refine ⟨b, hvertex, ?_⟩
    ring
  have hlower : ∀ y ∈ {y : ℝ | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c}, c ≤ y := by
    intro y hy
    rcases hy with ⟨x, hx, rfl⟩
    have hsq : 0 ≤ (x - b) ^ 2 := sq_nonneg (x - b)
    have hprod : 0 ≤ a * (x - b) ^ 2 := mul_nonneg (le_of_lt ha) hsq
    linarith
  have hleast : IsLeast {y : ℝ | ∃ x ∈ Set.Icc l u, y = a * (x - b) ^ 2 + c} c := by
    exact ⟨hmem, hlower⟩
  have hunique : ∀ x ∈ Set.Icc l u, (a * (x - b) ^ 2 + c = c ↔ x = b) := by
    intro x hx
    constructor
    · intro heq
      have hzero : a * (x - b) ^ 2 = 0 := by
        linarith
      have hsquare : (x - b) ^ 2 = 0 := by
        exact (mul_eq_zero.mp hzero).resolve_left (ne_of_gt ha)
      nlinarith
    · intro hxb
      subst x
      ring
      exact ⟨hleast, hunique⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__me.me__c2bdca76

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f1638f917bd7a73d

4.81 081. translated_completed_square_range

The problem

For real numbers a, b, c with $a > 0$, if $f(x) = a(x - b)^2 + c$ for all real x , prove that the range of f is exactly $[c, \infty)$; that is, every value of f is at least c , and every real number at least c is attained.

Lean statement

```
theorem translated_completed_square_range (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↦ a * (x - b) ^ 2 + c) : Set.range f =
  ↪ Set.Ici c
```

Parent. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$ with $a > 0$. If $f(x) = a(x - b)^2 + c$, what is the minimum possible value of $f(x)$? Show that it is c .

Parent in Lean

```
theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↦ a * (x - b) ^ 2 + c) : IsLeast
  ↪ (Set.range f) c
```

What it contributes. Supplies the positivity and square-nonnegativity lower-bound argument, adapted from least-value membership to global range containment.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes least-value existence into a full range characterization. Its reverse inclusion requires constructing a preimage using a symbolic square root and proving the scaled square equals $y - c$, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the entire range, requiring construction of a preimage using a square root and division by the positive scale factor. The parent only proves a lower bound and provides no surjectivity argument for every value at least c .

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a least-value claim into a full range characterization, requiring a new reverse-inclusion argument that constructs a preimage using a square root; the parent's lower-bound proof does not supply this.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1777 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_completed_square_range.extracted_1_1 (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a)
  (h₀ : f = fun x => a * (x - b) ^ 2 + c) : Set.range f = Set.Ici c
```

Read back by a model that saw only the Lean. Let f be a function from $\mathbb{R}$ to $\mathbb{R}$, and let a , b , and c be real numbers. Assume that $a > 0$ and that for every real number x , $f(x) = a(x - b)^2 + c$. Then the range of f is the set of all real numbers greater than or equal to c .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_completed_square_range (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x => a * (x - b) ^ 2 + c) : Set.range f =
  ↪ Set.Ici c := by
  subst f
  ext y
  constructor
  · rintro ⟨x, rfl⟩
    change c ≤ a * (x - b) ^ 2 + c
    have hparent_lower_bound : 0 ≤ a * (x - b) ^ 2 :=
      mul_nonneg (le_of_lt ha) (sq_nonneg (x - b))
    linarith
  · intro hy
    change c ≤ y at hy
    have hyc : 0 ≤ y - c := by linarith
    have ht : 0 ≤ (y - c) / a := div_nonneg hyc (le_of_lt ha)
    have hsqrt : (Real.sqrt ((y - c) / a)) ^ 2 = (y - c) / a :=
      Real.sq_sqrt ht
    have hdiv : a * ((y - c) / a) = y - c := by
      field_simp [ne_of_gt ha]
    have hcalc : a * (Real.sqrt ((y - c) / a)) ^ 2 = y - c := by
      rw [hsqrt, hdiv]
    refine ⟨b + Real.sqrt ((y - c) / a), ?_⟩
    dsimp
    nlinarith [hcalc]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__me__c6571295

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b03f0c818f06c94

4.82 082. translated_completed_square_least

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$ with $a > 0$. If $f(x) = a(x - b)^2 + c$, what is the minimum possible value of $f(x)$? Show that it is c .

Lean statement

```
theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x ↦ a * (x - b) ^ 2 + c) : IsLeast
  ↪ (Set.range f) c
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Generalizes the parent's completed-square witness and lower-bound argument from fixed coefficients to symbolic a , b , and c .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the quadratic and requires proving that the symbolic value c is attained at $x = b$ and is a lower bound using $a > 0$. The parent's fixed completing-square argument at 3 does not supply this generalized reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic and must prove the lower bound using positivity of an arbitrary coefficient and nonnegativity of a square. This requires the new multiplicative nonnegativity argument rather than recalling the parent's fixed completed-square proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic: it must use the symbolic minimizer b and establish nonnegativity of $a \cdot (x - b)^2$ from the new positivity hypothesis. The parent's fixed witness and square argument do not directly supply this generalized reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1771 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_completed_square_least.extracted_1_1 (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a)
(h₀ : f = fun x => a * (x - b) ^ 2 + c) : IsLeast (Set.range f) c
```

Read back by a model that saw only the Lean. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and let $a, b, c \in \mathbb{R}$. Assume that $a > 0$ and that for every real number x , $f(x) = a(x - b)^2 + c$. Then c is the least element of the range of f ; that is, c belongs to the range of f and $c \leq y$ for every y in the range of f .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_completed_square_least (f : ℝ → ℝ) (a b c : ℝ) (ha : 0 < a) (h₀ : f = fun x => a * (x - b) ^ 2 + c) : IsLeast
  (Set.range f) c := by
  subst f
  constructor
  · refine ⟨b, ?_⟩
    simp
  · rintro y ⟨x, rfl⟩
    have hprod : 0 ≤ a * (x - b) ^ 2 := mul_nonneg (le_of_lt ha) (sq_nonneg (x - b))
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__me__3cb02b9d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 93b50f44115c430f

4.83 083. quadratic_range_characterization

The problem

For the function $f(x) = x^2 - 6x + 13$, characterize its range. Show that a real number y lies in the range of f exactly when $y \geq 4$.

Lean statement

```
theorem quadratic_range_characterization
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  ∀ y : ℝ, y ∈ Set.range f ↔ 4 ≤ y := by
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Uses the parent quadratic and its lower-bound completion-of-the-square argument, extending it with an explicit witness for every admissible range value.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the minimum assertion into a full range characterization. Its converse direction requires constructing the explicit witness $3 + \sqrt{y-4}$ and proving it realizes every admissible value, which is not supplied by the parent's lower-bound argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum assertion into a full range characterization. Its reverse implication requires constructing the explicit witness using $\sqrt{y-4}$, reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child characterizes the entire range, including a new reverse-direction construction using the explicit witness $3 + \sqrt{y-4}$. The parent's minimum proof supplies only attainment at 4 and a lower bound, so recalling it does not solve the child's substantive existence argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2665 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_range_characterization.extracted_1_1 : ∀ (f : ℝ → ℝ),  
  (f = fun x => x ^ 2 - 6 * x + 13) → ∀ (y : ℝ), y ∈ Set.range f ↔ 4 ≤ y
```

Read back by a model that saw only the Lean. For every function f from $\mathbb{R}$ to $\mathbb{R}$, if $f(x) = x^2 - 6x + 13$ for all real x , then for every real number y , y belongs to the range of f if and only if $4 \leq y$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_range_characterization  
  (f : ℝ → ℝ)  
  (h₀ : f = fun x => x ^ 2 - 6 * x + 13) :  
  ∀ y : ℝ, y ∈ Set.range f ↔ 4 ≤ y := by  
  subst f  
  intro y  
  constructor  
  · rintro ⟨x, rfl⟩  
    have hsquare : 0 ≤ (x - 3) ^ 2 := sq_nonneg (x - 3)  
    nlinarith  
  · intro hy  
    have hnonneg : 0 ≤ y - 4 := by  
      linarith  
    have hsq : (Real.sqrt (y - 4)) ^ 2 = y - 4 := by  
      simpa using Real.sq_sqrt hnonneg  
    refine ⟨3 + Real.sqrt (y - 4), ?_⟩  
    nlinarith [hsq]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__me__b16e00b9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b16e00b9f30acda3

4.84 084. translated_square_strict_positivity

The problem

For real numbers c, d, x , prove that $d \leq (x - c)^2 + d$, with equality if and only if $x = c$.

Lean statement

```
theorem translated_square_strict_positivity (c d x : ℝ) :
  d ≤ (x - c) ^ 2 + d ∧ ((x - c) ^ 2 + d = d ↔ x = c)
```

Parent. For real numbers c, d and $r \geq 0$, the minimum of $(x - c)^2 + d$ on the interval $[c - r, c + r]$ is d , attained exactly at $x = c$.

Parent in Lean

```
theorem translated_square_minimum_interval (c d r : ℝ) (hr : 0 ≤ r) :
  IsLeast ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧
  ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →
    ((x - c) ^ 2 + d = d ↔ x = c)
```

What it contributes. Uses the parent's translated-square and equality-characterization checkpoints, generalized from interval-restricted minimization to a pointwise lower-bound characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuinely new universal pointwise characterization: it removes the interval restriction and `IsLeast` image claim, requiring the lower bound and equality condition for every real x . Its proof directly uses square nonnegativity and zero-square elimination, so recalling the parent's interval-minimum theorem does not supply the result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's interval-image minimum statement into a pointwise inequality and equality characterization for arbitrary real x . Its proof directly derives nonnegativity and zero-square elimination, without relying on the parent's interval or `IsLeast` argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the interval and `IsLeast` structure, proving a global pointwise lower-bound and equality characterization for arbitrary real x . Its proof independently derives nonnegativity and zero-square elimination, so recalling the parent theorem is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2727 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_strict_positivity.extracted_1_1 (c d x : ℝ) :  
  d ≤ (x - c) ^ 2 + d ∧ ((x - c) ^ 2 + d = d ↔ x = c)
```

Read back by a model that saw only the Lean. For all real numbers c , d , and x , d is less than or equal to $(x - c)^2 + d$, and $(x - c)^2 + d$ equals d if and only if x equals c .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem translated_square_strict_positivity (c d x : ℝ) :  
  d ≤ (x - c) ^ 2 + d ∧ ((x - c) ^ 2 + d = d ↔ x = c) := by  
  have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)  
  have hbound : d ≤ (x - c) ^ 2 + d := by  
    linarith  
  have heq : (x - c) ^ 2 + d = d ↔ x = c := by  
    constructor  
    · intro h  
      have hzero : x - c = 0 := by  
        nlinarith  
      linarith  
    · intro hxc  
      rw [hxc]  
      ring  
  exact ⟨hbound, heq⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.me.me.me__d0a0cd76

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d0a0cd76d20e84ec

4.85 085. translated_square_range_iff

The problem

For real numbers c , d , and y , the number y is in the range of $(x - c)^2 + d$ exactly when $d \leq y$.

Lean statement

```
theorem translated_square_range_iff (c d y : ℝ) :
  y ∈ Set.range (fun x : ℝ => (x - c) ^ 2 + d) ↔ d ≤ y
```

Parent. For real numbers c and d , the least value of $(x - c)^2 + d$ over all real x is d .

Parent in Lean

```
theorem translated_square_minimum_uniform (c d : ℝ) :
  IsLeast (Set.range (fun x : ℝ => (x - c) ^ 2 + d)) d
```

What it contributes. Supplied the lower-bound argument via nonnegativity of the translated square; the mutation changes the minimum claim into a full range characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from identifying a minimum to characterizing the entire range. Its reverse direction requires constructing an arbitrary preimage with a square-root witness, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a minimum-existence result into a full range characterization. Although the lower-bound direction resembles the parent, the reverse direction requires constructing an explicit square-root witness and applying `Real.sq_sqrt`, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum assertion into a full range characterization. Its reverse direction requires constructing the explicit witness $c + \sqrt{y - d}$ and using `Real.sq_sqrt`, which the parent proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2721 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_range_iff.extracted_1_1 (c d y : ℝ) :
  (y ∈ Set.range fun x => (x - c) ^ 2 + d) ↔ d ≤ y
```

Read back by a model that saw only the Lean. For real numbers c , d , and y , y belongs to the range of the function that maps each real number x to $(x - c)^2 + d$ if and only if $d \leq y$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_range_iff (c d y : ℝ) :
  y ∈ Set.range (fun x : ℝ => (x - c) ^ 2 + d) ↔ d ≤ y := by
  constructor
  · intro hy
    rcases hy with ⟨x, hx⟩
    have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)
    nlinarith
  · intro hdy
    have hnonneg : 0 ≤ y - d := sub_nonneg.mpr hdy
    have hsquare : (Real.sqrt (y - d)) ^ 2 = y - d := Real.sq_sqrt hnonneg
    have hwitness : ((c + Real.sqrt (y - d)) - c) ^ 2 + d = y := by
      nlinarith
    exact ⟨c + Real.sqrt (y - d), hwitness⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.me.me__3cf2d608

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3cf2d608076357ba

4.86 086. translated_square_minimum_interval

The problem

For real numbers c, d and $r \geq 0$, the minimum of $(x - c)^2 + d$ on the interval $[c - r, c + r]$ is d , attained exactly at $x = c$.

Lean statement

```
theorem translated_square_minimum_interval (c d r : ℝ) (hr : 0 ≤ r) :
  IsLeast ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧
  ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →
    ((x - c) ^ 2 + d = d ↔ x = c)
```

Parent. For real numbers c and d , the least value of $(x - c)^2 + d$ over all real x is d .

Parent in Lean

```
theorem translated_square_minimum_uniform (c d : ℝ) :
  IsLeast (Set.range (fun x : ℝ => (x - c) ^ 2 + d)) d
```

What it contributes. Reuses the translated-square attainment and nonnegative-square lower-bound reasoning, strengthened to an interval image and an equality-locus conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the domain to a bounded interval and adds a genuine equality-locus characterization, requiring proof that equality occurs exactly at $x = c$. This is not recoverable by merely recalling the parent's minimum proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine equality-locus characterization, requiring a new argument that a square vanishes only when $x = c$. The parent's minimum proof does not establish this claim, and the interval restriction is correctly handled by proving that c lies in the interval.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine equality-locus characterization on a restricted interval, requiring proving that a square vanishes exactly at the center. This is not supplied by recalling the parent's minimum proof, and the interval restriction also requires establishing center membership.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2719 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_minimum_interval.extracted_1_1 (c d r : ℝ) (hr : 0 ≤ r) :  
  IsLeast ((fun x => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧  
    ∀ x ∈ Set.Icc (c - r) (c + r), (x - c) ^ 2 + d = d ↔ x = c
```

Read back by a model that saw only the Lean. For real numbers c, d , and r with $0 \leq r$, the number d is the least element of the set of values $\{(x - c)^2 + d \mid c - r \leq x \leq c + r\}$; moreover, for every x satisfying $c - r \leq x \leq c + r$, $(x - c)^2 + d = d$ if and only if $x = c$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem translated_square_minimum_interval (c d r : ℝ) (hr : 0 ≤ r) :  
  IsLeast ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) d ∧  
    ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →  
      ((x - c) ^ 2 + d = d ↔ x = c) := by  
  have hcenter : c ∈ Set.Icc (c - r) (c + r) := by  
    constructor <=> linarith  
  have h_attain : d ∈ ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) := by  
    refine ⟨c, hcenter, ?_⟩  
    ring  
  have h_lower : ∀ {y : ℝ},  
    y ∈ ((fun x : ℝ => (x - c) ^ 2 + d) '' Set.Icc (c - r) (c + r)) → d ≤ y := by  
    intro y hy  
    rcases hy with ⟨x, hx, rfl⟩  
    have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)  
    nlinarith  
  have h_eq : ∀ x : ℝ, x ∈ Set.Icc (c - r) (c + r) →  
    ((x - c) ^ 2 + d = d ↔ x = c) := by  
    intro x hx  
    rcases hx with ⟨hl, hu⟩  
    constructor  
    · intro hval  
      have hzero : x - c = 0 := by  
        nlinarith [sq_nonneg (x - c), hl, hu]  
      linarith  
    · intro hxc  
      rw [hxc]  
      ring  
  constructor  
  · exact ⟨h_attain, h_lower⟩  
  · exact h_eq
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.me.me__fe304b63

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fe304b6318eac288

4.87 087. completed_square_level_right

The problem

For $y = (x - 3)^2 + 4$, show that every level $t \geq 4$ is attained at exactly one point $x \geq 3$.

Lean statement

```
theorem completed_square_level_right
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4)
  (t : ℝ)
  (ht : 4 ≤ t) :
  ∃! x : ℝ, 3 ≤ x ∧ f x = t := by
```

Parent. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Retains the parent's completed-square representation and nonnegativity checkpoint, while replacing the fixed minimum claim with a parameterized inverse-level existence and uniqueness theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child parameterizes the attained level and proves existence and uniqueness on the right-hand ray using square-root construction and sign elimination. The parent only establishes the minimum value and supplies neither the level witness nor uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the attained level and proves existence and uniqueness on the ray $x \geq 3$, requiring square-root construction and exclusion of the negative root. The parent's minimum argument does not supply this characterization or its uniqueness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the unique right-hand point attaining every level $t \geq 4$, requiring square-root construction and a two-factor uniqueness argument beyond the parent's minimum proof. It follows the planned parameterized bounded generalization and is not solvable by recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2695 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem completed_square_level_right.extracted_1.1 (f : ℝ → ℝ) (h : f = fun x => (x - 3) ^ 2 + 4) (t : ℝ) (ht : 4 ≤ t) :
  ∃! x, 3 ≤ x ∧ f x = t
```

Read back by a model that saw only the Lean. *Let f be a function from the real numbers to the real numbers such that $f(x) = (x - 3)^2 + 4$ for every real number x . For any real number t with $4 \leq t$, there exists exactly one real number x such that $3 \leq x$ and $f(x) = t$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem completed_square_level_right
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4)
  (t : ℝ)
  (ht : 4 ≤ t) :
  ∃! x : ℝ, 3 ≤ x ∧ f x = t := by
  subst f
  have hnonneg : 0 ≤ t - 4 := by
    linarith
  have hsqrt : 0 ≤ Real.sqrt (t - 4) := Real.sqrt_nonneg _
  have hsquare : (Real.sqrt (t - 4)) ^ 2 = t - 4 := by
    rw [Real.sq_sqrt hnonneg]
  refine (3 + Real.sqrt (t - 4), ?_, ?_)
  · constructor
    · linarith
    · nlinarith
  · intro y hy
    rcases hy with ⟨hy3, hfy⟩
    have hy_sq : (y - 3) ^ 2 = t - 4 := by
      nlinarith [hfy]
    have hprod : ((y - 3) - Real.sqrt (t - 4)) * ((y - 3) + Real.sqrt (t - 4)) = 0 := by
      nlinarith [hy_sq, hsquare]
    rcases mul_eq_zero.mp hprod with hdiff | hsum
    · nlinarith
    · nlinarith [hy3, hsqrt]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me__0ee00499

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 0ee00499097b34fd

4.88 088. translated_square_minimum

The problem

For any real number c , the least value of $(x - c)^2 + 4$ over all real x is 4.

Lean statement

```
theorem translated_square_minimum
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4
```

Parent. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x => (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Generalized the parent's completed-square range-minimum argument by replacing the fixed vertex 3 with an arbitrary real translation parameter c .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the vertex: the range witness must change from the fixed value 3 to the arbitrary real c , requiring symbolic ring reasoning rather than direct normalization. No hypothesis is unused, and the parent proof cannot be copied unchanged.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the vertex: the least-value witness changes from 3 to the symbolic parameter c , and the lower-bound argument must remain symbolic. A solver recalling the parent's fixed-3 proof cannot apply it unchanged.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed vertex 3 is genuinely replaced by an arbitrary real parameter c , and the proof must establish attainment at the symbolic point $x=c$ while preserving the lower-bound argument. This is the requested parameterization, not a decorative wrapper or unused hypothesis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2700 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_minimum.extracted_1_1 (c : ℝ) (f : ℝ → ℝ) (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4
```

Read back by a model that saw only the Lean. *Let c be a real number and let f be a function from the real numbers to the real numbers such that, for every real number x , $f(x) = (x - c)^2 + 4$. Then 4 is the least element of the range of f : 4 belongs to the range of f , and for every y in the range of f , $4 \leq y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_minimum
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
  subst f
  constructor
  · use c
    ring
  · rintro y ⟨x, rfl⟩
    have hsq : 0 ≤ (x - c) ^ 2 := sq_nonneg (x - c)
    nlinarith
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me__2590095a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 2590095a9abbd33b

4.89 089. positive_base_cube_logb

The problem

For a real number $b > 1$, evaluate $\log_b(b^3)$. Show that it is 3.

Lean statement

```
theorem positive_base_cube_logb (b : ℝ) (hb : 1 < b) : Real.logb b (b ^ 3) = 3
```

Parent. Evaluate $\log_3 27$. Show that it is 3.

Parent in Lean

```
theorem mathd_algebra_484 : Real.logb (3 : ℝ) (27 : ℝ) = 3 := by
```

What it contributes. Generalized the parent's concrete logb evaluation pattern from base 3 and input 27 to the parameterized cube input b^3 , retaining the logarithm nonzero obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed-base evaluation: it must derive ' $\text{Real.log } b \neq 0$ ' from the symbolic hypothesis ' $1 < b$ ' and then establish the cube law for every such base. The parent's concrete normalization cannot be reused without this variable-dependent argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed-base computation to arbitrary real $b > 1$. Its proof must derive nonzero $\log b$ from the symbolic positivity hypothesis, so the parent's numerical normalization is insufficient by itself.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and proves the cube law for every real $b > 1$. Its proof requires deriving $\log b \neq 0$ from positivity and applying Real.log_pow symbolically, so recalling the fixed-base parent does not supply the argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1708 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem positive_base_cube_logb.extracted_1_1 (b : ℝ) (hb : 1 < b) : logb b (b ^ 3) = 3
```

Read back by a model that saw only the Lean. For a real number b with $1 < b$, the logarithm base b of b cubed is equal to 3.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem positive_base_cube_logb (b : ℝ) (hb : 1 < b) : Real.logb b (b ^ 3) = 3 := by
  have hlogpos : 0 < Real.log b := Real.log_pos hb
  have hlog : Real.log b ≠ 0 := ne_of_gt hlogpos
  rw [Real.logb, Real.log_pow]
  field_simp [hlog]
  norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_484__me__a80c851b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8fd9ebcfe21a83f9

4.90 090. gcd_lcm_set_empty

The problem

Let x be a positive integer. Then the set of integers b such that the greatest common divisor of b and x is $x + 3$ and the least common multiple of b and x is $x(x + 3)$ is empty.

Lean statement

```
theorem gcd_lcm_set_empty
  (x : ℤ)
  (h₀ : 0 < x)
  (Sa : Set ℤ)
  (hSa : Sa = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)}) :
  Sa = ∅
```

Parent. The greatest common divisor of two integers is $(x + 3)$ and their least common multiple is $x(x + 3)$, where x is a positive integer. If one of the integers is 40, what is the smallest possible value of the other one? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_126
  (x a : ℤ)
  (h₀ : 0 < x)
  (h₁ : x = 40 ∨ a = 40)
  (Sa Sx : Set ℤ)
  (hSa : Sa = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)})
  (hSx : Sx = {y | 0 < y ∧ Int.gcd a y = y + 3 ∧ Int.lcm a y = y * (y + 3)})
  (h₂ : Nonempty Sa)
  (h₃ : Nonempty Sx) :
  IsLeast Sa 8 ∨ IsLeast Sx 8 := by
```

What it contributes. Uses the parent's gcd-divides-right checkpoint and the same symbolic gcd/lcm set definition, while changing the final obligation to set emptiness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent's fixed $x = 40$ contradiction to arbitrary positive x : $x + 3$ divides x , which is impossible by positivity and multiplier sign cases. Its set-emptiness conclusion and proof structure are materially different from the parent's disjunctive least-element argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent's fixed $x = 40$ contradiction to arbitrary positive x , proving that $x + 3$ cannot divide x via a sign and multiplier argument. Its set-emptiness conclusion and proof are not recoverable by recalling the parent's disjunctive least-element proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's vacuous disjunction with a symbolic emptiness theorem and generalizes the divisor contradiction from $x = 40$ to arbitrary positive x . Its proof requires a new sign-and-divisibility argument, so recalling the parent's case split and final contradiction is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1793 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gcd_lcm_set_empty.extracted_1_1 (x : ℤ) (h₀ : 0 < x) (Sa : Set ℤ)
  (hSa : Sa = {b | ↑(b.gcd x) = x + 3 ∧ ↑(b.lcm x) = x * (x + 3)}) : Sa = ∅
```

Read back by a model that saw only the Lean. *For an integer x with $0 < x$, and a set Sa of integers satisfying $Sa = \{b \in \mathbb{Z} \mid \gcd(b, x) = x + 3 \text{ and } \text{lcm}(b, x) = x(x + 3)\}$, the set Sa is empty.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem gcd_lcm_set_empty
  (x : ℤ)
  (h₀ : 0 < x)
  (Sa : Set ℤ)
  (hSa : Sa = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)}) :
  Sa = ∅ := by
  rw [hSa]
  ext b
  constructor
  · intro hb
    have hd : (Int.gcd b x : ℤ) | x := Int.gcd_dvd_right b x
    have heq : (Int.gcd b x : ℤ) = x + 3 := by
      simpa using hb.1
    rw [heq] at hd
    rcases hd with ⟨k, hk⟩
    have hpos : 0 < x + 3 := by omega
    rcases le_or_gt k 0 with hk_nonpos | hk_pos
    · have hmul : (x + 3) * k ≤ 0 :=
        mul_nonpos_of_nonneg_of_nonpos (by omega) hk_nonpos
      omega
    · have hdiff : 0 ≤ (x + 3) * (k - 1) :=
        mul_nonneg (by omega) (by omega)
      have hbound : x + 3 ≤ (x + 3) * k := by
        nlinarith
      omega
  · intro hb
    simp at hb
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__me__b6f20679

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 14622aac7d81ca1c

4.91 091. bounded_offset_gcd_lcm_sign

The problem

Let y be positive and let the integer offset c satisfy $-10 \leq c \leq 10$. If the greatest common divisor of 40 and y is $y + c$, while their least common multiple is $y(y + c)$, prove that $c \leq 0$.

Lean statement

```
theorem bounded_offset_gcd_lcm_sign
  (y c : ℤ)
  (hy : 0 < y)

  (hg : (Int.gcd 40 y : ℤ) = y + c)
  :
  c ≤ 0
```

Parent. There is no positive integer y and positive integer offset $c \leq 10$ such that the greatest common divisor of 40 and y is $y + c$ while their least common multiple is $y(y + c)$.

Parent in Lean

```
theorem bounded_offset_gcd_lcm_impossible
  (y c : ℤ)
  (hy : 0 < y)
  (hc : 0 < c)

  (hg : (Int.gcd 40 y : ℤ) = y + c)
  (hl : (Int.lcm 40 y : ℤ) = y * (y + c)) :
  False
```

What it contributes. Uses the parent's gcd divisibility checkpoint and the same gcd/lcm configuration, while changing the conclusion from impossibility to the signed characterization $c \leq 0$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child drops the positivity and lcm hypotheses and proves a new sign conclusion from gcd divisibility alone. Its proof uses a divisibility and quotient-sign argument, so recalling the parent's contradiction proof does not substantially help.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded-sign characterization: from the gcd divisibility it derives that a positive offset would make a positive divisor larger than y , without using the parent's lcm identity or square contradiction. It therefore cannot be solved by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different sign characterization: from the gcd equation alone, it derives that a positive offset would force the positive divisor $y+c$ to divide the smaller positive y . It drops the lcm contradiction entirely and requires a new divisibility and quotient-sign argument, so recalling the parent proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2545 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_offset_gcd_lcm_sign.extracted_1_1 (y c : ℤ) (hy : 0 < y) (hbound : -10 ≤ c ∧ c ≤ 10)
(hg : ↑(Int.gcd 40 y) = y + c) (hl : ↑(Int.lcm 40 y) = y * (y + c)) : c ≤ 0
```

Read back by a model that saw only the Lean. *For integers y and c , if y is positive, c lies between -10 and 10 inclusive, the greatest common divisor of 40 and y equals $y + c$, and the least common multiple of 40 and y equals $y(y + c)$, then $c \leq 0$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_offset_gcd_lcm_sign
  (y c : ℤ)
  (hy : 0 < y)

  (hg : (Int.gcd 40 y : ℤ) = y + c)
  :
  c ≤ 0 := by
  by_contra hnot
  have hc : 0 < c := by omega
  have hgc : (Int.gcd 40 y : ℤ) | y :=
    Int.gcd_dvd_right 40 y
  have hdiv : y + c | y := by
    rw [← hg]
    exact hgc
  rcases hdiv with ⟨k, hk⟩
  have hsum : 0 < y + c := by omega
  have hk_nonneg : 0 ≤ k := by
    by_contra hkn
    have hkneg : k < 0 := by omega
    have hprodneg : (y + c) * k < 0 :=
      mul_neg_of_pos_of_neg hsum hkneg
    nlinarith [hk, hprodneg, hy]
  have hkpos : 0 < k := by
    by_contra hknot
    have hkzero : k = 0 := by omega
    subst k
    norm_num at hk
    omega
  have hkone : 1 ≤ k := by omega
  have hmul_nonneg : 0 ≤ (y + c) * (k - 1) := by
    apply mul_nonneg
    · omega
    · omega
  have hle : y + c ≤ (y + c) * k := by
    nlinarith [hmul_nonneg]
  nlinarith [hk, hle, hc]
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh.me.me__77c02b0d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1d9ab47563e631cc

4.92 092. gcd_strictly_exceeds_candidate_impossible

The problem

There is no positive integer y whose greatest common divisor with 40 is greater than y and whose least common multiple with 40 equals the product of y and that greatest common divisor.

Lean statement

```
theorem gcd_strictly_exceeds_candidate_impossible
  (y : ℤ)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : ℤ) > y)
  :
  False
```

Parent. There is no positive integer y whose greatest common divisor with 40 is $y + 3$ and whose least common multiple with 40 is $y(y + 3)$.

Parent in Lean

```
theorem mathd_numbertheory_126_specialized
  (y : ℤ)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : ℤ) = y + 3)
  (hl : (Int.lcm 40 y : ℤ) = y * (y + 3)) :
  False
```

What it contributes. Retains the positive integer and gcd/lcm configuration and reuses the parent's gcd-divisibility check-point, while replacing the offset equation with a strict gcd-versus-candidate obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's fixed-offset gcd/lcm argument with a genuinely different contradiction: since $\text{gcd}(40, y)$ divides positive y , it cannot exceed y . The lcm hypothesis and the parent's product identity are entirely unnecessary.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed gcd/lcm equations with a strict inequality and requires a new argument that the gcd divides y , hence cannot exceed positive y . The parent's equation-based proof does not provide this reasoning, and the child is not a wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed gcd offset and lcm coupling with the independent condition that $\text{gcd}(40, y) > y$, then proves impossibility directly from $\text{gcd}(40, y)$ dividing y and $y > 0$. This requires a different divisibility-and-bound argument and does not recall the parent's product identity or case analysis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2525 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gcd_strictly_exceeds_candidate_impossible.extracted_1_1 (y : Z) (hy : 0 < y) (hg : ↑(Int.gcd 40 y) > y)
(hl : ↑(Int.lcm 40 y) = y * ↑(Int.gcd 40 y)) : False
```

Read back by a model that saw only the Lean. *Let y be an integer such that $0 < y$. Suppose that the integer cast of $\text{gcd}(40, y)$ is greater than y , and that the integer cast of $\text{lcm}(40, y)$ equals y multiplied by the integer cast of $\text{gcd}(40, y)$. Then a contradiction follows; that is, False.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem gcd_strictly_exceeds_candidate_impossible
  (y : Z)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : Z) > y)
  :
  False := by
  have hd : (Int.gcd 40 y : Z) | y :=
    Int.gcd_dvd_right 40 y
  have hnonneg : 0 ≤ (Int.gcd 40 y : Z) := by
    exact_mod_cast (Nat.zero_le (Int.gcd 40 y))
  have hbound : (Int.gcd 40 y : Z) ≤ y := by
    rcases hd with ⟨k, hk⟩
    have hk_pos : 0 < k := by
      by_contra hkp
      have hk_nonpos : k ≤ 0 := le_of_not_gt hkp
      have hprod : (Int.gcd 40 y : Z) * k ≤ 0 :=
        mul_nonpos_of_nonneg_of_nonpos hnonneg hk_nonpos
      have hy_nonpos : y ≤ 0 := by
        calc
          y = (Int.gcd 40 y : Z) * k := hk
          _ ≤ 0 := hprod
      omega
    have hk_one : (1 : Z) ≤ k := by omega
    have hmul : (Int.gcd 40 y : Z) * 1 ≤ (Int.gcd 40 y : Z) * k :=
      mul_le_mul_of_nonneg_left hk_one hnonneg
    calc
      (Int.gcd 40 y : Z) = (Int.gcd 40 y : Z) * 1 := by ring
      _ ≤ (Int.gcd 40 y : Z) * k := hmul
      _ = y := by symm; exact hk
    exact (not_lt_of_ge hbound) hg
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh.me__a7839e6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9116a430db6e9204

4.93 093. odd_power_sum_last_digit

The problem

Show that for every natural number n , the sum $19^{2n+1} + 99^{2n+1}$ has last digit 8.

Lean statement

```
theorem odd_power_sum_last_digit : ∀ n : ℕ, (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) % 10 = 8
```

Parent. What is the units digit of $19^{19} + 99^{99}$? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_202 : (19 ^ 19 + 99 ^ 99) % 10 = 8 := by
```

What it contributes. The parent supplies the fixed two-base last-digit pattern; this mutation generalizes it to all explicitly odd exponents and proves periodicity for both bases.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponents to all odd exponents and requires an induction showing multiplication by the square preserves the last digit. The parent's 'norm_num' computation supplies no such parametric argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent to all odd exponents and requires an induction showing each power remains 9 modulo 10. The parent's one-shot norm_num computation provides no proof of this law, so recalling it does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponents to all odd exponents and requires an induction showing multiplication by the square preserves the last digit. The parent's one-shot norm_num computation cannot be recalled to supply this universal law.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1867 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_power_sum_last_digit.extracted_1_1 : ∀ (n : ℕ), (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) % 10 = 8
```

Read back by a model that saw only the Lean. For every natural number n , the remainder when 19 raised to the power $2n+1$ plus 99 raised to the power $2n+1$ is divided by 10 is 8.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem odd_power_sum_last_digit : ∀ n : ℕ, (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) % 10 = 8 := by
  intro n
  have h19 : ∀ k : ℕ, 19 ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  have h99 : ∀ k : ℕ, 99 ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  have h19n := h19 n
  have h99n := h99 n
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__me__2aac1b65

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropt, Quot.sound, fingerprint e434ffffdaed70cc

4.94 094. gcd_odd_power_sum

The problem

For every natural number n , the greatest common divisor of $19^{2n+1} + 99^{2n+1}$ and 10 is 2.

Lean statement

```
theorem gcd_odd_power_sum (n : ℕ) : Nat.gcd (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) 10 = 2 := by
```

Parent. Show that the units digit of $19^{19} + 99^{99}$ is congruent to 8 modulo 10.

Parent in Lean

```
theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from one fixed modular computation to a universal exact gcd characterization. It generalizes the modulo-10 periodicity to arbitrary odd exponents and uses that residue to determine the gcd, so recalling the parent does not substantially solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponents to every odd exponent and changes the conclusion to an exact gcd characterization. Its proof derives a reusable modulo-10 power pattern, establishes the gcd through a nontrivial modular reduction, and cannot be obtained by recalling the parent's one-shot norm_num proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent congruence to all odd exponents and changes the conclusion to an exact gcd characterization. Its proof must establish divisibility by 2 and exclusion of divisibility by 5, rather than recalling the parent's single numerical normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2762 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gcd_odd_power_sum.extracted_1_1 (n : ℕ) : (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)).gcd 10 = 2
```

Read back by a model that saw only the Lean. For every natural number n , the greatest common divisor of $19^{(2n+1)} + 99^{(2n+1)}$ and 10 is equal to 2.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem gcd_odd_power_sum (n : ℕ) : Nat.gcd (19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)) 10 = 2 := by
  let x := 19 ^ (2 * n + 1) + 99 ^ (2 * n + 1)
  change Nat.gcd x 10 = 2
  have h19 : 19 ≡ 9 [MOD 10] := by
    norm_num [Nat.ModEq]
  have h99 : 99 ≡ 9 [MOD 10] := by
    norm_num [Nat.ModEq]
  have hbase : 9 ^ 2 ≡ 1 [MOD 10] := by
    norm_num [Nat.ModEq]
  have hpow : 9 ^ (2 * n) ≡ 1 [MOD 10] := by
    rw [pow_mul]
    simpa using hbase.pow n
  have hodd : 9 ^ (2 * n + 1) ≡ 9 [MOD 10] := by
    rw [pow_add]
    simpa using hpow.mul (by norm_num [Nat.ModEq] : 9 ^ 1 ≡ 9 [MOD 10])
  have hsum : x ≡ 9 ^ (2 * n + 1) + 9 ^ (2 * n + 1) [MOD 10] := by
    dsimp [x]
    exact (h19.pow (2 * n + 1)).add (h99.pow (2 * n + 1))
  have hmod : x % 10 = 8 := by
    have ht : x ≡ 9 + 9 [MOD 10] := hsum.trans (hodd.add hodd)
    norm_num [Nat.ModEq] at ht
    exact ht
  have hxmod : x ≡ 8 [MOD 10] := by
    simpa [Nat.ModEq] using hmod
  calc
    Nat.gcd x 10 = Nat.gcd 8 10 := Nat.ModEq.gcd_eq hxmod
    _ = 2 := by norm_num
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__ms.me__a5ada452

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a5ada4520e402853

4.95 095. bounded_first_period_exponent

The problem

Let n be a positive integer with $n \leq 6$. Show that 5^n is congruent to 1 modulo 7 if and only if $n = 6$.

Lean statement

```
theorem bounded_first_period_exponent (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Supplies the modular-power checkpoint pattern, specialized to the first period representative $5^6 \equiv 1$ modulo 7.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child characterizes all positive exponents up to 6 for which $5^n \equiv 1 \pmod{7}$, requiring a case analysis and establishing the minimal period 6. The parent only verifies the isolated exponent 30 and supplies none of this bounded characterization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single fixed residue computation into a bounded first-period characterization. It requires ruling out exponents 1 through 5 and separately proving the exponent-6 case, which the parent's computation of $5^{30} \equiv 1$ does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed residue computation into a bounded characterization of the first positive exponent, requiring an explicit case split over $n = 1$ through 6 and exclusion of the first five cases. The parent's proof of $5^{30} \equiv 1$ does not supply this minimality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1783 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_first_period_exponent.extracted_1_1 (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) :
  5 ^ n ≡ 1 [ZMOD 7] ↔ n = 6
```

Read back by a model that saw only the Lean. For a natural number n with $0 < n$ and $n \leq 6$, 5^n is congruent to 1 modulo 7 if and only if $n = 6$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_first_period_exponent (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 := by
  have hperiod : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  constructor
  · intro h
    have hcases : n = 1 ∨ n = 2 ∨ n = 3 ∨ n = 4 ∨ n = 5 ∨ n = 6 := by
      omega
    rcases hcases with h1 | h2 | h3 | h4 | h5 | h6
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · subst n
      norm_num [Int.ModEq] at h
    · exact h6
  · intro h
    subst n
    exact hperiod
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__me__a3c3ca0e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 9a5af6fc5c1aa54d

4.96 096. mathd_numbertheory_229_mutation

The problem

For every integer base whose residue modulo 7 is nonzero, its sixth power is congruent to 1 modulo 7.

Lean statement

```
theorem mathd_numbertheory_229_mutation (n : ℤ) (h : (n : ℤMod 7) ≠ 0) : n ^ 6 ≡ 1 [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Generalizes the parent’s modulo-7 power computation from base 5 and exponent 30 to an arbitrary nonzero residue and the underlying exponent-six period.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a single numerical computation with the general Fermat-law claim that every nonzero residue modulo 7 has sixth power 1. Its proof requires reasoning in $\mathbb{ZMod}\ 7$ and translating that result back to integer congruence, none of which is supplied by recalling the parent’s normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed computation to every integer base nonzero modulo 7, requiring Fermat-style reasoning in $\mathbb{ZMod}\ 7$ and conversion back to integer congruence. The parent’s direct numerical normalization supplies none of this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed computation to every integer nonzero modulo 7 and exponent 6, requiring Fermat’s little theorem and a conversion back to integer congruence. The parent’s direct normalization proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2667 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_229_mutation.extracted_1_1 (n : ℤ) (h : ↑n ≠ 0) : n ^ 6 ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer n whose residue modulo 7 is nonzero, the sixth power n^6 is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_229_mutation (n : ℤ) (h : (n : ℤMod 7) ≠ 0) : n ^ 6 ≡ 1 [ZMOD 7] := by
  letI : Fact (Nat.Prime 7) := ⟨by norm_num⟩
  have hn : (n : ℤMod 7) ^ 6 = 1 := by
    simpa using (ℤMod.pow_card_sub_one_eq_one (p := 7) h)
  apply (Int.modEq_iff_dvd).2
  have hz : ((1 - n ^ 6 : ℤ) : ℤMod 7) = 0 := by
    norm_num [Int.cast_sub, Int.cast_pow]
    exact sub_eq_zero.mpr hn.symm
  exact (ℤMod.intCast_zmod_eq_zero_iff_dvd (1 - n ^ 6) 7).mp hz
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__me__c5ab2543

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c5ab2543f0de475e

4.97 097. bounded_five_power_add_one_gcd_iff

The problem

For an integer k with $0 \leq k \leq 6$, show that the greatest common divisor of 7 and $5^k + 1$ equals 7 exactly when $k = 3$.

Lean statement

```
theorem bounded_five_power_add_one_gcd_iff (k : ℕ) (hbound : k ≤ 6) :
  Nat.gcd 7 (5 ^ k + 1) = 7 ↔ k = 3
```

Parent. For an integer k with $0 \leq k \leq 6$, show that 7 divides $5^k + 1$ exactly when $k = 3$.

Parent in Lean

```
theorem bounded_five_power_add_one_dvd_iff (k : ℕ) (hbound : k ≤ 6) : 7 ∣ (5 ^ k + 1 : ℕ) ↔ k = 3 := by
```

What it contributes. Supplies the bounded iff characterization that 7 divides 5^k+1 exactly when $k=3$; this is consumed as `hdiv` and converted into the new `gcd` target.

Review

Every judgement this row received. Admission required `strong` from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine `gcd-divisibility` equivalence, requiring `divisibility antisymmetry` and `gcd properties` before applying the bounded power characterization. Recalling the parent supplies only one checkpoint and does not prove the new `gcd` statement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes `divisibility` into a genuine `gcd` characterization and proves the nontrivial bridge between `gcd equality` and `divisibility`. Although it rederives the bounded `divisibility` checkpoint, the parent's proof does not supply the `gcd` argument, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from `divisibility` to a `gcd equality` and proves a genuine bridge between the two claims via `gcd divisibility properties`. Although it reproves the bounded `divisibility` checkpoint by case analysis, the parent proof does not supply the new `gcd` argument, so recalling the parent alone is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2432 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_five_power_add_one_gcd_iff.extracted_1_1 (k : ℕ) (hbound : k ≤ 6) :  
  Nat.gcd 7 (5 ^ k + 1) = 7 ↔ k = 3
```

Read back by a model that saw only the Lean. For a natural number k satisfying $k \leq 6$, the greatest common divisor of 7 and $5^k + 1$ is equal to 7 if and only if $k = 3$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_five_power_add_one_gcd_iff (k : ℕ) (hbound : k ≤ 6) :  
  Nat.gcd 7 (5 ^ k + 1) = 7 ↔ k = 3 := by  
  have hdiv : 7 ∣ (5 ^ k + 1 : ℕ) ↔ k = 3 := by  
    interval_cases k <|> norm_num  
  have hrel : Nat.gcd 7 (5 ^ k + 1) = 7 ↔ 7 ∣ (5 ^ k + 1 : ℕ) := by  
    constructor  
    · intro heq  
      have hg : Nat.gcd 7 (5 ^ k + 1) ∣ (5 ^ k + 1 : ℕ) :=  
        Nat.gcd_dvd_right 7 (5 ^ k + 1)  
      rw [heq] at hg  
      exact hg  
    · intro hd  
      apply Nat.dvd_antisymm  
      · exact Nat.gcd_dvd_left 7 (5 ^ k + 1)  
      · exact Nat.dvd_gcd (dvd_refl 7) hd  
  exact hrel.trans hdiv
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__ms.mh.mh.me__e1eec4bc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint e1eec4bc2fd5b65f

4.98 098. fibonacci_mod_four_period

The problem

Let (a_n) begin with $a_1 = a_2 = 1$ and satisfy $a_{n+2} = a_{n+1} + a_n$. Prove that for every nonnegative integer k , the remainders of $a_{6k}, a_{6k+1}, \dots, a_{6k+5}$ upon division by 4 are respectively 0, 1, 1, 2, 3, 1.

Lean statement

```
theorem fibonacci_mod_four_period (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  ∀ k : ℕ,
    a (6 * k) % 4 = 0 ∧
    a (6 * k + 1) % 4 = 1 ∧
    a (6 * k + 2) % 4 = 1 ∧
    a (6 * k + 3) % 4 = 2 ∧
    a (6 * k + 4) % 4 = 3 ∧
    a (6 * k + 5) % 4 = 1
```

Parent. The Fibonacci sequence is the sequence 1, 1, 2, 3, 5, ... where each term is the sum of the previous two terms. What is the remainder when the 100th term of the sequence is divided by 4? Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_483 (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) : a 100 % 4 = 3 := by
```

What it contributes. Preserves the parent's Fibonacci recurrence setup and initial-value derivation, while generalizing the isolated residue computation into a six-class periodic induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a complete six-class modulo-4 periodic classification via recurrence-based induction, requiring modular transport across all six residues. This is substantive new reasoning rather than recalling the parent's identification of the sequence with `Nat.fib` and evaluating one fixed index.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a complete six-class modulo-4 classification for all indices, requiring a new induction that propagates six consecutive residues through six recurrence steps. This is genuine bounded generalization and cannot be solved by recalling the parent's isolated computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's isolated computation at index 100 with a complete six-class modulo-4 classification for every index block. Its induction derives and transports six new recurrence residues, so recalling the parent's Fibonacci-identification proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2556 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fibonacci_mod_four_period.extracted_1_1 : ∀ (a : ℕ → ℕ),
  a 1 = 1 →
  a 2 = 1 →
  (∀ (n : ℕ), a (n + 2) = a (n + 1) + a n) →
  ∀ (k : ℕ),
    a (6 * k) % 4 = 0 ∧
    a (6 * k + 1) % 4 = 1 ∧
    a (6 * k + 2) % 4 = 1 ∧ a (6 * k + 3) % 4 = 2 ∧ a (6 * k + 4) % 4 = 3 ∧ a (6 * k + 5) % 4 = 1
```

Read back by a model that saw only the Lean. *For every function a from the natural numbers to the natural numbers, if $a(1) = 1$, $a(2) = 1$, and for every natural number n , $a(n + 2) = a(n + 1) + a(n)$, then for every natural number k , the following congruences hold modulo 4: $a(6k)$ has remainder 0; $a(6k + 1)$ has remainder 1; $a(6k + 2)$ has remainder 1; $a(6k + 3)$ has remainder 2; $a(6k + 4)$ has remainder 3; and $a(6k + 5)$ has remainder 1.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem fibonacci_mod_four_period (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  ∀ k : ℕ,
    a (6 * k) % 4 = 0 ∧
    a (6 * k + 1) % 4 = 1 ∧
    a (6 * k + 2) % 4 = 1 ∧
    a (6 * k + 3) % 4 = 2 ∧
    a (6 * k + 4) % 4 = 3 ∧
    a (6 * k + 5) % 4 = 1 := by
  have ha0 : a 0 = 0 := by
    have h := h₂ 0
    norm_num [h₀, h₁] at h
    omega
  have ha3 : a 3 = 2 := by
    have h := h₂ 1
    norm_num [h₀, h₁] at h
    exact h
  have ha4 : a 4 = 3 := by
    have h := h₂ 2
    norm_num [h₁, ha3] at h
    exact h
  have ha5 : a 5 = 5 := by
    have h := h₂ 3
    norm_num [ha3, ha4] at h
    exact h
  intro k
  induction k with
  | zero =>
    norm_num [ha0, h₀, h₁, ha3, ha4, ha5]
  | succ k ih =>
    rcases ih with ⟨h0, h1, h2, h3, h4, h5⟩
    have r6 : a (6 * k + 6) = a (6 * k + 5) + a (6 * k + 4) := by
      have h := h₂ (6 * k + 4)
      convert h using 1 <|> omega
    have r7 : a (6 * k + 7) = a (6 * k + 6) + a (6 * k + 5) := by
      have h := h₂ (6 * k + 5)
      convert h using 1 <|> omega
    have r8 : a (6 * k + 8) = a (6 * k + 7) + a (6 * k + 6) := by
```

```

have h := h2 (6 * k + 6)
convert h using 1 <|> omega
have r9 : a (6 * k + 9) = a (6 * k + 8) + a (6 * k + 7) := by
  have h := h2 (6 * k + 7)
  convert h using 1 <|> omega
have r10 : a (6 * k + 10) = a (6 * k + 9) + a (6 * k + 8) := by
  have h := h2 (6 * k + 8)
  convert h using 1 <|> omega
have r11 : a (6 * k + 11) = a (6 * k + 10) + a (6 * k + 9) := by
  have h := h2 (6 * k + 9)
  convert h using 1 <|> omega
have h6 : a (6 * k + 6) % 4 = 0 := by omega
have h7 : a (6 * k + 7) % 4 = 1 := by omega
have h8 : a (6 * k + 8) % 4 = 1 := by omega
have h9 : a (6 * k + 9) % 4 = 2 := by omega
have h10 : a (6 * k + 10) % 4 = 3 := by omega
have h11 : a (6 * k + 11) % 4 = 1 := by omega
refine {?_, ?_, ?_, ?_, ?_, ?_}
· convert h6 using 1 <|> omega
· convert h7 using 1 <|> omega
· convert h8 using 1 <|> omega
· convert h9 using 1 <|> omega
· convert h10 using 1 <|> omega
· convert h11 using 1 <|> omega

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_483__me__d29b0035

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d29b0035a34bd2fc

4.99 099. four_consecutive_residue_law

The problem

For every natural number n , prove that there is a natural number k such that $n + (n + 1) + (n + 2) + (n + 3) = 4k + 2$ and the remainder when this sum is divided by 4 is 2.

Lean statement

```
theorem four_consecutive_residue_law (n : ℕ) : ∃ k : ℕ, n + (n + 1) + (n + 2) + (n + 3) = 4 * k + 2 ∧ (n + (n + 1) + (n + 2) + (n + 3)) % 4 = 2 := by
```

Parent. Find the remainder when $91145 + 91146 + 91147 + 91148$ is divided by 4. Show that it is 2.

Parent in Lean

```
theorem mathd_numbertheory_640 : (91145 + 91146 + 91147 + 91148) % 4 = 2 := by
```

What it contributes. Retains the four-consecutive-natural-number sum and modulo-4 conclusion while replacing the fixed numerical instance with a symbolic initial term and explicit algebraic decomposition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child establishes the symbolic identity $n + (n + 1) + (n + 2) + (n + 3) = 4(n + 1) + 2$ for arbitrary n , then derives the residue law. The parent's numerical norm_num proof provides no reasoning for this parameterized decomposition.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed residue computation: it proves symbolically that every four-consecutive-integer sum is $4(n+1)+2$, then derives the modulus law. The parent's numerical norm_num proof provides no reusable argument for this universal statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed numerical computation into a law for arbitrary n , requiring symbolic arithmetic and an explicit decomposition. The parent's norm_num proof offers no reusable argument for this general case.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1851 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem four_consecutive_residue_law.extracted_1_1 (n : ℕ) :  
  ∃ k, n + (n + 1) + (n + 2) + (n + 3) = 4 * k + 2 ∧ (n + (n + 1) + (n + 2) + (n + 3)) % 4 = 2
```

Read back by a model that saw only the Lean. *For every natural number n , there exists a natural number k such that $n + (n + 1) + (n + 2) + (n + 3) = 4k + 2$ and the remainder when $n + (n + 1) + (n + 2) + (n + 3)$ is divided by 4 is 2.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem four_consecutive_residue_law (n : ℕ) :  
  ∃ k : ℕ,  
    n + (n + 1) + (n + 2) + (n + 3) = 4 * k + 2 ∧  
    (n + (n + 1) + (n + 2) + (n + 3)) % 4 = 2 := by  
  refine ⟨n + 1, ?_, ?_⟩  
  · omega  
  · have hdecomp : n + (n + 1) + (n + 2) + (n + 3) = 4 * (n + 1) + 2 := by  
      omega  
    rw [hdecomp]  
    omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_640__me__625ab733

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 8ccadd1d869afcb4

4.100 100. modular_power_difference_general

The problem

Let a and b be the residues of integers A and B modulo 7. Show that for every natural number t , $A^{6t+1} - B^{6t+1} \equiv a - b \pmod{7}$.

Lean statement

```
theorem modular_power_difference_general (t : ℕ) (A B a b : ℤ) (hA : A ≡ a [ZMOD 7]) (hB : B ≡ b [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7]
```

Parent. For natural numbers $t \leq 100$, let A and B be integers with $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that $A^{6t+1} - B^{6t+1} \equiv 3 \pmod{7}$.

Parent in Lean

```
theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Supplied the period-reduction argument modulo 7 and the congruence-subtraction checkpoint; the fixed residues 1 and 5 were generalized to arbitrary residues a and b .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed residues to arbitrary symbolic residues a and b , so the final congruence must be transported symbolically rather than concluding the parent's fixed value 3. It also independently uses the exponent-period reduction and has no redundant hypotheses.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed residues 1 and 5 to arbitrary residues a and b , requiring both congruence hypotheses to be transported through the reduced powers and yielding the symbolic difference $a-b$. This is not decoration or recall because the parent cannot establish the generalized conclusion with its fixed-residue hypotheses.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed residues 1 and 5 to arbitrary symbolic residues a and b , requiring both congruence hypotheses to be transported through the reduced powers and combined symbolically. The parent theorem cannot be recalled to close this generalized conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2500 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_difference_general.extracted_1_1 (t : ℕ) (A B a b : ℤ) (hA : A ≡ a [ZMOD 7])  
(hB : B ≡ b [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number t and integers A, B, a , and b , if A is congruent to a modulo 7 and B is congruent to b modulo 7, then A raised to the power $6t+1$ minus B raised to the power $6t+1$ is congruent to a minus b modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_difference_general (t : ℕ) (A B a b : ℤ) (hA : A ≡ a [ZMOD 7]) (hB : B ≡ b [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7] := by
  have hperiodA : A ^ (6 * t + 1) ≡ A ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hperiodB : B ^ (6 * t + 1) ≡ B ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hAred : A ^ (6 * t + 1) ≡ a [ZMOD 7] := by
    simpa using hperiodA.trans (hA.pow 1)
  have hBred : B ^ (6 * t + 1) ≡ b [ZMOD 7] := by
    simpa using hperiodB.trans (hB.pow 1)
  have hdifference : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ a - b [ZMOD 7] := by
    exact hAred.sub hBred
  exact hdifference
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.me__ec6d6429

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ec6d6429c3fc1b30

4.101 101. modular_power_difference_characterization

The problem

For integers A , B , and C , and any natural number t , show that $A^{6t+1} - B^{6t+1} \equiv C \pmod{7}$ if and only if $A - B \equiv C \pmod{7}$.

Lean statement

```
theorem modular_power_difference_characterization (t : ℕ) (A B C : ℤ) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ C [ZMOD 7] ↔ A - B ≡ C [ZMOD 7]
```

Parent. Two integer readings differ from 1 and 5 by multiples of 7, respectively. Show that the difference of their $(6t+1)$ st powers leaves remainder 3 upon division by 7.

Parent in Lean

```
theorem power_difference_residue_law_divisibility (t : ℕ) (A B : ℤ) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Reuses the parent’s modulus-7 exponent reduction checkpoint, generalized from fixed residue representatives to arbitrary integers and a latent target residue C .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent’s fixed residue hypotheses and proves a reusable congruence equivalence for arbitrary A , B , and latent target C . Its forward and reverse directions require establishing and using the symbolic difference congruence, so the parent’s fixed-residue conclusion cannot be recalled directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the fixed residue hypotheses and proves a general latent-target equivalence. Its key periodicity argument applies to arbitrary A and B , then uses transitivity in both directions; the parent’s fixed-residue specialization cannot directly supply this characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the fixed residue hypotheses and proves a reusable equivalence for arbitrary latent target C . Its proof must establish the general power reduction and both directions of the residue characterization, which the parent’s fixed-residue transitivity proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2530 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_difference_characterization.extracted_1_1 (t : ℕ) (A B C : ℤ) :  
  A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ C [ZMOD 7] ↔ A - B ≡ C [ZMOD 7]
```

Read back by a model that saw only the Lean. *For every natural number t and integers A , B , and C , the difference A raised to the power $6t+1$ minus B raised to the power $6t+1$ is congruent to C modulo 7 if and only if A minus B is congruent to C modulo 7.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_difference_characterization (t : ℕ) (A B C : ℤ) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ C [ZMOD 7] ↔ A - B ≡ C [ZMOD 7] := by
  have hAperiod : A ^ (6 * t + 1) ≡ A [ZMOD 7] := by
    have h : A ^ (6 * t + 1) ≡ A ^ 1 [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
      · norm_num
      · use t
      · omega
      · omega
      · omega
    simp using h
  have hBperiod : B ^ (6 * t + 1) ≡ B [ZMOD 7] := by
    have h : B ^ (6 * t + 1) ≡ B ^ 1 [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
      · norm_num
      · use t
      · omega
      · omega
      · omega
    simp using h
  have hdifference : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ A - B [ZMOD 7] := by
    exact hAperiod.sub hBperiod
  constructor
  · intro h
    exact hdifference.symm.trans h
  · intro h
    exact hdifference.trans h
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.ms.me__cb5235b5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint cb5235b5b834e1eb

4.102 102. power_difference_residue_law_mod_class

The problem

Let n be a natural number congruent to 1 modulo 6. If integers A and B differ from 1 and 5 by multiples of 7, respectively, show that $A^n - B^n \equiv 3 \pmod{7}$.

Lean statement

```
theorem power_difference_residue_law_mod_class (n : ℕ) (A B : ℤ) (hn : n % 6 = 1) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ n - B ^ n ≡ 3 [ZMOD 7]
```

Parent. Two integer readings differ from 1 and 5 by multiples of 7, respectively. Show that the difference of their $(6t+1)$ st powers leaves remainder 3 upon division by 7.

Parent in Lean

```
theorem power_difference_residue_law_divisibility (t : ℕ) (A B : ℤ) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Reuses the parent's modulo-7 period reduction, congruence transport, subtraction, and residue computation, with the exponent independently quantified by $n \% 6 = 1$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponent from the explicit form $6t+1$ to an arbitrary n satisfying $n \bmod 6 = 1$. Its proof must derive the quotient decomposition and divisibility of $n-1$ before applying periodicity, so the parent proof cannot be directly recalled.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent: it must derive a representation of any bounded-congruence exponent n as $6(n/6)+1$ before applying periodicity. This is not supplied by recalling the parent, whose exponent already has the required form.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from the syntactic form $6t+1$ to an arbitrary exponent satisfying $n \bmod 6 = 1$. Its proof must derive the decomposition and divisibility needed for periodicity, so recalling the parent's proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2527 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_residue_low_mod_class.extracted_1_1 (n : ℕ) (A B : ℤ) (hn : n % 6 = 1) (hA : 7 ∣ 1 - A)
(hB : 7 ∣ 5 - B) : A ^ n - B ^ n ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For a natural number n and integers A and B , if n leaves remainder 1 upon division by 6, 7 divides $1 - A$, and 7 divides $5 - B$, then A raised to the n th power minus B raised to the n th power is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem power_difference_residue_low_mod_class (n : ℕ) (A B : ℤ) (hn : n % 6 = 1) (hA : (7 : ℤ) ∣ 1 - A) (hB : (7 : ℤ) ∣ 5 - B) : A ^ n -
  B ^ n ≡ 3 [ZMOD 7] := by
  have hAcon : A ≡ 1 [ZMOD 7] := (Int.modEq_iff_dvd).2 hA
  have hBcon : B ≡ 5 [ZMOD 7] := (Int.modEq_iff_dvd).2 hB
  have hdecomp : n = 6 * (n / 6) + 1 := by
    omega
  have hdiv : 6 ∣ n - 1 := by
    use n / 6
    omega
  have hpos : 0 < n := by
    omega
  have hle : 1 ≤ n := by
    omega
  have hperiodA : A ^ n ≡ A ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := n) (y := 1)
    · norm_num
    · exact hdiv
    · exact hle
    · norm_num
  have hperiodB : B ^ n ≡ B ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := n) (y := 1)
    · norm_num
    · exact hdiv
    · exact hle
    · norm_num
  have hAred : A ^ n ≡ (1 : ℤ) [ZMOD 7] := by
    simpa using hperiodA.trans (hAcon.pow 1)
  have hBred : B ^ n ≡ (5 : ℤ) [ZMOD 7] := by
    simpa using hperiodB.trans (hBcon.pow 1)
  have hdifference : A ^ n - B ^ n ≡ (1 : ℤ) - 5 [ZMOD 7] := by
    exact hAred.sub hBred
  calc
    A ^ n - B ^ n ≡ (1 : ℤ) - 5 [ZMOD 7] := hdifference
    _ ≡ 3 [ZMOD 7] := by norm_num [Int.ModEq]
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.ms.me__f1a7f310

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f1a7f3100b6b19a6

4.103 103. power_difference_residue_law

The problem

For natural numbers $t \leq 100$, let A and B be integers with $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that $A^{6t+1} - B^{6t+1} \equiv 3 \pmod{7}$.

Lean statement

```
theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Parent. For every natural number t with $t \leq 100$, show that $29^{6t+1} - 5^{6t+1}$ is congruent to 3 modulo 7.

Parent in Lean

```
theorem mathd_numbertheory_728_mutation (t : ℕ) : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 3 [ZMOD 7] := by
```

What it contributes. Generalizes the parent's period-6 power reduction and subtraction checkpoints from fixed bases to arbitrary integers constrained to the same residue classes.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both bases and requires transporting their modular residues through the power-period argument before subtracting. Its hypotheses are essential, so recalling the parent's concrete proof does not directly solve the generalized claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both concrete bases and must use the hypotheses that they lie in residue classes 1 and 5 modulo 7. The parent's proof does not supply this symbolic residue-transfer step, so recalling it is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both concrete bases and requires using the new residue hypotheses to derive the final difference congruence. Although the periodicity argument is inherited, the parent proof cannot be applied directly because its concrete bases and residue computation no longer match.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2481 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_residue_law.extracted_1_1 (t : ℕ) (ht : t ≤ 100) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7])  
(hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number t with $t \leq 100$ and all integers A and B such that A is congruent to 1 modulo 7 and B is congruent to 5 modulo 7, the difference $A^{(6t+1)} - B^{(6t+1)}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡  
  3 [ZMOD 7] := by
  have hperiodA : A ^ (6 * t + 1) ≡ A ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hperiodB : B ^ (6 * t + 1) ≡ B ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hAred : A ^ (6 * t + 1) ≡ (1 : ℤ) [ZMOD 7] := by
    simpa using hperiodA.trans (hA.pow 1)
  have hBred : B ^ (6 * t + 1) ≡ (5 : ℤ) [ZMOD 7] := by
    simpa using hperiodB.trans (hB.pow 1)
  have hdifference : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ (1 : ℤ) - 5 [ZMOD 7] := by
    exact hAred.sub hBred
  have hresidue : (1 : ℤ) - 5 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  calc
    A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ (1 : ℤ) - 5 [ZMOD 7] := hdifference
    _ ≡ 3 [ZMOD 7] := hresidue
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me__6dbad337

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint ee3078c8502a4e6b

4.104 104. coupled_ap_modular_term

The problem

Let an arithmetic sequence have first four terms p , 9 , $3p - q$, and $3p + q$. If n is congruent modulo 7 to $29^{13} - 5^{13}$, show that the $(n + 1)$ st term is congruent to 3 modulo 7.

Lean statement

```
theorem coupled_ap_modular_term (p q : ℤ) (a : ℕ → ℤ) (n : ℕ)
  (hrec : ∀ k : ℕ, a (k + 2) - a (k + 1) = a (k + 1) - a k)
  (h1 : a 1 = p) (h2 : a 2 = 9) (h3 : a 3 = 3 * p - q)
  (h4 : a 4 = 3 * p + q)
  (hidx : (n : ℤ) ≡ (29 : ℤ)^13 - 5^13 [ZMOD 7]) :
  a (n + 1) ≡ 3 [ZMOD 7]
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Supplies the modulus-7 computation that $29^{13} - 5^{13}$ is congruent to 3, consumed through the explicit index hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires deriving the arithmetic progression explicitly, forcing $p = 5$ and $a(k) = 4k + 1$, then combining that formula with the modular condition on n . This is genuinely new reasoning and is not recoverable by recalling the parent's direct numerical normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires deriving the entire arithmetic progression from the recurrence and initial constraints, then coupling its index modulo 7 to the parent residue. A solver recalling the parent's fixed modular computation would not obtain this progression argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires deriving the entire arithmetic progression from the recurrence and initial constraints, then coupling its index modulo 7 to the parent residue. The parent's direct normalization of a fixed modular expression supplies none of this reasoning, and all hypotheses are materially used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2366 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coupled_ap_modular_term.extracted_1_1 (p q : ℤ) (a : ℕ → ℤ) (n : ℕ)
  (hrec : ∀ (k : ℕ), a (k + 2) - a (k + 1) = a (k + 1) - a k) (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q) (hidₓ : ↑n ≡ 29 ^ 13 - 5 ^ 13 [ZMOD 7]) : a (n + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For integers p and q , an integer-valued sequence a indexed by the natural numbers, and a natural number n , suppose that for every natural number k , $a(k + 2) - a(k + 1) = a(k + 1) - a(k)$, that $a(1) = p$, $a(2) = 9$, $a(3) = 3p - q$, and $a(4) = 3p + q$. If the integer n is congruent modulo 7 to $29^{13} - 5^{13}$, then $a(n + 1)$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem coupled_ap_modular_term (p q : ℤ) (a : ℕ → ℤ) (n : ℕ)
  (hrec : ∀ k : ℕ, a (k + 2) - a (k + 1) = a (k + 1) - a k)
  (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q)
  (h₄ : a 4 = 3 * p + q)
  (hidₓ : (n : ℤ) ≡ (29 : ℤ)^13 - 5^13 [ZMOD 7]) :
  a (n + 1) ≡ 3 [ZMOD 7] := by
  have hA := hrec 1
  have hB := hrec 2
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith [hA, hB]
  have hzero := hrec 0
  rw [h₂, h₁] at hzero
  have ha0 : a 0 = 1 := by
    linarith [hzero, hp]
  have hform : ∀ k : ℕ, a k = 4 * (k : ℤ) + 1 := by
    intro k
    induction k using Nat.strong_induction_on with
    | h k ih =>
      by_cases hk0 : k = 0
      · subst k
        rw [ha0]
        norm_num
      · by_cases hk1 : k = 1
        · subst k
          rw [h₁, hp]
          norm_num
        · have hk2 : 2 ≤ k := by omega
          let j := k - 2
          have hk : j + 2 = k := by
            dsimp [j]
            omega
          rw [← hk]
          have hj := ih j (by omega)
          have hj1 := ih (j + 1) (by omega)
          have hr := hrec j
          calc
            a (j + 2) = a (j + 1) + (a (j + 1) - a j) := by
              linarith [hr]
            _ = 4 * (↑(j + 2) : ℤ) + 1 := by
              rw [hj1, hj]
              norm_num [Nat.cast_add] <|> ring
  rw [hform]
  norm_num [Int.ModEq] at hidₓ
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me__43ecb415

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-

gerprint 43ecb4153200f83c

4.105 105. mathd_numbertheory_728_mutation

The problem

For every natural number t with $t \leq 100$, show that $29^{6t+1} - 5^{6t+1}$ is congruent to 3 modulo 7.

Lean statement

```
theorem mathd_numbertheory_728_mutation (t : ℕ) : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 3 [ZMOD 7] := by
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Retains the parent's modulo-7 power-difference target and residue computation, while adding a bounded exponent parameter and explicit period checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed exponent is genuinely parameterized to $6t+1$, requiring periodicity arguments for both powers and modular subtraction; the parent's direct normalization proof does not supply this reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 13 to every exponent of the form $6t+1$. Its proof requires periodicity and modular-power reasoning, which the parent's direct norm_num proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2475 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_728_mutation.extracted_1_1 (t : ℕ) (ht : t ≤ 100) :
  29 ^ (6 * t + 1) - 5 ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number t with $t \leq 100$, the natural-number difference $29^{6t+1} - 5^{6t+1}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_728_mutation (t : ℕ) : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 3 [ZMOD 7] := by
  have hperiod29 : (29 : ℤ) ^ (6 * t + 1) ≡ 29 ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hperiod5 : (5 : ℤ) ^ (6 * t + 1) ≡ 5 ^ 1 [ZMOD 7] := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1)
    · norm_num
    · use t
    · omega
    · omega
    · omega
  have hdifference : ((29 : ℤ) ^ (6 * t + 1) - 5 ^ (6 * t + 1)) ≡ 29 ^ 1 - 5 ^ 1 [ZMOD 7] := by
    exact hperiod29.sub hperiod5
  have hresidue : ((29 : ℤ) ^ 1 - 5 ^ 1) ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hdifference.trans hresidue
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me__e5a2f50a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8112b513089cfe57

4.106 106. affine_power_residue

The problem

For integers r, s and a natural number k , prove that $(29 + 7r)^{6k+1} - (5 + 7s)^{6k+1} \equiv 3 \pmod{7}$.

Lean statement

```
theorem affine_power_residue (r s : ℤ) (k : ℕ) : (29 + 7 * r) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
```

Parent. If integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{13} - b^{13}$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem thirteenth_power_difference_mod_seven (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Uses the parent's congruence-power transport and subtraction structure, strengthened to explicit affine base parameters and a variable exponent.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponent to every $6k+1$ and requires proving the new periodicity $(5)^6 \equiv 1 \pmod{7}$, while also deriving the base congruences from affine parameterizations. This is not recoverable by recalling the parent's fixed-exponent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The variable exponent $6k+1$ requires a new periodicity argument, including proving $5^6 \equiv 1 \pmod{7}$ and transporting powers through multiplication and addition. The parent's fixed-exponent numerical normalization cannot establish this generalized residue.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes both bases to independent multiples of seven and the exponent to $6k+1$, requiring periodicity reasoning for the powered residue of 5. The parent's fixed-exponent normalization proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2419 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_power_residue.extracted_1_1 (r s : ℤ) (k : ℕ) :
  (29 + 7 * r) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For all integers r and s and every natural number k , $(29 + 7r)^{(6k+1)} - (5 + 7s)^{(6k+1)}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem affine_power_residue (r s : ℤ) (k : ℕ) : (29 + 7 * r) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
  have ha : (29 + 7 * r : ℤ) ≡ 29 [ZMOD 7] := by
    norm_num [Int.ModEq, Int.add_emod, Int.mul_emod]
  have hb : (5 + 7 * s : ℤ) ≡ 5 [ZMOD 7] := by
    norm_num [Int.ModEq, Int.add_emod, Int.mul_emod]
  have haPow : (29 + 7 * r : ℤ) ^ (6 * k + 1) ≡ 29 ^ (6 * k + 1) [ZMOD 7] := ha.pow (6 * k + 1)
  have hbPow : (5 + 7 * s : ℤ) ^ (6 * k + 1) ≡ 5 ^ (6 * k + 1) [ZMOD 7] := hb.pow (6 * k + 1)
  have hdiff : (29 + 7 * r : ℤ) ^ (6 * k + 1) - (5 + 7 * s) ^ (6 * k + 1) ≡ 29 ^ (6 * k + 1) - 5 ^ (6 * k + 1) [ZMOD 7] := haPow.sub
    <| hbPow
  have h29 : (29 : ℤ) ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have h29Pow : (29 : ℤ) ^ (6 * k + 1) ≡ 1 [ZMOD 7] := by
    simpa using h29.pow (6 * k + 1)
  have h5six : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have h5mult : (5 : ℤ) ^ (6 * k) ≡ 1 [ZMOD 7] := by
    simpa [pow_mul] using h5six.pow k
  have h5Pow : (5 : ℤ) ^ (6 * k + 1) ≡ 5 [ZMOD 7] := by
    calc
      (5 : ℤ) ^ (6 * k + 1) = 5 ^ (6 * k) * 5 := by rw [pow_add]; simp
      _ = (1 : ℤ) * 5 [ZMOD 7] := h5mult.mul (Int.ModEq.refl 5)
      _ = 5 := by simp
  have hfixed : (29 : ℤ) ^ (6 * k + 1) - 5 ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
    calc
      (29 : ℤ) ^ (6 * k + 1) - 5 ^ (6 * k + 1) ≡ 1 - 5 [ZMOD 7] := h29Pow.sub h5Pow
      _ = 3 [ZMOD 7] := by norm_num [Int.ModEq]
  exact hdiff.trans hfixed
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.me__641f9146

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 641f91465ff59538

4.107 107. power_difference_mod_seven_periodic

The problem

For every natural number t , if integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{6t+1} - b^{6t+1}$ leaves remainder 3 modulo 7.

Lean statement

```
theorem power_difference_mod_seven_periodic (a b : ℤ) (t : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ 3 [ZMOD 7] := by
```

Parent. If integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{13} - b^{13}$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem thirteenth_power_difference_mod_seven (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
```

What it contributes. Retains the parent’s residue hypotheses and subtraction-of-powered-congruences checkpoint, while replacing the fixed exponent with the six-period parameterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponent to an unbounded parameter and requires a new period-six power reduction modulo 7 before transporting and subtracting the congruences. This reasoning is absent from the parent’s direct fixed-exponent computation.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent to arbitrary $6t+1$ and requires a new period-six modular argument to reduce both powers before subtracting. The parent’s fixed thirteenth-power calculation does not supply this parameterized reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent to arbitrary unbounded t and requires a period-six modular-power argument to reduce both powers. The parent’s direct numerical normalization for exponent 13 does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2096 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_mod_seven_periodic.extracted_1_1 (a b : ℤ) (t : ℕ) (ha : a ≡ 29 [ZMOD 7])  
  (hb : b ≡ 5 [ZMOD 7]) : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For all integers a and b and every natural number t , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then $a^{6t+1} - b^{6t+1}$ is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_mod_seven_periodic (a b : ℤ) (t : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ 3 [ZMOD 7] := by
  have hperiod : 6 ∣ (6 * t + 1) - 1 := by
    refine ⟨t, ?_⟩
    omega
  have hle : 1 ≤ 6 * t + 1 := by
    omega
  have h29period : (29 : ℤ) ^ (6 * t + 1) ≡ (29 : ℤ) ^ 1 [ZMOD 7] := by
    exact Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1) (by norm_num) hperiod hle (by norm_num) 29
  have h5period : (5 : ℤ) ^ (6 * t + 1) ≡ (5 : ℤ) ^ 1 [ZMOD 7] := by
    exact Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + 1) (y := 1) (by norm_num) hperiod hle (by norm_num) 5
  have ha_period : a ^ (6 * t + 1) ≡ (29 : ℤ) ^ (6 * t + 1) [ZMOD 7] := ha.pow (6 * t + 1)
  have hb_period : b ^ (6 * t + 1) ≡ (5 : ℤ) ^ (6 * t + 1) [ZMOD 7] := hb.pow (6 * t + 1)
  have hdiff : a ^ (6 * t + 1) - b ^ (6 * t + 1) ≡ (29 : ℤ) ^ (6 * t + 1) - (5 : ℤ) ^ (6 * t + 1) [ZMOD 7] := ha_period.sub hb_period
  have hbase : (29 : ℤ) ^ (6 * t + 1) - (5 : ℤ) ^ (6 * t + 1) ≡ (29 : ℤ) - 5 [ZMOD 7] := by
    calc
      (29 : ℤ) ^ (6 * t + 1) - (5 : ℤ) ^ (6 * t + 1) ≡ (29 : ℤ) ^ 1 - (5 : ℤ) ^ 1 [ZMOD 7] := h29period.sub h5period
      _ ≡ (29 : ℤ) - 5 [ZMOD 7] := by norm_num
  have hnum : (29 : ℤ) - 5 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hdiff.trans (hbase.trans hnum)
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.me__a73264dd

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a73264ddf683dbf4

4.108 108. generalized_residue_divisibility

The problem

Let a and b be integers with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that, for every nonnegative integer k , 7 divides $a^{6k+1} - b^{6k+1} - 3$.

Lean statement

```
theorem generalized_residue_divisibility (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ (6 * k + 1) - b ^ (6 * k + 1) - 3) := by
```

Parent. Two sensors report integers a and b , with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that 7 divides $a^{13} - b^{13} - 3$.

Parent in Lean

```
theorem variable_base_residue_divisibility (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ 13 - b ^ 13 - 3) := by
```

What it contributes. Supplied the power-transport, subtraction, fixed-residue, congruence-to-divisibility, and witness-negation checkpoints; the exponent is generalized to $6k+1$ and the periodic power checkpoint is proved inductively.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponent 13 to every exponent of the form $6k+1$. Its proof must establish the exponent periodicity modulo 7, especially for 5^{6k+1} , which is not supplied by recalling the parent's fixed-exponent calculation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 13 to every exponent of the form $6k+1$. Its proof must establish periodicity of powers modulo 7 and derive the residue for the symbolic exponent, which is not supplied by the parent's numerical normalization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the isolated exponent 13 to every exponent of the form $6k+1$, requiring a new periodicity argument for $5^{6k+1} \equiv 5 \pmod{7}$. The parent's direct normalization of fixed powers does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2421 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem generalized_residue_divisibility.extracted_1_1 (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :  
  7 ∣ a ^ (6 * k + 1) - b ^ (6 * k + 1) - 3
```

Read back by a model that saw only the Lean. For integers a and b and a natural number k , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then 7 divides $a^{6k+1} - b^{6k+1} - 3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem generalized_residue_divisibility (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ (6 * k + 1) - b ^ (6 * k + 1) - 3) := by
  have ha_exp : a ^ (6 * k + 1) ≡ (29 : ℤ) ^ (6 * k + 1) [ZMOD 7] := ha.pow (6 * k + 1)
  have hb_exp : b ^ (6 * k + 1) ≡ (5 : ℤ) ^ (6 * k + 1) [ZMOD 7] := hb.pow (6 * k + 1)
  have h29base : (29 : ℤ) ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have h29 : (29 : ℤ) ^ (6 * k + 1) ≡ 1 [ZMOD 7] := by
    simpa using h29base.pow (6 * k + 1)
  have h5 : (5 : ℤ) ^ (6 * k + 1) ≡ 5 [ZMOD 7] := by
    have hdiv : 6 ∣ (6 * k + 1 - 1) := by omega
    have hle : 1 ≤ 6 * k + 1 := by omega
    simpa using
      (Int.ModEq.pow_eq_pow (p := 7) (x := 6 * k + 1) (y := 1)
        (by norm_num : Nat.Prime 7) hdiv hle (by norm_num : 0 < (1 : ℕ)) (5 : ℤ)))
  have hdiff : a ^ (6 * k + 1) - b ^ (6 * k + 1) ≡ (29 : ℤ) ^ (6 * k + 1) - (5 : ℤ) ^ (6 * k + 1) [ZMOD 7] := ha_exp.sub hb_exp
  have hconst : (29 : ℤ) ^ (6 * k + 1) - (5 : ℤ) ^ (6 * k + 1) ≡ 3 [ZMOD 7] := by
    calc
      (29 : ℤ) ^ (6 * k + 1) - (5 : ℤ) ^ (6 * k + 1) ≡ 1 - 5 [ZMOD 7] := h29.sub h5
      _ ≡ 3 [ZMOD 7] := by norm_num [Int.ModEq]
  have hres : a ^ (6 * k + 1) - b ^ (6 * k + 1) ≡ 3 [ZMOD 7] := hdiff.trans hconst
  rw [Int.modEq_iff_dvd] at hres
  rcases hres with ⟨m, hm⟩
  refine ⟨-m, ?_⟩
  omega
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.ms.me__ad3f0fdf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ad3f0fdcf255c771

4.109 109. ternary_prefix_decode

The problem

Let n be a positive natural number. If a natural number has ternary digit list consisting of a terminal digit d , followed by n copies of 2 and then 1, prove that its positional value is $d + 3 \left(2 \sum_{i=0}^{n-1} 3^i + 3^n \right)$; in particular, the terminal digit is retained.

Lean statement

```
theorem ternary_prefix_decode
  (n d x : ℕ)

  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * Finset.sum (Finset.range n) (fun i => 3 ^ i) + 3 ^ n)
```

Parent. Let n be positive with $n \leq 3$. If a natural number has ternary digit list consisting of a terminal digit d , followed by n copies of 2 and then 1, prove its positional value is $d + 3 \left(2 \sum_{i=0}^{n-1} 3^i + 3^n \right)$; in particular, the terminal digit is retained.

Parent in Lean

```
theorem ternary_bounded_prefix_decode
  (n d x : ℕ)
  (hn : 0 < n)
  (hbound : n ≤ 3)
  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * Finset.sum (Finset.range n) (fun i => 3 ^ i) + 3 ^ n)
```

What it contributes. Retains the parent's ternary digit-list encoding and terminal-digit positional-decoding checkpoint, while replacing its finite case split with induction over an arbitrary prefix length.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the finite bound on n and proves the positional decoding law uniformly by induction, requiring a genuinely new recursive argument rather than the parent's finite case split. Its proof does not recall or reuse the parent's bounded conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the bound on n and proves the decoding identity uniformly by induction, requiring a geometric-sum lemma and recursive evaluation of `ofDigits`. This is genuine reasoning absent from the parent's finite case split and is not a forced wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the finite bound and replaces the parent's three-case computation with a genuine induction proving the positional decoding law for every prefix length. A solver recalling the parent's bounded case split would not supply the new uniform induction argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2548 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_prefix_decode.extracted_1_1 (n d x : ℕ) (hn : 0 < n)
  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * ∑ i ∈ Finset.range n, 3 ^ i + 3 ^ n)
```

Read back by a model that saw only the Lean. For natural numbers n , d , and x , if n is positive and the base-3 digit list of x is d followed by n copies of 2 and then 1, then $x = d + 3(2\sum_{i \in \{0, \dots, n-1\}} 3^i + 3^n)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem ternary_prefix_decode
  (n d x : ℕ)

  (h : Nat.digits 3 x = d :: (List.replicate n 2 ++ [1])) :
  x = d + 3 * (2 * Finset.sum (Finset.range n) (fun i => 3 ^ i) + 3 ^ n) := by
  have hgeom : ∀ m : ℕ,
    2 * Finset.sum (Finset.range m) (fun i => 3 ^ i) + 1 = 3 ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      simp only [Finset.sum_range_succ, pow_succ]
      omega
  have hsum : ∀ m : ℕ,
    Nat.ofDigits 3 (List.replicate m 2 ++ [1]) =
      2 * Finset.sum (Finset.range m) (fun i => 3 ^ i) + 3 ^ m := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      simp [List.replicate_succ, Nat.ofDigits_cons, ih,
        Finset.sum_range_succ, pow_succ]
      have hg := hgeom m
      omega
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=
    (Nat.ofDigits_digits 3 x).symm
  rw [h] at hdecode
  rw [Nat.ofDigits_cons, hsum n] at hdecode
  exact hdecode
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me.me.me__7899a23f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c6c3a577dd4c7ba1

4.110 110. ternary_terminal_digit_decode

The problem

Let $d \in \{0, 1, 2\}$. If a natural number has base-three representation $122d_3$, show that its value is $d + 51$; in particular, when $d = 2$, its value is 53.

Lean statement

```
theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

What it contributes. Uses the parent's `Nat.ofDigits_digits` recovery and positional evaluation approach, generalized from the fixed least-significant digit 2 to $d < 3$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the terminal ternary digit and requires deriving the symbolic decoding formula $x = d + 51$; the fixed-value parent proof does not supply that algebra. The specialization $x = 53$ is only a consequence of the new general law.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the leading digit and proves the symbolic positional-decoding law $x = d + 51$, rather than merely replaying the fixed computation. The specialization implication is immediate from this new general result, but the main generalized claim requires symbolic decoding absent from the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the terminal digit and proves the new symbolic decoding law $x = d + 51$; the parent only supplies the fixed case $d = 2$. The parent's proof cannot be recalled verbatim to establish the generalized arithmetic identity.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2511 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_terminal_digit_decode.extracted_1_1 (d x : ℕ) (hd : d < 3) (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

Read back by a model that saw only the Lean. For natural numbers d and x , if $d < 3$ and the base-3 digit expansion of x , listed from least significant to most significant, is $[d, 2, 2, 1]$, then $x = d + 51$, and if $d = 2$, then $x = 53$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53) := by
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=
    (Nat.ofDigits_digits 3 x).symm
  have hvalue : x = d + 51 := by
    calc
      x = Nat.ofDigits 3 (Nat.digits 3 x) := hdecode
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [h]
      _ = d + 51 := by norm_num [Nat.ofDigits_cons]
  constructor
  · exact hvalue
  · intro hdigit
    rw [hvalue, hdigit]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me__0fd2ae3f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 46d57d77b1203e42

4.111 111. choose_translated_square_level_iff

The problem

For positive integers n and k , show that $\left[\left(x - \binom{n}{k} \right)^2 - \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1} \right]$ if and only if $x = \binom{n}{k}$, for real x .

Lean statement

```
theorem choose_translated_square_level_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  ((x - (Nat.choose n k : ℝ)) ^ 2 + (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
  x = (Nat.choose n k : ℝ)
```

Parent. For a positive integer n and $0 \leq k \leq n$, show that $\binom{n}{k} = 1$ when $k = 0$, while for $k > 0$, $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem choose_boundary_pascal (n k : ℕ) (hn : 0 < n) :
  Nat.choose n k = if k = 0 then 1 else Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

What it contributes. Supplied the positive-index Pascal recurrence, used as the load-bearing rewrite of the vertical offset.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child retains the Pascal recurrence but adds a genuine translated-square classification: the solver must normalize the recursive level and prove that a real square vanishes exactly at its center. Memorizing the parent's boundary-case proof does not supply this argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a Pascal identity into a real-valued level-set classification. It must cast the recursive identity and prove that a translated square equals zero exactly at its center, reasoning not supplied by the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child combines Pascal recursion with a genuinely new real-algebraic classification: it reduces the translated-square equation to a zero square and proves an iff statement. Memorizing the parent's boundary-case Pascal proof would not supply the square-equation reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2415 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem choose_translated_square_level_iff.extracted_1_1 (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :  
(x - ↑(n.choose k)) ^ 2 + ↑(n.choose k) = ↑((n - 1).choose k + (n - 1).choose (k - 1)) ↔ x = ↑(n.choose k)
```

Read back by a model that saw only the Lean. For natural numbers n and k with $0 < n$ and $0 < k$, and for a real number x , the equation $(x - C(n,k))^2 + C(n,k) = C(n-1,k) + C(n-1,k-1)$ holds if and only if $x = C(n,k)$, where $C(a,b)$ denotes the binomial coefficient and subtraction of natural numbers is truncated subtraction.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem choose_translated_square_level_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :  
  ((x - (Nat.choose n k : ℝ)) ^ 2 + (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔  
  x = (Nat.choose n k : ℝ) := by  
  have hpascal : Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by  
    simpa [Nat.add_comm] using Nat.choose_eq_choose_pred_add hn hk  
  have hoff : (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) := by  
    exact_mod_cast hpascal  
  constructor  
  · intro h  
    nlinarith [sq_nonneg (x - (Nat.choose n k : ℝ))]  
  · intro hx  
    rw [hx]  
    simp [hoff]
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__me.me__ae9ead38

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ae9ead389b8cfd91

4.112 112. choose_boundary_pascal

The problem

For a positive integer n and $0 \leq k \leq n$, show that $\binom{n}{k} = 1$ when $k = 0$, while for $k > 0$, $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Lean statement

```
theorem choose_boundary_pascal (n k : ℕ) (hn : 0 < n) :
  Nat.choose n k = if k = 0 then 1 else Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

Parent. Show that for positive integers n and k with $k \leq n$, we have $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem numbertheory_nckeqnm1ckpm1ckm1 (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
```

What it contributes. Supplies the positive-index Pascal recurrence via `Nat.choose_eq_choose_pred_add`; the new zero-index branch is handled separately.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds the genuinely new boundary case $k=0$, where Pascal's positive-index recurrence does not apply, and removes the unnecessary bound $k \leq n$. Its proof requires a case split and separately establishes the base value.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the recurrence to the boundary case $k = 0$, requiring a case split and a separate proof that the choose value is 1. The parent's positive-index lemma cannot establish this boundary branch, and the child also removes the unnecessary bound $k \leq n$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the recurrence to the boundary case $k=0$, requiring a case split and removal of the parent's positivity hypothesis on k . The parent's proof only handles the positive-index case and does not establish the new boundary behavior.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2406 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem choose_boundary_pascal.extracted_1_1 (n k : ℕ) (hn : 0 < n) (hk : k ≤ n) :  
  n.choose k = if k = 0 then 1 else (n - 1).choose k + (n - 1).choose (k - 1)
```

Read back by a model that saw only the Lean. For natural numbers n and k , if n is positive and k is at most n , then the binomial coefficient n choose k equals 1 when $k = 0$, and otherwise equals the sum of $(n - 1)$ choose k and $(n - 1)$ choose $(k - 1)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem choose_boundary_pascal (n k : ℕ) (hn : 0 < n) :  
  Nat.choose n k = if k = 0 then 1 else Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by  
  by_cases hzero : k = 0  
  · subst k  
    simp  
  · have hkpos : 0 < k := Nat.pos_of_ne_zero hzero  
    have hpascal : Nat.choose n k = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k :=  
      Nat.choose_eq_choose_pred_add hn hkpos  
    rw [if_neg hzero]  
    calc  
      Nat.choose n k = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k := hpascal  
      _ = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := Nat.add_comm _ _
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__me__ab239085

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint c0f43117760bc230

4.113 113. complementary_choose_eq_one_iff

The problem

For integers $n \geq 2$ and $0 < k \leq n$, classify all k such that the complementary binomial coefficient $\binom{n}{n-k}$ equals 1.

Lean statement

```
theorem complementary_choose_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose n (n - k) = 1 ↔ k = n
```

Parent. For positive integers n and k with $k \leq n$, show that $\binom{n}{n-k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem complementary_pascal (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

What it contributes. Reused the parent’s complementary-index symmetry checkpoint, while changing the conclusion to a bounded classification of unit-valued coefficients.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the Pascal identity into a classification theorem for when a complementary binomial coefficient equals one. Its proof requires the characterization `Nat.choose_eq_one_iff` and excludes the zero-index branch using `hk`; recalling the parent does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes from a Pascal recurrence to a classification of when a complementary binomial coefficient equals 1. Its proof requires the distinct theorem ‘`Nat.choose_eq_one_iff`’ and a case split using the positivity hypothesis, so recalling the parent’s Pascal proof does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the Pascal recurrence into a classification theorem for when the complementary binomial coefficient equals one. Although it reuses binomial symmetry, its essential argument relies on the distinct characterization `Nat.choose n k = 1` and a case split, so recalling the parent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2629 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem complementary_choose_eq_one_iff.extracted_1_1 (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :  
  n.choose (n - k) = 1 ↔ k = n
```

Read back by a model that saw only the Lean. For natural numbers n and k satisfying $2 \leq n$, $0 < k$, and $k \leq n$, the binomial coefficient " n choose $(n - k)$ " equals 1 if and only if $k = n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem complementary_choose_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :  
  Nat.choose n (n - k) = 1 ↔ k = n := by
  have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
    Nat.choose_symm hkn
  have hone : Nat.choose n k = 1 ↔ k = 0 ∨ n = k :=
    Nat.choose_eq_one_iff
  constructor
  · intro h
    have hcases : k = 0 ∨ n = k := hone.mp (hsymm.symm.trans h)
    rcases hcases with hk0 | hnk
    · omega
    · exact hnk.symm
  · intro h
    subst k
    simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms.me__6b4f9b19

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 803ae78f47813c8c

4.114 114. prime_dvd_iff_coupled_product_uniform_offset

The problem

For a prime number p and a positive exponent e , characterize the integer offsets m for which, for every integer n , p divides n exactly when $(n(n + m))^e \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_coupled_product_uniform_offset
  (m : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e) :
  ((∀ n : ℤ, (↑p : ℤ) ∣ n ↔ (n * (n + m)) ^ e ≡ 0 [ZMOD p]) ↔
   (↑p : ℤ) ∣ m)
```

Parent. For a prime number p , integers n, k , and a positive exponent $e \leq B$, show that p divides n if and only if $(n(n + kp))^e \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n * (n + k * (↑p : ℤ))) ^ e ≡ 0 [ZMOD p]
```

What it contributes. Supplied the prime-modulus, positive-power, natAbs product-splitting, and divisibility-cancellation proof checkpoints; the fixed multiple $k \cdot p$ was replaced by the active hypothesis $p \mid m$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine offset characterization: its forward direction specializes the universal criterion at $n = -m$ to derive $p \mid m$, while the reverse direction uses $p \mid m$ to recover the coupled-product criterion. This introduces arbitrary offsets and a new necessity argument beyond the parent's fixed multiple-of- p translate.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded generalization: it characterizes exactly which offsets work uniformly, adding a new forward argument by specializing at $n = -m$. The reverse direction generalizes the parent's divisibility transfer, but recalling the parent alone does not establish the offset characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed translated-product equivalence into a uniform offset characterization. Its forward direction requires the new specialization $n = -m$ to derive $p \mid m$, so recalling the parent cannot solve the theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2635 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_coupled_product_uniform_offset.extracted_1_1 (m : ℤ) (p e : ℕ) (hp : Nat.Prime p)
  (he_pos : 0 < e) : (∀ (n : ℤ), ↑p ∣ n ↔ (n * (n + m)) ^ e ≡ 0 [ZMOD ↑p]) ↔ ↑p ∣ m
```

Read back by a model that saw only the Lean. *Let m be an integer and let p and e be natural numbers such that p is prime and e is positive. Then the following are equivalent: for every integer n , p divides n if and only if the e -th power of $n(n + m)$ is congruent to 0 modulo p ; and p divides m .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem prime_dvd_iff_coupled_product_uniform_offset
  (m : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e) :
  ((∀ n : ℤ, (↑p : ℤ) ∣ n ↔ (n * (n + m)) ^ e ≡ 0 [ZMOD p]) ↔
   (↑p : ℤ) ∣ m) := by
  simp only [Int.modEq_zero_iff_dvd]
  constructor
  · intro h
    have hneg : (↑p : ℤ) ∣ -m := by
      apply (h (~m)).mpr
      simp [Nat.ne_of_gt he_pos]
    obtain ⟨a, ha⟩ := hneg
    refine ⟨-a, ?_⟩
    calc
      m = -(-m) := by ring
      _ = -((↑p : ℤ) * a) := by rw [ha]
      _ = (↑p : ℤ) * (-a) := by ring
  · intro hm n
    have he_ne : e ≠ 0 := Nat.ne_of_gt he_pos
    constructor
    · intro hdiv
      have hproduct : (↑p : ℤ) ∣ n * (n + m) := by
        exact dvd_mul_of_dvd_left hdiv _
      exact dvd_pow hproduct he_ne
    · intro hpower
      have hproduct : (↑p : ℤ) ∣ n * (n + m) := by
        exact Int.Prime.dvd_pow' hp hpower
      have hproduct_nat : p ∣ (n * (n + m)).natAbs :=
        (Int.natCast_dvd).mp hproduct
      rw [Int.natAbs_mul] at hproduct_nat
      rcases hp.dvd_mul.mp hproduct_nat with hdiv | htranslated
      · exact (Int.natCast_dvd).mpr hdiv
      · have htranslated_int : (↑p : ℤ) ∣ n + m :=
          (Int.natCast_dvd).mpr htranslated
        have hcancel : (↑p : ℤ) ∣ (n + m) - m := by
          exact dvd_sub htranslated_int hm
        convert hcancel using 1 <|> ring
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh.mh.me__0286cdbe

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0286cdbe3cb2b06d

4.115 115. prime_dvd_iff_even_bounded_power_modEq_zero

The problem

For a prime number p , an integer n , and a natural number e with $1 \leq e$, prove that $p \mid n$ if and only if $n^{2e} \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_even_bounded_power_modEq_zero (n : ℤ) (p e : ℕ) (h₀ : Nat.Prime p) (he₁ : 1 ≤ e) :
  (↑p : ℤ) ∣ n ↔ n ^ (2 * e) ≡ 0 [ZMOD p]
```

Parent. Show that for any prime p and any integer n , we have $p \mid n$ if and only if $n^2 \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem numbertheory_prmdvsneqnsqmodpeq0 (n : ℤ) (p : ℕ) (h₀ : Nat.Prime p) :
  ↑p ∣ n ↔ n ^ 2 ≡ 0 [ZMOD p] := by
```

What it contributes. Preserves the prime-modulus and integer-divisibility setting and reuses its square-divisibility bridge on n^e , while adding the bounded exponent parameter and a power-divisibility step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the exponent and requires reducing divisibility of $n^{(2e)}$ to a square, then using primality to descend from $p \mid n^e$ to $p \mid n$. This is not recoverable by simply recalling or applying the parent proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the square criterion to every positive even exponent. Its reverse direction requires introducing n^e , applying the prime square-divisibility lemma, and then lifting divisibility through a power—reasoning absent from the parent proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the square criterion to a symbolic positive exponent. Its converse requires rewriting the even power as a square of n^e , applying the prime square-divisibility lemma, and then using prime divisibility of a power to recover $p \mid n$; this is not mere recall of the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2084 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_even_bounded_power_modEq_zero.extracted_1_1 (n : ℤ) (p e : ℕ) (h₀ : Nat.Prime p) (he₁ : 1 ≤ e) :  
  ↑p ∣ n ↔ n ^ (2 * e) ≡ 0 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For an integer n and natural numbers p and e , if p is prime and $1 \leq e$, then p divides n if and only if n raised to the power $2e$ is congruent to 0 modulo p .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem prime_dvd_iff_even_bounded_power_modEq_zero (n : ℤ) (p e : ℕ) (h₀ : Nat.Prime p) (he₁ : 1 ≤ e) :  
  (↑p : ℤ) ∣ n ↔ n ^ (2 * e) ≡ 0 [ZMOD p] := by  
  rw [Int.modEq_zero_iff_dvd]  
  constructor  
  · intro h  
    exact dvd_pow h (Nat.ne_of_gt (Nat.mul_pos (by norm_num) (Nat.zero_lt_of_lt he₁)))  
  · intro h  
    apply Int.natCast_dvd.mpr  
    have hs : (↑p : ℤ) ∣ (n ^ e) ^ 2 := by  
      simpa [pow_mul, Nat.mul_comm] using h  
    have hpabs : p ∣ (n ^ e).natAbs :=  
      Int.Prime.dvd_natAbs_of_coe_dvd_sq h₀ (n ^ e) hs  
    apply h₀.dvd_of_dvd_pow  
    simpa [Int.natAbs_pow] using hpabs
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me__dfe2a291

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 960dc5475db79eb2

4.116 116. seven_not_dvd_five_pow_twenty_one

The problem

Show that 7 does not divide 5^{21} by determining its remainder modulo 7.

Lean statement

```
theorem seven_not_dvd_five_pow_twenty_one : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 21
```

Parent. Show that 7 does not divide 5^{15} .

Parent in Lean

```
theorem seven_not_dvd_five_pow_fifteen : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 15
```

What it contributes. Supplied the divisibility-to-modular-contradiction checkpoint and theorem-proof style; the child changes the exponent and computes the direct residue \$6\$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the exponent and independently computes the residue $(5^{21} \bmod 7) = 6$, then derives a contradiction from divisibility implying residue 0. The parent's square-based argument for exponent 15 does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the new exponent case directly by converting divisibility to a zero residue and computing $(5 : \mathbb{Z})^{21} \equiv 6 \pmod{7}$, rather than reusing the parent's square criterion and exponent-doubling argument. It satisfies the requested fresh modular contradiction and cannot be solved by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses a genuinely different modular argument: it derives the exponent-21 residue as 6 directly and contradicts divisibility via the zero-residue criterion. The parent's squaring argument and residue-1 conclusion do not supply this proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2425 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem seven_not_dvd_five_pow_twenty_one.extracted_1_1 : ¬ 7 ∣ 5 ^ 21
```

Read back by a model that saw only the Lean. 7 does not divide 5 raised to the 21st power.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem seven_not_dvd_five_pow_twenty_one : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 21 := by
  intro hdiv
  have hzero : (5 : ℤ) ^ 21 ≡ 0 [ZMOD 7] := by
    exact (Int.modEq_zero_iff_dvd).2 hdiv
  have hresidue : (5 : ℤ) ^ 21 ≡ 6 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have contradiction_mod : (0 : ℤ) ≡ 6 [ZMOD 7] := by
    exact hzero.symm.trans hresidue
  norm_num [Int.ModEq] at contradiction_mod
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.me__787dad62

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 787dad628ffb8836

4.117 117. dvd_add_one_iff_exp_eq_three

The problem

Let x be an integer congruent to 5 modulo 7. For $0 \leq k \leq 3$, show that 7 divides $x^k + 1$ exactly when $k = 3$.

Lean statement

```
theorem dvd_add_one_iff_exp_eq_three (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) : (7 : ℤ) ∣ x ^ k + 1 ↔ k = 3
```

Parent. Let x be an integer congruent to 5 modulo 7. Show that 7 does not divide x^{15} .

Parent in Lean

```
theorem not_dvd_pow_fifteen_of_modEq_five (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ (7 : ℤ) ∣ x ^ 15
```

What it contributes. Supplied the invariant $x \equiv 5$ modulo 7 and the modular-power transport method.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes nondivisibility of a fixed fifteenth power into a bounded exact characterization for all exponents up to 3. It requires transporting a variable power and performing a finite exponent analysis, so the parent's proof cannot be recalled to solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a valid bounded characterization: it transports the residue for arbitrary k and separately checks $k=0,1,2,3$, with divisibility occurring only at $k=3$. The parent's fixed-exponent contradiction proof does not supply this case analysis or characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes nondivisibility of a fixed fifteenth power into a bounded exact characterization for all exponents up to 3. It requires transporting a variable power and checking each bounded exponent, so recalling the parent proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2439 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dvd_add_one_iff_exp_eq_three.extracted_1_1 (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) :  
  7 ∣ x ^ k + 1 ↔ k = 3
```


Read back by a model that saw only the Lean. For an integer x satisfying $x \equiv 5$ modulo 7 and a natural number k with $k \leq 3$, 7 divides $x^k + 1$ if and only if $k = 3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem dvd_add_one_iff_exp_eq_three (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) : (7 : ℤ) ∣ x ^ k + 1 ↔ k = 3 := by
  have htransport : (7 : ℤ) ∣ x ^ k + 1 ↔ (7 : ℤ) ∣ (5 : ℤ) ^ k + 1 := by
    have hp : x ^ k + 1 ≡ (5 : ℤ) ^ k + 1 [ZMOD 7] := by
      exact (h.pow k).add_right 1
    constructor
    · intro hd
      have hz : x ^ k + 1 ≡ 0 [ZMOD 7] := by
        rw [Int.modEq_zero_iff_dvd]
        exact hd
      have hz' : (5 : ℤ) ^ k + 1 ≡ 0 [ZMOD 7] := hp.symm.trans hz
      rw [Int.modEq_zero_iff_dvd] at hz'
      exact hz'
    · intro hd
      have hz : (5 : ℤ) ^ k + 1 ≡ 0 [ZMOD 7] := by
        rw [Int.modEq_zero_iff_dvd]
        exact hd
      have hz' : x ^ k + 1 ≡ 0 [ZMOD 7] := hp.trans hz
      rw [Int.modEq_zero_iff_dvd] at hz'
      exact hz'
  have hbase : (7 : ℤ) ∣ (5 : ℤ) ^ k + 1 ↔ k = 3 := by
    interval_cases k <|> norm_num
  exact htransport.trans hbase
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.mh.me__814266dc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 814266dc73c2bfad

4.118 118. square_translation_modulus

The problem

For positive integers n and all integers a, k , prove that translating a by a multiple of n preserves its square modulo n : $(a + nk)^2 \equiv a^2 \pmod{n}$.

Lean statement

```
theorem square_translation_modulus (a k n : ℤ) : (a + n * k) ^ 2 ≡ a ^ 2 [ZMOD n] := by
```

Parent. For all integers a and k , prove that translating a by a multiple of 3 preserves its square modulo 3: $(a + 3k)^2 \equiv a^2 \pmod{3}$.

Parent in Lean

```
theorem square_mod_three_translation (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3] := by
```

What it contributes. Generalizes the parent's square-expansion and modular-translation checkpoint from fixed modulus 3 to an arbitrary positive integer modulus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the modulus from the fixed value 3 to an arbitrary integer n , and its proof requires explicit divisibility reasoning rather than reusing the parent's modulo-3 simplification. The claim is valid even without a positivity assumption on n .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the modulus from the fixed value 3 to an arbitrary integer n , so recalling the parent does not solve it. Its proof must establish divisibility for a symbolic modulus rather than reuse the parent's concrete modular simplification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed modulus 3 to an arbitrary integer modulus and proves the result via divisibility and a congruence calculation, rather than recalling the parent's simp-based modulo-3 proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2687 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_translation_modulus.extracted_1_1 (a k n : ℤ) (hn : 0 < n) : (a + n * k) ^ 2 ≡ a ^ 2 [ZMOD n]
```

Read back by a model that saw only the Lean. For integers a , k , and n with $n > 0$, $(a + nk)^2$ is congruent to a^2 modulo n .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_translation_modulus (a k n : ℤ) : (a + n * k) ^ 2 ≡ a ^ 2 [ZMOD n] := by
  have h_expand : (a + n * k) ^ 2 = a ^ 2 + n * (2 * a * k + n * k ^ 2) := by
    ring
  have h_multiple : n * (2 * a * k + n * k ^ 2) ≡ 0 [ZMOD n] := by
    apply Int.modEq_zero_iff_dvd.mpr
    refine ⟨2 * a * k + n * k ^ 2, ?_⟩
    ring
  calc
    (a + n * k) ^ 2 = a ^ 2 + n * (2 * a * k + n * k ^ 2) := h_expand
    _ ≡ a ^ 2 + 0 [ZMOD n] := h_multiple.add_left (a ^ 2)
    _ = a ^ 2 := by ring
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__me__7c947b95

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6948b265b39ea149

4.119 119. square_mod_three_translation

The problem

For all integers a and k , prove that translating a by a multiple of 3 preserves its square modulo 3: $(a + 3k)^2 \equiv a^2 \pmod{3}$.

Lean statement

```
theorem square_mod_three_translation (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3] := by
```

Parent. Show that the square of any integer is congruent to 0 or 1 modulo 3.

Parent in Lean

```
theorem numbertheory_sqmod3in01d (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ∨ a ^ 2 ≡ 1 [ZMOD 3] := by
```

What it contributes. Uses the parent's modulus-3 square context, while replacing its residue classification with a new congruence-expansion checkpoint for arbitrary translations by $3k$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different translation-invariance congruence: expanding $(a+3k)^2$ and showing the difference is divisible by 3. The parent only classifies square residues modulo 3, so recalling its case split does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different claim: translating by a multiple of 3 preserves the square modulo 3. Its algebraic expansion and divisibility argument do not reuse the parent's residue-class case split, so recalling the parent would not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a distinct translation-invariance congruence, requiring polynomial expansion and showing the added multiple-of-3 terms vanish modulo 3. The parent's residue case split only classifies square residues and does not directly supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2680 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_mod_three_translation.extracted_1_1 (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3]
```

Read back by a model that saw only the Lean. For all integers a and k , $(a + 3k)^2$ is congruent to a^2 modulo 3.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_mod_three_translation (a k : ℤ) : (a + 3 * k) ^ 2 ≡ a ^ 2 [ZMOD 3] := by
  have h_expand : (a + 3 * k) ^ 2 = a ^ 2 + 3 * (2 * a * k + 3 * k ^ 2) := by
    ring
  rw [h_expand]
  simp [Int.ModEq, Int.add_emod, Int.mul_emod]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__me__46dc4c00

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 46dc4c00eaa36df3

4.120 120. aime_1983_p9_mutation

The problem

Let $x \in \mathbb{R}$, let $S = (0, \pi)$, and define $f(t) = \frac{9t^2 \sin^2 t + 4}{t \sin t}$. For every y of the form $f(z)$ with $z \in S$, we have $12 \leq y$, and $y = 12$ if and only if there exists $z \in S$ such that $z \sin z = \frac{2}{3}$ and $y = f(z)$.

Lean statement

```
theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  ∀ y ∈ Set.image f S,
    12 ≤ y ∧
    (y = 12 ↔ ∃ z ∈ S, z * Real.sin z = 2 / 3 ∧ y = f z) := by
```

Parent. Find the minimum value of $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ for $0 < x < \pi$. Show that it is 12.

Parent in Lean

```
theorem aime_1983_p9
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from an IsLeast assertion to a pointwise characterization of exactly when an image value equals 12. Besides reproving the lower bound, it must derive the equality condition $t = z \cdot \sin z = 2/3$ and prove both directions of the equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum claim into a full equality characterization: it must prove that equality forces $z \cdot \sin z = 2/3$, then establish the converse for any witness. This equality-case reasoning is not supplied by the parent's IsLeast proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum claim into an equality characterization for every image value. Proving that equality forces $z \cdot \sin z = 2/3$ requires a new algebraic equality-case argument beyond the parent's lower-bound and witness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1833 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1983_p9_mutation.extracted_1.1 : ∀ (S : Set ℝ) (f : ℝ → ℝ),
  (f = fun x => (9 * (x ^ 2 * sin x ^ 2) + 4) / (x * sin x)) →
  S = Set.Ioo 0 π → ∀ y ∈ f '' S, 12 ≤ y ∧ (y = 12 ↔ ∃ z ∈ S, z * sin z = 2 / 3 ∧ y = f z)
```

Read back by a model that saw only the Lean. For every set S of real numbers and every function f from the real numbers to the real numbers, if $f(x) = (9(x^2 \sin^2 x) + 4)/(x \sin x)$ for all real x , and S is the open interval $(0, \pi)$, then for every real number y in the image $f(S)$, we have $12 \leq y$, and $y = 12$ if and only if there exists z in S such that $z \sin z = 2/3$ and $y = f(z)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  ∀ y ∈ Set.image f S,
    12 ≤ y ∧
    (y = 12 ↔ ∃ z ∈ S, z * Real.sin z = 2 / 3 ∧ y = f z) := by
  rw [h₀, h₁]
  intro y hy
  rcases hy with ⟨z, hz, rfl⟩
  have hvalue : ∀ q : ℝ, q ∈ Set.Ioo 0 Real.pi → q * Real.sin q = 2 / 3 →
    (9 * (q ^ 2 * Real.sin q ^ 2) + 4) / (q * Real.sin q) = 12 := by
    intro q hq hqval
    have hsq : q ^ 2 * Real.sin q ^ 2 = (q * Real.sin q) ^ 2 := by
      ring
    rw [hsq, hqval]
    norm_num
  have hzpos : 0 < z * Real.sin z := by
    exact mul_pos hz.1 (Real.sin_pos_of_pos_of_lt_pi hz.1 hz.2)
  let t : ℝ := z * Real.sin z
  have hsq : z ^ 2 * Real.sin z ^ 2 = t ^ 2 := by
    dsimp [t]
    ring
  have hbound : 12 ≤ (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) := by
    apply (le_div_iff hzpos).2
    rw [hsq]
    nlinarith [sq_nonneg (3 * t - 2)]
  constructor
  · exact hbound
  · constructor
    · intro heq
      have hmul := (div_eq_iff (ne_of_gt hzpos)).mp heq
      rw [hsq] at hmul
      have ht : t = 2 / 3 := by
        dsimp [t] at hmul
        nlinarith [sq_nonneg (3 * t - 2)]
      refine ⟨z, hz, ?, rfl⟩
      simp [t] using ht
    · rintro ⟨w, hw, hwt, hyw⟩
```

```
calc
  (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) =
    (9 * (w ^ 2 * Real.sin w ^ 2) + 4) / (w * Real.sin w) := hyw
  _ = 12 := hvalue w hw hwt
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p9__mh__1afd2f1c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8f8979c2b397776c

4.121 121. aime_1983_p9_mutation

The problem

For $0 < x < \pi$, prove that $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ has minimum value 12, that this value is attained, and that equality holds exactly when $x \sin x = \frac{2}{3}$.

Lean statement

```
theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 ∧
    (∃ z, z ∈ S ∧ f z = 12) ∧
    (∀ z, z ∈ S → (f z = 12 ↔ z * Real.sin z = 2 / 3))
```

Parent. Find the minimum value of $\frac{9x^2 \sin^2 x + 4}{x \sin x}$ for $0 < x < \pi$. Show that it is 012.

Parent in Lean

```
theorem aime_1983_p9
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The added equivalence is a genuine characterization: proving that any point attaining the minimum must satisfy $z \backslash \sin z = 2/3$ requires a new reverse algebraic argument, not supplied by the parent's existence and lower-bound proof. The conjunction also retains the parent result, but the appended claim is mathematically substantive rather than decorative.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine characterization of every point attaining the minimum: proving $f(z)=12$ iff $z \backslash \sin z = 2/3$ requires an equality-case algebra argument not supplied by the parent's least-value proof. The existential conjunct is redundant, but the universal iff changes the proof obligation substantively.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2368 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1983_p9_mutation.extracted_1_1 (S : Set ℝ) (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * sin x ^ 2) + 4) / (x * sin x)) (h₁ : S = Set.Ioo 0 π) :
  IsLeast (f '' S) 12 ∧ (∃ z ∈ S, f z = 12) ∧ ∀ z ∈ S, f z = 12 ↔ z * sin z = 2 / 3
```

Read back by a model that saw only the Lean. Let S be a subset of the real numbers and let f be a real-valued function on the real numbers. Suppose that, for every real number x , $f(x) = (9(x^2 \sin^2 x) + 4)/(x \sin x)$, and that S is the open interval $(0, \pi)$. Then 12 is the least element of the image $f(S)$; there exists some z in S such that $f(z) = 12$; and for every z in S , $f(z) = 12$ if and only if $z \sin z = 2/3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem aime_1983_p9_mutation
  (x : ℝ)
  (S : Set ℝ)
  (f : ℝ → ℝ)
  (h₀ : f = fun x => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x))
  (h₁ : S = Set.Ioo 0 Real.pi) :
  IsLeast (Set.image f S) 12 ∧
    (∃ z, z ∈ S ∧ f z = 12) ∧
    (∀ z, z ∈ S → (f z = 12 ↔ z * Real.sin z = 2 / 3)) := by
  subst f
  subst S
  let g : ℝ → ℝ := fun t => t * Real.sin t
  have hg : Continuous g := continuous_id.mul Real.continuous_sin
  have hpi : 0 ≤ Real.pi / 2 := by positivity
  have htarget : (2 / 3 : ℝ) ∈ Set.Icc (g 0) (g (Real.pi / 2)) := by
    constructor
    · norm_num [g]
    · simp [g, Real.sin_pi_div_two]
      nlinarith [Real.pi_gt_three]
  obtain ⟨z, hz, hzval⟩ :=
    intermediate_value_Icc hpi hg.continuousOn htarget
  have hzval' : z * Real.sin z = 2 / 3 := by
    simp [g] using hzval
  have hz0 : 0 < z := by
    have hzle : 0 ≤ z := hz.1
    have hzne : z ≠ 0 := by
      intro h
      subst z
      norm_num at hzval'
      exact lt_of_le_of_ne hzle (Ne.symm hzne)
  have hzpi : z < Real.pi := by
    nlinarith [hz.2, Real.pi_pos]
  have hleast : IsLeast (Set.image (fun x : ℝ => (9 * (x ^ 2 * Real.sin x ^ 2) + 4) / (x * Real.sin x)) (Set.Ioo 0 Real.pi)) 12 := by
    constructor
    · refine ⟨z, ⟨hz0, hzpi⟩, ?_⟩
      change (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) = 12
      have hsq : z ^ 2 * Real.sin z ^ 2 = (z * Real.sin z) ^ 2 := by ring
      rw [hsq, hzval']
      norm_num
    · rintro y ⟨w, hw, rfl⟩
      have hwpos : 0 < w * Real.sin w := by
        exact mul_pos hw.1 (Real.sin_pos_of_pos_of_lt_pi hw.1 hw.2)
```

```

have hsq : w ^ 2 * Real.sin w ^ 2 = (w * Real.sin w) ^ 2 := by ring
apply (le_div_iff hwpos).2
rw [hsq]
nlinarith [sq_nonneg (3 * (w * Real.sin w) - 2)]
refine (hleast, ?_, ?_)
· rcases hleast.1 with ⟨z, hz, hzval⟩
  exact ⟨z, hz, hzval⟩
· intro z hz
  have hsin : 0 < Real.sin z := by
    exact Real.sin_pos_of_pos_of_lt_pi hz.1 hz.2
  have hypos : 0 < z * Real.sin z := by
    exact mul_pos hz.1 hsin
  have hsq : z ^ 2 * Real.sin z ^ 2 = (z * Real.sin z) ^ 2 := by ring
  constructor
  · intro heq
    have heq' : (9 * (z * Real.sin z) ^ 2 + 4) / (z * Real.sin z) = 12 := by
      simpa only [hsq] using heq
    have hmul : 9 * (z * Real.sin z) ^ 2 + 4 = 12 * (z * Real.sin z) := by
      exact (div_eq_iff (ne_of_gt hypos)).mp heq'
    have hzero : (3 * (z * Real.sin z) - 2) ^ 2 = 0 := by
      nlinarith [hmul]
    nlinarith [hzero]
  · intro heq
    calc
      (9 * (z ^ 2 * Real.sin z ^ 2) + 4) / (z * Real.sin z) =
        (9 * (z * Real.sin z) ^ 2 + 4) / (z * Real.sin z) := by rw [hsq]
      _ = 12 := by rw [heq]; norm_num

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. aime_1983_p9__mh__6f3029bc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint 6f3029bc9d9f901a

4.122 122. aime_1984_p15_squared_tuple

The problem

Suppose real numbers x, y, z, w satisfy the four weighted-square equations for $n = 2, 4, 6, 8$. Determine the exact values of x^2, y^2, z^2, w^2 .

Lean statement

```
theorem aime_1984_p15_squared_tuple (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hy : y ^ 2 = 10395 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hz : z ^ 2 = 9009 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hw : w ^ 2 = 6435 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  exact ⟨hx, hy, hz, hw⟩
```

Parent. Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = 1$. What exact value must x^2 equal?

Parent in Lean

```
theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from determining only x^2 to determining the complete tuple of squared variables. The parent's proof supplies only the x^2 component; proving the three additional exact values requires extending the linear-system reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from determining only x^2 to determining the complete tuple (x^2, y^2, z^2, w^2) . The parent's proof supplies only the first component, so the additional three values require genuinely new deductions from the coupled equations.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from determining only x^2 to determining the complete tuple (x^2, y^2, z^2, w^2) . Proving the additional coordinates requires extracting new information from the coupled linear system, beyond the parent's single-coordinate result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2722 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_squared_tuple.extracted_1_1 (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024
```

Read back by a model that saw only the Lean. Let x, y, z , and w be real numbers. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x^2 = 11025/1024$, $y^2 = 10395/1024$, $z^2 = 9009/1024$, and $w^2 = 6435/1024$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_squared_tuple (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 ∧ y ^ 2 = 10395 / 1024 ∧ z ^ 2 = 9009 / 1024 ∧ w ^ 2 = 6435 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hy : y ^ 2 = 10395 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hz : z ^ 2 = 9009 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hw : w ^ 2 = 6435 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  exact ⟨hx, hy, hz, hw⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh.ms.mh__5434b06d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5434b06d384cdcad

4.123 123. aime_1984_p15_x_two_roots

The problem

Suppose real numbers x, y, z, w satisfy the four weighted-square equations above. Show that $x = 105/32$ or $x = -105/32$.

Lean statement

```
theorem aime_1984_p15_x_two_roots (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x = 105 / 32 ∨ x = -(105 / 32) := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hfactor : (x - 105 / 32) * (x + 105 / 32) = 0 := by
    nlinarith [hx_sq]
  rcases mul_eq_zero.mp hfactor with hpos | hnég
  · left
    nlinarith
  · right
    nlinarith
```

Parent. Suppose x, y, z, w are real numbers satisfying the four weighted-square equations above, and $x \geq 0$. Determine x .

Parent in Lean

```
theorem aime_1984_p15_x (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1)
  (hx : 0 ≤ x) : x = 105 / 32 := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  nlinarith [hx_sq]
```

What it contributes. Reuses the parent’s weighted-square hypotheses and linear-elimination checkpoint, while removing the sign hypothesis and adding an explicit square-factorization step for the two-root conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the sign hypothesis and replaces the single signed-value conclusion with a genuine two-root classification. After deriving the squared value as in the parent, it must factor the quadratic and perform a zero-product case split, which the parent’s final nonnegativity argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the nonnegativity hypothesis and correctly classifies both possible real roots from the derived squared value. Its factorization and case split require reasoning not supplied by the parent’s sign-based conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the sign hypothesis and changes the conclusion to a complete two-root classification. The parent's nonnegativity-based final step cannot establish the negative branch, so recalling the parent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2701 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x_two_roots.extracted_1_1 (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x = 105 / 32 ∨ x = -(105 / 32)
```

Read back by a model that saw only the Lean. For real numbers $x, y, z,$ and w , suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x = 105/32$ or $x = -(105/32)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x_two_roots (x y z w : ℝ)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
x = 105 / 32 ∨ x = -(105 / 32) := by
norm_num at h₀ h₁ h₂ h₃
have hx_sq : x ^ 2 = 11025 / 1024 := by
  linarith [h₀, h₁, h₂, h₃]
have hfactor : (x - 105 / 32) * (x + 105 / 32) = 0 := by
  nlinarith [hx_sq]
rcases mul_eq_zero.mp hfactor with hpos | hneg
· left
  nlinarith
· right
  nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh__9b2d27e0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9b2d27e03931a264

4.124 124. unique_affine_coefficients

The problem

For any complex numbers a , b , and c , show that there exists a unique pair of complex numbers (u, v) such that, for every complex number d , $(a - d)(a - c)(a - b) = ud + v$.

Lean statement

```
theorem unique_affine_coefficients (a b c : ℂ) :
  ∃! p : ℂ × ℂ, ∀ d : ℂ,
    (a - d) * (a - c) * (a - b) = p.1 * d + p.2
```

Parent. Show that for any complex numbers a, b, c, d , $(a-d)(a-c)(a-b) = -(((a^2-(b+c)a)+cb)d)+(a^2-(b+c)a+cb)a$.

Parent in Lean

```
theorem algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta (a b c d : ℂ) :
  (a - d) * (a - c) * (a - b) =
  -((a ^ 2 - (b + c) * a + c * b) * d) + (a ^ 2 - (b + c) * a + c * b) * a := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the goal from an algebraic identity to a uniqueness statement for affine coefficients. Although the parent suggests the witness and helps with existence, the child additionally proves coefficient uniqueness by evaluating at $d = 0$ and $d = 1$, which the parent's ring proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an algebraic identity into a genuine existence-and-uniqueness characterization of the affine coefficients. Its proof must establish uniqueness by evaluating at two distinct values and solving the resulting equations, which the parent's ring proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a genuine uniqueness theorem for affine coefficients. Its proof requires extracting the coefficients by evaluating at $d = 0$ and $d = 1$ and proving product equality, reasoning not supplied by the parent's direct ring normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1837 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_affine_coefficients.extracted_1_1 (a b c : ℂ) :  
  ∃! p, ∀ (d : ℂ), (a - d) * (a - c) * (a - b) = p.1 * d + p.2
```

Read back by a model that saw only the Lean. For any complex numbers a , b , and c , there exists a unique ordered pair p of complex numbers such that, for every complex number d , $(a - d)(a - c)(a - b) = p_1 d + p_2$, where p_1 and p_2 are the first and second components of p .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem unique_affine_coefficients (a b c : ℂ) :  
  ∃! p : ℂ × ℂ, ∀ d : ℂ,  
    (a - d) * (a - c) * (a - b) = p.1 * d + p.2 := by  
  refine ⟨(-(a - c) * (a - b)), a * (a - c) * (a - b), ?_, ?_⟩  
  · intro d  
    ring  
  · intro p hp  
    have h0 := hp 0  
    have h1 := hp 1  
    apply Prod.ext  
    · change p.1 = -(a - c) * (a - b)  
      linear_combination h0 - h1  
    · change p.2 = a * (a - c) * (a - b)  
      linear_combination -h0
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_3rootspoly_amdtamctambeqnasqmbpctapcbtdpasqmbpctapcbta__mh__9b14d385

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8a5c6f9045c9c663

4.125 125. reciprocal_equality_characterization

The problem

For positive real numbers x , y , and z satisfying $x + y + z = 1$, prove that $9/(x + y + z) = 2/(x + y) + 2/(y + z) + 2/(z + x)$ if and only if $x = y = z = 1/3$.

Lean statement

```
theorem reciprocal_equality_characterization (x y z : ℝ) (hpos : 0 < x ∧ 0 < y ∧ 0 < z) (hsum : x + y + z = 1) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔
  x = 1 / 3 ∧ y = 1 / 3 ∧ z = 1 / 3
```

Parent. Show that for any three positive real numbers x , y , and z , $9/(x + y + z) \leq 2/(x + y) + 2/(y + z) + 2/(z + x)$.

Parent in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (he : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Supplies the reciprocal-sum lower-bound structure and its denominator-clearing bound checkpoint; the child adds a new equality-gap rigidity argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the inequality into an equality characterization, requiring extraction of equality from a sum-of-squares identity and then using $x+y+z=1$ to derive $x=y=z=1/3$. The parent's proof establishes nonnegativity but does not supply the zero-case analysis or uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality into an equality characterization: it must extract vanishing of each squared-difference term and then derive $x=y=z=1/3$. The parent only proves nonnegativity and supplies no equality-case reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an inequality into a genuine equality characterization. Its forward direction requires proving that equality forces every nonnegative squared-difference term to vanish, then deriving $x=y=z=1/3$; the parent only establishes nonnegativity and supplies no equality-case reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1844 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_equality_characterization.extracted_1_1 (x y z : ℝ) (hpos : 0 < x ∧ 0 < y ∧ 0 < z)
(hsum : x + y + z = 1) :
9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = 1 / 3 ∧ y = 1 / 3 ∧ z = 1 / 3
```

Read back by a model that saw only the Lean. For real numbers x, y , and z satisfying $x > 0, y > 0, z > 0$, and $x + y + z = 1$, the equality $9/(x + y + z) = 2/(x + y) + 2/(y + z) + 2/(z + x)$ holds if and only if $x = 1/3, y = 1/3$, and $z = 1/3$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem reciprocal_equality_characterization (x y z : ℝ) (hpos : 0 < x ∧ 0 < y ∧ 0 < z) (hsum : x + y + z = 1) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔
  x = 1 / 3 ∧ y = 1 / 3 ∧ z = 1 / 3 := by
  rcases hpos with ⟨hx, hy, hz⟩
  have hxy : 0 < x + y := by linarith
  have hyz : 0 < y + z := by linarith
  have hzx : 0 < z + x := by linarith
  have hxyz : 0 < x + y + z := by linarith
  constructor
  · intro heq
    have hpoly : 9 * (x + y) * (y + z) * (z + x) =
      2 * (x + y + z) * ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) := by
      field_simp [ne_of_gt hxyz, ne_of_gt hxy, ne_of_gt hyz, ne_of_gt hzx] at heq
      nlinarith [heq]
    have hid : 2 * (x + y + z) * ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -
      9 * (x + y) * (y + z) * (z + x) =
      (x + y) * ((y + z) - (z + x)) ^ 2 +
      (y + z) * ((z + x) - (x + y)) ^ 2 +
      (z + x) * ((x + y) - (y + z)) ^ 2 := by ring
    have hzero : (x + y) * ((y + z) - (z + x)) ^ 2 +
      (y + z) * ((z + x) - (x + y)) ^ 2 +
      (z + x) * ((x + y) - (y + z)) ^ 2 = 0 := by
      rw [← hid]
      nlinarith [hpoly]
    have ht1 : 0 ≤ (x + y) * ((y + z) - (z + x)) ^ 2 := by positivity
    have ht2 : 0 ≤ (y + z) * ((z + x) - (x + y)) ^ 2 := by positivity
    have ht3 : 0 ≤ (z + x) * ((x + y) - (y + z)) ^ 2 := by positivity
    have h1 : (x + y) * ((y + z) - (z + x)) ^ 2 = 0 := by
      nlinarith [hzero, ht2, ht3]
    have h2 : (y + z) * ((z + x) - (x + y)) ^ 2 = 0 := by
      nlinarith [hzero, ht1, ht3]
    have h3 : (z + x) * ((x + y) - (y + z)) ^ 2 = 0 := by
      nlinarith [hzero, ht1, ht2]
    have hs1 : ((y + z) - (z + x)) ^ 2 = 0 := by
      rcases mul_eq_zero.mp h1 with h | h
      · exfalso; linarith
      · exact h
    have hs2 : ((z + x) - (x + y)) ^ 2 = 0 := by
      rcases mul_eq_zero.mp h2 with h | h
      · exfalso; linarith
      · exact h
    have hs3 : ((x + y) - (y + z)) ^ 2 = 0 := by
      rcases mul_eq_zero.mp h3 with h | h
      · exfalso; linarith
      · exact h
    have hxy' : x = y := by nlinarith [hs3]
    have hyz' : y = z := by nlinarith [hs1]
    have hxval : x = 1 / 3 := by nlinarith [hsum, hxy', hyz']
    exact ⟨hxval, by nlinarith [hxy'], by nlinarith [hyz']⟩
  · rintro ⟨rfl, rfl, rfl⟩
    norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__mh__4daebd7c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 67c46be14930e086

4.126 126. reciprocal_inequality_eq_iff

The problem

For positive real numbers x , y , and z , equality holds in the reciprocal inequality exactly when $x = y = z$.

Lean statement

```
theorem reciprocal_inequality_eq_iff (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = y ∧ y = z
```

Parent. Show that for any three positive real numbers x , y , and z , $9/(x + y + z) \leq 2/(x + y) + 2/(y + z) + 2/(z + x)$.

Parent in Lean

```
theorem algebra_9onxpyzleqsum2onxpy (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :
  9 / (x + y + z) ≤ 2 / (x + y) + 2 / (y + z) + 2 / (z + x) := by
```

What it contributes. Reuses the parent reciprocal expression, positivity setup, common-denominator normalization, and numerator comparison, while replacing the inequality conclusion with a full equality-locus characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the inequality into an exact equality characterization, requiring the converse direction to extract vanishing squared differences and prove $x = y = z$. The parent's nonnegative-polynomial argument only establishes the inequality and does not supply this equality-locus reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality into an exact equality characterization. Its reverse direction requires deriving vanishing square terms from a sum-of-squares decomposition, reasoning not supplied by the parent's one-sided inequality proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality into an exact equality characterization, requiring the new direction that equality forces each squared-difference term to vanish and hence $x=y=z$. The parent only proves the inequality and does not supply this equality-locus argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2569 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_inequality_eq_iff.extracted_1_1 (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :  
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = y ∧ y = z
```

Read back by a model that saw only the Lean. For real numbers x, y , and z satisfying $0 < x$, $0 < y$, and $0 < z$, the equality $9/(x + y + z) = 2/(x + y) + 2/(y + z) + 2/(z + x)$ holds if and only if $x = y$ and $y = z$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem reciprocal_inequality_eq_iff (x y z : ℝ) (h₀ : 0 < x ∧ 0 < y ∧ 0 < z) :  
  9 / (x + y + z) = 2 / (x + y) + 2 / (y + z) + 2 / (z + x) ↔ x = y ∧ y = z := by  
  rcases h₀ with ⟨hx, hy, hz⟩  
  have hxy : 0 < x + y := by linarith  
  have hyz : 0 < y + z := by linarith  
  have hzx : 0 < z + x := by linarith  
  have hxyz : 0 < x + y + z := by linarith  
  have hden : 0 < (x + y + z) * (x + y) * (y + z) * (z + x) := by  
    positivity  
  have hleft :  
    9 / (x + y + z) =  
      (9 * (x + y) * (y + z) * (z + x)) /  
      ((x + y + z) * (x + y) * (y + z) * (z + x)) := by  
    field_simp [ne_of_gt hxyz, ne_of_gt hxy, ne_of_gt hyz, ne_of_gt hzx] <|> ring  
  have hright :  
    2 / (x + y) + 2 / (y + z) + 2 / (z + x) =  
      (2 * (x + y + z) *  
      ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y))) /  
      ((x + y + z) * (x + y) * (y + z) * (z + x)) := by  
    field_simp [ne_of_gt hxyz, ne_of_gt hxy, ne_of_gt hyz, ne_of_gt hzx] <|> ring  
  constructor  
  · intro heq  
    rw [hleft, hright] at heq  
    have hcross := (div_eq_div_iff (ne_of_gt hden) (ne_of_gt hden)).mp heq  
    have hprod :  
      (2 * (x + y + z) *  
      ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -  
      9 * (x + y) * (y + z) * (z + x)) *  
      ((x + y + z) * (x + y) * (y + z) * (z + x)) = 0 := by  
      nlinarith [hcross]  
    have hzero :  
      2 * (x + y + z) *  
      ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -  
      9 * (x + y) * (y + z) * (z + x) = 0 := by  
      rcases mul_eq_zero.mp hprod with h | h  
      · exact h  
      · exact False.elim (ne_of_gt hden h)  
    have hdecomp :  
      2 * (x + y + z) *  
      ((x + y) * (y + z) + (y + z) * (z + x) + (z + x) * (x + y)) -  
      9 * (x + y) * (y + z) * (z + x) =  
      (x + y) * (x - y) ^ 2 + (y + z) * (y - z) ^ 2 +  
      (z + x) * (z - x) ^ 2 := by  
      ring  
    have ht1 : 0 ≤ (x + y) * (x - y) ^ 2 :=  
      mul_nonneg (le_of_lt hxy) (sq_nonneg _)  
    have ht2 : 0 ≤ (y + z) * (y - z) ^ 2 :=  
      mul_nonneg (le_of_lt hyz) (sq_nonneg _)  
    have ht3 : 0 ≤ (z + x) * (z - x) ^ 2 :=  
      mul_nonneg (le_of_lt hzx) (sq_nonneg _)  
    have hs1 : (x - y) ^ 2 = 0 := by  
      nlinarith [hzero, hdecomp]  
    have hs2 : (y - z) ^ 2 = 0 := by  
      nlinarith [hzero, hdecomp]  
    have hxy' : x = y := by nlinarith [hs1]
```

```

have hyz' : y = z := by nlinarith [hs2]
exact ⟨hxy', hyz'⟩
· rintro ⟨hxy', hyz'⟩
  subst y
  subst z
  field_simp [ne_of_gt hx] <|> ring

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_9onxpypzleqsum2onxpy__mh__bcee330d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint bcee330ddcbc6305

4.127 127. reciprocal_amgm_eq_iff

The problem

Let x be a positive real number. Show that $2 - \sqrt{2} = 2 - x - \frac{1}{2x}$ if and only if $x = \frac{\sqrt{2}}{2}$.

Lean statement

```
theorem reciprocal_amgm_eq_iff :
  ∀ x : ℝ, x > 0 → (2 - Real.sqrt 2 = 2 - x - 1 / (2 * x) ↔ x = Real.sqrt 2 / 2)
```

Parent. Let x be a positive real number. Show that $2 - \sqrt{2} \geq 2 - x - \frac{1}{2x}$.

Parent in Lean

```
theorem algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x :
  ∀ x > 0, 2 - Real.sqrt 2 ≥ 2 - x - 1 / (2 * x) := by
```

What it contributes. Reuses the parent's positive-real reciprocal expression and square-root normalization, while replacing the inequality target with an equality-case characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the inequality into an equality-case characterization, requiring a new factorization and zero-square argument to derive $x = \sqrt{2}/2$, plus a converse verification. The parent's proof establishes only the bound and does not supply this uniqueness reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child characterizes the equality case, proving that equality forces the square term $(2x - \sqrt{2})^2$ to vanish and hence $x = \sqrt{2}/2$, then verifies the converse. The parent only proves the reciprocal AM-GM inequality and supplies no equality analysis.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the equality case, requiring a square-factorization argument to show equality forces $2x = \sqrt{2}$ and a separate verification of the converse. The parent's inequality proof does not supply this uniqueness/equality reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1732 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_amgm_eq_iff.extracted_1_1 : ∀ x > 0, 2 - √2 = 2 - x - 1 / (2 * x) ↔ x = √2 / 2
```

Read back by a model that saw only the Lean. *For every real number $x > 0$, $2 - \sqrt{2} = 2 - x - 1/(2x)$ if and only if $x = \sqrt{2}/2$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem reciprocal_amgm_eq_iff :
  ∀ x : ℝ, x > 0 → (2 - Real.sqrt 2 = 2 - x - 1 / (2 * x) ↔ x = Real.sqrt 2 / 2) := by
  intro x hx
  have hs2 : (Real.sqrt 2) ^ 2 = (2 : ℝ) := by
    norm_num
  have hx0 : x ≠ 0 := ne_of_gt hx
  constructor
  · intro heq
    have hsum : x + 1 / (2 * x) = Real.sqrt 2 := by
      linarith
    have hfactor :
      (2 * x - Real.sqrt 2) ^ 2 =
        4 * x * (x + 1 / (2 * x) - Real.sqrt 2) := by
      field_simp [hx0]
      nlinarith [hs2]
    have hzero : (2 * x - Real.sqrt 2) ^ 2 = 0 := by
      rw [hfactor, hsum]
      ring
      nlinarith [hzero]
    · intro hxval
      rw [hxval]
      have hspos : 0 < Real.sqrt 2 := by positivity
      have hs0 : Real.sqrt 2 ≠ 0 := ne_of_gt hspos
      field_simp [hs0]
      nlinarith [hs2]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. algebra_amgm_faxinrrp2msqrt2geq2mxm1div2x__mh__7650d2bf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2c19cbaa9f79a4a6

4.128 128. bounded_subtractive_prime_balance_unique_complement

The problem

Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that every integer root of $z^2 - (p + q - 2)z + t + 1$ has a unique complementary integer root whose sum with it is $p + q - 2$ and whose product with it is $t + 1$.

Lean statement

```
theorem bounded_subtractive_prime_balance_unique_complement :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 →
      ∃! w : ℤ,
        w^2 - ((p : ℤ) + (q : ℤ) - 2) * w + (t : ℤ) + 1 = 0 ∧
        z + w = (p : ℤ) + (q : ℤ) - 2 ∧
        z * w = (t : ℤ) + 1
```

Parent. Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root characterization into a genuine existence-and-uniqueness theorem for the complementary root. Its witness construction, symmetry argument, product derivation, and uniqueness from the sum are new obligations not supplied by the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from characterizing all roots to proving existence and uniqueness of a complementary root, requiring a symmetry argument and a separate uniqueness proof from the sum relation. The parent's root-factorization proof does not directly supply this construction or uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2463 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_prime_balance_unique_complement.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
  p * q - (p + q) = t →
  ∀ (z : ℤ),
    z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1 = 0 →
    ∃! w, w ^ 2 - (↑p + ↑q - 2) * w + ↑t + 1 = 0 ∧ z + w = ↑p + ↑q - 2 ∧ z * w = ↑t + 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime numbers, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, and if the natural-number subtraction $p \cdot q - (p + q)$ equals t , then for every integer z satisfying $z^2 - (p + q - 2)z + t + 1 = 0$, there exists a unique integer w such that $w^2 - (p + q - 2)w + t + 1 = 0$, $z + w = p + q - 2$, and $z \cdot w = t + 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_prime_balance_unique_complement :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 →
      ∃! w : ℤ,
        w^2 - ((p : ℤ) + (q : ℤ) - 2) * w + (t : ℤ) + 1 = 0 ∧
        z + w = (p : ℤ) + (q : ℤ) - 2 ∧
        z * w = (t : ℤ) + 1 := by
  intro t p q hdom hbal z hz
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q) : ℕ) : ℤ) = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hconstant : (t : ℤ) + 1 = ((p : ℤ) - 1) * ((q : ℤ) - 1) :=
    hfactor.symm
  refine ⟨(p : ℤ) + (q : ℤ) - 2 - z, ?_, ?_⟩
  · constructor
    · have hsymm :
        ((p : ℤ) + (q : ℤ) - 2 - z)^2 -
          ((p : ℤ) + (q : ℤ) - 2) * ((p : ℤ) + (q : ℤ) - 2 - z) +
          (t : ℤ) + 1 =
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
          ring
        rw [hsymm]
        exact hz
    constructor
    · ring
    · nlinarith [hz, hconstant]
  · intro y hy
    rcases hy with ⟨hyroot, hysum, hyprod⟩
    linarith [hysum]
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh__5bc81901

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5bc8190112540d25

4.129 129. bounded_subtractive_prime_balance_positive_right_of_larger_root

The problem

Let $p < q$ be distinct primes between 4 and 18, and suppose their product minus their sum is t . Prove that $z^2 - (p + q - 2)z + t + 1 > 0$ for every integer $z > q - 1$.

Lean statement

```
theorem bounded_subtractive_prime_balance_positive_right_of_larger_root :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (q : ℤ) - 1 < z →
      0 < z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1
```

Parent. Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

What it contributes. Reused the parent's bounded-subtraction cast and factorization checkpoint, then consumed it in a new quantified strict-sign argument using the added ordering $p < q$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root characterization into a strict positivity claim and requires using the new ordering hypothesis to prove both factors are positive to the right of the larger root. The parent's zero-product argument does not supply this sign reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces an ordered pair and proves strict positivity beyond the larger root, requiring a new sign argument for both factored terms. The parent only characterizes the roots and does not supply this order-and-inequality reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2449 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_prime_balance_positive_right_of_larger_root.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
  p * q - (p + q) = t → ∀ (z : ℤ), ↑q - 1 < z → 0 < z ^ 2 - (↑p + ↑q - 2) * z + ↑t + 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, and $p < q$, and if $p \cdot q - (p + q) = t$ (with subtraction in the natural numbers), then for every integer z satisfying $q - 1 < z$, one has $0 < z^2 - (p + q - 2)z + t + 1$, where p , q , and t are viewed as integers in the inequality and expression.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_prime_balance_positive_right_of_larger_root :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p < q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (q : ℤ) - 1 < z →
      0 < z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
  intro t p q hdom hbal z hz
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne, hpq⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hpz : 0 < z - ((p : ℤ) - 1) := by
    have hp_int : (p : ℤ) < q := by
      exact_mod_cast hpq
    linarith
  have hqz : 0 < z - ((q : ℤ) - 1) := by
    linarith
  have hprod : 0 < (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := by
    exact mul_pos hpz hqz
  calc
    0 < (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) := hprod
    _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
      calc
        (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + ((p : ℤ) - 1) * ((q : ℤ) - 1) := by ring
        _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
          rw [hfactor]
          ring
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh__8f23d403

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8f23d4038c1f35ce

4.130 130. bounded_subtractive_balance_quadratic_nonnegative_outside

The problem

Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that $z^2 - (p + q - 2)z + t + 1$ is nonnegative whenever z lies outside the interval with endpoints $p - 1$ and $q - 1$.

Lean statement

```
theorem bounded_subtractive_balance_quadratic_nonnegative_outside :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (z ≤ (p : ℤ) - 1 ∧ z ≤ (q : ℤ) - 1) ∨
      ((p : ℤ) - 1 ≤ z ∧ (q : ℤ) - 1 ≤ z) →
      0 ≤ z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1
```

Parent. Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Parent in Lean

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from an exact root characterization to a nonnegativity result outside the interval between the two roots. Its proof requires new order and sign reasoning for the factored quadratic, which the parent's zero-product argument does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root-characterization equivalence into a sign claim outside both roots. Its proof requires new order and product-sign reasoning that the parent's root extraction argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from characterizing the two integer roots to proving nonnegativity on both exterior regions. Although it reuses the parent's factorization identity, it requires new sign reasoning for the two factors and is not recoverable by recalling the parent's iff proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2443 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_balance_quadratic_nonnegative_outside.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
  p * q - (p + q) = t →
  ∀ (z : ℤ), z ≤ tp - 1 ∧ z ≤ tq - 1 ∨ tp - 1 ≤ z ∧ tq - 1 ≤ z → 0 ≤ z ^ 2 - (tp + tq - 2) * z + tt + 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, and if the natural-number subtraction $p \cdot q - (p + q)$ equals t , then for every integer z satisfying either ($z \leq p - 1$ and $z \leq q - 1$) or ($p - 1 \leq z$ and $q - 1 \leq z$), one has $0 \leq z^2 - (p + q - 2)z + t + 1$, where the natural numbers p , q , and t are regarded as integers in the latter expression and inequalities.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_balance_quadratic_nonnegative_outside :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      (z ≤ (p : ℤ) - 1 ∧ z ≤ (q : ℤ) - 1) ∨
      ((p : ℤ) - 1 ≤ z ∧ (q : ℤ) - 1 ≤ z) →
      0 ≤ z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
  intro t p q hdom hbal z hout
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  have hproduct :
    (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
    z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
    calc
      (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
        z^2 - ((p : ℤ) + (q : ℤ) - 2) * z +
          ((p : ℤ) - 1) * ((q : ℤ) - 1) := by ring
      _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
        rw [hfactor]
        ring
  rcases hout with hout | hout
  · have hleft : z - ((p : ℤ) - 1) ≤ 0 := by linarith
    have hright : z - ((q : ℤ) - 1) ≤ 0 := by linarith
    have hnonneg : 0 ≤ (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) :=
      mul_nonneg_of_nonpos_of_nonpos hleft hright
    rw [hproduct] at hnonneg
    exact hnonneg
  · have hleft : 0 ≤ z - ((p : ℤ) - 1) := by linarith
    have hright : 0 ≤ z - ((q : ℤ) - 1) := by linarith
    have hnonneg : 0 ≤ (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) :=
```



```
mul_nonneg hleft hright  
rw [hproduct] at hnonneg  
exact hnonneg
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh.mh__9296a78a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9296a78a4f4fca68

4.131 131. bounded_subtractive_prime_balance_integer_roots

The problem

Let p and q be distinct primes between 4 and 18, and suppose their product minus their sum is t . Show that the integer roots of $z^2 - (p + q - 2)z + t + 1$ are exactly $p - 1$ and $q - 1$.

Lean statement

```
theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1
```

Parent. For every nonnegative integer t , characterize when t can be obtained by subtracting the sum of two distinct primes between 4 and 18 from their product. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Parent in Lean

```
theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

What it contributes. Reuses the bounded distinct-prime domain and the cast-based treatment of truncated natural subtraction, while changing the conclusion to an exhaustive integer-root classification with an explicit factorization checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from an existential arithmetic equivalence to an exact characterization of all integer roots, requiring polynomial factorization and the integer zero-product argument. The parent supplies only the coefficient identity, not the root analysis, so recalling its proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the equivalence of two arithmetic expressions into an integer-root characterization, requiring a new factorization argument and zero-product case split. Although it reuses the parent's algebraic identity, the parent proof does not supply the polynomial-root reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new root-characterization claim: after deriving the shifted-factor identity, it must factor the integer quadratic and use the integral-domain zero-product property to prove both directions. The parent only establishes an existential reformulation and does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2428 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_subtractive_prime_balance_integer_roots.extracted_1_1 : ∀ (t p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
  p * q - (p + q) = t → ∀ (z : ℤ), z ^ 2 - (tp + tq - 2) * z + tt + 1 = 0 ↔ z = tp - 1 ∨ z = tq - 1
```

Read back by a model that saw only the Lean. For all natural numbers t , p , and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, and if $p \cdot q - (p + q) = t$, then for every integer z , $z^2 - (p + q - 2)z + t + 1 = 0$ if and only if $z = p - 1$ or $z = q - 1$, where the natural numbers p , q , and t are regarded as integers in the equation.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_subtractive_prime_balance_integer_roots :
  ∀ t : ℕ, ∀ p q : ℕ,
    Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q →
    p * q - (p + q) = t →
    ∀ z : ℤ,
      z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 = 0 ↔
      z = (p : ℤ) - 1 ∨ z = (q : ℤ) - 1 := by
  intro t p q hdom hbal z
  rcases hdom with ⟨hp, hq, hp4, hp18, hq4, hq18, hne⟩
  have hle : p + q ≤ p * q := by
    nlinarith
  have hbal_z : (p : ℤ) * q - ((p : ℤ) + q) = (t : ℤ) := by
    have hcast : ((p * q - (p + q)) : ℕ) : ℤ = (t : ℤ) := by
      exact_mod_cast hbal
    rw [Nat.cast_sub hle] at hcast
    norm_num [Nat.cast_mul, Nat.cast_add] at hcast
    exact hcast
  have hfactor : ((p : ℤ) - 1) * ((q : ℤ) - 1) = (t : ℤ) + 1 := by
    nlinarith [hbal_z]
  constructor
  · intro hz
    have hprod : (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) = 0 := by
      calc
        (z - ((p : ℤ) - 1)) * (z - ((q : ℤ) - 1)) =
          z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + ((p : ℤ) - 1) * ((q : ℤ) - 1) := by ring
        _ = z^2 - ((p : ℤ) + (q : ℤ) - 2) * z + (t : ℤ) + 1 := by
          rw [hfactor]
          ring
        _ = 0 := hz
    rcases mul_eq_zero.mp hprod with hroot | hroot
    · left
      linarith
    · right
      linarith
  · intro hroot
    rcases hroot with hroot | hroot
    · subst z
      nlinarith [hfactor]
    · subst z
      nlinarith [hfactor]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.mh__1520daff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1520daff89b22cbd

4.132 132. bounded_prime_product_balance_value_classification

The problem

For every nonnegative integer t , characterize when two distinct primes between 4 and 18 satisfy $pq = t + (p + q)$. Show that the possible values of t are exactly 23, 39, 47, 59, 63, 71, 95, 119, 159, and 191.

Lean statement

```
theorem bounded_prime_product_balance_value_classification :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
      t = 23 ∨ t = 39 ∨ t = 47 ∨ t = 59 ∨ t = 63 ∨ t = 71 ∨ t = 95 ∨ t = 119 ∨ t = 159 ∨ t = 191
```

Parent. For every nonnegative integer t , characterize when t satisfies the additive balance $pq = t + (p + q)$ for two distinct primes between 4 and 18. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Parent in Lean

```
theorem bounded_prime_product_additive_balance_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an algebraic equivalence into an exact finite classification of all possible values of t . Its proof requires enumerating the bounded primes, checking all pairings, and constructing witnesses, none of which is supplied by the parent's normalization argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential balance equation into an exact classification of all possible values of t , requiring bounded prime enumeration and a ten-case witness construction. The parent's algebraic equivalence proof does not supply this finite classification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from an algebraic equivalence to a complete finite classification of all admissible values of t . Its proof requires enumerating the bounded primes, analyzing all distinct pairs, and proving the converse witnesses; the parent's factoring argument does not supply this classification.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2447 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_product_balance_value_classification.extracted_1_1 : ∀ (t : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = t + (p + q)) ↔
  t = 23 ∨ t = 39 ∨ t = 47 ∨ t = 59 ∨ t = 63 ∨ t = 71 ∨ t = 95 ∨ t = 119 ∨ t = 159 ∨ t = 191
```

Read back by a model that saw only the Lean. *For every natural number t , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $p \cdot q = t + (p + q)$, if and only if t is one of 23, 39, 47, 59, 63, 71, 95, 119, 159, or 191.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_prime_product_balance_value_classification :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
      t = 23 ∨ t = 39 ∨ t = 47 ∨ t = 59 ∨ t = 63 ∨ t = 71 ∨ t = 95 ∨ t = 119 ∨ t = 159 ∨ t = 191 := by
  intro t
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hbal⟩
    have hp_cases : p = 5 ∨ p = 7 ∨ p = 11 ∨ p = 13 ∨ p = 17 := by
      interval_cases p <|> norm_num at hp <|> omega
    have hq_cases : q = 5 ∨ q = 7 ∨ q = 11 ∨ q = 13 ∨ q = 17 := by
      interval_cases q <|> norm_num at hq <|> omega
    rcases hp_cases with rfl | rfl | rfl | rfl | rfl <|>
    rcases hq_cases with rfl | rfl | rfl | rfl | rfl <|>
    omega
  · intro ht
    rcases ht with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
    · exact ⟨5, 7, by norm_num⟩
    · exact ⟨5, 11, by norm_num⟩
    · exact ⟨5, 13, by norm_num⟩
    · exact ⟨7, 11, by norm_num⟩
    · exact ⟨5, 17, by norm_num⟩
    · exact ⟨7, 13, by norm_num⟩
    · exact ⟨7, 17, by norm_num⟩
    · exact ⟨11, 13, by norm_num⟩
    · exact ⟨11, 17, by norm_num⟩
    · exact ⟨13, 17, by norm_num⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.ms.mh__9e1c3aa2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9e1c3aa207fabbf1

4.133 133. prime_pair_quadratic_sign_characterization

The problem

Let $p < q$ be natural numbers. For an integer x , show that $x^2 - (p + q)x + pq > 0$ exactly when $x < p$ or $x > q$.

Lean statement

```
theorem prime_pair_quadratic_sign_characterization (p q : ℕ) (x : ℤ)
  (hpq : p < q) :
  (0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)) ↔
  x < (p : ℤ) ∨ (q : ℤ) < x
```

Parent. Let $p < q$ be primes between 4 and 18 satisfying $pq - p - q = 119$. Show that $x^2 - (p + q)x + pq > 0$ for every integer $x > q$.

Parent in Lean

```
theorem prime_pair_quadratic_positive_outside (p q : ℕ) (x : ℤ)
  (hpq : p < q)
  (hx : (q : ℤ) < x) :
  0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a one-sided positivity result into a full sign characterization, requiring a new forward argument to exclude the interval between the roots and a reverse argument for the left-of-root case. The parent's right-of-root positivity proof alone cannot establish this biconditional.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-sided positivity result into a full sign characterization: it proves positivity both below p and above q , and proves nonpositivity throughout the interval. This requires a new converse and an interval sign argument beyond the parent's outside- q factor positivity proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-sided positivity result into a full sign characterization, proving both exterior cases and nonpositivity on the interval between the roots. This requires new case analysis and contradiction reasoning beyond the parent's factorization-and-positive-product argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2520 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_quadratic_sign_characterization.extracted_1_1 (p q : ℕ) (x : ℤ) (hpq : p < q) :
  0 < x ^ 2 - (↑p + ↑q) * x + ↑p * ↑q ↔ x < ↑p ∨ ↑q < x
```

Read back by a model that saw only the Lean. For natural numbers p and q and an integer x , assuming $p < q$, the inequality $0 < x^2 - (p + q)x + pq$ holds if and only if $x < p$ or $q < x$, where p and q are regarded as integers in the inequality.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_pair_quadratic_sign_characterization (p q : ℕ) (x : ℤ)
  (hpq : p < q) :
  (0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)) ↔
  x < (p : ℤ) ∨ (q : ℤ) < x := by
  have hpq' : (p : ℤ) ≤ (q : ℤ) := by
    omega
  have hfac :
    x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) =
    (x - (p : ℤ)) * (x - (q : ℤ)) := by
    ring
  rw [hfac]
  constructor
  · intro hpos
    by_cases hxp : x < (p : ℤ)
    · exact Or.inl hxp
    · right
      by_contra hqx
      have hpx : (p : ℤ) ≤ x := by
        omega
      have hxq : x ≤ (q : ℤ) := by
        omega
      have hleft : 0 ≤ x - (p : ℤ) := by
        omega
      have hright : x - (q : ℤ) ≤ 0 := by
        omega
      have hnonpos : (x - (p : ℤ)) * (x - (q : ℤ)) ≤ 0 := by
        exact mul_nonpos_of_nonneg_of_nonpos hleft hright
      exact (not_lt_of_ge hnonpos) hpos
  · intro h
    rcases h with hxp | hqx
    · have hleft : x - (p : ℤ) < 0 := by
        omega
      have hright : x - (q : ℤ) < 0 := by
        omega
      exact mul_pos_of_neg_of_neg hleft hright
    · have hleft : 0 < x - (p : ℤ) := by
        omega
      have hright : 0 < x - (q : ℤ) := by
        omega
      exact mul_pos hleft hright
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh.mh__c31d4118

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c31d4118491553ff

4.134 134. shifted_prime_product_mod_four

The problem

For every natural number n , if distinct primes p and q between 4 and 18 satisfy $pq - (p + q) = n$, then n is congruent to 3 modulo 4.

Lean statement

```
theorem shifted_prime_product_mod_four (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 4 = 3
```

Parent. Let n be a natural number. If two distinct prime numbers between 4 and 18 satisfy $pq - (p + q) = n$, prove that n is odd.

Parent in Lean

```
theorem shifted_prime_product_mod_two (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 2 = 1
```

What it contributes. Reused the bounded distinct-prime hypothesis, shifted factorization checkpoint, and elimination of the even-prime branches; strengthened the final residue conclusion from modulo 2 to modulo 4.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the parity conclusion to a modulo-four obstruction by proving both shifted factors $p-1$ and $q-1$ are even, hence $n+1$ is divisible by 4. This extra divisibility reasoning is not supplied by the parent's modulo-two proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parity conclusion to a modulo-four obstruction by proving both shifted prime factors $p-1$ and $q-1$ are even, hence $n+1$ is divisible by 4. This requires new factorization arithmetic beyond the parent's proof of mere evenness.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens parity to the modulo-four obstruction $n+1 = 4ab$, requiring both shifted odd-prime factors $p-1$ and $q-1$ to be explicitly shown even. The parent only establishes $n+1$ is even and does not supply this stronger factorization reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2053 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_prime_product_mod_four.extracted_1_1 (n : ℕ) :  
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) →  
    n % 4 = 3
```

Read back by a model that saw only the Lean. *For a natural number n , if there exist distinct natural numbers p and q such that p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and the natural-number subtraction $p \cdot q - (p + q)$ equals n , then the remainder when n is divided by 4 is 3.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem shifted_prime_product_mod_four (n : ℕ) :  
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 4 = 3 := by  
  rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩  
  have hsum : p + q ≤ p * q := by  
    nlinarith  
  have hmul : p * q = n + (p + q) := by  
    omega  
  have hp_sub : p - 1 + 1 = p := by  
    omega  
  have hq_sub : q - 1 + 1 = q := by  
    omega  
  have hfac : (p - 1) * (q - 1) = n + 1 := by  
    nlinarith [hmul, hp_sub, hq_sub]  
  obtain ⟨a, ha⟩ | ⟨a, ha⟩ := Nat.even_or_odd p  
  · have hp2 : p = 2 := (Nat.Prime.even_iff hp).mp ⟨a, ha⟩  
    omega  
  · obtain ⟨b, hb⟩ | ⟨b, hb⟩ := Nat.even_or_odd q  
    · have hq2 : q = 2 := (Nat.Prime.even_iff hq).mp ⟨b, hb⟩  
      omega  
    · have hp_shift : p - 1 = 2 * a := by  
        omega  
      have hq_shift : q - 1 = 2 * b := by  
        omega  
      have hn1 : n + 1 = 4 * (a * b) := by  
        nlinarith [hfac, hp_shift, hq_shift]  
      omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh__1522f78c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d1529e38793ea1e9

4.135 135. prime_pair_quadratic_positive_outside

The problem

Let $p < q$ be primes between 4 and 18 satisfying $pq - p - q = 119$. Show that $x^2 - (p+q)x + pq > 0$ for every integer $x > q$.

Lean statement

```
theorem prime_pair_quadratic_positive_outside (p q : ℕ) (x : ℤ)
  (hpq : p < q)
  (hx : (q : ℤ) < x) :
  0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ)
```

Parent. Let $p < q$ be distinct primes between 4 and 18 satisfying $pq - p - q = 119$. Show that the integer roots of $x^2 - (p+q)x + pq = 0$ are exactly p and q .

Parent in Lean

```
theorem prime_pair_quadratic_roots (p q : ℕ)
  (x : ℤ) :
  (x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) = 0) ↔
  x = (p : ℤ) ∨ x = (q : ℤ)
```

What it contributes. Uses the parent's quadratic factorization checkpoint and the ordered relation $p < q$, while changing the conclusion to a strict outside-root sign claim.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an equality characterization into a strict-positivity claim outside the larger root. Its proof requires ordered-root reasoning and positivity of both factors via 'mul_pos', which the parent's zero-product argument does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root-equation conclusion into strict positivity beyond the larger root. It uses the factorization from the parent but additionally reasons from the ordered hypotheses to show both factors are positive, which the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from characterizing roots to proving strict positivity beyond the larger root. Its proof requires sign reasoning for both factors, which the parent's zero-product argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2482 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_quadratic_positive_outside.extracted_1_1 (p q : ℕ) (x : ℤ) (hp : Nat.Prime p) (hq : Nat.Prime q)
  (hp4 : 4 < p) (hq18 : q < 18) (hpq : p < q) (hrel : ↑p * ↑q - ↑p - ↑q = 119) (hx : ↑q < x) :
  0 < x ^ 2 - (↑p + ↑q) * x + ↑p * ↑q
```

Read back by a model that saw only the Lean. For natural numbers p and q and an integer x , if p and q are prime, $4 < p$, $q < 18$, $p < q$, $p \cdot q - p - q = 119$, and $q < x$, then $0 < x^2 - (p + q)x + p \cdot q$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_pair_quadratic_positive_outside (p q : ℕ) (x : ℤ)

  (hpq : p < q)

  (hx : (q : ℤ) < x) :
  0 < x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) := by
have hfac :
  x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) =
  (x - (p : ℤ)) * (x - (q : ℤ)) := by
  ring
have hxp : 0 < x - (p : ℤ) := by
  omega
have hxq : 0 < x - (q : ℤ) := by
  omega
have hprod : 0 < (x - (p : ℤ)) * (x - (q : ℤ)) := by
  exact mul_pos hxp hxq
rw [hfac]
exact hprod
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh__af2d4b34

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9a55ebe023da8640

4.136 136. shifted_quadratic_equal_values

The problem

Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that two integers give the same value of $x^2 - (p + q)x + (t + p + q)$ exactly when they are equal or their sum is $p + q$.

Lean statement

```
theorem shifted_quadratic_equal_values :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    ∀ x y : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
        y ^ 2 - ((p : ℤ) + q) * y + ((t : ℤ) + p + q) ↔
      x = y ∨ x + y = (p : ℤ) + q := by
```

Parent. Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that $(p - 1)(q - 1) = t + 1$, and that p and q are exactly the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$.

Parent in Lean

```
theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from characterizing the polynomial's roots to characterizing all pairs of integers with equal polynomial values. Although factorization from the parent remains useful, the new difference-of-squares argument and two-variable case split are not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from identifying the quadratic's roots to characterizing all pairs of integers with equal quadratic values. Its proof requires a new difference-of-squares argument, deriving $(x-y)(x+y-(p+q))=0$, which is not supplied by the parent's root proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the root characterization into a distinct two-variable equal-value characterization. Its proof requires deriving and solving the factored difference equation, which is not supplied by the parent's root-solving argument.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2538 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_quadratic_equal_values.extracted_1_1 : ∀ (p q t : ℕ),
  Nat.Prime p →
  Nat.Prime q →
  4 ≤ p →
  p ≤ 18 →
  4 ≤ q →
  q ≤ 18 →
  p ≠ q →
  t ≤ 200 →
  p * q - (p + q) = t →
  ∀ (x y : ℤ),
    x ^ 2 - (↑p + ↑q) * x + (↑t + ↑p + ↑q) = y ^ 2 - (↑p + ↑q) * y + (↑t + ↑p + ↑q) ↔
    x = y ∨ x + y = ↑p + ↑q
```

Read back by a model that saw only the Lean. For all natural numbers p , q , and t , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $t \leq 200$, and $p \cdot q - (p + q) = t$ (with subtraction in the natural numbers), then for all integers x and y , $x^2 - (p + q)x + (t + p + q) = y^2 - (p + q)y + (t + p + q)$ if and only if $x = y$ or $x + y = p + q$, where the natural numbers p , q , and t are regarded as integers in the quadratic expressions.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem shifted_quadratic_equal_values :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    ∀ x y : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
        y ^ 2 - ((p : ℤ) + q) * y + ((t : ℤ) + p + q) ↔
        x = y ∨ x + y = (p : ℤ) + q := by
  intro p q t hp hq hp4 hp18 hq4 hq18 hne htbound hrel
  have hsum_le : p + q ≤ p * q := by
    interval_cases p <|> interval_cases q <|> norm_num
  have hprod : p * q = t + (p + q) :=
    (Nat.sub_eq_iff_eq_add hsum_le).mp hrel
  have hrelZ : (p : ℤ) * q = (t : ℤ) + p + q := by
    have hprodZ : (p : ℤ) * q = (t : ℤ) + ((p + q) : ℤ) := by
      exact_mod_cast hprod
    simpa [Nat.cast_add, add_assoc] using hprodZ
  have hfactor : ∀ z : ℤ,
    z ^ 2 - ((p : ℤ) + q) * z + ((t : ℤ) + p + q) =
      (z - p) * (z - q) := by
    intro z
    nlinarith [hrelZ]
  intro x y
  constructor
  · intro hxy
    have hzero : (x - y) * (x + y - ((p : ℤ) + q)) = 0 := by
      nlinarith [hxy, hfactor x, hfactor y]
```

```

rcases mul_eq_zero.mp hzero with hzero | hzero
· left
  omega
· right
  omega
· intro hxy
  rcases hxy with rfl | hxy
· rfl
· have hy : y = (p : ℤ) + q - x := by
  omega
  rw [hy]
  ring

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.mh__f398235d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f398235dde5761c8

4.137 137. prime_pair_quadratic_nonnegative_iff

The problem

Let $p \leq q$ be natural numbers and let x be an integer. Show that $x^2 - (p + q)x + pq \geq 0$ exactly when $x \leq p$ or $q \leq x$.

Lean statement

```
theorem prime_pair_quadratic_nonnegative_iff (p q : ℕ) (x : ℤ) (hpq : (p : ℤ) ≤ (q : ℤ)) :
  0 ≤ x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) ↔
  x ≤ (p : ℤ) ∨ (q : ℤ) ≤ x
```

Parent. For natural numbers p, q and an integer x , show that x is neither p nor q exactly when $x^2 - (p + q)x + pq$ is nonzero.

Parent in Lean

```
theorem prime_pair_quadratic_nonzero_iff (p q : ℕ) (x : ℤ) :
  (¬ (x = (p : ℤ) ∨ x = (q : ℤ))) ↔
  x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) ≠ 0
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a zero-characterization into a sign characterization over the ordered roots. Its proof requires new interval and product-sign reasoning; the parent's root/nonzero argument does not supply this.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from nonvanishing to a full sign characterization, requiring order reasoning and a case split between the roots. The parent's factorization is useful but its root-equivalence argument does not supply the new sign proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the zero-root characterization into a sign characterization: proving nonnegativity requires ordering the roots and reasoning about the sign of both factors, including exclusion of the interval between them. The parent's factorization alone does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2532 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_quadratic_nonnegative_iff.extracted_1_1 (p q : ℕ) (x : ℤ) (hpq : ↑p ≤ ↑q) :  
  0 ≤ x ^ 2 - (↑p + ↑q) * x + ↑p * ↑q ↔ x ≤ ↑p ∨ ↑q ≤ x
```

Read back by a model that saw only the Lean. For natural numbers p and q , an integer x , and the hypothesis that $p \leq q$, the inequality $0 \leq x^2 - (p + q)x + pq$ holds if and only if $x \leq p$ or $q \leq x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_pair_quadratic_nonnegative_iff (p q : ℕ) (x : ℤ) (hpq : (p : ℤ) ≤ (q : ℤ)) :  
  0 ≤ x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) ↔  
  x ≤ (p : ℤ) ∨ (q : ℤ) ≤ x := by  
  have hfac :  
    x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) =  
      (x - (p : ℤ)) * (x - (q : ℤ)) := by  
    ring  
  constructor  
  · intro hnon  
    by_cases hleft : x ≤ (p : ℤ)  
    · exact Or.inl hleft  
    · right  
      by_contra hright  
      have hxp : (p : ℤ) < x := by omega  
      have hxq : x < (q : ℤ) := by omega  
      have hprodneg : (x - (p : ℤ)) * (x - (q : ℤ)) < 0 := by  
        exact mul_neg_of_pos_of_neg (sub_pos.mpr hxp) (sub_neg.mpr hxq)  
      have hpolyneg : x ^ 2 - ((p : ℤ) + (q : ℤ)) * x + (p : ℤ) * (q : ℤ) < 0 := by  
        rw [hfac]  
        exact hprodneg  
      linarith  
  · intro hout  
    rcases hout with hleft | hright  
    · have h1 : x - (p : ℤ) ≤ 0 := by linarith  
      have h2 : x - (q : ℤ) ≤ 0 := by linarith  
      rw [hfac]  
      exact mul_nonneg_of_nonpos_of_nonpos h1 h2  
    · have h1 : 0 ≤ x - (p : ℤ) := by linarith  
      have h2 : 0 ≤ x - (q : ℤ) := by linarith  
      rw [hfac]  
      exact mul_nonneg h1 h2
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh.ms.mh__94f43e84

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 94f43e8478d5e09c

4.138 138. prime_pair_119_characterization

The problem

For any natural numbers p and q that are distinct primes between 4 and 18, $pq - (p + q) = 119$ if and only if $(p, q) = (11, 13)$ or $(p, q) = (13, 11)$.

Lean statement

```
theorem prime_pair_119_characterization :
  ∀ p q : ℕ,
    Nat.Prime p → Nat.Prime q → 4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    (p * q - (p + q) = 119 ↔ (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11))
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? **(A)** 22 **(B)** 60 **(C)** 119 **(D)** 180 **(E)** 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the nested existence claim into a full iff characterization, requiring both verification of the two ordered prime pairs and exclusion of every other bounded pair. The parent proof only derives a contradiction from the 22 case by enumerating p and does not establish any characterization of the 119 case.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a 119-witness into a full iff characterization, requiring exhaustive reasoning about both p and q and identifying the only ordered pairs. The parent only rules out several target values and supplies no uniqueness or classification argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential fact into a biconditional characterization for every bounded prime pair, requiring exhaustive reasoning over both p and q and proving both directions. The parent's contradiction proof only handles the surrounding conjunction and cannot supply this universal uniqueness characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1726 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_pair_119_characterization.extracted_1.1 : ∀ (p q : ℕ),
  Nat.Prime p →
  Nat.Prime q →
  4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q → (p * q - (p + q) = 119 ↔ p = 11 ∧ q = 13 ∨ p = 13 ∧ q = 11)
```

Read back by a model that saw only the Lean. *For all natural numbers p and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \neq q$, then $p \cdot q - (p + q) = 119$ if and only if $(p = 11 \text{ and } q = 13) \text{ or } (p = 13 \text{ and } q = 11)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem prime_pair_119_characterization :
  ∀ p q : ℕ,
    Nat.Prime p → Nat.Prime q → 4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    (p * q - (p + q) = 119 ↔ (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11)) := by
  intro p q hp hq hp4 hp18 hq4 hq18 hne
  interval_cases p <|> interval_cases q <|> norm_num at *
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__02d6b54d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 51e450964c9682a5

4.139 139. amc12_2000_p6_mutation_range

The problem

For two distinct primes between 4 and 18, the possible values of $pq - (p + q)$ are exactly 23, 39, 47, 59, 63, 71, 95, 119, 159, and 191.

Lean statement

```
theorem amc12_2000_p6_mutation_range :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = n) ↔
      n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨ n = 119 ∨ n = 159 ∨ n = 191
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes five isolated existence/nonexistence checks into a complete characterization of every attainable value. Its forward direction must exhaustively classify all distinct prime pairs and its reverse direction supplies ten witnesses, so the parent proof does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces five selected existence/nonexistence checks with a complete characterization of every attainable value, requiring exhaustive forward enumeration and explicit witnesses for the reverse direction. Its proof does not reuse the parent's contradiction argument and establishes a genuinely stronger range statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from refuting a specific conjunction of equations to characterizing the complete range of $p*q-(p+q)$ for distinct primes in $[4,18]$. Its proof must perform exhaustive enumeration and provide witnesses for ten values, which the parent's single contradiction proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2524 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12_2000_p6_mutation_range.extracted_1_1 : ∀ (n : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) ↔
  n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨ n = 119 ∨ n = 159 ∨ n = 191
```

Read back by a model that saw only the Lean. *For every natural number n , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that p times q minus the sum $p + q$ equals n , if and only if n is one of 23, 39, 47, 59, 63, 71, 95, 119, 159, or 191.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem amc12_2000_p6_mutation_range :
  ∀ n : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = n) ↔
      n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨ n = 119 ∨ n = 159 ∨ n = 191 := by
  intro n
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
    have hforward :
      n = 23 ∨ n = 39 ∨ n = 47 ∨ n = 59 ∨ n = 63 ∨ n = 71 ∨ n = 95 ∨
      n = 119 ∨ n = 159 ∨ n = 191 := by
      interval_cases p <|> interval_cases q <|>
      simp_all (config := {decide := true})
    exact hforward
  · intro hn
    rcases hn with rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl | rfl
    · exact ⟨5, 7, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 11, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨5, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨7, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 13, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨11, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
    · exact ⟨13, 17, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num, by norm_num⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__1995d807

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 1995d8075402f51d

4.140 140. amc12_2000_p6_mutation

The problem

Two different primes between 4 and 18 are chosen. If their product minus their sum is 119, then the primes are 11 and 13, in either order.

Lean statement

```
theorem amc12_2000_p6_mutation :
  ∀ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119 →
    (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11)
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes existence of one witness into a complete characterization of every admissible pair, requiring exclusion of all other bounded prime cases; the parent's witness proof does not supply that uniqueness argument. Its finite case analysis is a genuinely new proof obligation, not a recall of the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a witness for 119 into a complete uniqueness characterization of both ordered prime pairs. Its proof must enumerate and eliminate all bounded pairs, whereas the parent can close using the unrelated 22 contradiction and does not establish the pair classification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential claim into a uniqueness characterization of all bounded prime pairs satisfying the equation. Proving both possible orderings requires exhaustive reasoning about both variables, which the parent's contradiction proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2512 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12_2000_p6_mutation.extracted_1_1 : ∀ (p q : ℕ),
  Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = 119 →
  p = 11 ∧ q = 13 ∨ p = 13 ∧ q = 11
```

Read back by a model that saw only the Lean. *For all natural numbers p and q , if p and q are prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, and $p \cdot q - (p + q) = 119$, then either $p = 11$ and $q = 13$, or $p = 13$ and $q = 11$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem amc12_2000_p6_mutation :
  ∀ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119 →
    (p = 11 ∧ q = 13) ∨ (p = 13 ∧ q = 11) := by
  intro p q h
  rcases h with ⟨hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
  have hp_cases : 4 ≤ p ∧ p ≤ 18 := ⟨hp4, hp18⟩
  have hq_cases : 4 ≤ q ∧ q ≤ 18 := ⟨hq4, hq18⟩
  interval_cases p <|> interval_cases q <|> norm_num at *
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__502f3339

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 502f3339c29bdd78

4.141 141. shifted_prime_product_mod_two

The problem

Let n be a natural number. If two distinct prime numbers between 4 and 18 satisfy $pq - (p + q) = n$, prove that n is odd.

Lean statement

```
theorem shifted_prime_product_mod_two (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 2 = 1
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Uses the parent's bounded distinct-prime shifted product equation and its prime-modulus obstruction surface, but changes the endpoint to a parameterized parity congruence derived through the shifted factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent's fixed numerical contradiction into a parity theorem for every representable n . Its proof requires factoring the shifted product and using the oddness of primes greater than 2, reasoning absent from the parent's bounded interval_cases and norm_num proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new universal parity obstruction for arbitrary n by factoring $p*q - (p+q) + 1$ and using that distinct primes at least 4 are odd. The parent's finite interval enumeration of five fixed target values supplies none of this modular reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new universal parity obstruction for every represented n , using oddness of distinct bounded primes and a shifted factorization; the parent only performs a finite contradiction for the fixed value 22. Its argument is not recoverable by recalling the parent's witness-based interval case split.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2048 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_prime_product_mod_two.extracted_1_1 (n : ℕ) :
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) →
    n % 2 = 1
```

Read back by a model that saw only the Lean. *For every natural number n , if there exist distinct prime natural numbers p and q such that $4 \leq p \leq 18$, $4 \leq q \leq 18$, and $p \cdot q - (p + q) = n$, then n leaves remainder 1 upon division by 2.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem shifted_prime_product_mod_two (n : ℕ) :
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q - (p + q) = n) → n % 2 = 1 := by
  rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, heq⟩
  have hsum : p + q ≤ p * q := by
    nlinarith
  have hmul : p * q = n + (p + q) := by
    omega
  have hp_sub : p - 1 + 1 = p := by
    omega
  have hq_sub : q - 1 + 1 = q := by
    omega
  have hfac : (p - 1) * (q - 1) = n + 1 := by
    nlinarith [hmul, hp_sub, hq_sub]
  obtain ⟨a, ha⟩ | ⟨a, ha⟩ := Nat.even_or_odd p
  · have hp2 : p = 2 := (Nat.Prime.even_iff hp).mp ⟨a, ha⟩
    omega
  · obtain ⟨b, hb⟩ | ⟨b, hb⟩ := Nat.even_or_odd q
    · have hq2 : q = 2 := (Nat.Prime.even_iff hq).mp ⟨b, hb⟩
      omega
    · have hn1 : ∃ k : ℕ, n + 1 = 2 * k := by
        refine ⟨2 * a * b, ?_⟩
        nlinarith [hfac, hp_sub, hq_sub, ha, hb]
      rcases hn1 with ⟨k, hk⟩
      omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__92f062d2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint bd34dfb531ac1cab

4.142 142. parameterized_shifted_roots

The problem

Let p and q be distinct primes between 4 and 18, and let $t \leq 200$ satisfy $pq - (p + q) = t$. Show that $(p - 1)(q - 1) = t + 1$, and that p and q are exactly the integer roots of $x^2 - (p + q)x + (t + p + q) = 0$.

Lean statement

```
theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
    Nat.Prime p → Nat.Prime q →
    4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
    t ≤ 200 → p * q - (p + q) = t →
    (p - 1) * (q - 1) = t + 1 ∧
    ∀ x : ℤ,
      x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
      x = p ∨ x = q := by
```

Parent. Two different prime numbers between 4 and 18 are chosen. When their sum is subtracted from their product, which of the following numbers could be obtained? (A) 22 (B) 60 (C) 119 (D) 180 (E) 231. Show that 119 is possible, but options other than 119 cannot be obtained.

Parent in Lean

```
theorem amc12_2000_p6 :
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 22) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 60) ∧
  (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 119) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 180) ∧
  (¬ ∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
    p * q - (p + q) = 231) := by
```

What it contributes. Retains the parent's bounded distinct-prime domain and product-minus-sum relation, while replacing the fixed attainable value with the parameter t and proving the resulting factorization and root characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the attainable value with t and proves a root characterization for the resulting shifted quadratic. Its factorization and integer-root argument replace the parent's finite case enumeration, so recalling the parent does not substantially solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed-value existence test with a parameterized algebraic identity and a root characterization for the shifted quadratic. Its proof requires factorization and zero-product reasoning, which are not supplied by the parent's bounded interval enumeration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the attainable value and proves a new algebraic factorization and root characterization, requiring polynomial reasoning absent from the parent's finite contradiction proof. The parent's proof cannot be recalled to establish either conjunct.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2514 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_shifted_roots.extracted_1_1 : ∀ (p q t : ℕ),
  Nat.Prime p →
  Nat.Prime q →
  4 ≤ p →
  p ≤ 18 →
  4 ≤ q →
  q ≤ 18 →
  p ≠ q →
  t ≤ 200 →
  p * q - (p + q) = t →
  (p - 1) * (q - 1) = t + 1 ∧
  ∀ (x : ℤ), x ^ 2 - (↑p + ↑q) * x + (↑t + ↑p + ↑q) = 0 ↔ x = ↑p ∨ x = ↑q
```

Read back by a model that saw only the Lean. For all natural numbers p, q , and t , if p is prime, q is prime, $4 \leq p \leq 18$, $4 \leq q \leq 18$, $p \neq q$, $t \leq 200$, and $pq - (p + q) = t$, then $(p - 1)(q - 1) = t + 1$ and, for every integer x , $x^2 - (p + q)x + (t + p + q) = 0$ if and only if $x = p$ or $x = q$, where the coefficients p, q , and t in the integer equation are regarded as integers.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem parameterized_shifted_roots :
  ∀ p q t : ℕ,
  Nat.Prime p → Nat.Prime q →
  4 ≤ p → p ≤ 18 → 4 ≤ q → q ≤ 18 → p ≠ q →
  t ≤ 200 → p * q - (p + q) = t →
  (p - 1) * (q - 1) = t + 1 ∧
  ∀ x : ℤ,
  x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
  x = p ∨ x = q := by
  intro p q t hp hq hp4 hp18 hq4 hq18 hne htbound hrel
  have hsum_le : p + q ≤ p * q := by
    interval_cases p <|> interval_cases q <|> norm_num
  have hprod : p * q = t + (p + q) :=
    (Nat.sub_eq_iff_eq_add hsum_le).mp hrel
  have hfac : (p - 1) * (q - 1) = t + 1 := by
    interval_cases p <|> interval_cases q <|> norm_num at hrel <|> omega
  have hrelZ : (p : ℤ) * q = (t : ℤ) + p + q := by
    have hprodZ : (p : ℤ) * q = (t : ℤ) + ((p + q : ℕ) : ℤ) := by
      exact_mod_cast hprod
    simpa [Nat.cast_add, add_assoc] using hprodZ
  have hpoly : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) =
    (x - p) * (x - q) := by
    intro x
    nlinarith [hrelZ, hfac]
  have hroots : ∀ x : ℤ,
    x ^ 2 - ((p : ℤ) + q) * x + ((t : ℤ) + p + q) = 0 ↔
    x = p ∨ x = q := by
```

```

intro x
constructor
· intro hx
  have hzero : (x - p) * (x - q) = 0 := by
    rw [← hpoly x]
    exact hx
  rcases mul_eq_zero.mp hzero with hzero | hzero
  · left
    omega
  · right
    omega
· intro hx
  rcases hx with rfl | rfl
  · simp [hpoly]
  · simp [hpoly]
exact ⟨hfacs, hroots⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12_2000_p6__mh__f43b4d36

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint f43b4d361c08e0f6

4.143 143. characterize_bounded_threshold

The problem

For the sequence beginning with 4, 7, where each later term is the units digit of the sum of the preceding two, characterize the thresholds t for which 2 is the least index with $S_n > t$ and $n \leq 3$. Also show that the first three terms have sum 12.

Lean statement

```
theorem characterize_bounded_threshold
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (t : ℕ) :
  (IsLeast {n : ℕ | S n > t ∧ n ≤ 3} 2 ↔ 4 ≤ t ∧ t < 11) ∧ S 3 = 12
```

Parent. Let a sequence begin with 4, 7, and let each later term be the units digit of the sum of the two preceding terms. If S_n is the sum of the first n terms, show that 2 is the least n such that $S_n > 10$ and $n \leq 3$.

Parent in Lean

```
theorem bounded_threshold_mutation
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10 ∧ n ≤ 3}) :
  IsLeast Sₙ 2
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the threshold and proves a symbolic least-index characterization, requiring new lower-bound and upper-bound reasoning for arbitrary t ; it also derives the additional value $S(3)=12$. The parent's fixed-threshold IsLeast proof and hypothesis h_4 cannot be recalled to solve this child.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child parameterizes the threshold and proves a two-way characterization of exactly when 2 is the least qualifying index, requiring exclusion of competing indices and handling both threshold directions. It also consumes the recurrence to establish the additional value $S(3)=12$, so recalling the parent's fixed-threshold leastness proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the threshold and proves a full characterization of when 2 is the least qualifying index, while also deriving the new value $S 3 = 12$. This requires reasoning beyond the parent's fixed-threshold membership proof.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2753 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem characterize_bounded_threshold.extracted_1_1 (u S : ℕ → ℕ) (h₀ : u 0 = 4) (h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) (h₃ : ∀ (n : ℕ), S n = ∑ k ∈ Finset.range n, u k) (t : ℕ) :
(IsLeast {n | S n > t ∧ n ≤ 3} 2 ↔ 4 ≤ t ∧ t < 11) ∧ S 3 = 12
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers such that $u(0)=4$, $u(1)=7$, for every natural number $n \geq 2$ we have $u(n)=(u(n-2)+u(n-1)) \bmod 10$, and for every natural number n , $S(n)=\sum_{k \in \{0,1,\dots,n-1\}} u(k)$. Then, for every natural number t , 2 is the least natural number n satisfying $S(n) > t$ and $n \leq 3$ if and only if $4 \leq t$ and $t < 11$; moreover, $S(3)=12$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem characterize_bounded_threshold
(u S : ℕ → ℕ)
(h₀ : u 0 = 4)
(h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
(h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
(t : ℕ) :
(IsLeast {n : ℕ | S n > t ∧ n ≤ 3} 2 ↔ 4 ≤ t ∧ t < 11) ∧ S 3 = 12 := by
have hu2 : u 2 = 1 := by
  have h := h₂ 2 (by omega)
  norm_num [h₀, h₁] at h ⊢
  exact h
have hS0 : S 0 = 0 := by
  rw [h₃]
  simp
have hS1 : S 1 = 4 := by
  rw [h₃]
  simp [h₀]
have hS2 : S 2 = 11 := by
  rw [h₃]
  norm_num [Finset.sum_range_succ, h₀, h₁]
have hS3 : S 3 = 12 := by
  rw [h₃]
  norm_num [Finset.sum_range_succ, h₀, h₁, hu2]
constructor
· constructor
· intro h
  have hmem := h.1
  change S 2 > t ∧ 2 ≤ 3 at hmem
  have ht_upper : t < 11 := by
    rw [hS2] at hmem
    omega
  have ht_lower : 4 ≤ t := by
    by_contra hlt
    have ht : t < 4 := by omega
    have h1mem : 1 ∈ {n : ℕ | S n > t ∧ n ≤ 3} := by
      change S 1 > t ∧ 1 ≤ 3
      rw [hS1]
```

```

      omega
      have h1e := h.2 h1mem
      omega
    exact ⟨ht_lower, ht_upper⟩
  · rintro ⟨ht_lower, ht_upper⟩
  have hmem :  $2 \in \{n : \mathbb{N} \mid S\ n > t \wedge n \leq 3\}$  := by
    change  $S\ 2 > t \wedge 2 \leq 3$ 
    rw [hS2]
    omega
  refine ⟨hmem, ?_⟩
  intro n hn
  change  $S\ n > t \wedge n \leq 3$  at hn
  rcases hn with ⟨hgt, hn3⟩
  have hn0 :  $n \neq 0$  := by
    intro hn0
    subst n
    rw [hS0] at hgt
    omega
  have hn1 :  $n \neq 1$  := by
    intro hn1
    subst n
    rw [hS1] at hgt
    omega
  omega
  · exact hS3

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12a_2002_p21__me.mh__b19ac235

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint b19ac235ddcfcfed

4.144 144. sequence_periodic_twelve

The problem

Show that the sequence is periodic with period 12: for every nonnegative integer n , the $(n+12)$ -th term equals the n -th term.

Lean statement

```
theorem sequence_periodic_twelve
  (u : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) :
  ∀ n, u (n + 12) = u n := by
```

Parent. Consider the sequence of numbers: 4, 7, 1, 8, 9, 7, 6, ... For $n > 2$, the n -th term of the sequence is the units digit of the sum of the two previous terms. Let S_n denote the sum of the first n terms of this sequence. Show that the smallest value of n for which $S_n > 10,000$ is 1999.

Parent in Lean

```
theorem amc12a_2002_p21
  (u S : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sₙ : Set ℕ)
  (h₄ : Sₙ = {n | S n > 10000}) :
  IsLeast Sₙ 1999 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from locating a threshold of partial sums to proving a universal period-12 law for the recurrence. Its proof requires strong induction and recurrence alignment, which the parent's finite threshold computation does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a threshold minimum for cumulative sums to a global period-12 theorem for the recurrence. Its strong induction and recurrence-shift argument require mathematics absent from the parent's finite-sum threshold proof, so recalling the parent does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the threshold-set minimum into a global periodicity theorem. Its strong induction and recurrence-shifting argument require genuinely new reasoning and do not reuse the parent's cumulative-sum proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2771 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B universally quantifies over sequences and adds initial-value and recurrence hypotheses, whereas Statement A refers only to an unspecified particular sequence and states periodicity.

Goal round-trip: what Lean elaborated

```
theorem sequence_periodic_twelve.extracted_1_1 :  $\forall (u : \mathbb{N} \rightarrow \mathbb{N}),$   

 $u\ 0 = 4 \rightarrow u\ 1 = 7 \rightarrow (\forall n \geq 2, u\ n = (u\ (n - 2) + u\ (n - 1)) \% 10) \rightarrow \forall (n : \mathbb{N}), u\ (n + 12) = u\ n$ 
```

Read back by a model that saw only the Lean. *For every sequence u of natural numbers, if $u(0)=4$, $u(1)=7$, and for every natural number $n \geq 2$, $u(n)=(u(n-2)+u(n-1)) \bmod 10$, then for every natural number n , $u(n+12)=u(n)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem sequence_periodic_twelve
  (u :  $\mathbb{N} \rightarrow \mathbb{N}$ )
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ :  $\forall n \geq 2, u\ n = (u\ (n - 2) + u\ (n - 1)) \% 10$ ) :
 $\forall n, u\ (n + 12) = u\ n$  := by
  intro n
  refine Nat.strong_induction_on n ?_
  intro n ih
  cases n with
  | zero =>
    norm_num [h₂, h₀, h₁]
  | succ n =>
    cases n with
    | zero =>
      norm_num [h₂, h₀, h₁]
    | succ n =>
      have hp :  $u\ (n + 12) = u\ n$  := ih n (by omega)
      have hq :  $u\ (n + 1 + 12) = u\ (n + 1)$  := ih (n + 1) (by omega)
      have hleft := h₂ (n + 14) (by omega)
      have hright := h₂ (n + 2) (by omega)
      calc
         $u\ (n + 2 + 12) = (u\ (n + 12) + u\ (n + 13)) \% 10$  := by
          convert hleft using 1 <|> omega
        _ =  $(u\ n + u\ (n + 1)) \% 10$  := by
          rw [hp]
          exact congrArg (fun x => (u n + x) % 10) (by
            simpa [Nat.add_assoc, Nat.add_comm, Nat.add_left_comm] using hq)
        _ =  $u\ (n + 2)$  := by
          exact hright.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2002_p21__mh__ab69afd6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ab69afd68794e092

4.145 145. least_threshold_from_period_block

The problem

Let a sequence satisfy the same units-digit recurrence, and suppose it has period p with one-period sum b . If every partial block sum before $kp + r$ is at most T , while the sum at $kp + r$ exceeds T , prove that $kp + r$ is the least index whose partial sum exceeds T .

Lean statement

```
theorem least_threshold_from_period_block
  (u : ℕ → ℕ)

  (p b T k r : ℕ)
  (hp : 0 < p)
  (hperiod : ∀ n, u (n + p) = u n)
  (hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)

  (hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
  (hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
  IsLeast {n | S n > T} (k * p + r) := by
```

Parent. Consider the sequence of numbers: 4, 7, 1, 8, 9, 7, 6, ... For $n > 2$, the n -th term of the sequence is the units digit of the sum of the two previous terms. Let S_n denote the sum of the first n terms of this sequence. Show that the smallest value of n for which $S_n > 10,000$ is 1999.

Parent in Lean

```
theorem amc12a_2002_p21
  (u : ℕ → ℕ)
  (h₀ : u 0 = 4)
  (h₁ : u 1 = 7)
  (h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10)
  (h₃ : ∀ n, S n = ∑ k ∈ Finset.range n, u k)
  (Sn : Set ℕ)
  (h₄ : Sn = {n | S n > 10000}) :
  IsLeast Sn 1999 := by
```

What it contributes. Preserves the units-digit recurrence, initial values, partial-sum definition, and least-threshold conclusion shape while replacing the fixed threshold computation with a symbolic period-and-block-sum criterion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the threshold and crossing index and replaces the parent's concrete recurrence computation with a general period/block decomposition using quotient-remainder reasoning. Its proof must establish that every earlier index has the required block representation and bound, so recalling the parent's finite sum calculation does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete recurrence to arbitrary periodic sequences and symbolic block data, then uses quotient-remainder decomposition to prove minimality. This requires reasoning absent from the parent's finite numerical computation; the unused period hypothesis is redundant but does not collapse the new theorem to recall.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's finite recurrence computation with a symbolic quotient-and-remainder argument using an arbitrary period, block sum, and threshold-crossing bound. Its proof requires new decomposition reasoning and does not reuse the parent's explicit calculation or closing lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2723 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem least_threshold_from_period_block.extracted_1_1 (u S : ℕ → ℕ) (h₀ : u 0 = 4) (h₁ : u 1 = 7)
(h₂ : ∀ n ≥ 2, u n = (u (n - 2) + u (n - 1)) % 10) (h₃ : ∀ (n : ℕ), S n = ∑ i ∈ Finset.range n, u i) (p b T k r : ℕ)
(hp : 0 < p) (hperiod : ∀ (n : ℕ), u (n + p) = u n)
(hblock : ∀ (j s : ℕ), S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i) (hr : r < p)
(hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
(hbound : ∀ (j s : ℕ), j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
IsLeast {n | S n > T} (k * p + r)
```

Read back by a model that saw only the Lean. Let u and S be functions from the natural numbers to the natural numbers such that $u(0)=4$, $u(1)=7$, and for every natural number $n \geq 2$, $u(n)=(u(n-2)+u(n-1)) \bmod 10$. Suppose that for every natural number n , $S(n)=\sum_{i \in \text{Finset.range } n} u(i)$. Let p, b, T, k, r be natural numbers with $p > 0$, such that $u(n+p)=u(n)$ for every natural number n . Suppose also that for all natural numbers j and s , $S(jp+s)=jb+\sum_{i \in \text{Finset.range } s} u(i)$, that $r < p$, and that $kb+\sum_{i \in \text{Finset.range } r} u(i) > T$. Finally, assume that for all natural numbers j and s , if $jp+s < kp+r$, then $jb+\sum_{i \in \text{Finset.range } s} u(i) \leq T$. Then $kp+r$ is the least natural number n such that $S(n) > T$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem least_threshold_from_period_block
(u S : ℕ → ℕ)

(p b T k r : ℕ)
(hp : 0 < p)
(hperiod : ∀ n, u (n + p) = u n)
(hblock : ∀ j s, S (j * p + s) = j * b + ∑ i ∈ Finset.range s, u i)

(hcross : k * b + ∑ i ∈ Finset.range r, u i > T)
(hbound : ∀ j s, j * p + s < k * p + r → j * b + ∑ i ∈ Finset.range s, u i ≤ T) :
IsLeast {n | S n > T} (k * p + r) := by
have hperiod_checkpoint : ∀ n, u (n + p) = u n := hperiod
constructor
· change S (k * p + r) > T
  rw [hblock]
  exact hcross
· intro n hn
```

```

change S n > T at hn
by_contra hnot
have hnlt : n < k * p + r := Nat.lt_of_not_ge hnot
let j := n / p
let s := n % p
have hslt : s < p := Nat.mod_lt n hp
have hdecomp : j * p + s = n := by
  dsimp [j, s]
  simpa [Nat.mul_comm] using Nat.div_add_mod n p
have harg : j * p + s < k * p + r := by
  simpa [hdecomp] using hnlt
have hsum : j * b +  $\sum i \in \text{Finset.range } s, u i \leq T$  := hbound j s harg
have hSn : S n = j * b +  $\sum i \in \text{Finset.range } s, u i$  := by
  rw [← hdecomp]
  exact hblock j s
have hle : S n ≤ T := by
  rw [hSn]
  exact hsum
omega

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2002_p21__mh__c0433309

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 808f7f3adc74ba2b

4.146 146. sequence_characterization

The problem

Let $p, q \in \mathbb{R}$ and let a be a real sequence indexed by the natural numbers. Suppose that $a_{n+2} - a_{n+1} = a_{n+1} - a_n$ for every natural number n , and that $a_1 = p$, $a_2 = 9$, $a_3 = 3p - q$, and $a_4 = 3p + q$. Prove that $a_n = 4n + 1$ for every natural number n .

Lean statement

```
theorem sequence_characterization (p q : ℝ) (a : ℕ → ℝ) (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9)
  ↔ (h₃ : a 3 = 3 * p - q) (h₄ : a 4 = 3 * p + q) : ∀ n : ℕ, a n = 4 * (n : ℝ) + 1
```

Parent. The first four terms of an arithmetic sequence are p , 9 , $3p - q$, and $3p + q$. What is the 2010th term of this sequence? Show that it is 8041.

Parent in Lean

```
theorem amc12a_2010_p10 (p q : ℝ) (a : ℕ → ℝ) (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n)
  (h₁ : a 1 = p) (h₂ : a 2 = 9) (h₃ : a 3 = 3 * p - q) (h₄ : a 4 = 3 * p + q) : a 2010 = 8041 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child strengthens a single-value conclusion to a full characterization of the sequence, requiring the new boundary case $a_0 = 1$ and an induction over every natural index. The parent's formula covers positive indices, but it does not supply the argument at index 0.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single distant-value claim into a complete characterization for every natural index, including the newly required base case $a_0 = 1$. Its proof must derive the initial value and establish the recurrence globally, which the parent's proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single-index evaluation into a full characterization for every natural index, including the newly required base case $a_0 = 1$. Its proof must use the recurrence at index 0 and establish the step formula by induction, obligations absent from the parent's fixed-index argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2038 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sequence_characterization.extracted_1_1 : ∀ (p q : ℝ) (a : ℕ → ℝ),  
  (∀ (n : ℕ), a (n + 2) - a (n + 1) = a (n + 1) - a n) →  
    a 1 = p → a 2 = 9 → a 3 = 3 * p - q → a 4 = 3 * p + q → ∀ (n : ℕ), a n = 4 * ↑n + 1
```

Read back by a model that saw only the Lean. For all real numbers p and q and every sequence a of real numbers indexed by the natural numbers, if for every natural number n , $a(n + 2) - a(n + 1) = a(n + 1) - a(n)$, and $a(1) = p$, $a(2) = 9$, $a(3) = 3p - q$, and $a(4) = 3p + q$, then for every natural number n , $a(n) = 4n + 1$, where n is viewed as a real number.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem sequence_characterization (p q : ℝ) (a : ℕ → ℝ) (h₀ : ∀ n, a (n + 2) - a (n + 1) = a (n + 1) - a n) (h₁ : a 1 = p) (h₂ : a 2 = 9)
  ↪ (h₃ : a 3 = 3 * p - q) (h₄ : a 4 = 3 * p + q) : ∀ n : ℕ, a n = 4 * (n : ℝ) + 1 := by
  have hA := h₀ 1
  have hB := h₀ 2
  norm_num at hA hB
  rw [h₃, h₂, h₁] at hA
  rw [h₄, h₃, h₂] at hB
  have hp : p = 5 := by
    linarith
  have hzero := h₀ 0
  norm_num at hzero
  rw [h₂, h₁, hp] at hzero
  have ha0 : a 0 = 1 := by
    linarith
  have hstep : ∀ n : ℕ, a (n + 1) - a n = 4 := by
    intro n
    induction n with
    | zero =>
      norm_num [h₁, hp, ha0]
    | succ n ih =>
      have hrec : a (n + 2) - a (n + 1) = a (n + 1) - a n := h₀ n
      linarith
  intro n
  induction n with
  | zero =>
    norm_num [ha0]
  | succ n ih =>
    have hs := hstep n
    have ha : a (n + 1) = a n + 4 := by
      linarith
    rw [ha, ih]
    norm_num [Nat.cast_succ]
  ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2010_p10__mh__43a400bc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0927852decccfbe

4.147 147. bounded_power_sum_product

The problem

For every natural number n , show that there exists a sequence of natural numbers $(e_k)_{k \geq 0}$ such that $e_0 = 1$, $e_{k+1} = 2e_k$ for every natural number k , and $\prod_{k=0}^{n-1} (2^{e_k} + 3^{e_k}) = 3^{e_n} - 2^{e_n}$.

Lean statement

```
theorem bounded_power_sum_product :
  ∀ n : ℕ, ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, ((2 : ℤ) ^ e k + (3 : ℤ) ^ e k)) =
      (3 : ℤ) ^ e n - (2 : ℤ) ^ e n
```

Parent. Show that the following equality holds: $(2+3)(2^2+3^2)(2^4+3^4)(2^8+3^8)(2^{16}+3^{16})(2^{32}+3^{32})(2^{64}+3^{64}) = 3^{128}-2^{128}$.

Parent in Lean

```
theorem amc12a_2021_p9 :
  (∏ k ∈ Finset.range (7 : ℕ),
    ((2 : ℕ) ^ (2 ^ k : ℕ) + (3 : ℕ) ^ (2 ^ (k : ℕ)))) =
    (3 : ℕ) ^ (128 : ℕ) - (2 : ℕ) ^ (128 : ℕ) := by
```

What it contributes. Supplied the power-sum finite-product domain and the terminal difference-of-powers conclusion shape; its fixed seven-factor identity is generalized to arbitrary n .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed seven-factor identity to every finite index and explicitly proves the exponent recurrence. Its induction derives the telescoping step over $\mathbb{Z}$, whereas the parent is only a concrete ‘norm_num’ computation, so memorizing the parent does not supply the child’s proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor numerical identity with a universally bounded-index product law and explicitly requires constructing an exponent sequence satisfying a recurrence. Its inductive telescoping proof is genuinely new and is not recoverable by recalling the parent’s finite norm_num computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed seven-factor numerical identity with a universal finite-product law and explicitly proves the exponent recurrence. Its induction and telescoping argument require mathematical reasoning absent from the parent’s one-shot normalization proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1797 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_power_sum_product.extracted_1_1 : ∀ (n : ℕ),
  ∃ e,
    e 0 = 1 ∧ (∀ (k : ℕ), e (k + 1) = 2 * e k) ∧ ∏ k ∈ Finset.range n, (2 ^ e k + 3 ^ e k) = 3 ^ e n - 2 ^ e n
```

Read back by a model that saw only the Lean. *For every natural number n , there exists a sequence e of natural numbers such that $e(0) = 1$, for every natural number k we have $e(k + 1) = 2 \cdot e(k)$, and the product over all k in $\{0, 1, \dots, n - 1\}$ of $(2^{e(k)} + 3^{e(k)})$ equals $3^{e(n)} - 2^{e(n)}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_power_sum_product :
  ∀ n : ℕ, ∃ e : ℕ → ℕ,
    e 0 = 1 ∧
    (∀ k : ℕ, e (k + 1) = 2 * e k) ∧
    (∏ k ∈ Finset.range n, ((2 : ℤ) ^ e k + (3 : ℤ) ^ e k)) =
      (3 : ℤ) ^ e n - (2 : ℤ) ^ e n := by
  intro n
  let e : ℕ → ℕ := fun k => 2 ^ k
  refine (e, ?_, ?_, ?_)
  · simp [e]
  · intro k
    simp [e, pow_succ, Nat.mul_comm]
  · have htel : ∀ m : ℕ,
      (∏ k ∈ Finset.range m,
        ((2 : ℤ) ^ (2 ^ k) + (3 : ℤ) ^ (2 ^ k))) =
        (3 : ℤ) ^ (2 ^ m) - (2 : ℤ) ^ (2 ^ m) := by
    intro m
    induction m with
    | zero => simp
    | succ m ih =>
      rw [Finset.prod_range_succ, ih]
      have hexp : 2 ^ (Nat.succ m) = 2 * 2 ^ m := by
        simp [pow_succ, Nat.mul_comm]
      have h3 : (3 : ℤ) ^ (2 ^ (Nat.succ m)) =
        ((3 : ℤ) ^ (2 ^ m)) ^ 2 := by
        rw [hexp, Nat.mul_comm, pow_mul]
      have h2 : (2 : ℤ) ^ (2 ^ (Nat.succ m)) =
        ((2 : ℤ) ^ (2 ^ m)) ^ 2 := by
        rw [hexp, Nat.mul_comm, pow_mul]
      rw [h3, h2]
      ring
    simp [e] using htel n
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12a_2021_p9__mh__48b45c78

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0e021d853daa103f

4.148 148. generalized_amc12b_2020_p22

The problem

For every real number $a \geq 1$, the set of values $(2^t - a)t / 4^t$, as t ranges over $\mathbb{R}$, has greatest element $1/(4a)$.

Lean statement

```
theorem generalized_amc12b_2020_p22
  (a : ℝ) (ha : 1 ≤ a) (S : Set ℝ)
  (hS : S = {x : ℝ | ∃ t : ℝ, x = (2 ^ t - a * t) * t / 4 ^ t}) :
  IsGreatest S (1 / (4 * a))
```

Parent. What is the maximum value of $\frac{(2^t - 3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS : S = {x : ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12 : ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the coefficient and maximum, requiring the new hypothesis $a \geq 1$ to establish that $1/(2a)$ lies in the intermediate-value interval and to weight the square nonnegativity argument by a . The parent's fixed coefficient proof cannot supply the generalized theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the coefficient from 3 to arbitrary $a \geq 1$, requiring a new target value $1/(2a)$ in the intermediate-value argument and an a -dependent square-completion bound. The parent's fixed target and algebra do not directly supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the coefficient and maximum, requiring the new hypothesis $a \geq 1$ to establish positivity and a parameter-dependent square-completion identity. The parent's fixed coefficient 3 argument does not supply this symbolic reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2093 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem generalized_amc12b_2020_p22.extracted_1_1 (a : ℝ) (ha : 1 ≤ a) (S : Set ℝ)
(hS : S = {x | ∃ t, x = (2 ^ t - a * t) * t / 4 ^ t}) : IsGreatest S (1 / (4 * a))
```

Read back by a model that saw only the Lean. For every real number a with $1 \leq a$, if S is the set of all real numbers of the form $((2^t - a)t)/4^t$ as t ranges over the real numbers, then $1/(4a)$ is the greatest element of S .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem generalized_amc12b_2020_p22
(a : ℝ) (ha : 1 ≤ a) (S : Set ℝ)
(hS : S = {x : ℝ | ∃ t : ℝ, x = (2 ^ t - a * t) * t / 4 ^ t}) :
IsGreatest S (1 / (4 * a)) := by
have ha0 : 0 < a := by linarith
let g : ℝ → ℝ := fun t => t / 2 ^ t
have hg : Continuous g := by
dsimp [g]
exact continuous_id.div (Real.continuous_const_rpow (by norm_num))
(fun x => by positivity)
have hi := intermediate_value.Icc (show (0 : ℝ) ≤ 1 by norm_num) hg.continuousOn
have hleft : g 0 = 0 := by norm_num [g]
have hright : g 1 = (1 / 2 : ℝ) := by norm_num [g]
have hm : (1 / (2 * a) : ℝ) ∈ Set.Icc (g 0) (g 1) := by
rw [hleft, hright]
constructor
· positivity
· apply (div_le_iff (by positivity : (0 : ℝ) < 2 * a)).2
  nlinarith
obtain ⟨t, ht, hgt⟩ := hi hm
rw [hS]
constructor
· refine ⟨t, ?_⟩
have hden : (4 : ℝ) ^ t = (2 ^ t) ^ 2 := by
calc
(4 : ℝ) ^ t = (2 ^ 2) ^ t := by norm_num
_ = 2 ^ (2 * t) :=
(Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 t).symm
_ = 2 ^ t * 2 ^ t := by
rw [show (2 : ℝ) * t = t + t by ring, Real.rpow_add (by norm_num)]
_ = (2 ^ t) ^ 2 := by ring
rw [hden]
have hp : 0 < (2 : ℝ) ^ t := by positivity
have hrel : t / 2 ^ t = 1 / (2 * a) := by
simp [g] using hgt
have hid :
(2 ^ t - a * t) * t / (2 ^ t) ^ 2 =
t / 2 ^ t - a * (t / 2 ^ t) ^ 2 := by
field_simp [ne_of_gt hp] <|> ring
rw [hid, hrel]
field_simp [ne_of_gt ha0] <|> ring
· rintro x ⟨s, rfl⟩
have hden : (4 : ℝ) ^ s = (2 ^ s) ^ 2 := by
calc
(4 : ℝ) ^ s = (2 ^ 2) ^ s := by norm_num
_ = 2 ^ (2 * s) :=
(Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 s).symm
_ = 2 ^ s * 2 ^ s := by
rw [show (2 : ℝ) * s = s + s by ring, Real.rpow_add (by norm_num)]
_ = (2 ^ s) ^ 2 := by ring
rw [hden]
have hp : 0 < (2 : ℝ) ^ s := by positivity
have hid :
(2 ^ s - a * s) * s / (2 ^ s) ^ 2 =
s / 2 ^ s - a * (s / 2 ^ s) ^ 2 := by
field_simp [ne_of_gt hp] <|> ring
```

```

rw [hid]
have hsquare : 0 ≤ (s / 2 ^ s - 1 / (2 * a)) ^ 2 := sq_nonneg _
have hnonneg :
  0 ≤ a * (s / 2 ^ s - 1 / (2 * a)) ^ 2 :=
  mul_nonneg (le_of_lt ha0) hsquare
have hident :
  s / 2 ^ s - a * (s / 2 ^ s) ^ 2 =
  1 / (4 * a) - a * (s / 2 ^ s - 1 / (2 * a)) ^ 2 := by
  field_simp [ne_of_gt hp, ne_of_gt ha0] <|> ring
rw [hident]
linarith

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12b_2020_p22__mh__c39c930b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c39c930bbcd712e1

4.149 149. exact_maximizer_characterization

The problem

Prove that for every real number s , $\frac{(2^s-3s)s}{4^s} \leq \frac{1}{12}$, with equality if and only if $\frac{s}{2^s} = \frac{1}{6}$; moreover, the value $\frac{1}{12}$ is attained for some real number t .

Lean statement

```
theorem exact_maximizer_characterization
  (S : Set ℝ)
  :
  (∀ s : ℝ, (2 ^ s - 3 * s) * s / 4 ^ s ≤ 1 / 12) ∧
  (∀ s : ℝ,
    (2 ^ s - 3 * s) * s / 4 ^ s = 1 / 12 ↔
    s / 2 ^ s = 1 / 6) ∧
  (∃ t : ℝ, (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12)
```

Parent. What is the maximum value of $\frac{(2^t-3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS: S = {x:ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a greatest-element claim to a full equality characterization, requiring proof that equality occurs exactly when $s / 2^s = 1/6$, in addition to the bound and attainment. This characterization is not supplied by merely recalling the parent's IsGreatest proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a greatest-element claim to a full maximizer characterization, including the nontrivial iff identifying equality exactly with $s/2^s = 1/6$. The parent's inequality proof does not establish this equality condition, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from proving an IsGreatest set witness to a full analytic characterization: global upper bound, exact equality condition, and attainment. The equality condition requires new reasoning that the parent proof does not supply, namely showing equality in the square bound forces $s / 2^s = 1/6$.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2070 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exact_maximizer_characterization.extracted_1_1 :
  (∀ (s : ℝ), (2 ^ s - 3 * s) * s / 4 ^ s ≤ 1 / 12) ∧
  (∀ (s : ℝ), (2 ^ s - 3 * s) * s / 4 ^ s = 1 / 12 ↔ s / 2 ^ s = 1 / 6) ∧
  ∃ t, (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12
```

Read back by a model that saw only the Lean. For every real number s , $((2^s - 3s)s)/4^s \leq 1/12$; for every real number s , $((2^s - 3s)s)/4^s = 1/12$ if and only if $s/2^s = 1/6$; and there exists a real number t such that $((2^t - 3t)t)/4^t = 1/12$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem exact_maximizer_characterization
  (S : Set ℝ)
  :
  (∀ s : ℝ, (2 ^ s - 3 * s) * s / 4 ^ s ≤ 1 / 12) ∧
  (∀ s : ℝ,
    (2 ^ s - 3 * s) * s / 4 ^ s = 1 / 12 ↔
    s / 2 ^ s = 1 / 6) ∧
  (∃ t : ℝ, (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12) := by
  let g : ℝ → ℝ := fun t => t / 2 ^ t
  have hg : Continuous g := by
    dsimp [g]
    exact continuous_id.div (Real.continuous_const_rpow (by norm_num))
    (fun x => by positivity)
  have hi := intermediate_value_Icc (show (0 : ℝ) ≤ 1 by norm_num) hg.continuousOn
  have hm : (1 / 6 : ℝ) ∈ Set.Icc (g 0) (g 1) := by
    norm_num [g]
  obtain (t, ht, hgt) := hi hm
  have hattain : (2 ^ t - 3 * t) * t / 4 ^ t = 1 / 12 := by
    have hden : (4 : ℝ) ^ t = (2 ^ t) ^ 2 := by
      calc
        (4 : ℝ) ^ t = (2 ^ 2) ^ t := by norm_num
        _ = 2 ^ (2 * t) :=
          (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 t).symm
        _ = 2 ^ t * 2 ^ t := by
          rw [show (2 : ℝ) * t = t + t by ring, Real.rpow_add (by norm_num)]
        _ = (2 ^ t) ^ 2 := by ring
    rw [hden]
  have hp : 0 < (2 : ℝ) ^ t := by positivity
  dsimp [g] at hgt
  field_simp [ne_of_gt hp] at hgt
  nlinarith
constructor
· intro s
  have hden : (4 : ℝ) ^ s = (2 ^ s) ^ 2 := by
    calc
      (4 : ℝ) ^ s = (2 ^ 2) ^ s := by norm_num
      _ = 2 ^ (2 * s) :=
        (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 s).symm
      _ = 2 ^ s * 2 ^ s := by
        rw [show (2 : ℝ) * s = s + s by ring, Real.rpow_add (by norm_num)]
      _ = (2 ^ s) ^ 2 := by ring
  rw [hden]
  have hp : 0 < (2 : ℝ) ^ s := by positivity
  have hsquare : 0 ≤ (s / 2 ^ s - 1 / 6) ^ 2 := sq_nonneg _
  have hid :
    (2 ^ s - 3 * s) * s / (2 ^ s) ^ 2 =
```

```

      s / 2 ^ s - 3 * (s / 2 ^ s) ^ 2 := by
    field_simp [ne_of_gt hp] <|> ring
  rw [hid]
  nlinarith
· constructor
· intro s
  have hden : (4 : ℝ) ^ s = (2 ^ s) ^ 2 := by
    calc
      (4 : ℝ) ^ s = (2 ^ 2) ^ s := by norm_num
      _ = 2 ^ (2 * s) :=
        (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 s).symm
      _ = 2 ^ s * 2 ^ s := by
        rw [show (2 : ℝ) * s = s + s by ring, Real.rpow_add (by norm_num)]
      _ = (2 ^ s) ^ 2 := by ring
  rw [hden]
  have hp : 0 < (2 : ℝ) ^ s := by positivity
  have hsquare : 0 ≤ (s / 2 ^ s - 1 / 6) ^ 2 := sq_nonneg _
  have hid :
    (2 ^ s - 3 * s) * s / (2 ^ s) ^ 2 =
      s / 2 ^ s - 3 * (s / 2 ^ s) ^ 2 := by
    field_simp [ne_of_gt hp] <|> ring
  rw [hid]
  constructor
  · intro heq
    nlinarith
  · intro hq
    nlinarith
· exact ⟨t, hattain⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12b_2020_p22__mh__dfb91952

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fb4e4a95f2f0f3fb

4.150 150. amc12b_2020_p22_mutation

The problem

Show that every real value between 0 and $\frac{1}{12}$ is attained by $\frac{(2^t-3t)t}{4^t}$ for some $t \in [0, 1]$.

Lean statement

```
theorem amc12b_2020_p22_mutation
  : ∀ y : ℝ, y ∈ Set.Icc 0 (1 / 12 : ℝ) →
    ∃ t ∈ Set.Icc 0 1,
      y = ((2 : ℝ) ^ t - 3 * t) * t / (4 : ℝ) ^ t := by
```

Parent. What is the maximum value of $\frac{(2^t-3t)t}{4^t}$ for real values of t ? Show that it is $\frac{1}{12}$.

Parent in Lean

```
theorem amc12b_2020_p22
  (S : Set ℝ)
  (hS: S = {x:ℝ | ∃ t, x = (2 ^ t - 3 * t) * t / 4 ^ t}) :
  IsGreatest S (1 / 12:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the maximum-value claim into a substantially different range characterization: every value in $[0, 1/12]$ is attained on $[0, 1]$. Its proof requires a second continuity and intermediate-value argument, which the parent's upper-bound proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a greatest-value claim into a range-surjectivity statement: every y in $[0, 1/12]$ must be attained by an exponent t in $[0, 1]$. It requires a new continuity argument for the original expression and a second intermediate-value construction, beyond the parent's single extremum proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from proving a greatest element to proving that every value in the full interval $[0, 1/12]$ is attained. Its proof requires a new continuity argument for the original expression and a second intermediate-value application, which the parent's maximum proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2373 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem amc12b_2020_p22_mutation.extracted_1_1 : ∀ y ∈ Set.Icc 0 (1 / 12),  
  ∃ t ∈ Set.Icc 0 1, y = (2 ^ t - 3 * t) * t / 4 ^ t
```

Read back by a model that saw only the Lean. For every real number y in the closed interval $[0, 1/12]$, there exists a real number t in the closed interval $[0, 1]$ such that $y = ((2^t - 3t)t) / 4^t$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem amc12b_2020_p22_mutation
  : ∀ y : ℝ, y ∈ Set.Icc 0 (1 / 12 : ℝ) →
    ∃ t ∈ Set.Icc 0 1,
      y = ((2 : ℝ) ^ t - 3 * t) * t / (4 : ℝ) ^ t := by
  let g : ℝ → ℝ := fun t => t / 2 ^ t
  let f : ℝ → ℝ := fun t => ((2 : ℝ) ^ t - 3 * t) * t / (4 : ℝ) ^ t
  have hg : Continuous g := by
    dsimp [g]
    exact continuous_id.div (Real.continuous_const_rpow (by norm_num))
    (fun x => by positivity)
  have hf : Continuous f := by
    have h2 : Continuous (fun t : ℝ => (2 : ℝ) ^ t) :=
      Real.continuous_const_rpow (by norm_num)
    have h4 : Continuous (fun t : ℝ => (4 : ℝ) ^ t) :=
      Real.continuous_const_rpow (by norm_num)
    dsimp [f]
    exact ((h2.sub (continuous_const.mul continuous_id)).mul continuous_id).div h4
    (fun x => by positivity)
  have hi := intermediate_value_Icc (show (0 : ℝ) ≤ 1 by norm_num) hg.continuousOn
  have hm : (1 / 6 : ℝ) ∈ Set.Icc (g 0) (g 1) := by
    norm_num [g]
  obtain (t₀, ht₀, hgt₀) := hi hm
  have hformula : ∀ u : ℝ, u / 2 ^ u = 1 / 6 → f u = 1 / 12 := by
    intro u hu
    have hden : (4 : ℝ) ^ u = (2 ^ u) ^ 2 := by
      calc
        (4 : ℝ) ^ u = (2 ^ 2) ^ u := by norm_num
        _ = 2 ^ (2 * u) :=
          (Real.rpow_mul (by norm_num : (0 : ℝ) ≤ 2) 2 u).symm
        _ = 2 ^ u * 2 ^ u := by
          rw [show (2 : ℝ) * u = u + u by ring, Real.rpow_add (by norm_num)]
        _ = (2 ^ u) ^ 2 := by ring
    dsimp [f]
    rw [hden]
    have hp : 0 < (2 : ℝ) ^ u := by positivity
    field_simp [ne_of_gt hp] at hu ⊢
    nlinarith
  have hgt₀' : t₀ / 2 ^ t₀ = 1 / 6 := by
    simpa [g] using hgt₀
  have hft₀ : f t₀ = 1 / 12 := hformula t₀ hgt₀'
  intro y hy
  have hf0 : f 0 = 0 := by
    norm_num [f]
  have hi' := intermediate_value_Icc ht₀.1 hf.continuousOn
  have hy' : y ∈ Set.Icc (f 0) (f t₀) := by
    rw [hf0, hft₀]
    exact hy
  obtain (t, ht, hyt) := hi' hy'
  have ht01 : t ∈ Set.Icc 0 1 := by
    exact (ht.1, le_trans ht.2 ht₀.2)
  exact (t, ht01, hyt.symm)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. amc12b_2020_p22__mh__eba9b6e6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint eba9b6e66e48af7e

4.151 151. unique_lattice_coordinate_period

The problem

Let f have periods p and q , and suppose no nontrivial integer combination of p and q is zero. Show that every displacement represented as $mp + nq$ has unique integer coordinates (m, n) and is a period of f .

Lean statement

```
theorem unique_lattice_coordinate_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x)
  (hind : ∀ a b : ℤ, (a : ℝ) * p + (b : ℝ) * q = 0 → a = 0 ∧ b = 0) :
  ∀ d : ℝ, (∃ m n : ℤ, d = (m : ℝ) * p + (n : ℝ) * q) →
    ∃! z : ℤ × ℤ,
      d = (z.1 : ℝ) * p + (z.2 : ℝ) * q ∧
      ∀ x : ℝ, f (x + d) = f x
```

Parent. Let f be a real-valued function with periods p and q . Show that every integer linear combination $mp + nq$ is also a period of f .

Parent in Lean

```
theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x
```

What it contributes. Supplies the integer-linear-combination periodicity checkpoint, reproduced as the named have hcomb and consumed to prove periodicity of the represented displacement.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new uniqueness characterization of integer coordinates, proved using the integer-independence hypothesis; the parent only establishes periodicity of integer combinations. Although the periodicity argument is reused, the uniqueness proof requires reasoning unavailable from the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness characterization of lattice coordinates, requiring the integer-independence hypothesis to show two representations coincide. The parent only establishes periodicity for integer combinations and does not supply this uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness theorem for lattice coordinates, requiring the integer-independence hypothesis and a difference-of-coordinates argument. Although it reuses the parent's periodicity result for existence, the parent does not supply the essential uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2502 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_lattice_coordinate_period.extracted_1_1 : ∀ (f : ℝ → ℝ) (p q : ℝ),
  (∀ (x : ℝ), f (x + p) = f x) →
  (∀ (x : ℝ), f (x + q) = f x) →
  (∀ (a b : ℤ), ↑a * p + ↑b * q = 0 → a = 0 ∧ b = 0) →
  ∀ (d : ℝ), (∃ m n, d = ↑m * p + ↑n * q) → ∃! z, d = ↑z.1 * p + ↑z.2 * q ∧ ∀ (x : ℝ), f (x + d) = f x
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers and every real numbers p and q , if p and q are periods of f —that is, $f(x + p) = f(x)$ and $f(x + q) = f(x)$ for every real x —and if the only integers a and b satisfying $a \cdot p + b \cdot q = 0$ are $a = 0$ and $b = 0$, then for every real number d that can be written as $d = m \cdot p + n \cdot q$ for some integers m and n , there exists a unique pair $z = (z_1, z_2)$ of integers such that $d = z_1 \cdot p + z_2 \cdot q$ and d is a period of f , meaning $f(x + d) = f(x)$ for every real x .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem unique_lattice_coordinate_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x)
  (hind : ∀ a b : ℤ, (a : ℝ) * p + (b : ℝ) * q = 0 → a = 0 ∧ b = 0) :
  ∀ d : ℝ, (∃ m n : ℤ, d = (m : ℝ) * p + (n : ℝ) * q) →
  ∃! z : ℤ × ℤ,
    d = (z.1 : ℝ) * p + (z.2 : ℝ) * q ∧
    ∀ x : ℝ, f (x + d) = f x := by
  have hcomb : ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x := by
    have hp_periodic : Function.Periodic f p := hp
    have hq_periodic : Function.Periodic f q := hq
    intro m n x
    have hpm : Function.Periodic f ((m : ℝ) * p) := hp_periodic.int_mul m
    have hqn : Function.Periodic f ((n : ℝ) * q) := hq_periodic.int_mul n
    calc
      f (x + (m : ℝ) * p + (n : ℝ) * q) =
        f ((x + (n : ℝ) * q) + (m : ℝ) * p) := by
          congr 1
          ring
        _ = f (x + (n : ℝ) * q) := hpm (x + (n : ℝ) * q)
        _ = f x := hqn x
  intro d hd
  obtain ⟨m, n, hdn⟩ := hd
  have hperiod : ∀ x : ℝ, f (x + d) = f x := by
    intro x
    rw [hdn]
    simpa [add_assoc] using hcomb m n x
  refine ⟨(m, n), ⟨hdn, hperiod⟩, ?_⟩
  intro z hz
  have hrel : (z.1 : ℝ) * p + (z.2 : ℝ) * q =
    (m : ℝ) * p + (n : ℝ) * q := by
    linarith [hz.1, hdn]
  have hdiff : ((z.1 - m : ℤ) : ℝ) * p +
    ((z.2 - n : ℤ) : ℝ) * q = 0 := by
    push_cast
    linarith [hrel]
```

```
have hcoords := hind (z.1 - m) (z.2 - n) hdiff
apply Prod.ext
· exact sub_eq_zero.mp hcoords.1
· exact sub_eq_zero.mp hcoords.2
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.mh.mh__1d992577

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1d9925779b572639

4.152 152. finite_integer_combination_period

The problem

Let f be a real-valued function, and let each member of a finite indexed family be a period of f . Show that every integer linear combination of these periods is also a period of f .

Lean statement

```
theorem finite_integer_combination_period (f : ℝ → ℝ) {ι : Type} [Fintype ι]
  (p : ι → ℝ) (hperiod : ∀ i : ι, ∀ x : ℝ, f (x + p i) = f x) :
  ∀ c : ι → ℤ, ∀ x : ℝ,
    f (x + ∑ i : ι, (c i : ℝ) * p i) = f x
```

Parent. Let f be a real-valued function with periods p and q . Show that every integer linear combination $mp + nq$ is also a period of f .

Parent in Lean

```
theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x
```

What it contributes. Generalizes the parent's integer-multiple and period-composition argument from two named generators to an arbitrary finite indexed family and finite sum.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from two fixed periods to an arbitrary finite indexed family. Its proof requires finite-set induction and aggregation of periods, which the parent's two-generator argument does not directly supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from two fixed generators to an arbitrary finite indexed family and requires induction over the finite sum, combining each generator's integer multiple as a period. The parent's two-generator calculation does not supply this finite-family induction or its sum bookkeeping.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from two fixed generators to an arbitrary finite indexed family. Its proof requires finite-set induction and combining periods across the family, which the parent's two-generator argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2500 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_integer_combination_period.extracted_1_1 : ∀ (f : ℝ → ℝ) {ι : Type} [inst : Fintype ι] (p : ι → ℝ),
  (∀ (i : ι) (x : ℝ), f (x + p i) = f x) → ∀ (c : ι → ℤ) (x : ℝ), f (x + ∑ i, ↑(c i) * p i) = f x
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers, every finite type ι , and every family of real numbers p indexed by ι , if for every i in ι and every real number x we have $f(x + p(i)) = f(x)$, then for every family of integers c indexed by ι and every real number x , we have $f(x + \sum_{i \in \iota} (c(i) \cdot p(i))) = f(x)$, where each integer $c(i)$ is regarded as a real number.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem finite_integer_combination_period (f : ℝ → ℝ) {ι : Type} [Fintype ι]
  (p : ι → ℝ) (hperiod : ∀ i : ι, ∀ x : ℝ, f (x + p i) = f x) :
  ∀ c : ι → ℤ, ∀ x : ℝ,
    f (x + ∑ i : ι, (c i : ℝ) * p i) = f x := by
  classical
  intro c x
  have hsum : ∀ s : Finset ι,
    Function.Periodic f (∑ i ∈ s, (c i : ℝ) * p i) := by
    intro s
    induction s using Finset.induction_on with
    | empty =>
      intro y
      simp
    | @insert a s ha ih =>
      have hca : Function.Periodic f ((c a : ℝ) * p a) := by
        have hpa : Function.Periodic f (p a) := hperiod a
        exact hpa.int_mul (c a)
      have hadd := ih.add_period hca
      simp [Finset.sum_insert, ha, add_assoc, add_comm, add_left_comm] using hadd
  exact hsum Finset.univ x
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.mh.mh__5e6484a3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5e6484a32a7b7d00

4.153 153. integer_combination_period

The problem

Let f be a real-valued function with periods p and q . Show that every integer linear combination $mp + nq$ is also a period of f .

Lean statement

```
theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x
```

Parent. Let f be a real-valued function with a positive period p . Show that every natural multiple of p is also a period of f .

Parent in Lean

```
theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
```

What it contributes. Supplied the natural-multiple induction checkpoint for a single period; the child extends it to negative shifts and composes two independent period translations.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from one natural nonnegative iterate to arbitrary integer combinations of two independent periods. Its proof requires integer-period closure and composition of two translations, which the parent's induction argument does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires combining two independent periods and extending periodicity to arbitrary positive and negative integer multiples, using reasoning absent from the parent's natural-number induction. It is therefore not solvable by recalling the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from one positive natural iterate to arbitrary integer combinations of two independent periods. It requires handling integer multiples, including negative coefficients, and combining the two periodicities, none of which is supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2497 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem integer_combination_period.extracted_1_1 : ∀ (f : ℝ → ℝ) (p q : ℝ),
  (∀ (x : ℝ), f (x + p) = f x) →
  (∀ (x : ℝ), f (x + q) = f x) → ∀ (m n : ℤ) (x : ℝ), f (x + ↑m * p + ↑n * q) = f x
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers and every real numbers p and q , if $f(x + p) = f(x)$ for every real x and $f(x + q) = f(x)$ for every real x , then for every integers m and n and every real x , $f(x + m p + n q) = f(x)$, where the integers m and n are regarded as real numbers.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem integer_combination_period (f : ℝ → ℝ) (p q : ℝ)
  (hp : ∀ x : ℝ, f (x + p) = f x)
  (hq : ∀ x : ℝ, f (x + q) = f x) :
  ∀ m n : ℤ, ∀ x : ℝ,
    f (x + (m : ℝ) * p + (n : ℝ) * q) = f x := by
  have hp_periodic : Function.Periodic f p := hp
  have hq_periodic : Function.Periodic f q := hq
  intro m n x
  have hpm : Function.Periodic f ((m : ℝ) * p) := hp_periodic.int_mul m
  have hqn : Function.Periodic f ((n : ℝ) * q) := hq_periodic.int_mul n
  calc
    f (x + (m : ℝ) * p + (n : ℝ) * q) =
      f ((x + (n : ℝ) * q) + (m : ℝ) * p) := by
        congr 1
        ring
    _ = f (x + (n : ℝ) * q) := hqn (x + (n : ℝ) * q)
    _ = f x := hpm x
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.mh__4b068dd2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b068dd24ba4c73c

4.154 154. periodic_integer_subgroup_action

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ satisfy $f(x + p) = f(x)$ for every real x . Show that every translation belonging to the additive subgroup generated by p is a period of f , including negative integer multiples.

Lean statement

```
theorem periodic_integer_subgroup_action (f : ℝ → ℝ) (p : ℝ) (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ d : ℝ, d ∈ AddSubgroup.zmultiples p → Function.Periodic f d := by
```

Parent. For a real-valued function and a real shift p , the law $f(x + p) = f(x)$ for every real x is equivalent to saying that every natural multiple of p is a period of f .

Parent in Lean

```
theorem periodic_natural_multiple_equivalence (f : ℝ → ℝ) (p : ℝ) :
  (∀ x : ℝ, f (x + p) = f x) ↔
  ∀ n : ℕ, Function.Periodic f ((n : ℝ) * p) := by
```

What it contributes. Supplied the base-period conversion from the pointwise equation and the natural-multiple induction checkpoint; the child extends this to signed multiples and subgroup membership.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely strengthens natural multiples to all signed integer multiples represented via additive-subgroup membership. Its proof requires new reasoning for negative periods and extracting an integer representation from subgroup membership, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires extending period invariance from natural multiples to all signed integer multiples, then converting subgroup membership into such a representation. The parent's proof supplies only natural multiplication and does not establish negative-period invariance or the subgroup representation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens natural-multiple periodicity to every signed integer multiple represented by membership in `AddSubgroup.zmultiples p`. Its proof must handle negative integers and extract an integer representation from subgroup membership, neither of which is supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2501 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem periodic_integer_subgroup_action.extracted_1_1 : ∀ (f : ℝ → ℝ) (p : ℝ),  
  (∀ (x : ℝ), f (x + p) = f x) → ∀ d ∈ AddSubgroup.zmultiples p, Function.Periodic f d
```

Read back by a model that saw only the Lean. For every function $f : \mathbb{R} \rightarrow \mathbb{R}$ and every real number p , if for every real number x we have $f(x + p) = f(x)$, then for every d belonging to the additive subgroup of $\mathbb{R}$ consisting of the integer multiples of p , d is a period of f ; that is, for every real number x , $f(x + d) = f(x)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem periodic_integer_subgroup_action (f : ℝ → ℝ) (p : ℝ) (hperiod : ∀ x : ℝ, f (x + p) = f x) :  
  ∀ d : ℝ, d ∈ AddSubgroup.zmultiples p → Function.Periodic f d := by  
  intro d hd  
  have hp : Function.Periodic f p := by  
    intro x  
    exact hperiod x  
  have hnég : Function.Periodic f (-p) := hp.neg  
  have hsigned : ∀ n : ℤ, Function.Periodic f (n • p) := by  
    intro n  
    cases n with  
    | ofNat n =>  
      simpa using hp.nsmul n  
    | negSucc n =>  
      have hh := hnég.nsmul (n + 1)  
      convert hh using 1 <|> simp [negSucc_zsmul] <|> ring  
  have hrepr : ∃ n : ℤ, n • p = d := (AddSubgroup.mem_zmultiples_iff).mp hd  
  rcases hrepr with ⟨n, hn⟩  
  rw [← hn]  
  exact hsigned n
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.ms.mh__76336c3e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 76336c3ebf68c293

4.155 155. periodic_natural_multiple_equivalence

The problem

For a real-valued function and a real shift p , the law $f(x + p) = f(x)$ for every real x is equivalent to saying that every natural multiple of p is a period of f .

Lean statement

```
theorem periodic_natural_multiple_equivalence (f : ℝ → ℝ) (p : ℝ) :
  (∀ x : ℝ, f (x + p) = f x) ↔
  ∀ n : ℕ, Function.Periodic f ((n : ℝ) * p) := by
```

Parent. Let f be a real-valued function with a positive period p . Show that every natural multiple of p is also a period of f .

Parent in Lean

```
theorem natural_iterate_period (f : ℝ → ℝ) (p : ℝ)
  (hperiod : ∀ x : ℝ, f (x + p) = f x) :
  ∀ n : ℕ, ∀ x : ℝ, f (x + (n : ℝ) * p) = f x := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is mathematically equivalent: the parent's period law implies that every natural multiple is a period, while the case $n = 1$ recovers the original law. Recasting the conclusion through `Function.Periodic` and an equivalence is a substantial Lean-surface re-encoding.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent: the forward direction derives every natural multiple period, while the reverse direction recovers the original period from the case $n = 1$. Recasting the conclusion through `Function.Periodic` and an iff is a genuine surface re-encoding.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is logically equivalent to the parent: its forward direction is the natural-multiple period law, while the converse recovers the original period by taking $n = 1$. It substantially changes the Lean surface by expressing the conclusion through `Function.Periodic` and an iff.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2495 statements
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem periodic_natural_multiple_equivalence.extracted_1_1 (f : ℝ → ℝ) (p : ℝ) :  
  (∀ (x : ℝ), f (x + p) = f x) ↔ ∀ (n : ℕ), Function.Periodic f (↑n * p)
```

Read back by a model that saw only the Lean. For a function f from the real numbers to the real numbers and a real number p , the following are equivalent: for every real number x , $f(x + p) = f(x)$; and for every natural number n , the real number n multiplied by p is a period of f , meaning that for every real number x , $f(x + n p) = f(x)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem periodic_natural_multiple_equivalence (f : ℝ → ℝ) (p : ℝ) :  
  (∀ x : ℝ, f (x + p) = f x) ↔  
    ∀ n : ℕ, Function.Periodic f ((n : ℝ) * p) := by  
  constructor  
  · intro h n  
    have hp : Function.Periodic f p := by  
      intro x  
      exact h x  
    exact hp.nat_mul n  
  · intro h x  
    simpa using (h 1 x)
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. imo_1968_p5_1__me.ms__076225ef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 076225efb862a51a

4.156 156. odd_iff_three_dvd_pow_add_one

The problem

For every natural base b congruent to 2 modulo 3 and every natural exponent k , k is odd if and only if 3 divides $b^k + 1$.

Lean statement

```
theorem odd_iff_three_dvd_pow_add_one (b : ℕ) (hb : b % 3 = 2) (k : ℕ) : Odd k ↔ 3 ∣ b ^ k + 1
```

Parent. Prove that for all natural numbers b and n , if $b \equiv 2 \pmod{3}$, then $3 \mid b^{2n+1} + 1$.

Parent in Lean

```
theorem odd_exponent_divisibility_of_base_mod_three (b n : ℕ) (hb : b % 3 = 2) : 3 ∣ b ^ (2 * n + 1) + 1
```

What it contributes. Reused the residue-derived divisor $3 \mid b+1$ and the odd-power divisibility checkpoint, while adding an inductive even-power residue obstruction for the converse.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child adds a genuine converse: from $3 \mid b^k + 1$ it must prove k is odd, requiring a new modular-periodicity argument for even powers. The parent only establishes divisibility for known odd exponents, so recalling its proof does not solve the child.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way odd-exponent divisibility result into an iff for arbitrary exponents. Its reverse direction requires a new even/odd case split and modular induction showing even powers are congruent to 1 modulo 3, which the parent proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent from one guaranteed odd-exponent divisibility instance to an iff characterizing all exponents. Its reverse direction requires a new modular argument ruling out even exponents, which the parent's odd-power factorization proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2064 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_iff_three_dvd_pow_add_one.extracted.1.1 (b : ℕ) (hb : b % 3 = 2) (k : ℕ) : Odd k ↔ 3 ∣ b ^ k + 1
```

Read back by a model that saw only the Lean. For natural numbers b and k , if b has remainder 2 upon division by 3, then k is odd if and only if 3 divides $b^k + 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem odd_iff_three_dvd_pow_add_one (b : ℕ) (hb : b % 3 = 2) (k : ℕ) : Odd k ↔ 3 ∣ b ^ k + 1 := by
  have hbase : 3 ∣ b + 1 := by
    have hdecomp : b % 3 + 3 * (b / 3) = b := Nat.mod_add_div b 3
    rw [hb] at hdecomp
    use b / 3 + 1
    omega
  have hb2 : b ^ 2 % 3 = 1 := by
    rw [Nat.pow_two, Nat.mul_mod, hb]
  have h_even_pow : ∀ n : ℕ, b ^ (2 * n) % 3 = 1 := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      have hexp : 2 * Nat.succ n = 2 * n + 2 := by omega
      calc
        b ^ (2 * Nat.succ n) % 3 = (b ^ (2 * n) * b ^ 2) % 3 := by rw [hexp, pow_add]
        _ = ((b ^ (2 * n) % 3) * (b ^ 2 % 3)) % 3 := by rw [Nat.mul_mod]
        _ = 1 := by rw [ih, hb2]
  constructor
  · intro hk
    have hpow : b + 1 ∣ b ^ k + 1 ^ k := by
      exact Odd.nat_add_dvd_pow_add_pow b 1 hk
    have hpow2 : b + 1 ∣ b ^ k + 1 := by simp using hpow
    exact dvd_trans hbase hpow2
  · intro hdiv
    rcases even_or_odd k with he | ho
    · ex falso
      obtain ⟨n, hn⟩ := he
      have hkform : k = 2 * n := by omega
      have hzero : (b ^ k + 1) % 3 = 0 := Nat.dvd_iff_mod_eq_zero.mp hdiv
      have hbad : (b ^ (2 * n) + 1) % 3 = 2 := by
        rw [Nat.add_mod, h_even_pow]
      rw [hkform] at hzero
      omega
    · exact ho
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2toodnp1__me.mh__c1bdceed

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c1bdceed49f09898

4.157 157. fourth_periodic_power_divisibility_iff_base_congruent_four

The problem

For natural numbers b and n , prove that $5 \mid b^{4n+1} + 1$ if and only if b leaves remainder 4 upon division by 5.

Lean statement

```
theorem fourth_periodic_power_divisibility_iff_base_congruent_four (b n : ℕ) : 5 ∣ b ^ (4 * n + 1) + 1 ↔ b % 5 = 4
```

Parent. For natural numbers b and n , prove that $3 \mid b^{2n+1} + 1$ if and only if b leaves remainder 2 upon division by 3.

Parent in Lean

```
theorem odd_power_divisibility_iff_base_congruent_two (b n : ℕ) : 3 ∣ b ^ (2 * n + 1) + 1 ↔ b % 3 = 2
```

What it contributes. Preserves the parent's residue-class divisibility strategy and the reverse-direction factorization through $b+1$, while changing the modulus and exponent periodicity.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the modulus and the periodic exponent, requiring a five-way residue split and new periodicity arguments for residues 2 and 3 modulo 5. The parent's modulo-3 proof does not supply these cases, so this is substantive parameter-shift reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the modulus and periodic exponent, requiring new residue classes modulo 5 and inductive proofs that residues 2 and 3 retain their values under exponent shifts by 4. This reasoning is not supplied by the parent's modulo-3 argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Changing the modulus from 3 to 5 and the exponent pattern from odd to $4n+1$ requires new residue-cycle reasoning, including induction for powers of 2 and 3 modulo 5. The parent's mod-3 case split and divisibility argument do not supply this proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2479 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fourth_periodic_power_divisibility_iff_base_congruent_four.extracted_1_1 (b n : ℕ) :  
  5 ∣ b ^ (4 * n + 1) + 1 ↔ b % 5 = 4
```

Read back by a model that saw only the Lean. *For natural numbers b and n , 5 divides b raised to the power $4n + 1$ plus 1 if and only if the remainder when b is divided by 5 is 4.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem fourth_periodic_power_divisibility_iff_base_congruent_four (b n : ℕ) : 5 ∣ b ^ (4 * n + 1) + 1 ↔ b % 5 = 4 := by
  have hepos : 0 < 4 * n + 1 := by omega
  have hpow2 : ∀ k : ℕ, (2 : ℕ) ^ (4 * k + 1) % 5 = 2 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 4 * (Nat.succ k) + 1 = (4 * k + 1) + 4 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  have hpow3 : ∀ k : ℕ, (3 : ℕ) ^ (4 * k + 1) % 5 = 3 := by
    intro k
    induction k with
    | zero => norm_num
    | succ k ih =>
      rw [show 4 * (Nat.succ k) + 1 = (4 * k + 1) + 4 by omega, pow_add]
      norm_num [Nat.mul_mod, ih]
  constructor
  · intro hd
    have hzero : (b ^ (4 * n + 1) + 1) % 5 = 0 :=
      Nat.dvd_iff_mod_eq_zero.mp hd
    have hlt : b % 5 < 5 := Nat.mod_lt b (by norm_num)
    have hcases : b % 5 = 0 ∨ b % 5 = 1 ∨ b % 5 = 2 ∨ b % 5 = 3 ∨ b % 5 = 4 := by omega
    rcases hcases with hres | hres | hres | hres | hres
    · have hbmod : Nat.ModEq 5 b 0 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm0 : b ^ (4 * n + 1) % 5 = 0 := by
        simpa [Nat.ModEq] using hp
      rw [Nat.add_mod, hpm0] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 5 b 1 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm1 : b ^ (4 * n + 1) % 5 = 1 := by
        simpa [Nat.ModEq] using hp
      rw [Nat.add_mod, hpm1] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 5 b 2 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm2 : b ^ (4 * n + 1) % 5 = 2 := by
        calc
          b ^ (4 * n + 1) % 5 = (2 : ℕ) ^ (4 * n + 1) % 5 := by
            simpa [Nat.ModEq] using hp
          _ = 2 := hpow2 n
      rw [Nat.add_mod, hpm2] at hzero
      norm_num at hzero
    · have hbmod : Nat.ModEq 5 b 3 := by simpa [Nat.ModEq] using hres
      have hp := hbmod.pow (4 * n + 1)
      have hpm3 : b ^ (4 * n + 1) % 5 = 3 := by
        calc
          b ^ (4 * n + 1) % 5 = (3 : ℕ) ^ (4 * n + 1) % 5 := by
            simpa [Nat.ModEq] using hp
          _ = 3 := hpow3 n
      rw [Nat.add_mod, hpm3] at hzero
      norm_num at hzero
    · exact hres
  · intro hres
```



```

have hbase : 5 ∣ b + 1 := by
  refine ⟨b / 5 + 1, ?_⟩
  have hdecomp := Nat.mod_add_div b 5
  omega
have hodd : Odd (4 * n + 1) := by
  exact ⟨2 * n, by omega⟩
have hpow : b + 1 ∣ b ^ (4 * n + 1) + 1 ^ (4 * n + 1) := by
  exact Odd.nat_add_dvd_pow_add_pow b 1 hodd
have hpow' : b + 1 ∣ b ^ (4 * n + 1) + 1 := by simpa using hpow
exact dvd_trans hbase hpow'

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. induction_divisibility_3div2tooddp1__me.mh__e6074f46

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e6074f46c14b4bc1

4.158 158. factorial_polynomial_bound

The problem

Prove that for every natural number n with $4 \leq n$, we have $n^2 + n \leq n!$.

Lean statement

```
theorem factorial_polynomial_bound (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 + n ≤ n ! := by
```

Parent. Show that for any integer $n \geq 4$, we have $n^2 \leq n!$.

Parent in Lean

```
theorem induction_ineq_nsqlefactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the conclusion to $n^2 + n \leq n!$, which is not derivable from the parent's bound. Its proof requires a new factorial-growth argument via $(n-2)!$ and does not reuse the parent's closing lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the bound from n^2 to $n^2 + n$ and requires the new estimate $n+1 \leq (n-1)!$, proved through a separate factorial recurrence and arithmetic argument. The parent's proof only establishes $n \leq (n-1)!$ and cannot supply this stronger obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the bound by proving the additional linear term, requiring a new factorial-growth argument involving $(n-2)!$ and nonlinear arithmetic. The parent's proof only establishes $n^2 \leq n!$ and cannot supply this stronger obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2060 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem factorial_polynomial_bound.extracted_1_1 (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 + n ≤ n !
```

Read back by a model that saw only the Lean. For every natural number n satisfying $4 \leq n$, the sum $n^2 + n$ is at most $n!$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem factorial_polynomial_bound (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 + n ≤ n ! := by
  have hpred : n - 2 ≤ (n - 2)! := by
    exact Nat.self_le_factorial (n - 2)
  have hmul : (n - 1) * (n - 2) ≤ (n - 1) * (n - 2)! := by
    exact Nat.mul_le_mul_left (n - 1) hpred
  have hsub1 : n - 1 = n - 2 + 1 := by
    omega
  have hsub2 : n = n - 2 + 2 := by
    omega
  have hgrowth : n + 1 ≤ (n - 1)! := by
    have hrec : (n - 1) * (n - 2)! = (n - 1)! := by
      exact Nat.mul_factorial_pred (by omega)
    rw [← hrec]
    nlinarith [hmul]
  have hfact : n * (n - 1)! = n ! := by
    exact Nat.mul_factorial_pred (by omega)
  calc
    n ^ 2 + n = n * (n + 1) := by ring
    _ ≤ n * (n - 1)! := Nat.mul_le_mul_left n hgrowth
    _ = n ! := hfact
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_ineq_nsqlefactn__mh__aa61f93c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint aa61f93c981b6696

4.159 159. sq_le_factorial_iff

The problem

For which natural numbers n does $n^2 \leq n!$ hold? Show that it holds exactly when $n = 0$, $n = 1$, or $n \geq 4$.

Lean statement

```
theorem sq_le_factorial_iff (n : ℕ) : n ^ 2 ≤ Nat.factorial n ↔ n = 0 ∨ n = 1 ∨ 4 ≤ n
```

Parent. Show that for any integer $n \geq 4$, we have $n^2 \leq n!$.

Parent in Lean

```
theorem induction_ineq_nsqlefactn (n : ℕ) (h₀ : 4 ≤ n) : n ^ 2 ≤ n ! := by
```

What it contributes. Retained the parent's factorial-growth multiplication checkpoint as the induction step, while extending it to an all-natural-number classification with explicit exceptional cases.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child gives a complete iff characterization, requiring separate exceptional-case analysis for $n=0,1,2,3$ and an induction proving the inequality for all $n \geq 4$. This is substantive new reasoning beyond the parent's one-direction threshold argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child gives a complete iff characterization, requiring explicit exclusion of $n=2,3$ and a separate induction proving the bound for every $n \geq 4$. This substantially exceeds the parent's one-way threshold argument and cannot be solved by recalling it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child gives a complete iff characterization, including explicit exceptional cases $n = 0$ and $n = 1$ and exclusion of $n = 2,3$. Proving the forward cases and the all- $n \geq 4$ direction requires case analysis and induction beyond the parent's one-way threshold argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2714 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sq_le_factorial_iff.extracted_1_1 (n : ℕ) : n ^ 2 ≤ n ! ↔ n = 0 ∨ n = 1 ∨ 4 ≤ n
```

Read back by a model that saw only the Lean. For every natural number n , n squared is less than or equal to n factorial if and only if n equals 0, n equals 1, or n is at least 4.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sq_le_factorial_iff (n : ℕ) : n ^ 2 ≤ Nat.factorial n ↔ n = 0 ∨ n = 1 ∨ 4 ≤ n := by
  have htail : ∀ k : ℕ, 4 ≤ k → k ^ 2 ≤ Nat.factorial k := by
    intro k
    induction k with
    | zero =>
      intro hk
      omega
    | succ k ih =>
      intro hk
      by_cases hprev : 4 ≤ k
      · have hprevineq : k ^ 2 ≤ Nat.factorial k := ih hprev
        have hstep : (Nat.succ k) ^ 2 ≤ Nat.succ k * k ^ 2 := by
          nlinarith
        have hmul : Nat.succ k * k ^ 2 ≤ Nat.succ k * Nat.factorial k :=
          Nat.mul_le_mul_left (Nat.succ k) hprevineq
        calc
          (Nat.succ k) ^ 2 ≤ Nat.succ k * k ^ 2 := hstep
          _ ≤ Nat.succ k * Nat.factorial k := hmul
          _ = Nat.factorial (Nat.succ k) := by simp [Nat.factorial_succ]
      · have hk3 : k = 3 := by omega
        subst k
        norm_num
  constructor
  · intro h
    by_cases hs : n < 4
    · have hcases : n = 0 ∨ n = 1 ∨ n = 2 ∨ n = 3 := by omega
      rcases hcases with rfl | rfl | rfl | rfl
      · left
        rfl
      · exact Or.inr (Or.inl rfl)
      · norm_num at h
      · norm_num at h
    · exact Or.inr (Or.inr (by omega))
  · intro h
    rcases h with rfl | rfl | hn
    · norm_num
    · norm_num
    · exact htail n hn
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. induction_ineq_nsqulefactn__mh__c95b5f83

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c95b5f83df9dc725

4.160 160. quadratic_strict_sublevel_radius

The problem

For every positive radius and every real number x , the strict quadratic sublevel condition $x^2 - 5x - 4 < r^2 - 41/4$ holds exactly when x lies in the centered open interval $(5/2 - r, 5/2 + r)$.

Lean statement

```
theorem quadratic_strict_sublevel_radius (r x : ℝ) (hr : 0 < r) : x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔ x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r)
```

Parent. For every nonnegative radius at most 10, the quadratic sublevel set is exactly the centered interval determined by that radius.

Parent in Lean

```
theorem quadratic_sublevel_radius (r x : ℝ) (hr₀ : 0 ≤ r) (hr₁₀ : r ≤ 10) : x ^ 2 - 5 * x - 4 ≤ r ^ 2 - 41 / 4 ↔ x ∈ Set.Icc (5 / 2 - r) (5 / 2 + r)
```

What it contributes. Reused the parent's centered quadratic normalization and factor-product interval argument, strengthened from non-strict closed-radius bounds to strict positivity and open endpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the inequality and the interval from non-strict to strict, requiring boundary exclusion and strictly positive factor products. Its reverse direction also needs strict positivity from hr , so the parent's nonnegative-product argument does not directly supply the proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the non-strict sublevel characterization into a strict one, requiring endpoint exclusion and strictly positive factor products; the parent's nonnegative-product argument does not supply this. The positive-radius hypothesis is used to establish strict inequalities and the open interval conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the inequality and interval from closed to strict, requiring new boundary-exclusion reasoning and strict positivity arguments from hr . The parent's non-strict sublevel proof does not by itself establish this open-interval characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1778 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_strict_sublevel_radius.extracted_1_1 (r x : ℝ) (hr : 0 < r) :  
  x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔ x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r)
```

Read back by a model that saw only the Lean. For real numbers r and x , if $0 < r$, then $x^2 - 5x - 4 < r^2 - 41/4$ if and only if x lies in the open interval $(5/2 - r, 5/2 + r)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_strict_sublevel_radius (r x : ℝ) (hr : 0 < r) :  
  x ^ 2 - 5 * x - 4 < r ^ 2 - 41 / 4 ↔  
  x ∈ Set.Ioo (5 / 2 - r) (5 / 2 + r) := by  
  have hcompleted : x ^ 2 - 5 * x - 4 = (x - 5 / 2) ^ 2 - 41 / 4 := by  
    ring  
  have hfactor :  
    (x - 5 / 2 + r) * (x - 5 / 2 - r) =  
    (x - 5 / 2) ^ 2 - r ^ 2 := by  
    ring  
  constructor  
  · intro h  
    constructor  
    · by_contra hx  
      have hx' : x ≤ 5 / 2 - r := le_of_not_gt hx  
      have h₁ : x - 5 / 2 + r ≤ 0 := by linarith  
      have h₂ : x - 5 / 2 - r ≤ 0 := by linarith  
      have hp : 0 ≤ (x - 5 / 2 + r) * (x - 5 / 2 - r) :=  
        mul_nonneg_of_nonpos_of_nonpos h₁ h₂  
      linarith [hcompleted, hfactor]  
    · by_contra hx  
      have hx' : 5 / 2 + r ≤ x := le_of_not_gt hx  
      have h₁ : 0 ≤ x - 5 / 2 - r := by linarith  
      have h₂ : 0 ≤ x - 5 / 2 + r := by linarith  
      have hp : 0 ≤ (x - 5 / 2 - r) * (x - 5 / 2 + r) :=  
        mul_nonneg h₁ h₂  
      linarith [hcompleted, hfactor]  
  · rintro ⟨hleft, hright⟩  
    have h₁ : 0 < r - (x - 5 / 2) := by linarith  
    have h₂ : 0 < r + (x - 5 / 2) := by linarith  
    have hp : 0 < (r - (x - 5 / 2)) * (r + (x - 5 / 2)) :=  
      mul_pos h₁ h₂  
    have hfactor' :  
      (r - (x - 5 / 2)) * (r + (x - 5 / 2)) =  
      r ^ 2 - (x - 5 / 2) ^ 2 := by  
      ring  
    linarith [hcompleted, hfactor']
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_101__me.mh__f403dcac

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 67bb649ddbcaf76c

4.161 161. quadratic_interval_characterization

The problem

For real numbers a , b , and x with $a \leq b$, show that $(x - a)(x - b) \leq 0$ if and only if $x \in [a, b]$.

Lean statement

```
theorem quadratic_interval_characterization (a b x : ℝ) (hab : a ≤ b) : ((x - a) * (x - b) ≤ 0) ↔ x ∈ Set.Icc a b
```

Parent. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed quadratic bound to arbitrary endpoints and proves an iff characterization of the entire interval. It additionally requires the converse implication from interval membership to the product inequality, which the parent proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the specific quadratic bound to an iff characterization for arbitrary endpoints and requires proving both directions, including deriving interval membership from the product inequality. The parent's one-way contradiction proof does not supply the reverse implication or the symbolic endpoint reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely general equivalence characterizing an interval by the sign of a quadratic, requiring both-direction reasoning for arbitrary endpoints and a converse product-sign argument. The parent only establishes one specific inequality-to-interval implication and cannot supply this universal characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1775 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_interval_characterization.extracted_1_1 (a b x : ℝ) (hab : a ≤ b) :  
  (x - a) * (x - b) ≤ 0 ↔ x ∈ Set.Icc a b
```

Read back by a model that saw only the Lean. For real numbers a , b , and x , if $a \leq b$, then $(x - a)(x - b) \leq 0$ if and only if x lies in the closed interval $[a, b]$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_interval_characterization (a b x : ℝ) (hab : a ≤ b) :
  ((x - a) * (x - b) ≤ 0) ↔ x ∈ Set.Icc a b := by
  constructor
  · intro h
    constructor
    · by_contra hxa
      have hxa' : x < a := lt_of_not_ge hxa
      have hxb' : x < b := by linarith
      have hprod : 0 < (x - a) * (x - b) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      linarith
    · by_contra hxb
      have hxb' : b < x := lt_of_not_ge hxb
      have hxa' : a < x := by linarith
      have hprod : 0 < (x - a) * (x - b) :=
        mul_pos (by linarith) (by linarith)
      linarith
  · intro hx
    rcases hx with ⟨hxa, hxb⟩
    have hleft : 0 ≤ x - a := sub_nonneg.mpr hxa
    have hright : x - b ≤ 0 := sub_nonpos.mpr hxb
    have hprod : (x - a) * (x - b) ≤ 0 :=
      mul_nonpos_of_nonneg_of_nonpos hleft hright
    exact hprod
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_101__mh__4030f0e5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint e678c3b76bd1bd90

4.162 162. interval_from_factored_quadratic

The problem

Let $a \leq b$. Show that every real number x satisfying $(x - a)(x - b) \leq 0$ lies in the interval $[a, b]$.

Lean statement

```
theorem interval_from_factored_quadratic (a b x : ℝ) (hab : a ≤ b) (h : x ^ 2 - (a + b) * x + a * b ≤ 0) : x ∈ Set.Icc a b
```

Parent. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the two fixed endpoints and requires proving the symbolic factorization into $(x - a)(x - b)$, then using the arbitrary ordering $a \leq b$. The parent's fixed-constant sign argument cannot be recalled directly to establish this general theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed quadratic to arbitrary ordered roots a and b , requiring symbolic factorization and a parameterized sign argument. The parent's proof for the specific endpoints -2 and 7 does not directly supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed interval and quadratic to arbitrary endpoints $a \leq b$. It requires establishing the symbolic factorization and using endpoint ordering, reasoning not supplied by the parent's fixed-constant proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2467 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem interval_from_factored_quadratic.extracted_1_1 (a b x : ℝ) (hab : a ≤ b) (h : x ^ 2 - (a + b) * x + a * b ≤ 0) :  
  x ∈ Set.Icc a b
```

Read back by a model that saw only the Lean. For real numbers a , b , and x , if $a \leq b$ and $x^2 - (a + b)x + ab \leq 0$, then x belongs to the closed interval $[a, b]$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem interval_from_factored_quadratic (a b x : ℝ) (hab : a ≤ b) (h : x ^ 2 - (a + b) * x + a * b ≤ 0) : x ∈ Set.Icc a b := by
  have hfactor : x ^ 2 - (a + b) * x + a * b = (x - a) * (x - b) := by
    ring
  constructor
  · by_contra hx
    have hx' : x < a := lt_of_not_ge hx
    have hleft : x - a < 0 := by linarith
    have hright : x - b < 0 := by linarith
    have hprod : 0 < (x - a) * (x - b) :=
      mul_pos_of_neg_of_neg hleft hright
    nlinarith [h, hfactor]
  · by_contra hx
    have hx' : b < x := lt_of_not_ge hx
    have hleft : 0 < x - a := by linarith
    have hright : 0 < x - b := by linarith
    have hprod : 0 < (x - a) * (x - b) :=
      mul_pos hleft hright
    nlinarith [h, hfactor]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_101__mh__cd27eeb2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cd27eeb20e339bda

4.163 163. shifted_square_value_eq_twenty_nine_iff

The problem

For the shifted-square quadratic $f(x) = x^2 - 6x + 13$, prove that for every real number x satisfying $x^2 - 5x - 4 \leq 10$, $f(x) = 29$ if and only if $x = -2$.

Lean statement

```
theorem shifted_square_value_eq_twenty_nine_iff
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  ∀ x : ℝ, x ^ 2 - 5 * x - 4 ≤ 10 → (f x = 29 ↔ x = -2)
```

Parent. The shifted-square quadratic attains its maximum value 29 on the domain defined by $x^2 - 5x - 4 \leq 10$.

Parent in Lean

```
theorem shifted_square_isGreatest_on_sublevel
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsGreatest (f '' {x : ℝ | x ^ 2 - 5 * x - 4 ≤ 10}) 29 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a greatest-value statement into an equality characterization: it must factor equality at 29, analyze both roots, and use the feasible interval to exclude $x = 8$. The parent's upper-bound argument alone does not supply this iff proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a greatest-value claim into a uniqueness characterization. Proving equality forces a factorization and a zero-product case split, with the interval hypothesis needed to exclude the extraneous root $x = 8$; this reasoning is not supplied by the parent theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a greatest-value claim to a uniqueness characterization. Proving equality forces $(x + 2) * (x - 8) = 0$ and then uses the domain bound to exclude $x = 8$, reasoning not supplied by the parent's upper-bound proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1782 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_square_value_eq_twenty_nine_iff.extracted_1_1 : ∀ (f : ℝ → ℝ),  
  (f = fun x => x ^ 2 - 6 * x + 13) → ∀ (x : ℝ), x ^ 2 - 5 * x - 4 ≤ 10 → (f x = 29 ↔ x = -2)
```

Read back by a model that saw only the Lean. For every function f from the real numbers to the real numbers, if $f(x) = x^2 - 6x + 13$ for every real number x , then for every real number x satisfying $x^2 - 5x - 4 \leq 10$, $f(x) = 29$ if and only if $x = -2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem shifted_square_value_eq_twenty_nine_iff
  (f : ℝ → ℝ)
  (h₀ : f = fun x => x ^ 2 - 6 * x + 13) :
  ∀ x : ℝ, x ^ 2 - 5 * x - 4 ≤ 10 → (f x = 29 ↔ x = -2) := by
  have hformula : ∀ z : ℝ, f z = z ^ 2 - 6 * z + 13 := by
    intro z
    rw [h₀]
  have hcert : ∀ z : ℝ, z ^ 2 - 5 * z - 4 ≤ 10 → z ∈ Set.Icc (-2) 7 := by
    intro z hz
    constructor
    · by_contra h
      have hz' : z < -2 := lt_of_not_ge h
      have hp : 0 < (z + 2) * (z - 7) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      nlinarith
    · by_contra h
      have hz' : 7 < z := lt_of_not_ge h
      have hp : 0 < (z + 2) * (z - 7) :=
        mul_pos (by linarith) (by linarith)
      nlinarith
  intro x hx
  constructor
  · intro hvalue
    have hinterval := hcert x hx
    have hzero : (x + 2) * (x - 8) = 0 := by
      rw [hformula x] at hvalue
      nlinarith
    rcases mul_eq_zero.mp hzero with hleft | hright
    · linarith
    · linarith [hinterval.2]
  · intro hx2
    subst x
    rw [hformula (-2)]
    norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_101__x__mathd_algebra_410__xe.mh__9ef0003d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 418cae2b9cdfd644

4.164 164. circle_unique_y_coordinate

The problem

For which real values of x does the equation $x^2 + 8x + y^2 - 6y = 0$ determine a unique real value of y ? Show that these values are $x = -9$ and $x = 1$.

Lean statement

```
theorem circle_unique_y_coordinate (x : ℝ) :
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1
```

Parent. Which real x -coordinates occur on the circle $x^2 + 8x + y^2 - 6y = 0$? Show that they are exactly the numbers in $[-9, 1]$.

Parent in Lean

```
theorem circle_x_projection (x : ℝ) :
  (∃ y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔
  (-9 ≤ x ∧ x ≤ 1)
```

What it contributes. Reuses the parent circle's completed-square structure and endpoint factorization, but changes the target to an exact uniqueness characterization via reflection symmetry.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child requires a genuinely new uniqueness argument: it reflects any solution across $y=3$, uses uniqueness to force $y=3$, and then characterizes the boundary cases. The parent only establishes existence over the full interval and does not supply this reflection-and-uniqueness reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a circle point into an exact uniqueness characterization. Its proof requires a reflection argument showing uniqueness forces $y = 3$, followed by endpoint factorization, which is not supplied by the parent's projection proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a point on the circle into a uniqueness characterization. Its reflection argument shows that uniqueness forces $y=3$, followed by a zero-product endpoint case split, reasoning not supplied by the parent's projection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2634 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem circle_unique_y_coordinate.extracted_1_1 (x : ℝ) :  
  (∃! y, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1
```

Read back by a model that saw only the Lean. For every real number x , there exists exactly one real number y such that $x^2 + 8x + y^2 - 6y = 0$ if and only if $x = -9$ or $x = 1$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem circle_unique_y_coordinate (x : ℝ) :  
  (∃! y : ℝ, x ^ 2 + 8 * x + y ^ 2 - 6 * y = 0) ↔ x = -9 ∨ x = 1 := by  
  constructor  
  · rintro ⟨y, hy, hyuniq⟩  
    have hy_reflected : x ^ 2 + 8 * x + (6 - y) ^ 2 - 6 * (6 - y) = 0 := by  
      nlinarith [hy]  
    have heq : 6 - y = y := hyuniq (6 - y) hy_reflected  
    have hy3 : y = 3 := by  
      linarith  
    have hcircle : (x + 4) ^ 2 = 25 := by  
      nlinarith [hy, hy3]  
    have hprod : (x + 9) * (x - 1) = 0 := by  
      nlinarith [hcircle]  
    rcases mul_eq_zero.mp hprod with hleft | hright  
    · left  
      linarith  
    · right  
      linarith  
  · intro hx  
    rcases hx with rfl | rfl  
    · refine ⟨3, by norm_num, ?_⟩  
      intro y hy  
      nlinarith [hy]  
    · refine ⟨-9, by norm_num, ?_⟩  
      intro y hy  
      nlinarith [hy]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_107__me.mh__958b7837

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 958b783736d1ea6e

4.165 165. quadratic_sublevel_characterization

The problem

Let $a \leq v \leq b$, $k > 0$, and $\delta \geq 0$. Show that on $[a, b]$, the points where $k(x - v)^2 + c \leq c + k\delta^2$ are exactly those satisfying $|x - v| \leq \delta$.

Lean statement

```
theorem quadratic_sublevel_characterization
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  :
  ∀ x ∈ Set.Icc a b,
    k * (x - v)^2 + c ≤ c + k * δ^2 ↔ |x - v| ≤ δ
```

Parent. Let $a \leq v \leq b$, $k > 0$, and $\delta \geq 0$. For $f(x) = k(x - v)^2 + c$ on $[a, b]$, prove that v is the unique minimizer and that every point at distance at least δ from v has value at least $c + k\delta^2$.

Parent in Lean

```
theorem scaled_translated_square_gap
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => k * (x - v)^2 + c) (Set.Icc a b) v ∧
    (fun x : ℝ => k * (x - v)^2 + c) v = c ∧
    (∀ x ∈ Set.Icc a b, δ ≤ |x - v| →
      c + k * δ^2 ≤ k * (x - v)^2 + c) ∧
    (∀ x ∈ Set.Icc a b, k * (x - v)^2 + c = c → x = v)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from one-sided minimum/gap facts to an exact sublevel-set characterization, requiring a converse argument using absolute-value squares and positivity of k . The parent's minimization and equality-uniqueness proof does not supply this equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-sided lower-gap statement into an iff characterization of the entire quadratic sublevel set. Its proof requires establishing the reverse inequality and relating squared absolute values, reasoning not supplied by the parent's minimum and uniqueness arguments.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-sided lower-bound claim into an iff characterization of the entire quadratic sublevel set. Its converse requires absolute-value squaring and a nonnegative-factor product argument that the parent's minimum and gap proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2448 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_sublevel_characterization.extracted_1_1 : ∀ (a b v c k δ : ℝ),
  0 < k → 0 ≤ δ → a ≤ v → v ≤ b → ∀ x ∈ Set.Icc a b, k * (x - v) ^ 2 + c ≤ c + k * δ ^ 2 ↔ |x - v| ≤ δ
```

Read back by a model that saw only the Lean. For all real numbers a, b, v, c, k , and δ , if $k > 0$, $\delta \geq 0$, $a \leq v$, and $v \leq b$, then for every real number x in the closed interval $[a, b]$, the inequality $k(x-v)^2 + c \leq c + k\delta^2$ holds if and only if $|x-v| \leq \delta$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem quadratic_sublevel_characterization
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  :
  ∀ x ∈ Set.Icc a b,
    k * (x - v) ^ 2 + c ≤ c + k * δ ^ 2 ↔ |x - v| ≤ δ := by
  intro x hx
  have habs_sq : |x - v| ^ 2 = (x - v) ^ 2 := by
    by_cases h : 0 ≤ x - v
    · rw [abs_of_nonneg h]
    · have h' : x - v ≤ 0 := le_of_not_ge h
      rw [abs_of_nonpos h']
      ring
  constructor
  · intro hvalue
    have hsq : (x - v) ^ 2 ≤ δ ^ 2 := by
      nlinarith
    have habs_nonneg : 0 ≤ |x - v| := abs_nonneg (x - v)
    nlinarith [habs_sq]
  · intro hdist
    have hplus : 0 ≤ δ + |x - v| := by
      nlinarith [abs_nonneg (x - v)]
    have hprod : 0 ≤ (δ - |x - v|) * (δ + |x - v|) := by
      exact mul_nonneg (sub_nonneg.mpr hdist) hplus
    have habs_sq_le : |x - v| ^ 2 ≤ δ ^ 2 := by
      nlinarith [hprod]
    have hsq : (x - v) ^ 2 ≤ δ ^ 2 := by
      nlinarith [habs_sq, habs_sq_le]
    nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.me.mh__5c254eae

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 53fcdf72f7fb3b88

4.166 166. equal_values_scaled_translated_quadratic

The problem

Let $a \leq v \leq b$ and $k > 0$. If $x, y \in [a, b]$ satisfy $k(x - v)^2 + c = k(y - v)^2 + c$, prove that either $x = y$ or $x + y = 2v$.

Lean statement

```
theorem equal_values_scaled_translated_quadratic
  (a b v c k x y : ℝ)
  (hk : 0 < k)

  (heq : k * (x - v)^2 + c = k * (y - v)^2 + c) :
  x = y ∨ x + y = 2 * v
```

Parent. Let $a \leq v \leq b$, $k > 0$, and $\delta \geq 0$. For $f(x) = k(x - v)^2 + c$ on $[a, b]$, prove that v is the unique minimizer and that every point at distance at least δ from v has value at least $c + k\delta^2$.

Parent in Lean

```
theorem scaled_translated_square_gap
  (a b v c k δ : ℝ)
  (hk : 0 < k)
  (hδ : 0 ≤ δ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => k * (x - v)^2 + c) (Set.Icc a b) v ∧
  (fun x : ℝ => k * (x - v)^2 + c) v = c ∧
  (∀ x ∈ Set.Icc a b, δ ≤ |x - v| →
    c + k * δ^2 ≤ k * (x - v)^2 + c) ∧
  (∀ x ∈ Set.Icc a b, k * (x - v)^2 + c = c → x = v)
```

What it contributes. Retains the parent's positive scaled translated quadratic and vertex interval assumptions, while changing the conclusion to a two-point level-set characterization proved through factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the parent's minimum-at-the-vertex claim into a global characterization of all equal quadratic values. Its proof requires factoring a difference of squares and using positivity of the scale, reasoning not supplied by the parent's minimum argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from minimum and equality-at-the-vertex facts to a global characterization of equal quadratic values, requiring difference-of-squares factorization and a new equality case split. The parent's proof does not supply this reflection argument, and the child is not a repeat of the retained midpoint or sublevel results.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from minimum and uniqueness at the vertex to a global characterization of equal quadratic values. Its proof requires factoring the difference of squares and using positivity of k to derive equality or reflection, which the parent's minimum argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2467 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem equal_values_scaled_translated_quadratic.extracted_1_1 (a b v c k x y : ℝ) (hk : 0 < k) (hav : a ≤ v)
(hvb : v ≤ b) (hx : x ∈ Set.Icc a b) (hy : y ∈ Set.Icc a b) (heq : k * (x - v) ^ 2 + c = k * (y - v) ^ 2 + c) :
x = y ∨ x + y = 2 * v
```

Read back by a model that saw only the Lean. For real numbers a, b, v, c, k, x , and y , suppose that $k > 0$, $a \leq v$, $v \leq b$, x and y both lie in the closed interval $[a, b]$, and $k(x - v)^2 + c = k(y - v)^2 + c$. Then either $x = y$ or $x + y = 2v$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem equal_values_scaled_translated_quadratic
(a b v c k x y : ℝ)
(hk : 0 < k)

(hk : k * (x - v)^2 + c = k * (y - v)^2 + c) :
x = y ∨ x + y = 2 * v := by
have hdiff : k * ((x - v)^2 - (y - v)^2) = 0 := by
  nlinarith [hk]
have hfactor : k * ((x - y) * (x + y - 2 * v)) = 0 := by
  calc
    k * ((x - y) * (x + y - 2 * v)) = k * ((x - v)^2 - (y - v)^2) := by ring
    _ = 0 := hdiff
have hprod : (x - y) * (x + y - 2 * v) = 0 := by
  exact (mul_eq_zero.mp hfactor).resolve_left (ne_of_gt hk)
rcases mul_eq_zero.mp hprod with hxy | href
· left
  linarith
· right
  linarith
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.me.mh__efc5a618

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e84ca08787f4be0a

4.167 167. centered_quadratic_strict_growth_characterization

The problem

For any real numbers a and d , if $f(x) = a(x - 7)^2 + d$, then for every real number x , $f(x) > f(7)$ exactly when $a > 0$ and $x \neq 7$.

Lean statement

```
theorem centered_quadratic_strict_growth_characterization
  (a d : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x > f 7 ↔ 0 < a ∧ x ≠ 7
```

Parent. For real numbers a and d with $a \neq 0$, let $f(x) = a(x - 7)^2 + d$. Then $f(x) = f(7)$ holds exactly when $x = 7$.

Parent in Lean

```
theorem centered_quadratic_equality_characterization
  (a d : ℝ)
  (ha : a ≠ 0)
  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x = f 7 ↔ x = 7
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child strengthens the semantic characterization from equality at the vertex to a strict-inequality classification that also determines the sign of the coefficient, while dropping the nonzero hypothesis. Its proof requires new positivity and square-nonzero reasoning that the parent's equality proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes equality characterization into a strict-inequality characterization, proving both the necessary positivity of a and the exclusion of the vertex. This requires sign reasoning and does not follow from the parent's nonzero-coefficient argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes equality at the vertex into a strict-inequality classification and removes the nonzero-coefficient hypothesis. Proving the result requires a new sign analysis showing positivity exactly when $a > 0$ and $x \neq 7$, which the parent's proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2058 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem centered_quadratic_strict_growth_characterization.extracted_1_1 : ∀ (a d : ℝ) (f : ℝ → ℝ),
  (f = fun x => a * (x - 7) ^ 2 + d) → ∀ (x : ℝ), f x > f 7 ↔ 0 < a ∧ x ≠ 7
```

Read back by a model that saw only the Lean. For every real numbers a and d and every function f from $\mathbb{R}$ to $\mathbb{R}$, if f is the function defined by $f(x) = a(x - 7)^2 + d$ for every real x , then for every real x , $f(x) > f(7)$ if and only if $a > 0$ and $x \neq 7$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem centered_quadratic_strict_growth_characterization
  (a d : ℝ)

  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x > f 7 ↔ 0 < a ∧ x ≠ 7 := by
  have hfactor : ∀ x : ℝ, f x - f 7 = a * (x - 7)^2 := by
    intro x
    rw [hf]
    ring
  intro x
  constructor
  · intro h
    have hprodpos : 0 < a * (x - 7)^2 := by
      nlinarith [hfactor x]
    have hsqnonneg : 0 ≤ (x - 7)^2 := sq_nonneg (x - 7)
    have hapos : 0 < a := by
      by_contra hnot
      have hanonpos : a ≤ 0 := le_of_not_gt hnot
      have hprodnonpos : a * (x - 7)^2 ≤ 0 :=
        mul_nonpos_of_nonpos_of_nonneg hanonpos hsqnonneg
      linarith
    have hxne : x ≠ 7 := by
      intro hx
      subst x
      exact (lt_irrefl (f 7)) h
    exact ⟨hapos, hxne⟩
  · rintro ⟨hapos, hxne⟩
    have hne : x - 7 ≠ 0 := sub_ne_zero.mpr hxne
    have hsqpos : 0 < (x - 7)^2 := by
      have hmul : 0 < (x - 7) * (x - 7) := mul_self_pos.mpr hne
      simpa [pow_two] using hmul
    have hprodpos : 0 < a * (x - 7)^2 := mul_pos hapos hsqpos
    nlinarith [hfactor x]
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.mh.mh__865eb777

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 4e83bc9182afdcf7

4.168 168. centered_quadratic_equality_characterization

The problem

For real numbers a and d with $a \neq 0$, let $f(x) = a(x - 7)^2 + d$. Then $f(x) = f(7)$ holds exactly when $x = 7$.

Lean statement

```
theorem centered_quadratic_equality_characterization
  (a d : ℝ)
  (ha : a ≠ 0)
  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x = f 7 ↔ x = 7
```

Parent. For $f(x) = x^2 - 14x + 3$, the equality $f(x) = f(7)$ holds exactly when $x = 7$.

Parent in Lean

```
theorem quadratic_equality_characterization
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  ∀ x, f x = f 7 ↔ x = 7
```

What it contributes. Retains the parent's vertex-equality characterization and completed-square checkpoint, while replacing the fixed leading coefficient with an explicit nonzero parameter.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the leading coefficient and requires using the new nonzero hypothesis to infer that $a \cdot (x-7)^2 = 0$ forces $(x-7)^2 = 0$. The parent's fixed-coefficient square identity and proof do not supply this cancellation argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the leading coefficient: proving equality at the vertex forces $a \cdot (x-7)^2 = 0$, and the new hypothesis $a \neq 0$ is essential to derive $x = 7$. This reasoning is not supplied by the parent's fixed-coefficient square identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the leading coefficient and uses the new hypothesis $a \neq 0$ to infer from $a \cdot (x-7)^2 = 0$ that $x = 7$. The parent's fixed square identity and proof do not supply this cancellation reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2042 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem centered_quadratic_equality_characterization.extracted_1_1 : ∀ (a d : ℝ),
  a ≠ 0 → ∀ (f : ℝ → ℝ), (f = fun x => a * (x - 7) ^ 2 + d) → ∀ (x : ℝ), f x = f 7 ↔ x = 7
```

Read back by a model that saw only the Lean. *For every real number a and real number d , if a is nonzero, then for every function f from $\mathbb{R}$ to $\mathbb{R}$ such that $f(x) = a(x - 7)^2 + d$ for every real x , and for every real number x , $f(x) = f(7)$ if and only if $x = 7$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem centered_quadratic_equality_characterization
  (a d : ℝ)
  (ha : a ≠ 0)
  (f : ℝ → ℝ)
  (hf : f = fun x => a * (x - 7)^2 + d) :
  ∀ x, f x = f 7 ↔ x = 7 := by
  have hfactor : ∀ x : ℝ, f x - f 7 = a * (x - 7)^2 := by
    intro x
    rw [hf]
    ring
  intro x
  constructor
  · intro h
    have hprod : a * (x - 7)^2 = 0 := by
      nlinarith [hfactor x]
    have hsq : (x - 7)^2 = 0 := by
      rcases mul_eq_zero.mp hprod with ha0 | hs
      · exact (ha ha0).elim
      · exact hs
    have hxzero : x - 7 = 0 := by
      nlinarith [hsq]
    linarith
  · intro hx
    have hsame : f x = f 7 := by
      rw [hx]
    exact hsame
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh.mh__099ead40

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 099ead40ddfbbcf7

4.169 169. quadratic_unique_minimizer

The problem

What value of x will give the minimum value for $f(x) = x^2 - 14x + 3$? Show that x is a minimizer if and only if $x = 7$.

Lean statement

```
theorem quadratic_unique_minimizer
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  ∀ x, IsMinOn f Set.univ x ↔ x = 7
```

Parent. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a minimum-value claim into a uniqueness characterization: it must prove any minimizer equals 7, using both inequalities and strict quadratic equality. Although it reproves the parent's minimum as a sublemma, the parent proof alone does not establish the new uniqueness direction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum-value claim into a genuine uniqueness characterization. Proving the reverse direction requires showing any minimizer has equal value to the known minimizer and then deriving $x = 7$ via polynomial reasoning, which the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the minimum-value claim into a genuine uniqueness characterization. Beyond reproving the parent's minimum, it must compare an arbitrary minimizer with the known one and derive equality, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1738 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_unique_minimizer.extracted_1_1 : ∀ (f : ℝ → ℝ),  
  (f = fun x => x ^ 2 - 14 * x + 3) → ∀ (x : ℝ), IsMinOn f Set.univ x ↔ x = 7
```

Read back by a model that saw only the Lean. For every function f from $\mathbb{R}$ to $\mathbb{R}$, if f is equal to the function defined by $f(x) = x^2 - 14x + 3$ for all real x , then for every real number x , x is a global minimizer of f on $\mathbb{R}$ if and only if $x = 7$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_unique_minimizer  
  (f : ℝ → ℝ)  
  (hf : f = fun x => x^2 - 14 * x + 3) :  
  ∀ x, IsMinOn f Set.univ x ↔ x = 7 := by  
  have hmin : IsMinOn f Set.univ 7 := by  
    rw [hf, isMinOn_univ_iff]  
    intro x  
    nlinarith [sq_nonneg (x - 7)]  
  intro x  
  constructor  
  · intro hx  
    have hupper : f x ≤ f 7 := by  
      rw [isMinOn_univ_iff] at hx  
      exact hx 7  
    have hlower : f 7 ≤ f x := by  
      rw [isMinOn_univ_iff] at hmin  
      exact hmin x  
    rw [hf] at hupper hlower  
    nlinarith  
  · intro hx  
    subst x  
    exact hmin
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh__00ae8c07

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint df28ca854e198b0a

4.170 170. translated_square_quadratic_interval

The problem

Let $a \leq v \leq b$. For the quadratic $f(x) = (x - v)^2 + c$ on $[a, b]$, prove that v is its unique minimizer and that the minimum value is c .

Lean statement

```
theorem translated_square_quadratic_interval
  (a b v c : ℝ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => (x - v)^2 + c) (Set.Icc a b) v ∧
  (∀ x ∈ Set.Icc a b, (x - v)^2 + c = c → x = v) ∧
  (fun x : ℝ => (x - v)^2 + c) v = c
```

Parent. What value of x will give the minimum value for $x^2 - 14x + 3$? Show that it is 7.

Parent in Lean

```
theorem mathd_algebra_113
  (f : ℝ → ℝ)
  (hf : f = fun x => x^2 - 14 * x + 3) :
  IsMinOn f (Set.univ) 7 := by
```

What it contributes. Generalizes the parent's square-completion argument from a fixed global quadratic to a translated quadratic on an interval, while adding uniqueness and the explicit minimum value.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the vertex and constant, restricts the domain to an interval, and adds a uniqueness conclusion. Its proof requires establishing interval membership and showing equality of the square forces $x = v$, reasoning not supplied by the parent's minimum-only argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic and changes the conclusion to a bounded-interval minimum characterization with uniqueness and an explicit minimum value. The parent's fixed global minimization argument does not provide the symbolic translated-square reasoning or the uniqueness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent to arbitrary translated squares on a closed interval and adds a unique-minimizer characterization plus an explicit minimum value. Proving uniqueness and interval membership requires reasoning not supplied by the parent's global-minimum argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2400 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_quadratic_interval.extracted_1_1 (a b v c : ℝ) (hav : a ≤ v) (hvb : v ≤ b) :
  IsMinOn (fun x => (x - v) ^ 2 + c) (Set.Icc a b) v ∧
  (∀ x ∈ Set.Icc a b, (x - v) ^ 2 + c = c → x = v) ∧ (fun x => (x - v) ^ 2 + c) v = c
```

Read back by a model that saw only the Lean. For real numbers a, b, v , and c , if $a \leq v$ and $v \leq b$, then v is a point where the function $x \mapsto (x - v)^2 + c$ attains its minimum on the closed interval $[a, b]$; moreover, for every $x \in [a, b]$, if $(x - v)^2 + c = c$, then $x = v$; and the function's value at v is c .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_square_quadratic_interval
  (a b v c : ℝ)
  (hav : a ≤ v)
  (hvb : v ≤ b) :
  IsMinOn (fun x : ℝ => (x - v)^2 + c) (Set.Icc a b) v ∧
  (∀ x ∈ Set.Icc a b, (x - v)^2 + c = c → x = v) ∧
  (fun x : ℝ => (x - v)^2 + c) v = c := by
  have hv_mem : v ∈ Set.Icc a b := ⟨hav, hvb⟩
  have hcompletion (x : ℝ) : (x - v)^2 + c = c + (x - v)^2 := by
    ring
  constructor
  · rw [isMinOn_iff]
    intro x hx
    rw [hcompletion v, hcompletion x]
    nlinarith [sq_nonneg (x - v)]
  · constructor
    · intro x hx heq
      have hsq : (x - v)^2 = 0 := by
        nlinarith [heq]
      have hmul : (x - v) * (x - v) = 0 := by
        simpa [pow_two] using hsq
      have hzero : x - v = 0 := by
        rcases mul_eq_zero.mp hmul with h | h
        · exact h
        · exact h
      linarith
    · dsimp
      ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_113__mh__e0068f38

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint e0068f387c7839a9

4.171 171. weighted_square_remainder_parameterized

The problem

If x minimizes $t^2 - 2at + 3$ over the real numbers and x, y, z, w satisfy the four weighted-square equations with weights determined by 2, 4, 6, 8 and 1, 3, 5, 7, show that $y^2 + z^2 + w^2 = 36 - a^2$.

Lean statement

```
theorem weighted_square_remainder_parameterized (a x y z w : ℝ)
(hmin : IsMinOn (fun t : ℝ => t ^ 2 - 2 * a * t + 3) Set.univ x)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
y ^ 2 + z ^ 2 + w ^ 2 = 36 - a ^ 2
```

Parent. Show that no real numbers x, y, z, w satisfy the four weighted-square equations when x is a minimizer of $t^2 - 14t + 3$.

Parent in Lean

```
theorem weighted_square_infeasible_at_minimizer (x y z w : ℝ)
(hmin : IsMinOn (fun t : ℝ => t ^ 2 - 14 * t + 3) Set.univ x)
(h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
(h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
(h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
(h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
False
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the minimizer: it must derive the symbolic identity $x = a$ and retain a^2 through the final algebra, whereas the parent only substitutes the fixed value $x = 7$ to derive a contradiction. The conclusion is a nontrivial remainder identity rather than the parent's infeasibility claim.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The constant minimizer 7 is genuinely parameterized to an arbitrary minimizer a , and the contradictory endpoint in the parent is replaced by the symbolic identity $y^2 + z^2 + w^2 = 36 - a^2$. The parent's final infeasibility argument no longer applies; the proof must preserve a^2 symbolically.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2683 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem weighted_square_remainder_parameterized.extracted_1_1 (a x y z w : ℝ)
  (hmin : IsMinOn (fun t => t ^ 2 - 2 * a * t + 3) Set.univ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  y ^ 2 + z ^ 2 + w ^ 2 = 36 - a ^ 2
```

Read back by a model that saw only the Lean. Let a, x, y, z , and w be real numbers. Suppose that x minimizes the function $t \mapsto t^2 - 2at + 3$ over all real numbers, so that for every real t , $x^2 - 2ax + 3 \leq t^2 - 2at + 3$. Suppose also that $x^2/(2^2 - 1) + y^2/(2^2 - 3^2) + z^2/(2^2 - 5^2) + w^2/(2^2 - 7^2) = 1$, $x^2/(4^2 - 1) + y^2/(4^2 - 3^2) + z^2/(4^2 - 5^2) + w^2/(4^2 - 7^2) = 1$, $x^2/(6^2 - 1) + y^2/(6^2 - 3^2) + z^2/(6^2 - 5^2) + w^2/(6^2 - 7^2) = 1$, and $x^2/(8^2 - 1) + y^2/(8^2 - 3^2) + z^2/(8^2 - 5^2) + w^2/(8^2 - 7^2) = 1$. Then $y^2 + z^2 + w^2 = 36 - a^2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem weighted_square_remainder_parameterized (a x y z w : ℝ)
  (hmin : IsMinOn (fun t : ℝ => t ^ 2 - 2 * a * t + 3) Set.univ x)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  y ^ 2 + z ^ 2 + w ^ 2 = 36 - a ^ 2 := by
  rw [isMinOn_univ_iff] at hmin
  have hmin' : ∀ t : ℝ, x ^ 2 - 2 * a * x + 3 ≤ t ^ 2 - 2 * a * t + 3 := by
    exact hmin
  have hxa : x = a := by
    have ha' := hmin' a
    nlinarith [sq_nonneg (x - a)]
  norm_num at h₀ h₁ h₂ h₃
  have hnorm : x ^ 2 + y ^ 2 + z ^ 2 + w ^ 2 = 36 := by
    linarith
  have hxsq : x ^ 2 = a ^ 2 := by
    rw [hxa]
  nlinarith [hnorm, hxsq]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_113__x__aime_1984_p15__xe.mh__dc319d07

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint dc319d07409b0ac1

4.172 172. parameterized_nested_real_root_normalization

The problem

For real numbers a and b satisfying $0 \leq b$ and $a^2 = 64b^3$, show that $(16(a^2)^{1/3})^{1/3} = 4b^{1/3}$.

Lean statement

```
theorem parameterized_nested_real_root_normalization (a b : ℝ) (hb : 0 ≤ b) (h : a ^ 2 = 64 * b ^ 3) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 * b ^ ((1 : ℝ) / 3)
```

Parent. If $a = 8$, what is the value of $(16\sqrt[3]{a^2})^{\frac{1}{3}}$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_114 (a : ℝ) (h₀ : a = 8) :
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 := by
```

What it contributes. Preserves the parent nested real-radical structure and its cube-root normalization checkpoint, while replacing the fixed value hypothesis with the parameterized relation $a^2 = 64b^3$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the radicand and proves two new real-power identities using the variable b and its nonnegativity. Its proof depends on h , hb , and $rpow$ normalization, whereas the parent only substitutes the fixed value $a = 8$.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the radicand and requires symbolic real-power reasoning: it uses $b \geq 0$ to normalize $(b^3)^{1/3}$, applies multiplicativity of real powers, and proves the outer parameterized normalization. The parent's fixed-value `norm_num` argument does not supply these steps.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the root condition: it must derive the inner cube-root normalization from $64 * b^3$ and then normalize the outer real power symbolically in b . The parent only handles the fixed case $a = 8$ and its constant computation cannot supply these parameterized $rpow$ identities.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1710 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_nested_real_root_normalization.extracted_1_1 (a b : ℝ) (hb : 0 ≤ b) (h : a ^ 2 = 64 * b ^ 3) :  
  (16 * (a ^ 2) ^ (1 / 3)) ^ (1 / 3) = 4 * b ^ (1 / 3)
```

Read back by a model that saw only the Lean. For real numbers a and b , if b is nonnegative and $a^2 = 64b^3$, then $(16 \cdot (a^2)^{(1/3)})^{(1/3)} = 4 \cdot b^{(1/3)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem parameterized_nested_real_root_normalization (a b : ℝ) (hb : 0 ≤ b) (h : a ^ 2 = 64 * b ^ 3) :  
  (16 * (a ^ 2) ^ ((1 : ℝ) / 3)) ^ ((1 : ℝ) / 3) = 4 * b ^ ((1 : ℝ) / 3) := by  
  have hcube : 0 ≤ b ^ 3 := by positivity  
  have hprod : 0 ≤ 64 * b ^ 3 := by positivity  
  have hroot : (b ^ 3) ^ ((1 : ℝ) / 3) = b := by  
    convert Real.pow_rpow_inv_natCast hb (by norm_num : (3 : ℕ) ≠ 0) using 1 <|> norm_num  
  have hinner : (64 * b ^ 3) ^ ((1 : ℝ) / 3) = 4 * b := by  
    rw [Real.mul_rpow (by norm_num) hcube]  
    rw [show (64 : ℝ) ^ ((1 : ℝ) / 3) = 4 by  
      convert Real.pow_rpow_inv_natCast (x := (4 : ℝ)) (n := 3) (by norm_num) (by norm_num) using 1 <|> norm_num]  
    rw [hroot]  
  have houter : (64 * b) ^ ((1 : ℝ) / 3) = 4 * b ^ ((1 : ℝ) / 3) := by  
    rw [Real.mul_rpow (by norm_num) hb]  
    rw [show (64 : ℝ) ^ ((1 : ℝ) / 3) = 4 by  
      convert Real.pow_rpow_inv_natCast (x := (4 : ℝ)) (n := 3) (by norm_num) (by norm_num) using 1 <|> norm_num]  
    rw [h, hinner]  
  have hscale : 16 * (4 * b) = 64 * b := by ring  
  rw [hscale, houter]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_114__mh__af90cf1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b614b03f4056ea03

4.173 173. radical_difference_eq_zero_iff_zero_or_one

The problem

For a natural number b , characterize when $\sqrt{b^6} - \sqrt[3]{b^6} = 0$.

Lean statement

```
theorem radical_difference_eq_zero_iff_zero_or_one (b : ℕ) :
  Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) = 0 ↔ b = 0 ∨ b = 1
```

Parent. For a natural number $b \geq 2$, show that $\sqrt{b^6} - \sqrt[3]{b^6} > 0$.

Parent in Lean

```
theorem positive_radical_difference (b : ℕ) (hb : 2 ≤ b) :
  0 < Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes positivity under the hypothesis $b \geq 2$ into a full zero-set characterization over all naturals. It must analyze the factored expression's zero cases, including excluding $b = 0$ and deriving $b = 1$, which the parent's positivity proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes positivity into an exact zero-set characterization, including the new $b=0$ case and a reverse argument that factors the expression and analyzes zero products to force $b=1$. The parent's radical simplifications are reusable, but its positivity argument does not supply this equality-and-uniqueness reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a positivity result under the hypothesis $b \geq 2$ into a full zero-locus characterization for all natural numbers. Its proof requires handling the zero case, factoring the resulting polynomial, and proving both directions, none of which is supplied by the parent's positivity argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2549 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem radical_difference_eq_zero_iff_zero_or_one.extracted_1_1 (b : ℕ) :
  √(↑b ^ 6) - (↑b ^ 6) ^ (1 / 3) = 0 ↔ b = 0 ∨ b = 1
```


Read back by a model that saw only the Lean. For every natural number b , the square root of the sixth power of b minus the one-third power of the sixth power of b equals zero if and only if b equals 0 or b equals 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem radical_difference_eq_zero_iff_zero_or_one (b : ℕ) :
  Real.sqrt ((b : ℝ) ^ 6) - ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) = 0 ↔ b = 0 ∨ b = 1 := by
  by_cases hb0 : b = 0
  · subst b
    norm_num
  · have hbpos_nat : 0 < b := Nat.pos_of_ne_zero hb0
    have hbpos : 0 < (b : ℝ) := by
      exact_mod_cast hbpos_nat
    have hsqr : Real.sqrt ((b : ℝ) ^ 6) = (b : ℝ) ^ 3 := by
      calc
        Real.sqrt ((b : ℝ) ^ 6) = Real.sqrt (((b : ℝ) ^ 3) ^ 2) := by
          congr 1
        ring
        _ = |(b : ℝ) ^ 3| := by rw [Real.sqrt_sq_eq_abs]
        _ = (b : ℝ) ^ 3 := by rw [abs_of_nonneg] <|> positivity
    have hcuberoot : ((b : ℝ) ^ 6) ^ ((1 : ℝ) / 3) = (b : ℝ) ^ 2 := by
      have hroot := Real.pow_rpow_inv_natCast (x := (b : ℝ) ^ 2) (n := 3)
      (by positivity) (by norm_num)
      convert hroot using 1 <|> norm_num <|> ring
    constructor
    · intro h
      have hpoly : (b : ℝ) ^ 3 - (b : ℝ) ^ 2 = 0 := by
        rw [hsqr, hcuberoot] at h
        exact h
      have hfactor : (b : ℝ) ^ 3 - (b : ℝ) ^ 2 =
        (b : ℝ) ^ 2 * ((b : ℝ) - 1) := by ring
      have hprod : (b : ℝ) ^ 2 * ((b : ℝ) - 1) = 0 := by
        rw [← hfactor]
        exact hpoly
      have hsub : (b : ℝ) - 1 = 0 := by
        rcases mul_eq_zero.mp hprod with hsq | hsub
        · exact False.elim ((ne_of_gt (sq_pos_of_pos hbpos)) hsq)
        · exact hsub
      right
      have hreal : (b : ℝ) = 1 := by linarith
      exact_mod_cast hreal
    · intro h
      rcases h with hzero | hone
      · exact False.elim (hb0 hzero)
      · subst b
        norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_208__me.mh.mh__f2108e35

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f2108e35eb4bd027

4.174 174. sqrt_eq_iff_square

The problem

For nonnegative real numbers x and y , show that $\sqrt{x} = y$ if and only if $x = y^2$.

Lean statement

```
theorem sqrt_eq_iff_square : ∀ x y : ℝ, 0 ≤ x → 0 ≤ y → (Real.sqrt x = y ↔ x = y ^ 2) := by
```

Parent. What is the value of $\sqrt{1,000,000} - \sqrt[3]{1,000,000}$? Show that it is 900.

Parent in Lean

```
theorem mathd_algebra_208 : Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3) = 900 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent's single numerical identity to a full characterization of square roots for arbitrary nonnegative real numbers. Proving the biconditional requires reasoning absent from the parent's closed numerical normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single numerical evaluation with a general iff characterization of square roots for arbitrary nonnegative real numbers. Its proof obligation is not supplied by the parent's norm_num computation and does not overlap the retained bounded radical identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent's single numerical radical computation into a two-variable iff characterization of square roots under nonnegativity hypotheses. Its proof relies on a general theorem rather than recalling the parent's norm_num calculation, and it introduces a genuinely different proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2496 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sqrt_eq_iff_square.extracted_1_1 : ∀ (x y : ℝ), 0 ≤ x → 0 ≤ y → (√x = y ↔ x = y ^ 2)
```

Read back by a model that saw only the Lean. For all real numbers x and y , if x is nonnegative and y is nonnegative, then the square root of x equals y if and only if x equals y squared.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sqrt_eq_iff_square :  $\forall x y : \mathbb{R}, 0 \leq x \rightarrow 0 \leq y \rightarrow (\text{Real.sqrt } x = y \leftrightarrow x = y^2)$  := by
  intro x y hx hy
  have hcharacterization :  $\text{Real.sqrt } x = y \leftrightarrow x = y^2$  :=
    Real.sqrt_eq_iff_eq_sq hx hy
  exact hcharacterization
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_208__mh__10933a7a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 10933a7ac7747316

4.175 175. sqrt_sq_of_nonneg

The problem

For a nonnegative real number x , show that $\sqrt{x^2} = x$.

Lean statement

```
theorem sqrt_sq_of_nonneg (x : ℝ) (hx : 0 ≤ x) : Real.sqrt (x ^ 2) = x := by
```

Parent. What is the value of $\sqrt{1,000,000} - \sqrt[3]{1,000,000}$? Show that it is 900.

Parent in Lean

```
theorem mathd_algebra_208 : Real.sqrt 1000000 - 1000000 ^ ((1 : ℝ) / 3) = 900 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a closed numerical identity with a symbolic square-root identity requiring a nonnegativity hypothesis and an absolute-value argument. The parent's numerical normalization proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the numeric radical computation to an arbitrary nonnegative real and requires reasoning through the absolute-value identity, which the parent's normalization proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a fixed numerical computation with a universally quantified square-root identity, requiring symbolic reasoning about absolute values and nonnegativity. The parent's 'norm_num' proof supplies none of this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2481 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sqrt_sq_of_nonneg.extracted_1_1 (x : ℝ) (hx : 0 ≤ x) : √(x ^ 2) = x
```

Read back by a model that saw only the Lean. For a real number x satisfying $0 \leq x$, the square root of x squared equals x .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sqrt_sq_of_nonneg (x : ℝ) (hx : 0 ≤ x) : Real.sqrt (x ^ 2) = x := by
  have h_abs : |x| = x := abs_of_nonneg hx
  calc
    Real.sqrt (x ^ 2) = |x| := by simpa using Real.sqrt_sq_eq_abs x
    _ = x := h_abs
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_208__mh__fafacabf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fafacabf7dc7f991

4.176 176. mathd_algebra_214_range_characterization

The problem

For real numbers a, b, c and a function $f : \mathbb{R} \rightarrow \mathbb{R}$ of the form $f(x) = ax^2 + bx + c$ for every real x , attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, a real number h is attained by f if and only if $h \geq 3$.

Lean statement

```
theorem mathd_algebra_214_range_characterization
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ h : ℝ, (∃ x : ℝ, f x = h) ↔ 3 ≤ h
```

Parent. For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, and for any real number h , the equality $f(x) = h$ holds exactly when $(x - 2)^2 = 4(h - 3)$.

Parent in Lean

```
theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a pointwise fiber equivalence into a full range characterization, requiring both exclusion of values below the extremum and an explicit existence argument for every $h \geq 3$. The shared derivation of the quadratic's vertex parameters is groundwork, but the existential/surjectivity obligation is not supplied by merely recalling the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fiber equation into a full range characterization, requiring a new existence argument for every $h \geq 3$ (via solving a square-root condition) in addition to deriving the quadratic's orientation. This is substantive reasoning not supplied by the parent's pointwise equivalence proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fiber identity into a full characterization of the range, requiring both the lower-bound argument and a new existence construction for every $h \geq 3$. The parent's pointwise equivalence does not itself supply that proof obligation or its witness construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2092 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_214_range_characterization.extracted_1.1 : ∀ (a b c : ℝ) (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = a * x ^ 2 + b * x + c) →
  IsExtrOn f Set.univ 2 → f 2 = 3 → f 4 = 4 → ∀ (h : ℝ), (∃ x, f x = h) ↔ 3 ≤ h
```

Read back by a model that saw only the Lean. For all real numbers a , b , and c and every function f from the real numbers to the real numbers, if $f(x) = ax^2 + bx + c$ for every real x , if f has an extremum at $x = 2$ over all real numbers—meaning either $f(x) \leq f(2)$ for every real x or $f(2) \leq f(x)$ for every real x —if $f(2) = 3$, and if $f(4) = 4$, then for every real number h , there exists a real number x such that $f(x) = h$ if and only if $3 \leq h$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_214_range_characterization
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ h : ℝ, (∃ x : ℝ, f x = h) ↔ 3 ≤ h := by
  have hb : b = -4 * a := by
    rcases h₁ with hmin | hmax
    · have h1 : f 2 ≤ f 1 := by simpa using hmin (Set.mem_univ (1 : ℝ))
      have h3' : f 2 ≤ f 3 := by simpa using hmin (Set.mem_univ (3 : ℝ))
      rw [h₀ 1, h₂] at h1
      rw [h₀ 3, h₂] at h3'
      have e2 := h₀ 2
      have e4 := h₀ 4
      rw [h₂] at e2
      rw [h₃] at e4
      have ha : 0 < a := by
        by_contra hn
        have ha' : a = 0 := by nlinarith
        nlinarith [h1, h3', e2, e4]
      have ha_ne : a ≠ 0 := ne_of_gt ha
      let z : ℝ := 2 - (4 * a + b) / (2 * a)
      have hz : f 2 ≤ f z := by simpa using hmin (Set.mem_univ z)
      rw [h₀ z, h₂] at hz
      have hpoly :
        a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
        -(4 * a + b) ^ 2 / (4 * a) := by
        dsimp [z]
        field_simp [ha_ne]
        ring
      have hdiff : 0 ≤ -(4 * a + b) ^ 2 / (4 * a) := by
        nlinarith [hz, e2, hpoly]
      have hprod : (-(4 * a + b) ^ 2 / (4 * a)) * (4 * a) = -(4 * a + b) ^ 2 := by
        field_simp [ha_ne]
      have hd : (4 * a + b) ^ 2 = 0 := by
```

```

      have hp := mul_nonneg hdiff (le_of_lt (by nlinarith : 0 < 4 * a))
      nlinarith [hprod]
    nlinarith
  · have h1 : f 1 ≤ f 2 := by simpa using hmax (Set.mem_univ (1 : ℝ))
    have h3' : f 3 ≤ f 2 := by simpa using hmax (Set.mem_univ (3 : ℝ))
    rw [h0 1, h2] at h1
    rw [h0 3, h2] at h3'
    have e2 := h0 2
    have e4 := h0 4
    rw [h2] at e2
    rw [h3] at e4
    have ha : a < 0 := by
      by_contra hn
      have ha' : a = 0 := by nlinarith
      nlinarith [h1, h3', e2, e4]
    have ha_ne : a ≠ 0 := ne_of_lt ha
    let z : ℝ := 2 - (4 * a + b) / (2 * a)
    have hz : f z ≤ f 2 := by simpa using hmax (Set.mem_univ z)
    rw [h0 z, h2] at hz
    have hpoly :
      a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
      -(4 * a + b) ^ 2 / (4 * a) := by
      dsimp [z]
      field_simp [ha_ne]
      ring
    have hdiff : -(4 * a + b) ^ 2 / (4 * a) ≤ 0 := by
      nlinarith [hz, e2, hpoly]
    have hprod : (-(4 * a + b) ^ 2 / (4 * a)) * (4 * a) = -(4 * a + b) ^ 2 := by
      field_simp [ha_ne]
    have hd : (4 * a + b) ^ 2 = 0 := by
      have hp := mul_nonneg_of_nonpos_of_nonpos hdiff (le_of_lt (by nlinarith : 4 * a < 0))
      nlinarith [hprod]
    nlinarith
  have ha : a = (1 : ℝ) / 4 := by
    have e2 := h0 2
    have e4 := h0 4
    rw [h2] at e2
    rw [h3] at e4
    nlinarith [e2, e4, hb]
  have hc : c = 4 := by
    have e2 := h0 2
    rw [h2] at e2
    nlinarith [e2, ha, hb]
  have hform : ∀ x : ℝ, f x = (x - 2) ^ 2 / 4 + 3 := by
    intro x
    rw [h0 x, hb, ha, hc]
    ring
  intro h
  constructor
  · rintro ⟨x, hx⟩
    rw [hform x] at hx
    nlinarith [sq_nonneg (x - 2)]
  · intro hh
    let x : ℝ := 2 + Real.sqrt (4 * (h - 3))
    have hs : 0 ≤ 4 * (h - 3) := by nlinarith
    have hsqrt : (Real.sqrt (4 * (h - 3))) ^ 2 = 4 * (h - 3) := by
      exact Real.sq_sqrt hs
    refine ⟨x, ?_⟩
    rw [hform x]
    dsimp [x]
    nlinarith

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_214__me.mh.me.mh__79cf573c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 79cf573c44f2d2bf

4.177 177. mathd_algebra_214_strictMono_right

The problem

For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, whenever $2 \leq x < y$, we have $f(x) < f(y)$.

Lean statement

```
theorem mathd_algebra_214_strictMono_right
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x y : ℝ, 2 ≤ x < y → f x < f y
```

Parent. For a quadratic real function attaining an extremum at 2 with $f(2) = 3$ and $f(4) = 4$, and for any real number h , the equality $f(x) = h$ holds exactly when $(x - 2)^2 = 4(h - 3)$.

Parent in Lean

```
theorem mathd_algebra_214_bounded_height_fiber
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4)
  (h : ℝ)
  :
  ∀ x : ℝ, f x = h ↔ (x - 2) ^ 2 = 4 * (h - 3)
```

What it contributes. Reused the parent's extremum-based coefficient recovery and completed-square representation, while changing the final obligation to strict order on the right branch and adding the squared-distance checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fiber equation into strict monotonicity on the right of the vertex. After deriving the quadratic's centered form and positive leading coefficient, it must prove strict growth from $x < y$ and $x, y \geq 2$, which is reasoning absent from the parent's level-set equivalence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to strict monotonicity on the right of the vertex, requiring a new comparison-factorization argument after deriving the vertex and positive leading coefficient. The parent's fiber characterization does not supply this monotonicity proof, and the child is not a forced wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new argument showing strict growth from the vertex on the right, using the derived sign of the quadratic coefficient and factoring the difference between $f y$ and $f x$. The parent's fiber identity does not provide this monotonicity proof, so recalling the parent would not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2090 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_214_strictMono_right.extracted_1_1 : ∀ (a b c : ℝ) (f : ℝ → ℝ),
  (∀ (x : ℝ), f x = a * x ^ 2 + b * x + c) →
    IsExtrOn f Set.univ 2 → f 2 = 3 → f 4 = 4 → ∀ (x y : ℝ), 2 ≤ x → x < y → f x < f y
```

Read back by a model that saw only the Lean. For all real numbers a , b , and c , and every function f from $\mathbb{R}$ to $\mathbb{R}$, if for every real number x , $f(x) = ax^2 + bx + c$, if 2 is an extremum of f on $\mathbb{R}$, if $f(2) = 3$, and if $f(4) = 4$, then for all real numbers x and y , whenever $2 \leq x$ and $x < y$, we have $f(x) < f(y)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_214_strictMono_right
  (a b c : ℝ)
  (f : ℝ → ℝ)
  (h₀ : ∀ x, f x = a * x ^ 2 + b * x + c)
  (h₁ : IsExtrOn f Set.univ 2)
  (h₂ : f 2 = 3)
  (h₃ : f 4 = 4) :
  ∀ x y : ℝ, 2 ≤ x → x < y → f x < f y := by
  have hb : b = -4 * a := by
    rcases h₁ with hmin | hmax
    · have h1 : f 2 ≤ f 1 := by simpa using hmin (Set.mem_univ (1 : ℝ))
      have h3' : f 2 ≤ f 3 := by simpa using hmin (Set.mem_univ (3 : ℝ))
      rw [h₀ 1, h₂] at h1
      rw [h₀ 3, h₂] at h3'
      have e2 := h₀ 2
      have e4 := h₀ 4
      rw [h₂] at e2
      rw [h₃] at e4
      have ha : 0 < a := by
        by_contra ha'
        nlinarith [h1, h3', e2, e4]
      have ha_ne : a ≠ 0 := ne_of_gt ha
      let z : ℝ := 2 - (4 * a + b) / (2 * a)
      have hz : f 2 ≤ f z := by simpa using hmin (Set.mem_univ z)
      rw [h₀ z, h₂] at hz
      have hpoly :
        a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
          -(4 * a + b) ^ 2 / (4 * a) := by
          dsimp [z]
          field_simp [ha_ne]
          ring
      have hdiff : 0 ≤ -(4 * a + b) ^ 2 / (4 * a) := by
        nlinarith [hz, e2, hpoly]
      have hprod :
        (-(4 * a + b) ^ 2 / (4 * a)) * (4 * a) = -(4 * a + b) ^ 2 := by
          field_simp [ha_ne]
      have hsquare : (4 * a + b) ^ 2 = 0 := by
        have hp := mul_nonneg hdiff (le_of_lt (by nlinarith : 0 < 4 * a))
```

```

nlinarith [hprod]
nlinarith
· have h1 : f 1 ≤ f 2 := by simp using hmax (Set.mem_univ (1 : ℝ))
  have h3' : f 3 ≤ f 2 := by simp using hmax (Set.mem_univ (3 : ℝ))
  rw [h₀ 1, h₂] at h1
  rw [h₀ 3, h₂] at h3'
  have e2 := h₀ 2
  have e4 := h₀ 4
  rw [h₂] at e2
  rw [h₃] at e4
  have ha : a < 0 := by
    by_contra ha'
    nlinarith [h1, h3', e2, e4]
  have ha_ne : a ≠ 0 := ne_of_lt ha
  let z : ℝ := 2 - (4 * a + b) / (2 * a)
  have hz : f z ≤ f 2 := by simp using hmax (Set.mem_univ z)
  rw [h₀ z, h₂] at hz
  have hpoly :
    a * z ^ 2 + b * z + c - (a * (2 : ℝ) ^ 2 + b * 2 + c) =
      -(4 * a + b) ^ 2 / (4 * a) := by
    dsimp [z]
    field_simp [ha_ne]
    ring
  have hdiff : -(4 * a + b) ^ 2 / (4 * a) ≤ 0 := by
    nlinarith [hz, e2, hpoly]
  have hprod :
    -(4 * a + b) ^ 2 / (4 * a) * (4 * a) = -(4 * a + b) ^ 2 := by
    field_simp [ha_ne]
  have hsquare : (4 * a + b) ^ 2 = 0 := by
    have hp := mul_nonneg_of_nonpos_of_nonpos hdiff (le_of_lt (by nlinarith : 4 * a < 0))
    nlinarith [hprod]
  nlinarith
have ha : a = (1 : ℝ) / 4 := by
  have e2 := h₀ 2
  have e4 := h₀ 4
  rw [h₂] at e2
  rw [h₃] at e4
  nlinarith [e2, e4, hb]
have hc : c = 4 := by
  have e2 := h₀ 2
  rw [h₂] at e2
  nlinarith [e2, ha, hb]
have hform : ∀ t : ℝ, f t = (t - 2) ^ 2 / 4 + 3 := by
  intro t
  rw [h₀ t, hb, ha, hc]
  ring
intro x y hx hxy
have hsum : 0 < y + x - 4 := by
  nlinarith
have hprod : 0 < (y - x) * (y + x - 4) := by
  exact mul_pos (by linarith) hsum
have hfactor :
  (y - 2) ^ 2 - (x - 2) ^ 2 = (y - x) * (y + x - 4) := by
  ring
have hdist : (x - 2) ^ 2 < (y - 2) ^ 2 := by
  nlinarith [hprod, hfactor]
rw [hform x, hform y]
nlinarith [hdist]

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_algebra_214__me.mh.me.mh__a39ca298

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint a39ca298a16f4f50

4.178 178. mathd_algebra_234_mutation

The problem

A sequence of rational numbers satisfies a constant-ratio recurrence and begins with $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}$. Show that its n th term is $\frac{27}{125} \left(\frac{5}{3}\right)^n$ for every nonnegative integer n .

Lean statement

```
theorem mathd_algebra_234_mutation
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n, a n = (27 / 125) * (5 / 3) ^ n
```

Parent. What is the sixth term in the geometric sequence $\frac{27}{125}, \frac{9}{25}, \frac{3}{5}, \dots$? Express your answer as a common fraction. Show that it is $\frac{1}{25}$.

Parent in Lean

```
theorem mathd_algebra_234
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  (h₃ : a 2 = 3 / 5) :
  a 5 = 1 / 25 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fixed-index evaluation into a universal closed-form characterization for every n , requiring induction and recurrence transport beyond the parent’s finite three-step calculation. The proof is not a recall of the parent’s final computation, even though it independently derives the same ratio.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fixed-index evaluation into a universal closed-form theorem. It requires deriving the ratio and proving the formula by induction for every natural index, which the parent’s finite unfolding argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent’s single-index conclusion to a closed-form identity for every natural index. Its proof requires induction and a recurrence step, which the parent’s finite propagation argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2448 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_234_mutation.extracted_1_1 : ∀ (a : ℕ → ℚ) (d : ℚ),
  (∀ (n : ℕ), a (n + 1) = a n * d) →
    a 0 = 27 / 125 → a 1 = 9 / 25 → a 2 = 3 / 5 → ∀ (n : ℕ), a n = 27 / 125 * (5 / 3) ^ n
```

Read back by a model that saw only the Lean. For every sequence of rational numbers a indexed by the natural numbers and every rational number d , if $a(n+1) = a(n) \cdot d$ for every natural number n , $a(0) = 27/125$, $a(1) = 9/25$, and $a(2) = 3/5$, then for every natural number n , $a(n) = (27/125) \cdot (5/3)^n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_234_mutation
  (a : ℕ → ℚ)
  (d : ℚ)
  (h₀ : ∀ n, a (n + 1) = a n * d)
  (h₁ : a 0 = 27 / 125)
  (h₂ : a 1 = 9 / 25)
  :
  ∀ n, a n = (27 / 125) * (5 / 3) ^ n := by
  have hd : d = 5 / 3 := by
    have h := h₀ 0
    norm_num [h₁, h₂] at h
    linarith
  intro n
  induction n with
  | zero =>
    norm_num [h₁]
  | succ n ih =>
    calc
      a (Nat.succ n) = a (n + 1) := by rfl
      _ = a n * d := h₀ n
      _ = ((27 / 125) * (5 / 3) ^ n) * (5 / 3) := by rw [ih, hd]
      _ = (27 / 125) * (5 / 3) ^ (Nat.succ n) := by rw [pow_succ]; ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_234__mh__ff222c9f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9b5e4a16bef34ba4

4.179 179. reciprocal_solution_map_injective

The problem

Let $a \neq 0$. Show that the map sending each real number $b \neq 1$ to the unique nonzero solution of $a + \frac{1}{x} = \frac{b}{x}$ is injective.

Lean statement

```
theorem reciprocal_solution_map_injective (a : ℝ) (ha : a ≠ 0) :
  ∃ f : {b : ℝ // b ≠ 1} → ℝ,
    (∀ b, f b = (b - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = b / f b) ∧
    (∀ b x, x ≠ 0 ∧ a + 1 / x = b / x → x = f b) ∧
    Function.Injective f
```

Parent. Let a, b be real numbers with $a \neq 0$. The equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $b \neq 1$, and every such solution equals $\frac{b-1}{a}$.

Parent in Lean

```
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
```

What it contributes. Reuses the parent's reciprocal-equation solution formula, nonzero-witness argument, and denominator-clearing uniqueness step, while promoting them to a domain-level injective map.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new global claim: the solution assignment over all right-hand sides is injective. Proving injectivity requires comparing two solution values and recovering equality of their right-hand sides, which is not supplied by the parent's pointwise uniqueness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child promotes the scalar formula to a function on all right-hand sides other than 1 and additionally proves that this solution map is injective. Injectivity requires a new argument comparing two outputs and using $a \neq 0$, which is not supplied by the parent's per-instance uniqueness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child promotes the scalar solution formula to a function over all right-hand sides with $b \neq 1$ and additionally proves that this function is injective. Injectivity requires a new cancellation argument across two parameters, which is not supplied by the parent's pointwise uniqueness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2469 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_solution_map_injective.extracted_1_1 (a : ℝ) (ha : a ≠ 0) :
  ∃ f,
    (∀ (b : { b : ℝ // b ≠ 1 }), f b = (↑b - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = ↑b / f b) ∧
    (∀ (b : { b : ℝ // b ≠ 1 }) (x : ℝ), x ≠ 0 ∧ a + 1 / x = ↑b / x → x = f b) ∧ Function.Injective f
```

Read back by a model that saw only the Lean. *For every real number a with $a \neq 0$, there exists an injective function f from the set of real numbers b satisfying $b \neq 1$ to $\mathbb{R}$ such that, for every such b , $f(b) = (b - 1)/a$, $f(b) \neq 0$, and $a + 1/f(b) = b/f(b)$; moreover, for every such b and every real number x , if $x \neq 0$ and $a + 1/x = b/x$, then $x = f(b)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem reciprocal_solution_map_injective (a : ℝ) (ha : a ≠ 0) :
  ∃ f : {b : ℝ // b ≠ 1} → ℝ,
    (∀ b, f b = (b.1 - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = b.1 / f b) ∧
    (∀ b x, x ≠ 0 ∧ a + 1 / x = b.1 / x → x = f b) ∧
    Function.Injective f := by
  let f : {b : ℝ // b ≠ 1} → ℝ := fun b => (b.1 - 1) / a
  have hformula : ∀ b, f b = (b.1 - 1) / a ∧ f b ≠ 0 ∧ a + 1 / f b = b.1 / f b := by
    intro b
    have hnum : b.1 - 1 ≠ 0 := sub_ne_zero.mpr b.2
    have hnonzero : (b.1 - 1) / a ≠ 0 := div_ne_zero hnum ha
    refine ⟨rfl, hnonzero, ?_⟩
    dsimp [f]
    field_simp [hnonzero, ha]
    ring
  have hunique : ∀ b x, x ≠ 0 ∧ a + 1 / x = b.1 / x → x = f b := by
    intro b x hx
    rcases hx with ⟨hx0, hxeq⟩
    have hxeq' := hxeq
    field_simp [hx0] at hxeq'
    have htarget : x = (b.1 - 1) / a := by
      field_simp [ha]
      nlinarith [hxeq']
    simp [f] using htarget
  have hinjective : Function.Injective f := by
    intro b c hbc
    apply Subtype.ext
    have hquot : (b.1 - 1) / a = (c.1 - 1) / a := by
      simp [f] using hbc
    have hquot' := hquot
    field_simp [ha] at hquot'
    nlinarith [hquot']
  exact ⟨f, hformula, hunique, hinjective⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_251__me.me.mh__78561f8e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 78561f8ebf2706a6

4.180 180. reciprocal_equation_full_characterization

The problem

For real numbers a and b , the equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $a \neq 0$ and $b \neq 1$. In that case, the solution is $\frac{b-1}{a}$.

Lean statement

```
theorem reciprocal_equation_full_characterization (a b : ℝ) :
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1) ∧
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) →
    ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a)
```

Parent. Let a, b be real numbers with $a \neq 0$. The equation $a + \frac{1}{x} = \frac{b}{x}$ has a unique nonzero real solution exactly when $b \neq 1$, and every such solution equals $\frac{b-1}{a}$.

Parent in Lean

```
theorem reciprocal_equation_unique_solution (a b : ℝ) (ha : a ≠ 0) :
  (b ≠ 1 ↔ ∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ∧
  (b ≠ 1 → ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the hypothesis $a \neq 0$ and must prove it from unique existence, using a genuinely new contradiction via the distinct solutions x and $-x$ when $a=0$. It also establishes the full necessary-and-sufficient characterization, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the parent's hypothesis that $a \neq 0$ and proves it as a necessary condition from uniqueness, using a genuinely new x versus $-x$ argument. It also characterizes existence and uniqueness exactly by $a \neq 0$ and $b \neq 1$, so the parent's proof cannot simply be recalled.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the parent's fixed hypothesis $a \neq 0$ and proves it is necessary for existence and uniqueness, using a new $-x$ counterargument when $a = 0$. This full parameter characterization requires reasoning not supplied by the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2484 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_equation_full_characterization.extracted_1_1 (a b : ℝ) :
  ((∃! x, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1) ∧
  ((∃! x, x ≠ 0 ∧ a + 1 / x = b / x) → ∀ (x : ℝ), x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a)
```

Read back by a model that saw only the Lean. For all real numbers a and b , the following two assertions both hold: (1) there exists exactly one real number x such that x is nonzero and $a + 1/x = b/x$ if and only if $a \neq 0$ and $b \neq 1$; and (2) if there exists exactly one real number x such that x is nonzero and $a + 1/x = b/x$, then every real number x satisfying $x \neq 0$ and $a + 1/x = b/x$ is equal to $(b - 1)/a$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem reciprocal_equation_full_characterization (a b : ℝ) :
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1) ∧
  ((∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) →
    ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a) := by
  have hchar :
    (∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) ↔ a ≠ 0 ∧ b ≠ 1 := by
    constructor
    · rintro ⟨x, ⟨hx0, hxeq⟩, huniq⟩
      have hxeq' := hxeq
      field_simp [hx0] at hxeq'
      have ha : a ≠ 0 := by
        intro ha0
        have hb1 : b = 1 := by
          have ht := hxeq'
          rw [ha0] at ht
          nlinarith
        have hnegx : -x ≠ 0 := neg_ne_zero.mpr hx0
        have hnegEq : a + 1 / (-x) = b / (-x) := by
          simp [ha0, hb1]
        have heqneg := huniq (-x) ⟨hnegx, hnegEq⟩
        have hxzero : x = 0 := by
          nlinarith [heqneg]
        exact hx0 hxzero
      have hb : b ≠ 1 := by
        intro hb1
        have ht := hxeq'
        rw [hb1] at ht
        have hax : a * x = 0 := by
          nlinarith [ht]
        exact ha ((mul_eq_zero.mpr hax).resolve_right hx0)
      exact ⟨ha, hb⟩
    · rintro ⟨ha, hb⟩
      have hnum : b - 1 ≠ 0 := sub_ne_zero.mpr hb
      have hx0 : (b - 1) / a ≠ 0 := div_ne_zero hnum ha
      have heq : a + 1 / ((b - 1) / a) = b / ((b - 1) / a) := by
        field_simp [hx0, ha]
      ring
      refine ⟨(b - 1) / a, ⟨hx0, heq⟩, ?_⟩
      intro y hy
      rcases hy with ⟨hy0, hyeq⟩
      have hyeq' := hyeq
      field_simp [hy0] at hyeq'
      field_simp [ha]
      nlinarith [hyeq']
  have hformula :
    (∃! x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x) →
    ∀ x : ℝ, x ≠ 0 ∧ a + 1 / x = b / x → x = (b - 1) / a := by
```

```

intro hex x hx
have hab := (hchar.mp hex).1
rcases hx with ⟨hx0, hxeq⟩
have hxeq' := hxeq
field_simp [hx0] at hxeq'
field_simp [hab]
nlinarith [hxeq']
exact ⟨hchar, hformula⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_251__me.me.mh__91c74945

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 91c74945784f58fd

4.181 181. affine_reciprocal_solution

The problem

For real numbers a, b, c , and x with $c \neq 0$, if $c + \frac{a}{x} = \frac{b}{x}$, show that $x = \frac{b-a}{c}$.

Lean statement

```
theorem affine_reciprocal_solution (a b c x : ℝ) (hc : c ≠ 0) (h : c + a / x = b / x) : x = (b - a) / c
```

Parent. Three plus the reciprocal of a number equals 7 divided by that number. What is the number? Show that it is 2.

Parent in Lean

```
theorem mathd_algebra_251 (x : ℝ) (h₁ : 3 + 1 / x = 7 / x) : x = 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the reciprocal equation: it proves the symbolic solution $x = (b-a)/c$ for arbitrary real a, b, c with $c \neq 0$. The proof must establish $x \neq 0$ from the generalized hypotheses and perform symbolic field algebra, so memorizing the parent's numerical result is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal equation: arbitrary coefficients a, b , and nonzero c must be handled, including deriving $x \neq 0$ and solving symbolically. The parent's fixed constants and final linear arithmetic do not supply this generalized argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal equation: it must derive $x \neq 0$ from symbolic $c \neq 0$ and then solve for the general expression $(b-a)/c$. The parent's numerical algebra does not supply this generalized argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1831 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_reciprocal_solution.extracted_1_1 (a b c x : ℝ) (hc : c ≠ 0) (h : c + a / x = b / x) :  
  x = (b - a) / c
```

Read back by a model that saw only the Lean. Let a, b, c , and x be real numbers. Assume that c is nonzero and that $c + a/x = b/x$. Then $x = (b - a)/c$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem affine_reciprocal_solution (a b c x : ℝ) (hc : c ≠ 0) (h : c + a / x = b / x) : x = (b - a) / c := by
  have hx : x ≠ 0 := by
    intro hx
    subst x
    have hc0 : c = 0 := by
      simp using h
    exact hc hc0
  field_simp [hx] at h
  have hmul : c * x = b - a := by
    linarith [h]
  calc
    x = (c * x) / c := by
      field_simp [hc]
    _ = (b - a) / c := by rw [hmul]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_251__mh__9a5ed105

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6d0f241ac952258b

4.182 182. quadratic_discriminant_range

The problem

Show that the feasible values of c form exactly the interval $(-\infty, \frac{25}{4a}]$ when $a > 0$.

Lean statement

```
theorem quadratic_discriminant_range (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : S = Set.Iic (25 / (4 * a))
```

Parent. For a positive real number a , the values of c for which $ax^2 + 5x + c = 0$ has a real solution are exactly those satisfying $25 - 4ac \geq 0$. Show that their greatest value is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from identifying a greatest element to characterizing the entire set as an interval. Its reverse inclusion proves a new implication from the upper bound back to the discriminant inequality, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an extremum claim into an exact set characterization. Besides proving the parent's upper-bound direction, it must prove the converse that every c below the bound satisfies the discriminant inequality, which is a new proof obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a greatest-element claim into an exact characterization of the entire feasible range, requiring both inclusions and a reverse implication. The parent's proof supplies only the upper-bound direction and cannot be recalled to complete the set equality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2603 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_discriminant_range.extracted_1_1 (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | 0 ≤ 25 - 4 * a * c}) : S = Set.Iic (25 / (4 * a))
```

Read back by a model that saw only the Lean. For a real number a with $0 < a$, and a set S of real numbers satisfying $S = \{c \in \mathbb{R} \mid 0 \leq 25 - 4ac\}$, we have $S = \{c \in \mathbb{R} \mid c \leq 25/(4a)\}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem quadratic_discriminant_range (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : S = Set.Iic (25 / (4 *
  a)) := by
  rw [hS]
  ext c
  simp only [Set.mem_setOf_eq, Set.mem_Iic]
  have hpos : 0 < 4 * a := by
    nlinarith
  constructor
  · intro hc
    apply (le_div_iff₀ hpos).2
    nlinarith [hc]
  · intro hc
    have hmul : c * (4 * a) ≤ 25 := (le_div_iff₀ hpos).1 hc
    nlinarith [hmul]
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_28__me.ms.mh__a2cdcf5f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a2cdcf5f69651759

4.183 183. quadratic_solution_range

The problem

Describe exactly which real numbers c make the equation $2x^2 + 5x + c = 0$ have a real solution.

Lean statement

```
theorem quadratic_solution_range :
  ∀ c : ℝ, c ∈ {d : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + d = 0} ↔ c ≤ (25 / 8 : ℝ) := by
```

Parent. What is the largest number c such that $2x^2 + 5x + c = 0$ has at least one real solution? Express your answer as a common fraction. Show that it is $\frac{25}{8}$.

Parent in Lean

```
theorem mathd_algebra_28
  (S : Set ℝ)
  (hS : S = {c | ∃ x:ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25/8:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the greatest-element conclusion into a full characterization of the quadratic’s solution range. Its reverse implication requires constructing a real root via a square root, reasoning not supplied by the parent’s upper-bound-and-attainment proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying a greatest element to characterizing the entire solution range. Its reverse direction requires constructing a real root via a square root, reasoning not supplied by the parent’s maximum proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying the greatest element to characterizing the entire solution set via an iff. Its reverse direction requires constructing a real root using a square root, which is genuinely new reasoning beyond the parent’s upper-bound and witness argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2596 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_solution_range.extracted_1_1 : ∀ (c : ℝ),  
  c ∈ {d | ∃ x, 2 * x ^ 2 + 5 * x + d = 0} ↔ c ≤ 25 / 8
```

Read back by a model that saw only the Lean. *For every real number c , there exists a real number x satisfying $2x^2 + 5x + c = 0$ if and only if $c \leq 25/8$.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_solution_range :  
  ∀ c : ℝ, c ∈ {d : ℝ | ∃ x : ℝ, 2 * x ^ 2 + 5 * x + d = 0} ↔ c ≤ (25 / 8 : ℝ) := by  
  intro c  
  change (∃ x : ℝ, 2 * x ^ 2 + 5 * x + c = 0) ↔ c ≤ (25 / 8 : ℝ)  
  constructor  
  · rintro ⟨x, hx⟩  
    have hsq : 0 ≤ (4 * x + 5) ^ 2 := sq_nonneg (4 * x + 5)  
    nlinarith  
  · intro hc  
    have hd : 0 ≤ 25 - 8 * c := by  
      nlinarith  
    let s : ℝ := Real.sqrt (25 - 8 * c)  
    have hs : s ^ 2 = 25 - 8 * c := by  
      dsimp [s]  
      exact Real.sq_sqrt hd  
    refine ⟨(-5 + s) / 4, ?_⟩  
    nlinarith [hs]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_28__mh__22408ea6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 22408ea608393a55

4.184 184. quadratic_root_bound_and_equality

The problem

Show that if $2x^2 + 5x + c = 0$ has a real solution, then $c \leq \frac{25}{8}$, with equality exactly when $x = -\frac{5}{4}$.

Lean statement

```
theorem quadratic_root_bound_and_equality :
  ∀ (x c : ℝ), 2 * x ^ 2 + 5 * x + c = 0 →
    c ≤ (25 / 8 : ℝ) ∧ (c = (25 / 8 : ℝ) ↔ x = -5 / 4)
```

Parent. What is the largest number c such that $2x^2 + 5x + c = 0$ has at least one real solution? Express your answer as a common fraction. Show that it is $\frac{25}{8}$.

Parent in Lean

```
theorem mathd_algebra_28
  (S : Set ℝ)
  (hS : S = {c | ∃ x:ℝ, 2 * x ^ 2 + 5 * x + c = 0}) :
  IsGreatest S (25/8:ℝ) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a greatest-element statement about a set to a universal root characterization, adding the nontrivial equality case $c = 25/8$ iff $x = -5/4$. The parent's bound argument helps but does not supply this equality characterization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a set maximum to a root-wise bound plus an equality characterization. Proving equality forces $x = -5/4$ and proving the converse requires substituting that root, which is new reasoning beyond the parent's existence-and-bound argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from establishing a set maximum to characterizing equality for every quadratic root. Proving that attaining the bound forces $x = -5/4$ is additional reasoning beyond the parent's existence-and-upper-bound argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2612 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_root_bound_and_equality.extracted_1_1 : ∀ (x c : ℝ),  
  2 * x ^ 2 + 5 * x + c = 0 → c ≤ 25 / 8 ∧ (c = 25 / 8 ↔ x = -5 / 4)
```

Read back by a model that saw only the Lean. For every real numbers x and c , if $2x^2 + 5x + c = 0$, then $c \leq 25/8$ and ($c = 25/8$ if and only if $x = -5/4$).

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_root_bound_and_equality :  
  ∀ (x c : ℝ), 2 * x ^ 2 + 5 * x + c = 0 →  
    c ≤ (25 / 8 : ℝ) ∧ (c = (25 / 8 : ℝ) ↔ x = -5 / 4) := by  
  intro x c hx  
  have hsq : 0 ≤ (4 * x + 5) ^ 2 := sq_nonneg (4 * x + 5)  
  have hbound : c ≤ (25 / 8 : ℝ) := by  
    nlinarith [hsq]  
  constructor  
  · exact hbound  
  · constructor  
    · intro hc  
      nlinarith [hsq]  
    · intro hxval  
      subst x  
      norm_num at hx  
      linarith
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_28__mh__3871bd82

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 3871bd8251a48c27

4.185 185. generalized_abs_interval

The problem

For real numbers a, c and $r > 0$, solve $(1/r)|c + 2a| < 1$ for a in interval notation.

Lean statement

```
theorem generalized_abs_interval (a c r : ℝ) (hr : 0 < r) :
  (1 / r * abs (c + 2 * a) < 1) ↔
  a ∈ Set.Ioo ((-r - c) / 2) ((r - c) / 2)
```

Parent. Solve for a : $\frac{1}{5}|9 + 2a| < 1$. Express your answer in interval notation. Show that it is $(-7, -2)$.

Parent in Lean

```
theorem mathd_algebra_327
  (a : ℝ) :
  (1 / 5 * abs (9 + 2 * a) < 1) ↔ a ∈ Set.Ioo (-7:ℝ) (-2) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the center and radius and requires using the essential positivity hypothesis to transport an inequality through division. The parent's fixed-constant linear arithmetic proof does not directly supply this generalized argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the center and radius: proving the equivalence requires using the hypothesis $0 < r$ to multiply or divide inequalities safely. The parent's fixed-constant 'nlinarith' argument does not directly supply this variable-denominator reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the center term and radius, requiring positivity-aware division reasoning and preserving symbolic dependence on c and r . The parent's fixed-radius linear arithmetic proof does not directly supply this generalized obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2622 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem generalized_abs_interval.extracted_1_1 (a c r : ℝ) (hr : 0 < r) :  
  1 / r * |c + 2 * a| < 1 ↔ a ∈ Set.Ioo ((-r - c) / 2) ((r - c) / 2)
```

Read back by a model that saw only the Lean. For real numbers a , c , and r with $r > 0$, the inequality $(1/r) \cdot |c + 2a| < 1$ holds if and only if a lies in the open interval $((-r - c)/2, (r - c)/2)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem generalized_abs_interval (a c r : ℝ) (hr : 0 < r) :  
  (1 / r * abs (c + 2 * a) < 1) ↔  
    a ∈ Set.Ioo ((-r - c) / 2) ((r - c) / 2) := by  
  constructor  
  · intro h  
    have hdiv : abs (c + 2 * a) / r < 1 := by  
      simpa [div_eq_mul_inv, mul_comm] using h  
    have habs : abs (c + 2 * a) < r := by  
      have h' := (div_lt_iff₀ hr).mp hdiv  
      nlinarith  
    rw [abs_lt] at habs  
    exact ⟨by nlinarith [habs.1], by nlinarith [habs.2]⟩  
  · intro h  
    have hbounds : -r < c + 2 * a ∧ c + 2 * a < r := by  
      exact ⟨by nlinarith [h.1], by nlinarith [h.2]⟩  
    have habs : abs (c + 2 * a) < r := by  
      exact (abs_lt).2 hbounds  
    have hdiv : abs (c + 2 * a) / r < 1 := by  
      apply (div_lt_iff₀ hr).2  
      nlinarith  
    simpa [div_eq_mul_inv, mul_comm] using hdiv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_327__mh__12b4c4db

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 12b4c4dbb57a5ecc

4.186 186. bounded_quadratic_cutoff_classification

The problem

For every natural cutoff $c \leq 17$, the finite set of positive natural numbers whose quadratic expression $x^2 + 4x + 4$ is below c is empty when $c \leq 9$, is $\{1\}$ when $c \leq 16$, and is $\{1, 2\}$ otherwise.

Lean statement

```
theorem bounded_quadratic_cutoff_classification (c : ℕ) (hc : c ≤ 17) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
  S = if c ≤ 9 then ∅ else if c ≤ 16 then {1} else {1, 2}
```

Parent. For every natural number $c \leq 10$, let S be the finite set of positive natural numbers x such that $x^2 + 4x + 4 < c$. Show that S has cardinality 0 if $c \leq 9$ and cardinality 1 otherwise.

Parent in Lean

```
theorem bounded_quadratic_cutoff_cardinality (c : ℕ) (hc : c ≤ 10) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
  S.card = if c ≤ 9 then 0 else 1
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The bound is genuinely strengthened from 10 to 17, requiring a new cutoff analysis: proving that $x=2$ enters at $c=17$ while all $x \geq 3$ remain excluded. The added $\{1,2\}$ classification and extra case split impose mathematics absent from the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child raises the bound from 10 to 17 and must identify the new admissible case $x = 2$ at $c = 17$, requiring a genuinely new cutoff and case analysis beyond the parent's 0/1 cardinality argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the bound to 17 and classifies the entire finset, requiring a new case and proof that 2 is included while larger values are excluded. The parent only handles the $c \leq 10$ regime and cannot supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1816 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_quadratic_cutoff_classification.extracted_1_1 (c : ℕ) (hc : c ≤ 17) (S : Finset ℕ)
  (h₀ : ∀ (x : ℕ), x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
```

```
S = if c ≤ 9 then ∅ else if c ≤ 16 then {1} else {1, 2}
```

Read back by a model that saw only the Lean. For a natural number c with $c \leq 17$, and a finite set S of natural numbers such that, for every natural number x , x belongs to S if and only if $0 < x$ and $x^2 + 4x + 4 < c$, we have $S = \emptyset$ if $c \leq 9$, $S = \{1\}$ if $c \leq 16$ but $c > 9$, and $S = \{1, 2\}$ if $c > 16$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_quadratic_cutoff_classification (c : ℕ) (hc : c ≤ 17) (S : Finset ℕ) (h₀ : ∀ x, x ∈ S ↔ 0 < x ∧ x ^ 2 + 4 * x + 4 < c) :
  S = if c ≤ 9 then ∅ else if c ≤ 16 then {1} else {1, 2} := by
  by_cases h9 : c ≤ 9
  · have hS : S = ∅ := by
      ext x
      rw [h₀]
      constructor
      · rintro ⟨hx, hineq⟩
        have hsquare : 1 ≤ x ^ 2 := by
          simpa [pow_two] using Nat.mul_le_mul hx hx
        omega
      · intro hx
        simp at hx
  rw [hS]
  simp [h9]
  · by_cases h16 : c ≤ 16
    · have hS : S = {1} := by
        ext x
        rw [h₀]
        constructor
        · rintro ⟨hx, hineq⟩
          have hxle : x ≤ 1 := by
            by_contra h
            have hx2 : 2 ≤ x := by omega
            have hsquare : 4 ≤ x ^ 2 := by
              simpa [pow_two] using Nat.mul_le_mul hx2 hx2
            omega
          have hxone : x = 1 := by omega
          simpa [hxone]
        · intro hx
          have hxone : x = 1 := by simpa using hx
          subst x
          omega
      rw [hS]
      simp [h9, h16]
    · have hc17 : c = 17 := by omega
      have hS : S = {1, 2} := by
        ext x
        rw [h₀]
        constructor
        · rintro ⟨hx, hineq⟩
          have hxle : x ≤ 2 := by
            by_contra h
            have hx3 : 3 ≤ x := by omega
            have hsquare : 9 ≤ x ^ 2 := by
              simpa [pow_two] using Nat.mul_le_mul hx3 hx3
            omega
          have hxcases : x = 1 ∨ x = 2 := by omega
          rcases hxcases with rfl | rfl <|> simp
        · intro hx
          simp at hx
          rcases hx with rfl | rfl <|> norm_num [hc17]
      rw [hS]
      simp [h9, h16]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_405__me.mh__cb2846cc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3104249434c0c327

4.187 187. quadratic_minimum_general

The problem

For real numbers a and b , the minimum value of $y = (x - a)^2 + b$ is b .

Lean statement

```
theorem quadratic_minimum_general
  (a b : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - a) ^ 2 + b) :
  IsLeast (Set.range f) b
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both the vertex and minimum value, requiring a symbolic witness $x = a$ and a general nonnegativity argument. The parent's fixed constants 3 and 4 do not directly supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the minimizer and minimum, requiring a symbolic witness and algebraic verification rather than the parent's fixed numerical calculation. Its lower-bound argument must also remain valid for arbitrary real parameters, so this is not recall or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the vertex location and minimum value, so the parent's fixed numerical witness and normalization no longer suffice; symbolic algebra is required to establish the new minimum.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2682 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_minimum_general.extracted_1_1 (a b : ℝ) (f : ℝ → ℝ) (h : f = fun x => (x - a) ^ 2 + b) :  
  IsLeast (Set.range f) b
```

Read back by a model that saw only the Lean. For real numbers a and b and a function f from the real numbers to the real numbers, suppose that $f(x) = (x - a)^2 + b$ for every real number x . Then b is the least element of the range of f : b belongs to the range of f , and for every real number y in the range of f , $b \leq y$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_minimum_general  
  (a b : ℝ)  
  (f : ℝ → ℝ)  
  (h : f = fun x => (x - a) ^ 2 + b) :  
  IsLeast (Set.range f) b := by  
  subst f  
  constructor  
  · refine ⟨a, ?_⟩  
    ring  
  · rintro y ⟨x, rfl⟩  
    nlinarith [sq_nonneg (x - a)]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__mh__9adf79c2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9adf79c2a7ecc20b

4.188 188. translated_square_range

The problem

For any real number c , the range of $f(x) = (x - c)^2 + 4$ is exactly the set of real numbers at least 4.

Lean statement

```
theorem translated_square_range
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  Set.range f = Set.Ici 4
```

Parent. For any real number c , the least value of $(x - c)^2 + 4$ over all real x is 4.

Parent in Lean

```
theorem translated_square_minimum
  (c : ℝ)
  (f : ℝ → ℝ)
  (h : f = fun x => (x - c) ^ 2 + 4) :
  IsLeast (Set.range f) 4
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from attaining a lower bound to characterizing the entire range, requiring a surjectivity argument using square roots for every value at least 4. The parent's witness-and-lower-bound proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from identifying a minimum value to characterizing the entire range, requiring a new surjectivity argument using a square root. The parent's witness-and-lower-bound proof does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from proving a minimum to characterizing the entire range. Its reverse inclusion requires constructing a preimage using a square root, which is not supplied by the parent's minimum proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2720 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_square_range.extracted_1_1 (c : ℝ) (f : ℝ → ℝ) (h : f = fun x => (x - c) ^ 2 + 4) :  
  Set.range f = Set.Ici 4
```

Read back by a model that saw only the Lean. For a real number c and a function f from $\mathbb{R}$ to $\mathbb{R}$, if $f(x) = (x - c)^2 + 4$ for every real number x , then the range of f is the set of all real numbers greater than or equal to 4.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem translated_square_range  
  (c : ℝ)  
  (f : ℝ → ℝ)  
  (h : f = fun x => (x - c) ^ 2 + 4) :  
  Set.range f = Set.Ici 4 := by  
  subst f  
  ext y  
  constructor  
  · rintro ⟨x, rfl⟩  
    change 4 ≤ (x - c) ^ 2 + 4  
    nlinarith [sq_nonneg (x - c)]  
  · intro hy  
    change 4 ≤ y at hy  
    have hnonneg : 0 ≤ y - 4 := by linarith  
    have hsqrt : (Real.sqrt (y - 4)) ^ 2 = y - 4 := by  
      simp using Real.sq_sqrt hnonneg  
    refine ⟨c + Real.sqrt (y - 4), ?_⟩  
    nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.me.mh__07e632bf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 07e632bf03d13fb5

4.189 189. bounded_translated_square_unique_min

The problem

Let $a > 0$ and let $l \leq v \leq u$. For $f(x) = a(x - v)^2 + c$, show that the minimum of f on $[l, u]$ is $f(v)$ and that it is attained uniquely at $x = v$.

Lean statement

```
theorem bounded_translated_square_unique_min
  (f : ℝ → ℝ) (a c l u v : ℝ)
  (ha : 0 < a) (hlv : l ≤ v) (hvu : v ≤ u)
  (hf : f = fun x ↦ a * (x - v) ^ 2 + c) :
  IsLeast {y : ℝ | ∃ x : ℝ, l ≤ x ∧ x ≤ u ∧ y = f x} (f v) ∧
  ∃! x : ℝ, l ≤ x ∧ x ≤ u ∧ f x = f v
```

Parent. What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Parent in Lean

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

What it contributes. Uses the parent's completed-square nonnegativity argument for the lower-bound component, generalized to arbitrary center, coefficient, level, and an interval containing the vertex.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the quadratic with symbolic coefficient, center, offset, and interval, and adds a uniqueness characterization. Its proof requires positivity-based equality reasoning to show the sole minimizer is v , which the parent's fixed global-minimum proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds an essential bounded-interval generalization and proves uniqueness of the minimizer using the positive coefficient and equality of squares. The parent only establishes a global minimum for one fixed quadratic and supplies neither parameterization nor uniqueness reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the quadratic and interval, then proves a new uniqueness characterization of the minimizer. The parent's global minimum proof does not supply the interval-bound hypotheses or the argument that equality forces $x = v$.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2686 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_translated_square_unique_min.extracted_1_1 (f : ℝ → ℝ) (a c l u v : ℝ) (ha : 0 < a) (hlv : l ≤ v)
(hvu : v ≤ u) (hf : f = fun x => a * (x - v) ^ 2 + c) :
  IsLeast {y | ∃ x, l ≤ x ∧ x ≤ u ∧ y = f x} (f v) ∧ ∃! x, l ≤ x ∧ x ≤ u ∧ f x = f v
```

Read back by a model that saw only the Lean. Let f be a real-valued function on the real numbers, and let a, c, l, u , and v be real numbers. Assume that $a > 0$, $l \leq v$, $v \leq u$, and for every real x , $f(x) = a(x - v)^2 + c$. Then $f(v)$ is the least element of the set of all real numbers y for which there exists an x with $l \leq x \leq u$ and $y = f(x)$. Moreover, there exists a unique real number x such that $l \leq x \leq u$ and $f(x) = f(v)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_translated_square_unique_min
  (f : ℝ → ℝ) (a c l u v : ℝ)
  (ha : 0 < a) (hlv : l ≤ v) (hvu : v ≤ u)
  (hf : f = fun x => a * (x - v) ^ 2 + c) :
  IsLeast {y : ℝ | ∃ x : ℝ, l ≤ x ∧ x ≤ u ∧ y = f x} (f v) ∧
  ∃! x : ℝ, l ≤ x ∧ x ≤ u ∧ f x = f v := by
  subst f
  constructor
  · constructor
    · exact ⟨v, hlv, hvu, rfl⟩
    · rintro y ⟨x, hxL, hxU, rfl⟩
      have hbound : 0 ≤ (x - v) ^ 2 := sq_nonneg (x - v)
      nlinarith
  · refine ⟨v, ⟨hlv, hvu, rfl⟩, ?_⟩
    intro y hy
    have hvalue : a * (y - v) ^ 2 + c = a * (v - v) ^ 2 + c := by
      simpa using hy.2.2
    have hzero : (y - v) ^ 2 = 0 := by
      nlinarith
    nlinarith
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms.mh__60cdc9ff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 60cdc9ff998060d5

4.190 190. real_rpow_exponent_characterization

The problem

For a real base $b > 1$ and real exponents x and z , prove that $b^x = b^z$ if and only if $x = z$.

Lean statement

```
theorem real_rpow_exponent_characterization (b x z : ℝ) (hb : 1 < b) : b ^ x = b ^ z ↔ x = z
```

Parent. For a real base greater than 1, any two exponents in the interval $[0,10]$ that produce the same positive value are equal.

Parent in Lean

```
theorem bounded_power_exponent_unique (b y x z : ℝ) (hb : 1 < b) (hy : 0 < y) (hx : 0 ≤ x ∧ x ≤ 10) (hz : 0 ≤ z ∧ z ≤ 10) (hxp : b ^ x = y) (hxp : b ^ z = y) : x = z
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the boundedness and shared-value hypotheses and proves a global injectivity characterization of real exponentiation, including the reverse implication. This is a genuine generalization, and the parent's two logarithm-normalization arguments do not directly supply the child's iff proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the boundedness, positivity, and shared-value hypotheses and proves global injectivity of real exponentiation for every base $b > 1$. This is a genuine strengthening requiring a different injectivity lemma, not supplied by the parent's bounded logarithm argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the bounded uniqueness result to global injectivity of real exponentiation, removing the shared-value and exponent-bound hypotheses. Its proof uses the dedicated injectivity theorem and establishes positivity and non-unit conditions, so the parent's logarithm-based argument does not supply it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1738 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem real_rpow_exponent_characterization.extracted_1_1 (b x z : ℝ) (hb : 1 < b) : b ^ x = b ^ z ↔ x = z
```

Read back by a model that saw only the Lean. For real numbers b , x , and z , if b is greater than 1, then the real power of b with exponent x equals the real power of b with exponent z if and only if x equals z .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem real_rpow_exponent_characterization (b x z : ℝ) (hb : 1 < b) : b ^ x = b ^ z ↔ x = z := by
  have hbpos : 0 < b := by
    linarith
  have hbne : b ≠ 1 := by
    linarith
  exact Real.rpow_right_inj hbpos hbne
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_algebra_484__me.me.mh__86bdc8a3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint d42c1ea548d432a5

4.191 191. positive_integer_is_gcd_of_40_iff_dvd

The problem

For every positive integer y , the greatest common divisor of 40 and y equals y exactly when y divides 40.

Lean statement

```
theorem positive_integer_is_gcd_of_40_iff_dvd
  (y : ℤ)
  (hy : 0 < y) :
  ((Int.gcd 40 y : ℤ) = y ↔ y ∣ 40)
```

Parent. There is no positive integer y whose greatest common divisor with 40 is greater than y and whose least common multiple with 40 equals the product of y and that greatest common divisor.

Parent in Lean

```
theorem gcd_strictly_exceeds_candidate_impossible
  (y : ℤ)
  (hy : 0 < y)
  (hg : (Int.gcd 40 y : ℤ) > y)
  :
  False
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine gcd characterization, requiring divisibility in both directions and the greatest-common-divisor universal property. The parent only derives the upper bound $\text{gcd}(40, y) \leq y$ and does not supply the converse equality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an impossibility bound into a biconditional characterization: proving the reverse direction requires the gcd greatest-divisor theorem and the new hypothesis that y divides 40. The parent's proof of $\text{gcd}(40, y) \leq y$ does not establish this characterization or its converse.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child gives a genuine characterization: equality of the gcd with positive y is equivalent to y dividing 40. Its reverse direction requires the gcd greatestness theorem and divisibility reasoning absent from the parent's contradiction proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2550 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem positive_integer_is_gcd_of_40_iff_dvd.extracted_1_1 (y : ℤ) (hy : 0 < y) : ↑(Int.gcd 40 y) = y ↔ y ∣ 40
```

Read back by a model that saw only the Lean. *For an integer y with $0 < y$, the greatest common divisor of 40 and y , regarded as an integer, equals y if and only if y divides 40.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem positive_integer_is_gcd_of_40_iff_dvd
  (y : ℤ)
  (hy : 0 < y) :
  ((Int.gcd 40 y : ℤ) = y ↔ y ∣ 40) := by
  constructor
  · intro h_eq
    have h_gcd_dvd : (Int.gcd 40 y : ℤ) ∣ 40 :=
      Int.gcd_dvd_left 40 y
    rw [h_eq] at h_gcd_dvd
    exact h_gcd_dvd
  · intro h_div
    have h_nonneg : 0 ≤ y := by
      omega
    have h_greatest : y = (Int.gcd 40 y : ℤ) := by
      apply Int.gcd_greatest h_nonneg h_div (dvd_refl y)
    intro e _ h_e_y
    exact h_e_y
    exact h_greatest.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh.me.mh__d2494ad1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint d2494ad1427013e1

4.192 192. bounded_anchor_gcd_lcm_characterization

The problem

For an integer anchor a with $1 \leq a \leq 16$, consider positive integers b whose greatest common divisor and least common multiple with a are both a . Prove that this set has least element 8 exactly when $a = 8$.

Lean statement

```
theorem bounded_anchor_gcd_lcm_characterization
  (a : ℕ)
  :
  IsLeast {b : ℕ | 0 < b ∧ Nat.gcd b a = a ∧ Nat.lcm b a = a} 8 ↔ a = 8
```

Parent. The greatest common divisor of two integers is $(x + 3)$ and their least common multiple is $x(x + 3)$, where x is a positive integer. If one of the integers is 40, what is the smallest possible value of the other one? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_126
  (x a : ℤ)
  (h₀ : 0 < x)
  (h₁ : x = 40 ∨ a = 40)
  (Sₐ Sₓ : Set ℤ)
  (hSₐ : Sₐ = {b | Int.gcd b x = x + 3 ∧ Int.lcm b x = x * (x + 3)})
  (hSₓ : Sₓ = {y | 0 < y ∧ Int.gcd a y = y + 3 ∧ Int.lcm a y = y * (y + 3)})
  (h₂ : Nonempty Sₐ)
  (h₃ : Nonempty Sₓ) :
  IsLeast Sₐ 8 ∨ IsLeast Sₓ 8 := by
```

What it contributes. Generalizes the parent's integer gcd/lcm divisibility argument to a bounded natural-number anchor and replaces the fixed least-set disjunction with an anchor characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine parameterized characterization: the compatible set has least element 8 exactly when the anchor equals 8. Its divisibility and antisymmetry argument is unrelated to the parent's fixed-anchor disjunction and gcd-lcm contradiction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the anchor and proves a new iff characterization: the compatible set has least element 8 exactly when the anchor is 8. Its divisibility and antisymmetry argument is not supplied by the parent's fixed-40, disjunctive contradiction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed-anchor disjunction and shifted gcd/lcm equations with a genuine characterization over arbitrary anchors: the compatible set has least element 8 exactly when the anchor is 8. Its proof uses divisibility and antisymmetry to identify the anchor, reasoning not supplied by the parent's contradiction-based argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2493 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_anchor_gcd_lcm_characterization.extracted_1_1 (a : ℕ) (ha₁ : 1 ≤ a) (ha₁₆ : a ≤ 16) :
  IsLeast {b | 0 < b ∧ b.gcd a = a ∧ b.lcm a = a} 8 ↔ a = 8
```

Read back by a model that saw only the Lean. *For a natural number a satisfying $1 \leq a \leq 16$, 8 is the least natural number b such that $0 < b$, $\gcd(b,a) = a$, and $\text{lcm}(b,a) = a$, if and only if $a = 8$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_anchor_gcd_lcm_characterization
  (a : ℕ)
  :
  IsLeast {b : ℕ | 0 < b ∧ Nat.gcd b a = a ∧ Nat.lcm b a = a} 8 ↔ a = 8 := by
  constructor
  · intro h
    have h8mem := h.1
    have hg : Nat.gcd 8 a = a := h8mem.2.1
    have hl : Nat.lcm 8 a = a := h8mem.2.2
    have hdiv₁ : a | 8 := by
      have hd := Nat.gcd_dvd_left 8 a
      rw [hg] at hd
      exact hd
    have hdiv₂ : 8 | a := by
      have hd := Nat.dvd_lcm_left 8 a
      rw [hl] at hd
      exact hd
    exact Nat.dvd_antisymm hdiv₁ hdiv₂
  · intro ha
    subst a
    refine ⟨?, ?⟩
    · norm_num
    · intro b hb
      have hd : Nat.gcd b 8 | b := Nat.gcd_dvd_left b 8
      have hg : Nat.gcd b 8 = 8 := hb.2.1
      rw [hg] at hd
      rcases hd with ⟨k, hk⟩
      have hbpos : 0 < b := hb.1
      omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_126__mh__34f6bf04

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e87564ebb5c22cb4

4.193 193. odd_powers_ending_nine_sum

The problem

For natural numbers a, b, m, n , show that $(10a + 9)^{2m+1} + (10b + 9)^{2n+1}$ has remainder 8 upon division by 10.

Lean statement

```
theorem odd_powers_ending_nine_sum (a b m n : ℕ) : ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 = 8 := by
```

Parent. What is the units digit of $19^{19} + 99^{99}$? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_202 : (19 ^ 19 + 99 ^ 99) % 10 = 8 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed numerical computation: it proves every odd power of any number ending in 9 is congruent to 9 modulo 10, using an inductive periodicity argument before deriving the sum as 8. The parent's one-line norm_num proof supplies none of this symbolic reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes both fixed bases to arbitrary numbers ending in 9 and allows independent odd exponents. Its proof requires establishing a modular-periodicity invariant by induction, which the parent's one-shot norm_num proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes both bases from the fixed pair 19 and 99 to arbitrary numbers ending in 9 and requires an induction showing every odd power remains congruent to 9 modulo 10. The parent's one-step norm_num proof supplies none of this universal reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1871 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_powers_ending_nine_sum.extracted_1_1 (a b m n : ℕ) :
  ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 = 8
```

Read back by a model that saw only the Lean. For all natural numbers a, b, m , and n , the remainder when $(10a +$

$9)^{(2m+1)} + (10b+9)^{(2n+1)}$ is divided by 10 is 8.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem odd_powers_ending_nine_sum (a b m n : ℕ) :
  ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 = 8 := by
  have hsq_a : (10 * a + 9) ^ 2 % 10 = 1 := by
    norm_num [pow_two, Nat.mul_mod, Nat.add_mod]
  have hsq_b : (10 * b + 9) ^ 2 % 10 = 1 := by
    norm_num [pow_two, Nat.mul_mod, Nat.add_mod]
  have hpow_a : ∀ k : ℕ, (10 * a + 9) ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero =>
      norm_num [Nat.add_mod, Nat.mul_mod]
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega]
      rw [pow_add, Nat.mul_mod, ih, hsq_a]
  have hpow_b : ∀ k : ℕ, (10 * b + 9) ^ (2 * k + 1) % 10 = 9 := by
    intro k
    induction k with
    | zero =>
      norm_num [Nat.add_mod, Nat.mul_mod]
    | succ k ih =>
      rw [show 2 * (Nat.succ k) + 1 = (2 * k + 1) + 2 by omega]
      rw [pow_add, Nat.mul_mod, ih, hsq_b]
  calc
    ((10 * a + 9) ^ (2 * m + 1) + (10 * b + 9) ^ (2 * n + 1)) % 10 =
      (((10 * a + 9) ^ (2 * m + 1) % 10 + (10 * b + 9) ^ (2 * n + 1) % 10) % 10) := by
        rw [Nat.add_mod]
    _ = 8 := by
      rw [hpow_a m, hpow_b n]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__mh__d40adbf9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 7f775a4c9d8532cc

4.194 194. numbertheory_202_parity_characterization

The problem

For which natural numbers e and f is $19^e + 99^f$ congruent to 8 modulo 10? Show that this occurs exactly when both e and f are odd.

Lean statement

```
theorem numbertheory_202_parity_characterization (e f : ℕ) : (19 ^ e + 99 ^ f ≡ 8 [MOD 10]) ↔ (e % 2 = 1 ∧ f % 2 = 1) := by
```

Parent. Show that the units digit of $19^{19} + 99^{99}$ is congruent to 8 modulo 10.

Parent in Lean

```
theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
```

What it contributes. Retains the parent bases \$19\$, \$99\$, and modulus \$10\$, while replacing the fixed computation with the reusable parity classification underlying its congruence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponents and changes the conclusion into an exact characterization: the congruence holds precisely when both exponents are odd. Proving both directions requires parity-based residue reasoning that the parent's direct numerical normalization does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent from one fixed exponent pair to an exact characterization of all exponent parities. Its proof requires modular periodicity and a four-case parity analysis, which the parent's single 'norm_num' computation does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single fixed-exponent congruence with an exact characterization for arbitrary natural exponents. Proving both directions requires parity decomposition of the powers modulo 10, which is not supplied by the parent's one-line normalization proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2766 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem numbertheory_202_parity_characterization.extracted_1_1 (e f : ℕ) :  
  19 ^ e + 99 ^ f ≡ 8 [MOD 10] ↔ e % 2 = 1 ∧ f % 2 = 1
```

Read back by a model that saw only the Lean. For natural numbers e and f , 19 raised to the power e plus 99 raised to the power f is congruent to 8 modulo 10 if and only if the remainder when e is divided by 2 is 1 and the remainder when f is divided by 2 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem numbertheory_202_parity_characterization (e f : ℕ) :  
  (19 ^ e + 99 ^ f ≡ 8 [MOD 10]) ↔ (e % 2 = 1 ∧ f % 2 = 1) := by
  have residue19 : ∀ n : ℕ, 19 ^ n % 10 = (if n % 2 = 0 then 1 else 9) := by
    intro n
    have hsq : 19 ^ 2 ≡ 1 [MOD 10] := by norm_num [Nat.ModEq]
    have heven : ∀ k : ℕ, 19 ^ (2 * k) % 10 = 1 := by
      intro k
      have h := hsq.pow k
      simpa [Nat.ModEq, pow_mul] using h
    have hodd : ∀ k : ℕ, 19 ^ (2 * k + 1) % 10 = 9 := by
      intro k
      have h := (hsq.pow k).mul (show 19 ≡ 9 [MOD 10] by norm_num [Nat.ModEq])
      simpa [Nat.ModEq, pow_mul, pow_succ] using h
    rcases Nat.mod_two_eq_zero_or_one n with hn | hn
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using heven k
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k + 1 := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using hodd k
  have residue99 : ∀ n : ℕ, 99 ^ n % 10 = (if n % 2 = 0 then 1 else 9) := by
    intro n
    have hsq : 99 ^ 2 ≡ 1 [MOD 10] := by norm_num [Nat.ModEq]
    have heven : ∀ k : ℕ, 99 ^ (2 * k) % 10 = 1 := by
      intro k
      have h := hsq.pow k
      simpa [Nat.ModEq, pow_mul] using h
    have hodd : ∀ k : ℕ, 99 ^ (2 * k + 1) % 10 = 9 := by
      intro k
      have h := (hsq.pow k).mul (show 99 ≡ 9 [MOD 10] by norm_num [Nat.ModEq])
      simpa [Nat.ModEq, pow_mul, pow_succ] using h
    rcases Nat.mod_two_eq_zero_or_one n with hn | hn
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using heven k
    · have hd := Nat.mod_add_div n 2
      have hn2 : ∃ k, n = 2 * k + 1 := by
        refine ⟨n / 2, ?_⟩
        omega
      obtain ⟨k, rfl⟩ := hn2
      simpa [hn] using hodd k
  constructor
  · intro h
    by_cases he : e % 2 = 0 <|> by_cases hf : f % 2 = 0
    · have h19 := residue19 e
      have h99 := residue99 f
```

```

      simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h h19 h99
    omega
  · rintro ⟨he, hf⟩
    have h19 := residue19 e
    have h99 := residue99 f
    simp [Nat.ModEq, he, hf] at h19 h99
    omega

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__ms.mh__ad2e8e24

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ad2e8e24376466dd

4.195 195. bounded_period_exponents_twelve

The problem

Among the first twelve positive exponents, 6 and 12 are exactly the exponents for which 5 raised to that exponent is congruent to 1 modulo 7.

Lean statement

```
theorem bounded_period_exponents_twelve (n : ℕ) (hn : 0 < n) (hn12 : n ≤ 12) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 ∨ n = 12 := by
```

Parent. Let n be a positive integer with $n \leq 6$. Show that 5^n is congruent to 1 modulo 7 if and only if $n = 6$.

Parent in Lean

```
theorem bounded_first_period_exponent (n : ℕ) (hn : 0 < n) (hn6 : n ≤ 6) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The bound is genuinely extended from 6 to 12, requiring new residue checks for exponents 7–11 and proving the additional period at 12. Although the case-split scaffold mirrors the parent, the child's conclusion requires establishing a new exponent class rather than merely wrapping or decorating the parent result.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the bounded classification from exponents 1–6 to 1–12, requiring new case analysis and establishing the additional period instance $5^{12} \equiv 1 \pmod{7}$. The parent's theorem cannot handle $n = 12$, so its proof does not supply the child's full obligation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child expands the bound to 12 and characterizes both period multiples, requiring new modular checks for exponents 7–12 and a separate proof of the 12th-period fact. The parent's bounded case analysis only reaches exponent 6 and cannot supply this obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1816 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_period_exponents_twelve.extracted_1.1 (n : ℕ) (hn : 0 < n) (hn12 : n ≤ 12) :
  5 ^ n ≡ 1 [ZMOD 7] ↔ n = 6 ∨ n = 12
```

Read back by a model that saw only the Lean. For a natural number n satisfying $0 < n$ and $n \leq 12$, 5^n is congruent to 1 modulo 7 if and only if $n = 6$ or $n = 12$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_period_exponents_twelve (n : ℕ) (hn : 0 < n) (hn12 : n ≤ 12) : ((5 : ℤ) ^ n ≡ 1 [ZMOD 7]) ↔ n = 6 ∨ n = 12 := by
  have hperiod6 : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have hperiod12 : (5 : ℤ) ^ 12 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  constructor
  · intro h
    have hcases : n = 1 ∨ n = 2 ∨ n = 3 ∨ n = 4 ∨ n = 5 ∨ n = 6 ∨ n = 7 ∨ n = 8 ∨ n = 9 ∨ n = 10 ∨ n = 11 ∨ n = 12 := by
      omega
    rcases hcases with h1 | h2 | h3 | h4 | h5 | h6 | h7 | h8 | h9 | h10 | h11 | h12
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; exact Or.inl rfl
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; norm_num [Int.ModEq] at h
    · subst n; exact Or.inr rfl
  · intro h
    rcases h with h6 | h12
    · subst n
      simpa using hperiod6
    · subst n
      simpa using hperiod12
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__me.mh__004a2b02

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint db49e39731cde1db

4.196 196. bounded_six_period_power_mod_seven

The problem

Let n be an integer congruent to 5 modulo 7, and let k be a natural number. Prove that n^{6k} is congruent to 1 modulo 7.

Lean statement

```
theorem bounded_six_period_power_mod_seven (n : Z) (k : N) (h : n ≡ 5 [ZMOD 7]) : n ^ (6 * k) ≡ 1 [ZMOD 7]
```

Parent. Prove that if an integer n is congruent to 5 modulo 7, then n^{30} is congruent to 1 modulo 7.

Parent in Lean

```
theorem parameterized_power_mod_seven (n : Z) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Supplied the modulo-7 residue-lifting strategy, generalized from the fixed exponent to the bounded parameterized exponent $6k$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed exponent to every multiple of six, requiring the new periodicity argument that proves $5^6 \equiv 1$ and lifts it through an arbitrary k . The parent's direct computation of 5^{30} does not supply this parameterized reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent to an arbitrary six-period exponent and requires proving and iterating the modular period symbolically. The parent's numeric normalization at exponent 30 cannot supply this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 30 to an arbitrary six-period exponent and must establish the period $\backslash(5^6 \equiv 1 \backslash \text{pmod } 7\backslash)$ before lifting it to $\backslash(6k\backslash)$. The parent's direct fixed-power normalization cannot close this parameterized goal.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2085 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_six_period_power_mod_seven.extracted_1_1 (n : ℤ) (k : ℕ) (h : n ≡ 5 [ZMOD 7]) :  
  n ^ (6 * k) ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For any integer n and natural number k , if n is congruent to 5 modulo 7, then n raised to the power $6k$ is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem bounded_six_period_power_mod_seven (n : ℤ) (k : ℕ) (h : n ≡ 5 [ZMOD 7]) : n ^ (6 * k) ≡ 1 [ZMOD 7] := by  
  have hpow : n ^ (6 * k) ≡ (5 : ℤ) ^ (6 * k) [ZMOD 7] := h.pow (6 * k)  
  have hperiod : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by  
    norm_num [Int.ModEq]  
  have hperiod_pow : ((5 : ℤ) ^ 6) ^ k ≡ (1 : ℤ) ^ k [ZMOD 7] := hperiod.pow k  
  have hbase : (5 : ℤ) ^ (6 * k) ≡ 1 [ZMOD 7] := by  
    simpa [pow_mul] using hperiod_pow  
  have hresult : n ^ (6 * k) ≡ 1 [ZMOD 7] := hpow.trans hbase  
  exact hresult
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh.mh__91d988b9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5b4924b55b4cd723

4.197 197. five_pow_six_mul_mod_seven

The problem

Show that for every natural number n , the remainder when 5^{6n} is divided by 7 is 1.

Lean statement

```
theorem five_pow_six_mul_mod_seven (n : ℕ) : 5 ^ (6 * n) ≡ 1 [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed exponent 30 to every exponent of the form $6 \cdot n$. Its proof requires establishing the new base congruence $5^6 \equiv 1 \pmod{7}$ and lifting it through arbitrary powers, which the parent's direct norm_num proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed exponent 30 to every exponent of the form $6n$. Its proof must establish the new base congruence modulo 7 and then use preservation of congruence under powers, which the parent's direct computation does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed exponent 30 to every exponent of the form $6 \cdot n$, requiring the new modular-power argument via $5^6 \equiv 1$ and Int.ModEq.pow. The parent's one-off norm_num proof supplies no reasoning for this universal statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1780 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem five_pow_six_mul_mod_seven.extracted_1_1 (n : ℕ) : 5 ^ (6 * n) ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number n , 5 raised to the power $6n$ is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem five_pow_six_mul_mod_seven (n : ℕ) : 5 ^ (6 * n) ≡ 1 [ZMOD 7] := by
  have hbase : (5 : ℤ) ^ 6 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have hpow := Int.ModEq.pow n hbase
  simpa [pow_mul] using hpow
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__093b3f6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b34110a60f7d3d10

4.198 198. parameterized_power_mod_seven

The problem

Prove that if an integer n is congruent to 5 modulo 7, then n^{30} is congruent to 1 modulo 7.

Lean statement

```
theorem parameterized_power_mod_seven (n : ℤ) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

What it contributes. Supplied the modulus-7 power checkpoint that 5^{30} is congruent to 1; the child transports it from a parameterized base.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base and uses the new modular hypothesis via congruence preservation under powers before invoking the fixed numerical computation. The parent's proof supplies only the constant case and cannot establish the parameterized claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: it must use the hypothesis that n is congruent to 5 modulo 7 to transport the thirtieth-power congruence. The parent's numeral-only normalization proof supplies no reasoning for this variable case.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: it must transfer the congruence hypothesis through the thirtieth power before using the fixed-base computation. The parent's direct normalization proof supplies no way to handle an arbitrary integer congruent to 5 modulo 7.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2082 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_power_mod_seven.extracted_1_1 (n : ℤ) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer n , if n is congruent to 5 modulo 7, then n raised to the

30th power is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem parameterized_power_mod_seven (n : Z) (h : n ≡ 5 [ZMOD 7]) : n ^ 30 ≡ 1 [ZMOD 7] := by
  have hpow : n ^ 30 ≡ (5 : Z) ^ 30 [ZMOD 7] := h.pow 30
  have hbase : (5 : Z) ^ 30 ≡ 1 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hpow.trans hbase
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__3745eb24

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3745eb240b0fe738

4.199 199. every_int_seventh_power_mod_seven

The problem

Show that for every integer n , n^7 is congruent to n modulo 7.

Lean statement

```
theorem every_int_seventh_power_mod_seven (n : ℤ) : n ^ 7 ≡ n [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes a single numerical congruence to the polynomial identity $n^7 \equiv n$ for every integer, requiring a prime-modulus Fermat argument. The parent's norm_num computation for 5^{30} provides no proof of this universal statement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single numerical congruence into the universal Fermat congruence $n^7 \equiv n$ for every integer n . Its proof requires invoking primality and a prime-power modular theorem, none of which is supplied by the parent's direct normalization proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from one concrete computation to Fermat's universal congruence for every integer, requiring a prime-exponent theorem and quantified reasoning absent from the parent's norm_num proof. It is not a restatement of the kept $5^{30} \equiv 1 \pmod{7}$ result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1789 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem every_int_seventh_power_mod_seven.extracted_1_1 (n : ℤ) : n ^ 7 ≡ n [ZMOD 7]
```

Read back by a model that saw only the Lean. For every integer n , n raised to the seventh power is congruent to n modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem every_int_seventh_power_mod_seven (n : ℤ) : n ^ 7 ≡ n [ZMOD 7] := by
  have hprime : Nat.Prime 7 := by
    norm_num
  have hfermat : ∀ m : ℤ, m ^ 7 ≡ m [ZMOD 7] := by
    intro m
    exact Int.ModEq.pow_prime_eq_self hprime m
  exact hfermat n
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__43b55310

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 294a39dfac5ba0c5

4.200 200. pow_fifteen_mod_seven_of_congruent_five

The problem

Show that every integer congruent to 5 modulo 7 has its 15th power congruent to -1 modulo 7.

Lean statement

```
theorem pow_fifteen_mod_seven_of_congruent_five : ∀ n : ℤ, n ≡ 5 [ZMOD 7] → n ^ 15 ≡ (-1 : ℤ) [ZMOD 7] := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed residue to every integer congruent to 5 modulo 7 and changes the exponent to 15, requiring congruence transport plus the new computation $5^{15} \equiv -1$. The parent's computation of $5^{30} \equiv 1$ does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed numerical congruence into a universal congruence-class theorem: it must transfer $n \equiv 5$ through the fifteenth power and establish the new base result $5^{15} \equiv -1$. The parent's isolated $5^{30} \equiv 1$ result does not provide this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine universal congruence theorem: it must transport the hypothesis $n \equiv 5 [ZMOD 7]$ through the fifteenth power and separately establish $5^{15} \equiv -1$. The parent's isolated computation of 5^{30} provides neither this transport argument nor the new modular identity.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1807 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem pow_fifteen_mod_seven_of_congruent_five.extracted_1_1 : ∀ (n : ℤ),  
  n ≡ 5 [ZMOD 7] → n ^ 15 ≡ -1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every integer n , if n is congruent to 5 modulo 7, then n raised to the 15th power is congruent to -1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem pow_fifteen_mod_seven_of_congruent_five :  $\forall n : \mathbb{Z}, n \equiv 5 \pmod{7} \rightarrow n^{15} \equiv (-1 : \mathbb{Z}) \pmod{7} :=$  by
  intro n hn
  have hbase :  $(5 : \mathbb{Z})^{15} \equiv (-1 : \mathbb{Z}) \pmod{7} :=$  by
    norm_num [Int.ModEq]
  have hpow :  $n^{15} \equiv (5 : \mathbb{Z})^{15} \pmod{7} :=$  by
    exact hn.pow 15
  exact hpow.trans hbase
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__79de960b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4a1e3944da3d3752

4.201 201. modular_power_of_coprime_seven

The problem

Let n be an integer relatively prime to 7. Show that n^6 leaves remainder 1 when divided by 7.

Lean statement

```
theorem modular_power_of_coprime_seven (n : ℤ) (h : IsCoprime n 7) : n ^ 6 ≡ 1 [ZMOD 7]
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes from one numerical congruence to Fermat's theorem for every integer coprime to 7, requiring a substantive coprimality-based modular argument absent from the parent's direct normalization proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single computed residue with a universal Fermat-style theorem for every integer coprime to 7. Its proof requires invoking primality and a coprime modular-power theorem, reasoning absent from the parent's direct normalization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed numerical congruence to every integer coprime to 7 and exponent 6. The parent's direct normalization proof provides no argument for this universal theorem; the child requires invoking Fermat's little theorem with the coprimality hypothesis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2409 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_of_coprime_seven.extracted_1_1 (n : ℤ) (h : IsCoprime n 7) : n ^ 6 ≡ 1 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer n that is coprime to 7, the sixth power n^6 is congruent to 1 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_of_coprime_seven (n : ℤ) (h : IsCoprime n 7) : n ^ 6 ≡ 1 [ZMOD 7] := by
  have hprime : Nat.Prime 7 := by
    norm_num
  have hfermat : n ^ (7 - 1) ≡ 1 [ZMOD 7] :=
    Int.ModEq.pow_card_sub_one_eq_one hprime h
  simpa using hfermat
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__7a3c855f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7a3c855f070a2583

4.202 202. fermat_modEq_general

The problem

Let p be a prime natural number and let n be an integer relatively prime to p . Prove that $n^{p-1} \equiv 1 \pmod{p}$.

Lean statement

```
theorem fermat_modEq_general (p : ℕ) (n : ℤ) (hp : Nat.Prime p) (hcop : IsCoprime n p) : n ^ (p - 1) ≡ 1 [ZMOD p]
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child makes both the modulus and base variable and requires proving the general Fermat congruence from primality and coprimality. The parent's concrete normalization proof supplies none of this reasoning, and the child is not a repeat of the retained parameterized mod-seven result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed congruence to Fermat's theorem for arbitrary prime moduli and coprime integers. Its proof requires a general modular-arithmetic lemma and hypotheses absent from the parent, so recalling the parent cannot solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a genuine universal generalization from the fixed congruence to Fermat's theorem for arbitrary prime moduli and coprime integers. Its proof requires a modular-arithmetic theorem and the coprimality hypothesis, reasoning unavailable from the parent's numerical normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2089 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fermat_modEq_general.extracted_1_1 (p : ℕ) (n : ℤ) (hp : Nat.Prime p) (hcop : IsCoprime n ↑p) :
  n ^ (p - 1) ≡ 1 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For a natural number p and an integer n , if p is prime and n is coprime

to the integer p , then n raised to the natural-number exponent $p - 1$ is congruent to 1 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem fermat_modEq_general (p : ℕ) (n : ℤ) (hp : Nat.Prime p) (hcop : IsCoprime n p) :
  n ^ (p - 1) ≡ 1 [ZMOD p] := by
  have hprime : Nat.Prime p := hp
  have hunit : IsCoprime n p := hcop
  have hfermat : n ^ (p - 1) ≡ 1 [ZMOD p] :=
    Int.ModEq.pow_card_sub_one_eq_one hprime hunit
  exact hfermat
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__mh__8bb5ed7a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8bb5ed7a6cc95486

4.203 203. bounded_positional_multiplier_balance

The problem

For natural numbers $base$, $lead$, $unit$, and k with $1 < base$, $1 \leq lead < base$, $unit < base$, and $k \leq base$, the equation stating that k times the sum of the base- $base$ digits of the two-digit numeral $base \cdot lead + unit$ equals that numeral is equivalent to $(base - k) \cdot lead = (k - 1) \cdot unit$.

Lean statement

```
theorem bounded_positional_multiplier_balance
  (base lead unit k : ℕ)
  (hbase : 1 < base)
  (hlead : 1 ≤ lead)
  (hlead_lt : lead < base)
  (hunit_lt : unit < base)
  (hk : k ≤ base) :
  k * (Nat.digits base (base * lead + unit)).sum = base * lead + unit ↔
  (base - k) * lead = (k - 1) * unit
```

Parent. For natural numbers $base$, $lead$, $unit$, and k with $1 < base$, $1 \leq lead < base$, and $unit < base$, prove that k times the sum of the base- $base$ digits of $base \cdot lead + unit$ equals $base \cdot lead + unit$ if and only if $k(lead + unit) = base \cdot lead + unit$.

Parent in Lean

```
theorem bounded_positional_multiplier_characterization
  (base lead unit k : ℕ)
  (hbase : 1 < base)
  (hlead : 1 ≤ lead)
  (hlead_lt : lead < base)
  (hunit_lt : unit < base)
  :
  k * (Nat.digits base (base * lead + unit)).sum = base * lead + unit ↔
  k * (lead + unit) = base * lead + unit
```

What it contributes. Supplied the bounded-base digit decomposition and digit-sum reduction; the child replaces the parent's raw sum equation with the coefficient-balance characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child retains the positional-digit decomposition but replaces the fixed-point equation with a nontrivial Nat-subtraction balance law. Establishing equivalence requires new case analysis for $k = 0$ and arithmetic reasoning using $k \leq base$, which the parent's proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a substantive balance characterization. Its proof requires a new case split on $k=0$ and arithmetic reasoning with truncated natural subtraction, which the parent's digit-decomposition proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a balance equation and requires a substantive natural-number algebra argument, including cases for $k = 0$ and $k \geq 1$. The parent's digit decomposition does not supply this equivalence by recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2054 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_positional_multiplier_balance.extracted_1_1 (base lead unit k : ℕ) (hbase : 1 < base) (hlead : 1 ≤ lead)
(hlead_lt : lead < base) (hunit_lt : unit < base) (hk : k ≤ base) :
k * (base.digits (base * lead + unit)).sum = base * lead + unit ↔ (base - k) * lead = (k - 1) * unit
```

Read back by a model that saw only the Lean. For natural numbers $base$, $lead$, $unit$, and k , if $1 < base$, $1 \leq lead$, $lead < base$, $unit < base$, and $k \leq base$, then k multiplied by the sum of the base-base digits of $base \cdot lead + unit$ equals $base \cdot lead + unit$ if and only if $(base - k) \cdot lead = (k - 1) \cdot unit$, where the subtractions are natural-number subtractions.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_positional_multiplier_balance
  (base lead unit k : ℕ)
  (hbase : 1 < base)
  (hlead : 1 ≤ lead)
  (hlead_lt : lead < base)
  (hunit_lt : unit < base)
  (hk : k ≤ base) :
  k * (Nat.digits base (base * lead + unit)).sum = base * lead + unit ↔
  (base - k) * lead = (k - 1) * unit := by
  have hdigit : Nat.digits base (base * lead + unit) = unit :: Nat.digits base lead := by
    rw [Nat.add_comm (base * lead) unit]
    exact Nat.digits_add base hbase unit lead hunit_lt (Or.inr (Nat.ne_of_gt hlead))
  have hdecomp : (Nat.digits base (base * lead + unit)).sum = lead + unit := by
    rw [hdigit, Nat.digits_of_lt base lead (Nat.ne_of_gt hlead) hlead_lt]
    simp [Nat.add_comm]
  have hfixed_balance :
    k * (lead + unit) = base * lead + unit ↔
    (base - k) * lead = (k - 1) * unit := by
    by_cases hkzero : k = 0
    · subst k
      have hprod : 0 < base * lead := Nat.mul_pos (by omega) hlead
      constructor <=> intro h <=> simp at h <=> omega
    · have hkpos : 1 ≤ k := by omega
      have hklead : k * lead ≤ base * lead := Nat.mul_le_mul_right lead hk
      have hunit : unit ≤ k * unit := by
        calc
          unit = 1 * unit := by simp
          _ ≤ k * unit := Nat.mul_le_mul_right unit hkpos
      constructor
      · intro h
        simp only [Nat.sub_mul, Nat.one_mul]
        rw [Nat.mul_add] at h
        omega
      · intro h
        simp only [Nat.sub_mul, Nat.one_mul] at h
        rw [Nat.mul_add]
        omega
```

```
constructor
· intro h
  rw [hdecomp] at h
  exact hfixed_balance.mp h
· intro h
  rw [hdecomp]
  exact hfixed_balance.mpr h
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_284__me.me.mh__3832201b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint eaec694e7c7fb9d7

4.204 204. equal_nonzero_digits_fixed_point_impossible

The problem

Show that for every natural number k and every decimal digit a with $1 \leq a \leq 9$, the number $10a + a$ cannot equal k times the sum of its decimal digits.

Lean statement

```
theorem equal_nonzero_digits_fixed_point_impossible
  (k a : ℕ)
  (ha₁ : 1 ≤ a)
  (ha₉ : a ≤ 9) :
  ¬ (k * (Nat.digits 10 (10 * a + a)).sum = 10 * a + a)
```

Parent. For a bounded multiplier and a positive two-digit decimal pair, the multiplier fixed-point equation holds exactly when it holds after decomposing the number into its tens and units digits.

Parent in Lean

```
theorem bounded_decimal_multiplier_characterization
  (k a b : ℕ)
  (hk : k ≤ 10)
  (ha₁ : 1 ≤ a)
  (ha₉ : a ≤ 9)
  (hb₉ : b ≤ 9) :
  k * (Nat.digits 10 (10 * a + b)).sum = 10 * a + b ↔
  k * (a + b) = 10 * a + b
```

What it contributes. Uses the parent's bounded decimal digit-sum decomposition checkpoint, then specializes it to equal digits and consumes the resulting equation in an impossibility proof.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's bound on k and proves impossibility for every natural k , requiring a new arithmetic contradiction from $2*k = 11$ after specializing equal digits. The parent's bounded equivalence and finite digit computation do not supply this unbounded contradiction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to equal nonzero digits and derives the new contradiction $2*k = 11$ from the fixed-point equation, then uses arithmetic impossibility. The parent only establishes a digit-sum decomposition and an equivalence, so its proof does not supply the cancellation and contradiction argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to equal digits and adds a genuinely new impossibility argument: it transforms the fixed-point equation into $2*k = 11$ and derives a contradiction. The parent's digit-sum decomposition only rewrites the equation and does not supply this parity/cancellation reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2056 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem equal_nonzero_digits_fixed_point_impossible.extracted_1_1 (k a : ℕ) (ha₁ : 1 ≤ a) (ha₉ : a ≤ 9) :
  ¬k * (Nat.digits 10 (10 * a + a)).sum = 10 * a + a
```

Read back by a model that saw only the Lean. *For all natural numbers k and a such that $1 \leq a$ and $a \leq 9$, it is not the case that k multiplied by the sum of the base-10 digits of $10a + a$ equals $10a + a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem equal_nonzero_digits_fixed_point_impossible
  (k a : ℕ)
  (ha₁ : 1 ≤ a)
  (ha₉ : a ≤ 9) :
  ¬ (k * (Nat.digits 10 (10 * a + a)).sum = 10 * a + a) := by
  intro h
  have hdecomp : (Nat.digits 10 (10 * a + a)).sum = a + a := by
    interval_cases a <|>
    norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def]
  have hfixed : k * (a + a) = 10 * a + a := by
    simpa [hdecomp] using h
  have hfactor : (2 * k) * a = 11 * a := by
    calc
      (2 * k) * a = k * (a + a) := by ring
      _ = 10 * a + a := hfixed
      _ = 11 * a := by ring
  have hbad : 2 * k = 11 := Nat.mul_right_cancel (by omega) hfactor
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_284__me.mh__68fc7013

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a2e6c0da6e232809

4.205 205. characterize_digit_sum_multiplier

The problem

Let x be a positive two-digit number whose decimal digit sum, multiplied by a natural number k , equals x . Prove that $k \leq 10$, and that $k = 10$ exactly when x is divisible by 10.

Lean statement

```
theorem characterize_digit_sum_multiplier (k x : ℕ)
  (hlen : (Nat.digits 10 x).length = 2)
  (heq : k * (Nat.digits 10 x).sum = x) :
  k ≤ 10 ∧ (k = 10 ↔ x % 10 = 0)
```

Parent. What positive two-digit natural number is exactly twice the sum of its decimal digits? Show that it is uniquely determined and equals 18.

Parent in Lean

```
theorem unique_positive_two_digit_digit_sum_solution :
  ∃! x : ℕ, 0 < x ∧ (Nat.digits 10 x).length = 2 ∧ 2 * (Nat.digits 10 x).sum = x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed multiplier 2 to symbolic k and proves a new characterization: k is bounded by 10 and equals 10 exactly when x is divisible by 10. The parent's unique-witness argument does not establish this variable-parameter iff claim, even though both proofs use finite digit enumeration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed multiplier 2 and proves a new characterization: every two-digit solution has $k \leq 10$, with equality exactly when the units digit is zero. The parent's uniqueness proof for $k = 2$ does not supply this symbolic bound or equivalence.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the multiplier and proves a full characterization: every two-digit solution has $k \leq 10$, with equality exactly when the number ends in zero. The parent's fixed-multiplier uniqueness proof cannot establish this symbolic classification, even though the finite digit-bound technique is reused.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2520 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem characterize_digit_sum_multiplier.extracted_1_1 (k x : ℕ) (hpos : 0 < x) (hlen : (Nat.digits 10 x).length = 2)
  (heq : k * (Nat.digits 10 x).sum = x) : k ≤ 10 ∧ (k = 10 ↔ x % 10 = 0)
```

Read back by a model that saw only the Lean. For natural numbers k and x , if x is positive, the list of base-10 digits of x has length 2, and k multiplied by the sum of the base-10 digits of x equals x , then k is at most 10, and k equals 10 if and only if the remainder when x is divided by 10 is 0.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem characterize_digit_sum_multiplier (k x : ℕ)

  (hlen : (Nat.digits 10 x).length = 2)
  (heq : k * (Nat.digits 10 x).sum = x) :
  k ≤ 10 ∧ (k = 10 ↔ x % 10 = 0) := by
have hxlt : x < 100 := by
  have h := Nat.lt_base_pow_length_digits (m := x) (b := 10) (by norm_num)
  rw [hlen] at h
  norm_num at h
  exact h
interval_cases x <|>
  norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def] at hlen <|>
  norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def] at heq <|>
  norm_num [Nat.digits, Nat.digitsAux, Nat.digitsAux_def] <|>
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_284__me.mh__c0669468

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint f6439907ff880f2c

4.206 206. common_cube_fourth_characterization

The problem

Show that n is both the cube of x and the fourth power of t exactly when there is a natural number c such that $n = c^{12}$, $x = c^4$, and $t = c^3$.

Lean statement

```
theorem common_cube_fourth_characterization
  (n x t : ℕ) :
  (x ^ 3 = n ∧ t ^ 4 = n) ↔ ∃ c : ℕ, n = c ^ 12 ∧ x = c ^ 4 ∧ t = c ^ 3
```

Parent. Among positive integers other than 1 that are at most 4096, an integer is both a perfect cube and a perfect fourth power if and only if it equals 4096.

Parent in Lean

```
theorem bounded_common_cube_fourth_characterization
  (n : ℕ)
  (hn : n ≤ 4096) :
  (0 < n ∧ n ≠ 1 ∧ ∃ x, x ^ 3 = n ∧ ∃ t, t ^ 4 = n) ↔ n = 4096
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's bounded finite enumeration with a universal characterization of shared cube and fourth-power values. Its proof requires the coprime-exponent power decomposition theorem and symbolic exponent algebra, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's bounded finite search with a genuinely symbolic characterization of all common cube/fourth powers, requiring coprime-exponent factorization and deriving the twelfth-power structure.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the finite bound and proves a genuinely symbolic characterization: common cube and fourth powers must arise from a shared twelfth power. Its proof requires coprime-exponent reasoning, which is absent from the parent's bounded case split.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2633 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem common_cube_fourth_characterization.extracted_1_1 (n x t : ℕ) :  
  x ^ 3 = n ∧ t ^ 4 = n ↔ ∃ c, n = c ^ 12 ∧ x = c ^ 4 ∧ t = c ^ 3
```

Read back by a model that saw only the Lean. For natural numbers n , x , and t , x cubed equals n and t to the fourth power equals n if and only if there exists a natural number c such that n equals c to the twelfth power, x equals c to the fourth power, and t equals c cubed.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem common_cube_fourth_characterization  
  (n x t : ℕ) :  
  (x ^ 3 = n ∧ t ^ 4 = n) ↔ ∃ c : ℕ, n = c ^ 12 ∧ x = c ^ 4 ∧ t = c ^ 3 := by  
  constructor  
  · rintro ⟨hx, ht⟩  
    have hxt : x ^ 3 = t ^ 4 := by  
      calc  
        x ^ 3 = n := hx  
        _ = t ^ 4 := ht.symm  
    obtain ⟨c, hxc, htc⟩ :=  
      Nat.exists_eq_pow_of_exponent_coprime_of_pow_eq_pow  
        (a := x) (b := t) (m := 3) (n := 4) (by norm_num) hxt  
    refine ⟨c, ?_, hxc, htc⟩  
    calc  
      n = x ^ 3 := hx.symm  
      _ = (c ^ 4) ^ 3 := by rw [hxc]  
      _ = c ^ 12 := by ring  
  · rintro ⟨c, hn, hxc, htc⟩  
    constructor  
    · calc  
      x ^ 3 = (c ^ 4) ^ 3 := by rw [hxc]  
      _ = c ^ 12 := by ring  
      _ = n := hn.symm  
    · calc  
      t ^ 4 = (c ^ 3) ^ 4 := by rw [htc]  
      _ = c ^ 12 := by ring  
      _ = n := hn.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_296__me.mh__23bfbf4a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 23bfbf4a6a684f04

4.207 207. inverse_seven_is_right_inverse

The problem

For every natural number m relatively prime to 7, if n is the multiplicative inverse of 7 modulo m , show that $7n = 1$ modulo m .

Lean statement

```
theorem inverse_seven_is_right_inverse
  (m : ℕ)
  (n : ZMod m)
  (hm : Nat.Coprime 7 m)
  (h : n = (7 : ZMod m)⁻¹) :
  (7 : ZMod m) * n = 1
```

Parent. Find the integer n such that $0 \leq n < 398$ and n is a multiplicative inverse to 7 modulo 398. Show that it is 57.

Parent in Lean

```
theorem mathd_numbertheory_33
  (n : ZMod 398)
  (h₁ : n = 7⁻¹) :
  n = 57 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes from the fixed modulus 398 and computed inverse value 57 to every modulus coprime to 7, and changes the conclusion to a right-inverse identity. Its proof requires the general coprime-unit inverse lemma rather than recalling the parent's concrete modular calculation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from the fixed modulus 398 and computed inverse value 57 to arbitrary m coprime to 7, requiring the general ZMod inverse lemma. The parent's finite computation cannot supply this universal right-inverse claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed modulus and proves a right-inverse identity for every coprime modulus, requiring the new coprimality-based lemma 'ZMod.coe_mul_inv_eq_one'. The parent's computation at modulus 398 does not supply this argument, and the child is not merely the already-kept characterization at 398.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1880 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem inverse_seven_is_right_inverse.extracted_1_1 (m : ℕ) (n : ZMod m) (hm : Coprime 7 m) (h : n = 7⁻¹) :  
  7 * n = 1
```

Read back by a model that saw only the Lean. For a natural number m and an element n of $ZMod\ m$, if 7 is coprime to m and n equals the multiplicative inverse of 7 in $ZMod\ m$, then $7n = 1$ in $ZMod\ m$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem inverse_seven_is_right_inverse  
  (m : ℕ)  
  (n : ZMod m)  
  (hm : Nat.Coprime 7 m)  
  (h : n = (7 : ZMod m)⁻¹) :  
  (7 : ZMod m) * n = 1 := by  
  have hinv : (7 : ZMod m) * (7 : ZMod m)⁻¹ = 1 :=  
    ZMod.coe_mul_inv_eq_one 7 hm  
  rw [h]  
  exact hinv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_33__mh__8d11f807

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Quot.sound, fingerprint 6ab90dfa0d6926bb

4.208 208. modular_inverse_unique_398_for_nine

The problem

Show that 177 is the unique residue modulo 398 whose product with 9 is congruent to 1.

Lean statement

```
theorem modular_inverse_unique_398_for_nine (x : ZMod 398) :  
  ((9 : ZMod 398) * x = 1) ↔ x = 177
```

Parent. Find the integer n such that $0 \leq n < 398$ and n is a multiplicative inverse to 7 modulo 398. Show that it is 57.

Parent in Lean

```
theorem mathd_numbertheory_33  
  (n : ZMod 398)  
  (h₁ : n = 7⁻¹) :  
  n = 57 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the modulus equation and the conclusion's shape: it proves a biconditional characterizing the unique solution to $9x = 1$ modulo 398. Its forward direction requires a general cancellation/uniqueness argument, which is not supplied by the parent's direct computation of 7^{-1} .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the multiplier and conclusion form, requiring a bidirectional uniqueness proof and cancellation argument for solutions to $9x=1$ modulo 398. The parent's direct computation of $7^{-1}=57$ does not supply this reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the concrete inverse computation into a biconditional uniqueness theorem for a different residue, requiring a general cancellation argument and a new modular calculation. Remembering the parent's fact about 7^{-1} would not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2488 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_inverse_unique_398_for_nine.extracted_1_1 (x : ZMod 398) : 9 * x = 1 ↔ x = 177
```

Read back by a model that saw only the Lean. For an element x of the ring of integers modulo 398, $9x = 1$ if and only if $x = 177$ in $\mathbb{ZMod}\ 398$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem modular_inverse_unique_398_for_nine (x : ZMod 398) :
  ((9 : ZMod 398) * x = 1) ↔ x = 177 := by
  constructor
  · intro hx
    have h177 : (9 : ZMod 398) * 177 = 1 := by
      decide
    calc
      x = 1 * x := by simp
      _ = ((9 : ZMod 398) * 177) * x := by rw [h177]
      _ = 177 * ((9 : ZMod 398) * x) := by ring
      _ = 177 := by rw [hx]; simp
  · intro hx
    rw [hx]
    decide
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_33__mh__ae6d0d0e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Quot.sound, fingerprint ae6d0d0e8b69e631

4.209 209. fibonacci_recurrence_unique

The problem

Let a be a sequence of natural numbers with $a(1) = 1$, $a(2) = 1$, and $a(n + 2) = a(n + 1) + a(n)$ for every natural number n . Prove that any sequence b of natural numbers satisfying $b(1) = 1$, $b(2) = 1$, and $b(n + 2) = b(n + 1) + b(n)$ for every natural number n must satisfy $b(n) = a(n)$ for every natural number n .

Lean statement

```
theorem fibonacci_recurrence_unique (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1) (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) :
  ∀ b : ℕ → ℕ, b 1 = 1 → b 2 = 1 → (∀ n, b (n + 2) = b (n + 1) + b n) → ∀ n, b n = a n
```

Parent. The Fibonacci sequence is the sequence 1, 1, 2, 3, 5, ... where each term is the sum of the previous two terms. What is the remainder when the 100th term of the sequence is divided by 4? Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_483 (a : ℕ → ℕ) (h₀ : a 1 = 1) (h₁ : a 2 = 1)
  (h₂ : ∀ n, a (n + 2) = a (n + 1) + a n) : a 100 % 4 = 3 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the fixed-index modular conclusion into a uniqueness theorem for the entire recurrence. Its proof must establish equality at every index by strong induction, including deriving equality of the otherwise unstated zeroth terms from both recurrences; the parent's Fibonacci-identification and mod-4 argument supplies none of this.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from computing one fixed Fibonacci residue to proving uniqueness of the entire recurrence sequence. Its proof must compare two arbitrary sequences and derive equality at every index, which the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fixed-index modular conclusion into a uniqueness theorem for the entire recurrence. Its proof must establish equality of two arbitrary sequences at every index, including deriving equality at index 0 and carrying out a fresh strong-induction argument, so the parent's proof does not directly supply it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2052 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fibonacci_recurrence_unique.extracted_1_1 :  $\forall (a : \mathbb{N} \rightarrow \mathbb{N}),$   
   $a\ 1 = 1 \rightarrow$   
     $a\ 2 = 1 \rightarrow$   
       $(\forall (n : \mathbb{N}), a\ (n + 2) = a\ (n + 1) + a\ n) \rightarrow$   
         $\forall (b : \mathbb{N} \rightarrow \mathbb{N}), b\ 1 = 1 \rightarrow b\ 2 = 1 \rightarrow (\forall (n : \mathbb{N}), b\ (n + 2) = b\ (n + 1) + b\ n) \rightarrow \forall (n : \mathbb{N}), b\ n = a\ n$ 
```

Read back by a model that saw only the Lean. *For every function a from the natural numbers to the natural numbers, if $a(1) = 1$, $a(2) = 1$, and for every natural number n , $a(n + 2) = a(n + 1) + a(n)$, then for every function b from the natural numbers to the natural numbers, if $b(1) = 1$, $b(2) = 1$, and for every natural number n , $b(n + 2) = b(n + 1) + b(n)$, then for every natural number n , $b(n) = a(n)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem fibonacci_recurrence_unique (a :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (h₀ : a 1 = 1) (h₁ : a 2 = 1) (h₂ :  $\forall n, a\ (n + 2) = a\ (n + 1) + a\ n$ ) :
   $\forall b : \mathbb{N} \rightarrow \mathbb{N}, b\ 1 = 1 \rightarrow b\ 2 = 1 \rightarrow (\forall n, b\ (n + 2) = b\ (n + 1) + b\ n) \rightarrow \forall n, b\ n = a\ n :=$  by
  intro b hb₀ hb₁ hb₂
  have ha₀ : a 0 = b 0 := by
    have ha₂ := h₂ 0
    have hb₂' := hb₂ 0
    norm_num at ha₂ hb₂'
    omega
  intro n
  induction n using Nat.strong_induction_on with
  | h n ih =>
    by_cases hn0 : n = 0
    · subst n
      exact ha₀.symm
    by_cases hn1 : n = 1
    · subst n
      omega
    have hn2 : 2 ≤ n := by omega
    have ha_rec := h₂ (n - 2)
    have hb_rec := hb₂ (n - 2)
    have hi_prev := ih (n - 1) (by omega)
    have hi_prev2 := ih (n - 2) (by omega)
    rw [show n - 2 + 2 = n by omega, show n - 2 + 1 = n - 1 by omega] at ha_rec hb_rec
    omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_483__mh__1af05244

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 1af05244000ddf3b

4.210 210. bounded_residue_representation_unique

The problem

For all natural numbers q , k , and r , if $r < 6$ and $6q + 5 = 6k + r$, then $r = 5$ and $k = q$.

Lean statement

```
theorem bounded_residue_representation_unique (q k r : ℕ) (hr : r < 6) (hrep : 6 * q + 5 = 6 * k + r) : r = 5 ∧ k = q
```

Parent. For every natural number q at most 254, the number $6q + 5$ has remainder 5 upon division by 6.

Parent in Lean

```
theorem bounded_residue_law (q : ℕ) (hq : q ≤ 254) : (6 * q + 5) % 6 = 5 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a fixed modular residue to uniqueness of both quotient and remainder in a representation, requiring derivation of $r = 5$ and $k = q$ from the equation and $r < 6$. The parent's residue calculation does not supply this uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a residue computation into a uniqueness theorem: from an arbitrary representation with $r < 6$ it must derive both $r = 5$ and $k = q$. The parent's fixed-residue proof supplies neither quotient uniqueness nor the use of the representation hypothesis, so recalling it does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed residue computation into a uniqueness theorem, requiring derivation of both the remainder and quotient from an arbitrary representation. The parent's decomposition proof does not establish these new equalities.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed hq
corpus_dedup	hash	unique against 1873 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_residue_representation_unique.extracted_1_1 (q k r : ℕ) (hr : r < 6) (hrep : 6 * q + 5 = 6 * k + r) :  
  r = 5 ∧ k = q
```

Read back by a model that saw only the Lean. For natural numbers q , k , and r , if r is less than 6 and $6q + 5 = 6k + r$, then $r = 5$ and $k = q$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_residue_representation_unique (q k r : ℕ) (hr : r < 6) (hrep : 6 * q + 5 = 6 * k + r) : r = 5 ∧ k = q := by
  have hdecomp : 6 * q + 5 = 5 + 6 * q := by
    omega
  have hrem : r = 5 := by
    omega
  have hquo : k = q := by
    omega
  exact ⟨hrem, hquo⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_551__me.mh__c475d546

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 6155848cddaca817

4.211 211. nat_mod_six_eq_five_iff

The problem

For every natural number n , show that n leaves remainder 5 upon division by 6 if and only if $n = 6k + 5$ for some natural number k .

Lean statement

```
theorem nat_mod_six_eq_five_iff (n : ℕ) : n % 6 = 5 ↔ ∃ k : ℕ, n = 6 * k + 5 := by
```

Parent. Determine the remainder of 1529 (mod 6). Show that it is 5.

Parent in Lean

```
theorem mathd_numbertheory_551 : 1529 % 6 = 5 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child generalizes a single computed remainder to a full characterization for every natural number, requiring division/modulo decomposition and a converse arithmetic argument. The parent's 'norm_num' proof supplies none of this reasoning.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete remainder calculation to a characterization for every natural number. Its proof requires division-algorithm decomposition and a converse arithmetic argument, neither supplied by the parent's one-step norm_num proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces a single computed remainder with a universal characterization of all natural numbers congruent to 5 modulo 6. Its proof requires quotient-remainder decomposition and bidirectional existential reasoning, none of which is supplied by the parent's direct norm_num calculation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem nat_mod_six_eq_five_iff.extracted_1_1 (n : ℕ) : n % 6 = 5 ↔ ∃ k, n = 6 * k + 5
```

Read back by a model that saw only the Lean. For every natural number n , n modulo 6 equals 5 if and only if there exists a natural number k such that $n = 6k + 5$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem nat_mod_six_eq_five_iff (n : ℕ) : n % 6 = 5 ↔ ∃ k : ℕ, n = 6 * k + 5 := by
  constructor
  · intro h
    have hdecomp : n = 6 * (n / 6) + n % 6 := by
      omega
    have hform : n = 6 * (n / 6) + 5 := by
      omega
    exact ⟨n / 6, hform⟩
  · rintro ⟨k, hk⟩
    subst n
    omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_551__mh__b54fa7c4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 33d3d25490397417

4.212 212. remainder_uniqueness

The problem

Show that if $1529 = 6q + r$ with q, r nonnegative integers and $r < 6$, then $q = 254$ and $r = 5$.

Lean statement

```
theorem remainder_uniqueness : ∀ q r : ℕ, 1529 = 6 * q + r → r < 6 → q = 254 ∧ r = 5 := by
```

Parent. Determine the remainder of 1529 (mod 6). Show that it is 5.

Parent in Lean

```
theorem mathd_numbertheory_551 : 1529 % 6 = 5 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a fixed remainder computation into a uniqueness theorem requiring arbitrary natural-number decomposition hypotheses and deriving both quotient and remainder. The parent's numerical fact and 'norm_num' proof do not supply this quantified arithmetic argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a fixed remainder computation into a uniqueness theorem: from an arbitrary decomposition with a bounded remainder, it must derive both the quotient and remainder. This requires arithmetic reasoning about the hypotheses that the parent's direct normalization proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a concrete remainder computation into a uniqueness characterization of every valid quotient-remainder decomposition. Proving both the quotient and remainder from the hypotheses requires arithmetic reasoning not supplied by the parent's direct normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2432 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem remainder_uniqueness.extracted_1_1 : ∀ (q r : ℕ), 1529 = 6 * q + r → r < 6 → q = 254 ∧ r = 5
```

Read back by a model that saw only the Lean. For all natural numbers q and r , if $1529 = 6q + r$ and $r < 6$, then $q = 254$ and $r = 5$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem remainder_uniqueness :  $\forall q\ r : \mathbb{N}, 1529 = 6 * q + r \rightarrow r < 6 \rightarrow q = 254 \wedge r = 5$  := by
  intro q r hdecomp hrem
  have hq :  $q \leq 254$  := by
    omega
  have hresult :  $q = 254 \wedge r = 5$  := by
    omega
  exact hresult
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_551__mh__f928832a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f928832ab7633afd

4.213 213. power_difference_residue_bounded

The problem

Let t be a natural number and let $r \in \{0, 1, 2, 3, 4, 5\}$. If $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$, determine the residue of $A^{6t+r} - B^{6t+r}$ modulo 7.

Lean statement

```
theorem power_difference_residue_bounded (t r : ℕ) (A B : ℤ) (hr : r ≤ 5) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + r) - B ^ (6 * t + r) ≡ (match r with | 0 => 0 | 1 => 3 | 2 => 4 | 3 => 2 | 4 => 6 | 5 => 5 | _ => 0) [ZMOD 7]
```

Parent. For natural numbers $t \leq 100$, let A and B be integers with $A \equiv 1 \pmod{7}$ and $B \equiv 5 \pmod{7}$. Show that $A^{6t+1} - B^{6t+1} \equiv 3 \pmod{7}$.

Parent in Lean

```
theorem power_difference_residue_law (t : ℕ) (A B : ℤ) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ 3 [ZMOD 7]
```

What it contributes. Reuses the modulus-7 exponent-period argument, transport of both base congruences, and subtraction of congruences, while extending the exponent to all bounded residue classes.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponent from residue 1 to every bounded residue r and requires a six-way case split with distinct power residues modulo 7. The parent's fixed-residue proof does not supply these cases, especially the $r = 0$ treatment and the symbolic computation of $(1^r - 5^r)$ modulo 7.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from the parent's single residue 1 to all bounded residues 0 through 5, requiring a new residue case split and separate power computations. The parent's period argument alone does not supply these residue-specific conclusions.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the exponent from $6t+1$ to all bounded residues $6t+r$ and computes a different residue for each case. The parent's reduction to exponent 1 no longer applies; the proof must establish the period reduction for symbolic r and perform a six-way residue analysis.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2527 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_residue_bounded.extracted_1_1 (t r : ℕ) (A B : ℤ) (hr : r ≤ 5) (hA : A ≡ 1 [ZMOD 7])
(hB : B ≡ 5 [ZMOD 7]) :
A ^ (6 * t + r) - B ^ (6 * t + r) ≡
  match r, hr with
  | 0, hr => 0
  | 1, hr => 3
  | 2, hr => 4
  | 3, hr => 2
  | 4, hr => 6
  | 5, hr => 5
  | x, hr => 0 [ZMOD
7]
```

Read back by a model that saw only the Lean. For natural numbers t and r and integers A and B , if $r \leq 5$, A is congruent to 1 modulo 7, and B is congruent to 5 modulo 7, then $A^{(6t+r)} - B^{(6t+r)}$ is congruent modulo 7 to 0 when $r = 0$, 3 when $r = 1$, 4 when $r = 2$, 2 when $r = 3$, 6 when $r = 4$, 5 when $r = 5$, and 0 in any other matching case.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_residue_bounded (t r : ℕ) (A B : ℤ) (hr : r ≤ 5) (hA : A ≡ 1 [ZMOD 7]) (hB : B ≡ 5 [ZMOD 7]) : A ^ (6 * t + r) -
  B ^ (6 * t + r) ≡ (match r with | 0 => 0 | 1 => 3 | 2 => 4 | 3 => 2 | 4 => 6 | 5 => 5 | _ => 0) [ZMOD 7] := by
  have main_pos : ∀ n : ℕ, 0 < n → A ^ (6 * t + n) - B ^ (6 * t + n) ≡ (1 : ℤ) ^ n - 5 ^ n [ZMOD 7] := by
    intro n hn
    have hperiodA : A ^ (6 * t + n) ≡ A ^ n [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + n) (y := n)
      · norm_num
      · use t
      · omega
      · omega
      · exact hn
    have hperiodB : B ^ (6 * t + n) ≡ B ^ n [ZMOD 7] := by
      apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t + n) (y := n)
      · norm_num
      · use t
      · omega
      · omega
      · exact hn
    have hAred : A ^ (6 * t + n) ≡ (1 : ℤ) ^ n [ZMOD 7] := by
      exact hperiodA.trans (hA.pow n)
    have hBred : B ^ (6 * t + n) ≡ (5 : ℤ) ^ n [ZMOD 7] := by
      exact hperiodB.trans (hB.pow n)
    exact hAred.sub hBred
  have hcases : r = 0 ∨ r = 1 ∨ r = 2 ∨ r = 3 ∨ r = 4 ∨ r = 5 := by
    omega
  rcases hcases with rfl | rfl | rfl | rfl | rfl | rfl
  · by_cases ht : t = 0
    · subst t
      norm_num [Int.ModEq]
    · have hperiodA : A ^ (6 * t) ≡ A ^ 6 [ZMOD 7] := by
        apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t) (y := 6)
        · norm_num
        · use t - 1
        · omega
        · omega
        · norm_num
      have hperiodB : B ^ (6 * t) ≡ B ^ 6 [ZMOD 7] := by
        apply Int.ModEq.pow_eq_pow (p := 7) (x := 6 * t) (y := 6)
        · norm_num
        · use t - 1
        · omega
        · omega
        · norm_num
      have hAred : A ^ (6 * t) ≡ (1 : ℤ) ^ 6 [ZMOD 7] := by
        exact hperiodA.trans (hA.pow 6)
```

```

have hBred : B ^ (6 * t) ≡ (5 : Z) ^ 6 [ZMOD 7] := by
  exact hperiodB.trans (hB.pow 6)
calc
  A ^ (6 * t) - B ^ (6 * t) ≡ (1 : Z) ^ 6 - 5 ^ 6 [ZMOD 7] := hAred.sub hBred
  _ ≡ 0 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 1) - B ^ (6 * t + 1) ≡ (1 : Z) ^ 1 - 5 ^ 1 [ZMOD 7] := main_pos 1 (by omega)
  _ ≡ 3 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 2) - B ^ (6 * t + 2) ≡ (1 : Z) ^ 2 - 5 ^ 2 [ZMOD 7] := main_pos 2 (by omega)
  _ ≡ 4 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 3) - B ^ (6 * t + 3) ≡ (1 : Z) ^ 3 - 5 ^ 3 [ZMOD 7] := main_pos 3 (by omega)
  _ ≡ 2 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 4) - B ^ (6 * t + 4) ≡ (1 : Z) ^ 4 - 5 ^ 4 [ZMOD 7] := main_pos 4 (by omega)
  _ ≡ 6 [ZMOD 7] := by norm_num [Int.ModEq]
· calc
  A ^ (6 * t + 5) - B ^ (6 * t + 5) ≡ (1 : Z) ^ 5 - 5 ^ 5 [ZMOD 7] := main_pos 5 (by omega)
  _ ≡ 5 [ZMOD 7] := by norm_num [Int.ModEq]

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__me.me.mh__39acdc36

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 39acdc36e99f12f5

4.214 214. power_difference_period_six

The problem

Let a and b be integers with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that increasing any natural exponent by 6 preserves the residue of $a^k - b^k$ modulo 7.

Lean statement

```
theorem power_difference_period_six (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7]
```

Parent. Let a and b be integers with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. For $1 \leq k \leq 13$, prove that $a^k - b^k \equiv 3 \pmod{7}$ if and only if $k \equiv 1 \pmod{6}$.

Parent in Lean

```
theorem bounded_variable_base_residue_iff (a b : ℤ) (k : ℕ) (hk : k ≤ 13) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (a ^ k - b ^ k ≡ 3 [ZMOD 7]) ↔ k % 6 = 1 := by
```

What it contributes. Uses the parent's modulus-7 base-residue setup and six-cycle mechanism, replacing its bounded residue classification with a reusable exponent-transport invariant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's bounded case analysis and proves a reusable exponent-period law for every k . Its proof requires modular power transport and the period-six identity, which the parent's residue enumeration does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new unbounded period-six transport law, requiring derivation that both bases have sixth power congruent to 1 and rewriting powers via exponent addition. The parent's bounded case split and residue characterization do not supply this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a new unbounded period-six transport law for the power difference, using multiplicative periodicity of both bases. The parent only handles a bounded residue characterization by finite exponent cases and does not supply this reusable exponent-step argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2410 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem power_difference_period_six.extracted_1_1 (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :  
  a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7]
```

Read back by a model that saw only the Lean. For any integers a and b and any natural number k , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then a raised to the power $k + 6$ minus b raised to the power $k + 6$ is congruent modulo 7 to a raised to the power k minus b raised to the power k .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem power_difference_period_six (a b : ℤ) (k : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :  
  a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7] := by  
  have ha1 : a ≡ (1 : ℤ) [ZMOD 7] := by  
    exact ha.trans (by norm_num [Int.ModEq])  
  have ha6 : a ^ 6 ≡ (1 : ℤ) [ZMOD 7] := by  
    exact (ha1.pow 6).trans (by norm_num [Int.ModEq])  
  have hb6 : b ^ 6 ≡ (1 : ℤ) [ZMOD 7] := by  
    exact (hb.pow 6).trans (by norm_num [Int.ModEq])  
  have hpa : a ^ (k + 6) ≡ a ^ k [ZMOD 7] := by  
    rw [pow_add]  
    calc  
      a ^ k * a ^ 6 ≡ a ^ k * 1 [ZMOD 7] := (Int.ModEq.refl (n := 7) (a ^ k)).mul ha6  
      _ = a ^ k := by ring  
  have hpb : b ^ (k + 6) ≡ b ^ k [ZMOD 7] := by  
    rw [pow_add]  
    calc  
      b ^ k * b ^ 6 ≡ b ^ k * 1 [ZMOD 7] := (Int.ModEq.refl (n := 7) (b ^ k)).mul hb6  
      _ = b ^ k := by ring  
  have hdiff : a ^ (k + 6) - b ^ (k + 6) ≡ a ^ k - b ^ k [ZMOD 7] := hpa.sub hpb  
  exact hdiff
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.mh.mh.mh__bae561eb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint bae561eb51a0712b

4.215 215. bounded_power_difference_mod_seven_iff

The problem

Let integers a and b be congruent to 29 and 5, respectively, modulo 7, and let n be a nonnegative integer with $n \leq 19$. Prove that $a^n - b^n$ leaves remainder 3 modulo 7 if and only if $n \equiv 1 \pmod{6}$.

Lean statement

```
theorem bounded_power_difference_mod_seven_iff (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) (hn2 : n ≤ 19) : (a ^ n - b ^ n ≡ 3 [ZMOD 7]) ↔ n % 6 = 1 := by
```

Parent. Let integers a and b be congruent to 29 and 5, respectively, modulo 7. If $1 \leq n \leq 19$ and $n \equiv 1 \pmod{6}$, show that $a^n - b^n$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem bounded_power_difference_mod_seven (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) (hn2 : n ≤ 19) (hres : n % 6 = 1) : a ^ n - b ^ n ≡ 3 [ZMOD 7] := by
```

What it contributes. Supplies the power-preservation and subtraction checkpoint and the bounded sufficiency direction; the child adds the reverse bounded cycle classification.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child strengthens the parent from a one-way implication to an iff characterization. Its reverse direction requires an exhaustive modulo-7 cycle analysis over the bounded exponent range, which the parent's proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way implication into a bounded iff characterization. Its reverse direction requires ruling out every other exponent in the bounded range, reasoning not supplied by the parent's proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse: from the modular power-difference condition it must classify the bounded exponent and derive $n \bmod 6 = 1$. The parent only supplies the forward implication under that residue hypothesis, so recalling its proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2098 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_power_difference_mod_seven_iff.extracted_1_1 (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7])  
(hb : b ≡ 5 [ZMOD 7]) (hn₂ : n ≤ 19) : a ^ n - b ^ n ≡ 3 [ZMOD 7] ↔ n % 6 = 1
```

Read back by a model that saw only the Lean. For integers a and b and a natural number n , if a is congruent to 29 modulo 7, b is congruent to 5 modulo 7, and n is at most 19, then a raised to the n -th power minus b raised to the n -th power is congruent to 3 modulo 7 if and only if the remainder when n is divided by 6 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem bounded_power_difference_mod_seven_iff (a b : ℤ) (n : ℕ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) (hn₂ : n ≤ 19) : (a ^ n -  
↪ b ^ n ≡ 3 [ZMOD 7]) ↔ n % 6 = 1 := by
  have ha_pow : a ^ n ≡ (29 : ℤ) ^ n [ZMOD 7] := ha.pow n
  have hb_pow : b ^ n ≡ (5 : ℤ) ^ n [ZMOD 7] := hb.pow n
  have hdiff : a ^ n - b ^ n ≡ (29 : ℤ) ^ n - (5 : ℤ) ^ n [ZMOD 7] := ha_pow.sub hb_pow
  have hforward : n % 6 = 1 → a ^ n - b ^ n ≡ 3 [ZMOD 7] := by
    intro hres
    have hn_cases : n = 1 ∨ n = 7 ∨ n = 13 ∨ n = 19 := by
      omega
    rcases hn_cases with h | h | h | h
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
    · subst n
      exact hdiff.trans (by norm_num [Int.ModEq])
  constructor
  · intro htarget
    have hconst : (29 : ℤ) ^ n - (5 : ℤ) ^ n ≡ 3 [ZMOD 7] := hdiff.symm.trans htarget
    interval_cases n <|> norm_num [Int.ModEq] at hconst <|> norm_num
    · exact hforward
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.mh.mh__60d58253

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ce8bc703d9a577cd

4.216 216. numbertheory_728_generalized

The problem

Let a and b be integers with $a \equiv 1 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that $a^{13} - b^{13} \equiv 3 \pmod{7}$.

Lean statement

```
theorem numbertheory_728_generalized (a b : ℤ) (ha : a ≡ 1 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7]
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed integers to arbitrary integers specified only by their residue classes modulo 7. Its proof must transport congruences through exponentiation and subtraction before evaluating the resulting constant, which the parent's direct normalization does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed integers to arbitrary integers satisfying the corresponding congruences. Its proof must use congruence preservation under powers and subtraction before evaluating the constant residue, which the parent's direct norm_num proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed integers to arbitrary integers satisfying modular hypotheses. Its proof must transport congruence through powers and subtraction before evaluating the constant case, which the parent's direct norm_num proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1781 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem numbertheory_728_generalized.extracted_1.1 (a b : ℤ) (ha : a ≡ 1 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :
  a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For integers a and b , if a is congruent to 1 modulo 7 and b is congruent to 5 modulo 7, then a raised to the 13th power minus b raised to the 13th power is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem numbertheory_728_generalized (a b : ℤ) (ha : a ≡ 1 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
  have hpow_a : a ^ 13 ≡ (1 : ℤ) ^ 13 [ZMOD 7] := ha.pow 13
  have hpow_b : b ^ 13 ≡ (5 : ℤ) ^ 13 [ZMOD 7] := hb.pow 13
  have hdiff : a ^ 13 - b ^ 13 ≡ (1 : ℤ) ^ 13 - 5 ^ 13 [ZMOD 7] := hpow_a.sub hpow_b
  have hconst : (1 : ℤ) ^ 13 - 5 ^ 13 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  exact hdiff.trans hconst
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh__b4bc93a1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 694cb5f1a4d6738e

4.217 217. mod_seven_pow_sub_one

The problem

If an integer is congruent to 1 modulo 7, show that its thirteenth power minus 1 is congruent to 0 modulo 7.

Lean statement

```
theorem mod_seven_pow_sub_one (a : ℤ) (h : a ≡ 1 [ZMOD 7]) : a ^ 13 - 1 ≡ 0 [ZMOD 7] := by
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's fixed numerical computation with a quantified modular-arithmetic theorem. Its proof must transport the congruence through exponentiation and subtraction, which the parent's direct 'norm_num' computation does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed numerical congruence to an arbitrary integer congruent to 1 modulo 7 and requires transporting the congruence through the 13th power and subtraction. The parent's direct computation by norm_num does not provide this argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent's fixed numerical congruence to an arbitrary integer congruent to 1 modulo 7, requiring propagation of the hypothesis through exponentiation and subtraction. The parent's direct normalization proof supplies none of this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2362 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mod_seven_pow_sub_one.extracted_1_1 (a : ℤ) (h : a ≡ 1 [ZMOD 7]) : a ^ 13 - 1 ≡ 0 [ZMOD 7]
```

Read back by a model that saw only the Lean. For an integer a , if a is congruent to 1 modulo 7, then a raised to the 13th power minus 1 is congruent to 0 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mod_seven_pow_sub_one (a : ℤ) (h : a ≡ 1 [ZMOD 7]) : a ^ 13 - 1 ≡ 0 [ZMOD 7] := by
  have hpow : a ^ 13 ≡ (1 : ℤ) ^ 13 [ZMOD 7] := h.pow 13
  have hpow' : a ^ 13 ≡ (1 : ℤ) [ZMOD 7] := by
    simpa using hpow
  have hsub : a ^ 13 - 1 ≡ (1 : ℤ) - 1 [ZMOD 7] := hpow'.sub_right 1
  simpa using hsub
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh__c4656633

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c465663375f7b042

4.218 218. modular_power_family

The problem

For every nonnegative integer k with $k \equiv 1 \pmod{6}$, show that $29^k - 5^k \equiv 3 \pmod{7}$.

Lean statement

```
theorem modular_power_family : ∀ k : ℕ, k % 6 = 1 → (29 : ℤ)^k - 5^k ≡ 3 [ZMOD 7] := by
```

Parent. Compute $29^{13} - 5^{13}$ modulo 7. Show that it is 3.

Parent in Lean

```
theorem mathd_numbertheory_728 : 29^13 - 5^13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed exponent 13 to every natural exponent congruent to 1 modulo 6. Proving this requires periodic exponent reasoning modulo 7 for both bases, which the parent's direct norm_num computation does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed exponent to all natural exponents congruent to 1 modulo 6. Its proof must establish exponent-periodicity separately for both bases and transport the difference, which is not supplied by the parent's one-step norm_num proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed exponent 13 to every natural k congruent to 1 modulo 6. Its proof requires periodicity of powers modulo 7 and arithmetic on the exponent, which the parent's direct normalization proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2412 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem modular_power_family.extracted_1_1 : ∀ (k : ℕ), k % 6 = 1 → 29 ^ k - 5 ^ k ≡ 3 [ZMOD 7]
```

Read back by a model that saw only the Lean. For every natural number k , if the remainder when k is divided by 6 is 1, then 29 raised to the k -th power minus 5 raised to the k -th power is congruent to 3 modulo 7.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem modular_power_family :  $\forall k : \mathbb{N}, k \% 6 = 1 \rightarrow (29 : \mathbb{Z})^k - 5^k \equiv 3 \pmod{7}$  := by
  intro k hk
  have h29 :  $(29 : \mathbb{Z})^k \equiv 29^1 \pmod{7}$  := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := k) (y := 1) (n := 29)
    · norm_num
    · omega
    · omega
    · norm_num
  have h5 :  $(5 : \mathbb{Z})^k \equiv 5^1 \pmod{7}$  := by
    apply Int.ModEq.pow_eq_pow (p := 7) (x := k) (y := 1) (n := 5)
    · norm_num
    · omega
    · omega
    · norm_num
  have hdiff :  $(29 : \mathbb{Z})^k - 5^k \equiv 29^1 - 5^1 \pmod{7}$  := h29.sub h5
  calc
     $(29 : \mathbb{Z})^k - 5^k \equiv 29^1 - 5^1 \pmod{7}$  := hdiff
     $\equiv 3 \pmod{7}$  := by norm_num [Int.ModEq]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh__c7a850d6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c7a850d614e0edd2

4.219 219. nat_digits_injective_general

The problem

For any base and natural numbers, two numbers with the same digit representation are equal, and equal numbers have the same digit representation.

Lean statement

```
theorem nat_digits_injective_general
  (b x y : ℕ) :
  Nat.digits b x = Nat.digits b y ↔ x = y := by
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed base-3, fixed-digit reconstruction to an injectivity equivalence for arbitrary bases and numbers. It also requires a new converse argument via `congrArg`, while the parent's final numerical evaluation cannot apply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed base-3 evaluation into a full injectivity theorem for digit representations and adds a reverse implication. The parent's numerical normalization proof does not establish equality of arbitrary numbers from equal digit lists.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from one fixed ternary digit expansion to injectivity of digit representations for arbitrary bases and numbers. Its proof requires a bidirectional equivalence and reconstructs both numbers via `'Nat.ofDigits'`, which the parent's concrete calculation does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2519 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem nat_digits_injective_general.extracted_1_1 (b x y : ℕ) : b.digits x = b.digits y ↔ x = y
```

Read back by a model that saw only the Lean. *For all natural numbers b , x , and y , the sequence of digits of x in base b is equal to the sequence of digits of y in base b if and only if x equals y .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem nat_digits_injective_general
  (b x y : ℕ) :
  Nat.digits b x = Nat.digits b y ↔ x = y := by
  constructor
  · intro h
    have hx : Nat.ofDigits b (Nat.digits b x) = x := Nat.ofDigits.digits b x
    have hy : Nat.ofDigits b (Nat.digits b y) = y := Nat.ofDigits.digits b y
    have hrepr : Nat.ofDigits b (Nat.digits b x) = Nat.ofDigits b (Nat.digits b y) :=
      congrArg (Nat.ofDigits b) h
    have hxy : x = y := by
      calc
        x = Nat.ofDigits b (Nat.digits b x) := hx.symm
        _ = Nat.ofDigits b (Nat.digits b y) := hrepr
        _ = y := hy
    exact hxy
  · intro h
    exact congrArg (Nat.digits b) h
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__mh__8e90b5ce

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8e90b5cecf5f8291

4.220 220. digits_2221_value

The problem

For every natural number b and natural number x whose base- b digits, listed from least significant to most significant, are $[2, 2, 2, 1]$, show that $x = 2 + b(2 + b(2 + b \cdot 1))$.

Lean statement

```
theorem digits_2221_value (b x : ℕ) (h : Nat.digits b x = [2, 2, 2, 1]) : x = 2 + b * (2 + b * (2 + b * 1)) := by
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base: it derives the positional-value formula for every base b from the digit representation, rather than evaluating only the fixed base 3 case. The symbolic expansion must remain valid independently of the parent's numerical computation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the base from 3 to an arbitrary natural b and replaces the concrete value with a symbolic positional-value formula. The parent's specific numerical normalization to 53 does not establish this parameterized identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: instead of computing the fixed-base value 53, it derives the symbolic value for every base b satisfying the digit hypothesis. The parent's numerical normalization no longer applies, so the conclusion is not recoverable by recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1702 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem digits_2221_value.extracted_1_1 (b x : ℕ) (h : b.digits x = [2, 2, 2, 1]) :  
  x = 2 + b * (2 + b * (2 + b * 1))
```

Read back by a model that saw only the Lean. For natural numbers b and x , if the digits of x in base b are $[2, 2, 2, 1]$, then $x = 2 + b * (2 + b * (2 + b * 1))$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem digits_2221_value (b x : ℕ) (h : Nat.digits b x = [2, 2, 2, 1]) :  
  x = 2 + b * (2 + b * (2 + b * 1)) := by  
  have hrecover : Nat.ofDigits b (Nat.digits b x) = x := Nat.ofDigits_digits b x  
  have hdigits : Nat.ofDigits b (Nat.digits b x) = Nat.ofDigits b [2, 2, 2, 1] := by  
    rw [h]  
  have hrepr : x = Nat.ofDigits b [2, 2, 2, 1] := by  
    calc  
      x = Nat.ofDigits b (Nat.digits b x) := hrecover.symm  
      _ = Nat.ofDigits b [2, 2, 2, 1] := hdigits  
  calc  
    x = Nat.ofDigits b [2, 2, 2, 1] := hrepr  
    _ = 2 + b * (2 + b * (2 + b * 1)) := by simp [Nat.ofDigits_cons]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__mh__bcb2847

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 5053f27dbea21c3f

4.221 221. base_three_digits_value

The problem

If the base-three digits of a natural number are $[a, b, c, d]$, prove that the number is $a + 3b + 9c + 27d$.

Lean statement

```
theorem base_three_digits_value
  (x a b c d : ℕ)
  (h : Nat.digits 3 x = [a, b, c, d]) :
  x = a + 3 * b + 9 * c + 27 * d
```

Parent. My father's age is 1222_3 , in base three to represent his three lower limbs – two legs and a cane. How old is he in base ten? Show that it is 53.

Parent in Lean

```
theorem mathd_numbertheory_85
  (x : ℕ)
  (h₀ : Nat.digits 3 x = [2, 2, 2, 1]) :
  x = 53 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed digit tuple to symbolic digits and proves a positional-value identity, requiring symbolic simplification and algebra rather than merely evaluating the parent's constant expression. Its hypotheses are used and the proof is complete.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed digit sequence to symbolic digits and must derive and normalize a symbolic base-3 expansion, requiring algebraic reasoning beyond the parent's final numeric normalization. It is not a decoration or recall wrapper.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the fixed digit list and must symbolically expand its base-3 value for arbitrary digits. The parent's numeral-specific 'norm_num' step no longer applies; the child requires a general 'ofDigits' expansion and algebraic normalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2551 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem base_three_digits_value.extracted_1_1 (x a b c d : ℕ) (h : Nat.digits 3 x = [a, b, c, d]) :  
  x = a + 3 * b + 9 * c + 27 * d
```

Read back by a model that saw only the Lean. For natural numbers x , a , b , c , and d , if the base-3 digit expansion of x is the four-digit list $[a, b, c, d]$, then $x = a + 3b + 9c + 27d$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem base_three_digits_value  
  (x a b c d : ℕ)  
  (h : Nat.digits 3 x = [a, b, c, d]) :  
  x = a + 3 * b + 9 * c + 27 * d := by  
  have hrepr : x = Nat.ofDigits 3 (Nat.digits 3 x) :=  
    (Nat.ofDigits_digits 3 x).symm  
  have hdigits : Nat.ofDigits 3 (Nat.digits 3 x) = Nat.ofDigits 3 [a, b, c, d] := by  
    rw [h]  
  have hvalue : Nat.ofDigits 3 [a, b, c, d] = a + 3 * b + 9 * c + 27 * d := by  
    simp [Nat.ofDigits_cons] <|> ring  
  calc  
    x = Nat.ofDigits 3 (Nat.digits 3 x) := hrepr  
    _ = Nat.ofDigits 3 [a, b, c, d] := hdigits  
    _ = a + 3 * b + 9 * c + 27 * d := hvalue
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__mh__e2008f66

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e2008f661757762c

4.222 222. mod_eleven_succ_mul

The problem

For every natural number n , what is the remainder when $11n + 1$ is divided by 11? Show that it is 1.

Lean statement

```
theorem mod_eleven_succ_mul (n : ℕ) : (11 * n + 1) % 11 = 1 := by
```

Parent. What is the remainder when 2003 is divided by 11? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_961 : 2003 % 11 = 1 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the fixed computation to all natural n , requiring the prover to reason about the divisibility structure of $11*n+1$ rather than evaluate $2003 \% 11$. Its proof uses symbolic rearrangement and arithmetic reasoning, not the parent's numerical proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete computation to all natural n , so the parent's 'norm_num' proof cannot be reused. Proving the symbolic modular identity requires algebraic rearrangement and a general arithmetic argument via 'ring' and 'omega'.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the concrete numeral computation to all $n : \mathbb{N}$, requiring modular arithmetic and normalization of $11 * n + 1$. The parent's norm_num proof supplies no reasoning for this parameterized claim.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1832 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mod_eleven_succ_mul.extracted_1_1 (n : ℕ) : (11 * n + 1) % 11 = 1
```

Read back by a model that saw only the Lean. For every natural number n , the remainder when $11n + 1$ is divided by 11 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mod_eleven_succ_mul (n : ℕ) : (11 * n + 1) % 11 = 1 := by
  have hdecomp : 11 * n + 1 = n * 11 + 1 := by
    ring
  rw [hdecomp]
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_961__mh__71d2132c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint ed6aa6664ef6cfd2

4.223 223. mathd_numbertheory_961_mutation

The problem

Show that adding any multiple of 11 to 2003 leaves remainder 1 when divided by 11.

Lean statement

```
theorem mathd_numbertheory_961_mutation (n : ℕ) : (2003 + 11 * n) % 11 = 1 := by
```

Parent. What is the remainder when 2003 is divided by 11? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_961 : 2003 % 11 = 1 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed remainder to an arbitrary natural multiple of 11. Its proof must combine the computed decomposition of 2003 with symbolic modular arithmetic, which the parent's norm_num proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the dividend by an arbitrary natural number and proves the remainder remains 1, requiring symbolic modular arithmetic beyond the parent's closed computation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed remainder to every natural parameter n. The parent's norm_num proof supplies only the base computation; the child additionally requires symbolic modular reasoning to show adding any multiple of 11 preserves the remainder.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2714 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_961_mutation.extracted_1_1 (n : ℕ) : (2003 + 11 * n) % 11 = 1
```

Read back by a model that saw only the Lean. For every natural number n, the remainder when $2003 + 11n$ is divided by 11 is 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_961_mutation (n : ℕ) : (2003 + 11 * n) % 11 = 1 := by
  have h : 2003 = 11 * 182 + 1 := by
    norm_num
  rw [h]
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_961__mh__df101654

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint df101654ec090306

4.224 224. mathd_numbertheory_99_two_representatives

The problem

If $n < 94$ and $2n \equiv 15 \pmod{47}$, then n is either 31 or 78.

Lean statement

```
theorem mathd_numbertheory_99_two_representatives (n : ℕ) (hbound : n < 94) (h : 2 * n ≡ 15 [MOD 47]) : n = 31 ∨ n = 78
```

Parent. Among the natural numbers less than 47, the congruence $2n \equiv 15 \pmod{47}$ has the unique solution $n = 31$.

Parent in Lean

```
theorem mathd_numbertheory_99_bounded : ∃ n : ℕ, n = 31 ∧ n < 47 ∧ 2 * n ≡ 15 [MOD 47] ∧ ∀ m : ℕ, m < 47 → 2 * m ≡ 15 [MOD 47] → m = n
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The relaxed bound $n < 94$ requires deriving both possible representatives of the residue class, rather than using the parent's $m < 47$ step to identify the residue with the number itself. The proof therefore introduces a genuinely new bounded-case conclusion and reasoning obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the bounded uniqueness claim into a two-representative classification over the larger range $n < 94$. The parent's final step identifying a residue with the number itself no longer applies, and the new representative 78 must be derived from the modular residue and the expanded bound.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the uniqueness claim into a two-representative characterization under the larger bound $n < 94$. The parent's final step relies on $m < 47$ to identify the residue with the number itself, whereas the child must perform a new bounded residue case split yielding 31 or 78.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2712 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_99_two_representatives.extracted_1_1 (n : ℕ) (hbound : n < 94) (h : 2 * n ≡ 15 [MOD 47]) :  
  n = 31 ∨ n = 78
```

Read back by a model that saw only the Lean. For a natural number n less than 94, if $2n$ is congruent to 15 modulo 47, then n equals 31 or n equals 78.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_99_two_representatives (n : ℕ) (hbound : n < 94) (h : 2 * n ≡ 15 [MOD 47]) : n = 31 ∨ n = 78 := by
  have hmod : (2 * n) % 47 = 15 % 47 := h
  have hcanonical : n % 47 = 31 := by
    norm_num at hmod
    omega
  omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_99__mh.mh__3a778cec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 3a778cec4799c418

4.225 225. doubling_cancel_mod_47

The problem

Show that doubling is cancellable modulo 47: if $2n \equiv 2r \pmod{47}$, then $n \equiv r \pmod{47}$.

Lean statement

```
theorem doubling_cancel_mod_47 (n r : ℕ) (h : 2 * n ≡ 2 * r [MOD 47]) : n ≡ r [MOD 47] := by
```

Parent. Solve the congruence $2n \equiv 15 \pmod{47}$, as a residue modulo 47. (Give an answer between 0 and 46.) Show that it is 31.

Parent in Lean

```
theorem mathd_numbertheory_99 (n : ℕ) (h₀ : 2 * n ≡ 15 [MOD 47]) : n ≡ 31 [MOD 47] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the parent from solving one congruence to proving cancellation of the factor 2 modulo 47 for arbitrary n and r . Its proof requires a coprimality-based cancellation lemma, which the parent's direct arithmetic argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed residue equation to cancellation of a factor modulo 47 for arbitrary n and r . Its proof requires establishing coprimality and applying a cancellation theorem, which the parent's omega-based proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed congruence-solving task with a general cancellation theorem modulo 47, requiring a coprimality argument and a modular cancellation lemma. The parent's direct normalization and omega proof do not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2439 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem doubling_cancel_mod_47.extracted_1_1 (n r : ℕ) (h : 2 * n ≡ 2 * r [MOD 47]) : n ≡ r [MOD 47]
```

Read back by a model that saw only the Lean. For natural numbers n and r , if $2n$ is congruent to $2r$ modulo 47, then n is congruent to r modulo 47.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem doubling_cancel_mod_47 (n r : ℕ) (h : 2 * n ≡ 2 * r [MOD 47]) : n ≡ r [MOD 47] := by
  have hcop : Nat.gcd 47 2 = 1 := by
    norm_num
  have hcancel : n ≡ r [MOD 47] :=
    Nat.ModEq.cancel_left_of_coprime hcop h
  exact hcancel
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_99__mh__9367373a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 9367373a98ffd969

4.226 226. choose_translated_square_range_and_unique_min

The problem

For positive integers n and k , let $C = \binom{n}{k}$ and $D = \binom{n-1}{k} + \binom{n-1}{k-1}$. Show that every real y is attained by $(z - C)^2 + D$ exactly when $y \geq D$, and that its minimum D occurs only at $z = C$.

Lean statement

```
theorem choose_translated_square_range_and_unique_min (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  (∀ y : ℝ,
    ((∃ z : ℝ, (z - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) = y) ↔
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) ≤ y) ∧
    ((x - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) =
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
      x = (Nat.choose n k : ℝ)))
```

Parent. For positive integers n and k , show that $\lfloor \left(x - \binom{n}{k}\right)^2 + \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1} \rfloor$ if and only if $x = \binom{n}{k}$, for real x .

Parent in Lean

```
theorem choose_translated_square_level_iff (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  ((x - (Nat.choose n k : ℝ)) ^ 2 + (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
  x = (Nat.choose n k : ℝ)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine characterization of the entire range of the translated square, requiring a new existence proof via nonnegative differences and square roots; this is not supplied by the parent's pointwise minimum equivalence. It also retains the unique-minimizer claim, but the range component independently creates a harder proof obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine range characterization: every value at least the Pascal-sum constant must be attained, requiring a square-root construction, while also proving the minimum's uniqueness. This obligation is not supplied by the parent's fixed-level equivalence proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a single equality characterization to a full range characterization of the translated square, requiring a new square-root existence argument, while also retaining the minimum characterization. The parent's proof does not supply the surjectivity/range construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2465 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem choose_translated_square_range_and_unique_min.extracted_1_1 : ∀ (n k : ℕ),
  0 < n →
  0 < k →
  ∀ (x y : ℝ),
    ((∃ z, (z - ↑(n.choose k)) ^ 2 + ↑((n - 1).choose k + (n - 1).choose (k - 1)) = y) ↔
      ↑((n - 1).choose k + (n - 1).choose (k - 1)) ≤ y) ∧
    ((x - ↑(n.choose k)) ^ 2 + ↑((n - 1).choose k + (n - 1).choose (k - 1)) =
      ↑((n - 1).choose k + (n - 1).choose (k - 1)) ↔
      x = ↑(n.choose k))
```

Read back by a model that saw only the Lean. For all natural numbers n and k with $n > 0$ and $k > 0$, and for all real numbers x and y , both of the following hold: (1) there exists a real number z such that $(z - n.\text{choose } k)^2 + ((n-1).\text{choose } k + (n-1).\text{choose } (k-1)) = y$ if and only if $((n-1).\text{choose } k + (n-1).\text{choose } (k-1)) \leq y$; and (2) $(x - n.\text{choose } k)^2 + ((n-1).\text{choose } k + (n-1).\text{choose } (k-1)) = ((n-1).\text{choose } k + (n-1).\text{choose } (k-1))$ if and only if $x = n.\text{choose } k$, where the natural-number binomial coefficients and sums are regarded as real numbers in the real-valued expressions.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem choose_translated_square_range_and_unique_min (n k : ℕ) (hn : 0 < n) (hk : 0 < k) (x : ℝ) :
  (∀ y : ℝ,
    ((∃ z : ℝ, (z - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) = y) ↔
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) ≤ y) ∧
    (((x - (Nat.choose n k : ℝ)) ^ 2 + ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) =
      ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)) ↔
      x = (Nat.choose n k : ℝ))) := by
  have hpascal : Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
    simpa [Nat.add_comm] using Nat.choose_eq_choose_pred_add hn hk
  have hCD : (Nat.choose n k : ℝ) = ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) := by
    exact_mod_cast hpascal
  intro y
  constructor
  · constructor
    · rintro {z, hz}
      nlinarith [sq_nonneg (z - (Nat.choose n k : ℝ))]
    · intro hy
      have hs : 0 ≤ y - ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ) := by
        linarith
      have hsqrt := Real.sqrt hs
      refine ((Nat.choose n k : ℝ) + Real.sqrt (y - ((Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) : ℕ) : ℝ)), ?_)
      nlinarith
  · constructor
    · intro h
      nlinarith [sq_nonneg (x - (Nat.choose n k : ℝ))]
    · intro hx
      rw [hx]
      simp
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__me.me.mh__425f866a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 425f866a9309b486

4.227 227. complementary_pascal_sum_eq_n_iff

The problem

For integers $n \geq 2$ and $0 < k \leq n$, characterize exactly when $\binom{n-1}{k} + \binom{n-1}{k-1} = n$.

Lean statement

```
theorem complementary_pascal_sum_eq_n_iff (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = n ↔ k = 1 ∨ k = n - 1
```

Parent. For positive integers n and k with $k \leq n$, characterize exactly when $\binom{n-1}{k} + \binom{n-1}{k-1} = 1$.

Parent in Lean

```
theorem complementary_pascal_sum_eq_one_iff (n k : ℕ) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = 1 ↔ k = n
```

What it contributes. Reuses the parent's Pascal expansion and bounded positive-index setup, while replacing the unit-value classification with an inductive strict lower-bound argument for interior coefficients.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child changes the target from a unit binomial value to the nontrivial classification of indices where $\binom{n}{k} = n$. Its proof requires a new interior-binomial lower bound and excludes the endpoint $k=n$, so the parent's choose-equals-one argument cannot solve it.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from the sum being 1 to being n , yielding a new two-endpoint classification. Its proof requires a substantive lower-bound argument for interior binomial coefficients via induction, which is not supplied by the parent's Pascal-collapse and unit-value reasoning.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child transforms the Pascal sum into a binomial coefficient equal to n and proves a new classification: only the two endpoint indices $k = 1$ or $k = n - 1$ qualify. Its induction and interior lower-bound argument require mathematics absent from the parent's unit-value characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2655 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem complementary_pascal_sum_eq_n_iff.extracted_1_1 (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :
  (n - 1).choose k + (n - 1).choose (k - 1) = n ↔ k = 1 ∨ k = n - 1
```

Read back by a model that saw only the Lean. For natural numbers n and k satisfying $2 \leq n$, $0 < k$, and $k \leq n$, the equality $\binom{n-1}{k} + \binom{n-1}{k-1} = n$ holds if and only if $k = 1$ or $k = n - 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem complementary_pascal_sum_eq_n_iff (n k : ℕ) (hn : 2 ≤ n) (hk : 0 < k) (hkn : k ≤ n) :
  Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = n ↔ k = 1 ∨ k = n - 1 := by
  have hpascal : Nat.choose n k = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k := by
    have hn' : n - 1 + 1 = n := by omega
    have hk' : k - 1 + 1 = k := by omega
    calc
      Nat.choose n k = Nat.choose (n - 1 + 1) (k - 1 + 1) := by rw [hn', hk']
      _ = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) (k - 1 + 1) := by
        simpa [Nat.succ_eq_add_one] using Nat.choose_succ_succ (n - 1) (k - 1)
      _ = Nat.choose (n - 1) (k - 1) + Nat.choose (n - 1) k := by rw [hk']
  have hcollapse : Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) = Nat.choose n k := by
    rw [Nat.add_comm, hpascal]
  have hbig : ∀ m : ℕ, 2 ≤ m → ∀ j : ℕ, 2 ≤ j → j ≤ m - 2 → m < Nat.choose m j := by
    intro m
    induction m with
    | zero => intro hm; omega
    | succ m ih =>
      intro hm j hj hjm
      by_cases hm3 : m = 3
      · subst m
        have hj' : j = 2 := by omega
        subst j
        norm_num [Nat.choose]
      · have hm2 : 2 ≤ m := by omega
        have hjpos : 0 < j := by omega
        rw [Nat.choose_succ_left m j hjpos]
        by_cases hj2 : j = 2
        · subst j
          have hprev := ih hm2 2 (by omega) (by omega)
          have hpos : 0 < Nat.choose m 2 := Nat.choose_pos (by omega)
          rw [Nat.choose_one_right]
          omega
        · have hj3 : 3 ≤ j := by omega
          have hprev := ih hm2 (j - 1) (by omega) (by omega)
          have hpos : 0 < Nat.choose m j := Nat.choose_pos (by omega)
          omega
  constructor
  · intro h
    have hval : Nat.choose n k = n := hcollapse.symm.trans h
    by_cases hk1 : k = 1
    · exact Or.inl hk1
    by_cases hkn : k = n
    · subst k
      have hval' : Nat.choose n n = n := hval
      have : 1 = n := by simpa using hval'
      omega
    by_cases hkn1 : k = n - 1
    · exact Or.inr hkn1
    have hk2 : 2 ≤ k := by omega
    have hkupper : k ≤ n - 2 := by omega
    have hgt : n < Nat.choose n k := hbig n hn k hk2 hkupper
    omega
  · intro h
    rcases h with rfl | rfl
    · rw [hcollapse]
```

```

simp
· rw [hcollapse]
have hs : Nat.choose n (n - 1) = Nat.choose n 1 := by
  simpa using (Nat.choose_symm (n := n) (k := 1) (by omega))
rw [hs]
simp

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms.me.me.mh__8477755b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8477755ba5deb995

4.228 228. hockey_stick_binomial

The problem

For natural numbers n and k , show that the sum $\sum_{i=0}^n \binom{i}{k}$ equals $\binom{n+1}{k+1}$.

Lean statement

```
theorem hockey_stick_binomial (n k : ℕ) :
  (∑ i ∈ Finset.range (n + 1), Nat.choose i k) = Nat.choose (n + 1) (k + 1)
```

Parent. For predecessor indices p and q , Pascal's recursion expresses the explicitly named value $r = \text{choose } (p + 1) (q + 1)$ as $\text{choose } p (q + 1) + \text{choose } p q$.

Parent in Lean

```
theorem pascal_predecessor_surface (p q r : ℕ) (hr : r = Nat.choose (p + 1) (q + 1)) : r = Nat.choose p (q + 1) + Nat.choose p q
```

What it contributes. Supplied the Pascal recursion checkpoint, adapted from the named-value predecessor identity into the induction step of the variable-index summation theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the local Pascal recurrence into a variable-index finite-sum identity. Its induction must decompose the sum and combine the induction hypothesis with Pascal recursion, which the parent's single-step equality does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes Pascal's predecessor identity to a variable-index hockey-stick sum. Its inductive proof requires a new summation decomposition and repeatedly consumes Pascal recursion, so recalling the parent does not supply the argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded generalization: proving the hockey-stick sum requires induction, finite-sum decomposition, and Pascal recursion at each step. The parent supplies only a single local Pascal identity and does not provide the induction or summation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2056 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem hockey_stick_binomial.extracted_1_1 (n k : ℕ) :  
  ∑ i ∈ Finset.range (n + 1), i.choose k = (n + 1).choose (k + 1)
```

Read back by a model that saw only the Lean. For all natural numbers n and k , the sum of the binomial coefficients $\binom{n}{i}$ for integers i from 0 through n equals $\binom{n+1}{k+1}$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem hockey_stick_binomial (n k : ℕ) :  
  (∑ i ∈ Finset.range (n + 1), Nat.choose i k) = Nat.choose (n + 1) (k + 1) := by  
  induction n generalizing k with  
  | zero =>  
    cases k with  
    | zero => simp  
    | succ k => simp [Nat.choose]  
  | succ n ih =>  
    have hsum :  
      (∑ i ∈ Finset.range (n + 1 + 1), Nat.choose i k) =  
        (∑ i ∈ Finset.range (n + 1), Nat.choose i k) + Nat.choose (n + 1) k := by  
      rw [Finset.sum_range_succ]  
    have hpascal :  
      Nat.choose (n + 1 + 1) (k + 1) =  
        Nat.choose (n + 1) (k + 1) + Nat.choose (n + 1) k := by  
      simpa [Nat.add_assoc, Nat.add_comm, Nat.add_left_comm] using  
        (Nat.choose_succ_succ (n + 1) k)  
    calc  
      (∑ i ∈ Finset.range (Nat.succ n + 1), Nat.choose i k) =  
        (∑ i ∈ Finset.range (n + 1 + 1), Nat.choose i k) := by  
          congr 2 <|> omega  
      _ = (∑ i ∈ Finset.range (n + 1), Nat.choose i k) + Nat.choose (n + 1) k := hsum  
      _ = Nat.choose (n + 1) (k + 1) + Nat.choose (n + 1) k := by rw [ih k]  
      _ = Nat.choose (n + 1 + 1) (k + 1) := hpascal.symm  
      _ = Nat.choose (Nat.succ n + 1) (k + 1) := by congr 2 <|> omega
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms.mh__04540a98

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 04540a9858d9117f

4.229 229. translated_power_mul_quotient_iff_modEq_zero

The problem

Let p be prime, let n, a, k be integers, and let $e > 0$. Prove that there is a unique integer q satisfying $n^e a = pq$ if and only if $(n + kp)^e a \equiv 0 \pmod{p}$.

Lean statement

```
theorem translated_power_mul_quotient_iff_modEq_zero
  (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p) :
  (∃! q : ℤ, n ^ e * a = (↑p : ℤ) * q) ↔
  (n + k * (↑p : ℤ)) ^ e * a ≡ 0 [ZMOD (↑p : ℤ)]
```

Parent. Let p be prime, let n, a, k be integers with $p \nmid a$, and let $e > 0$. Prove that there is a unique integer q satisfying $n^e a = pq$ if and only if $(n + kp)^e a \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem unique_power_mul_quotient_iff_translated_modEq_zero
  (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  (hfactor : ¬ ((↑p : ℤ) ∣ a)) :
  (∃! q : ℤ, n ^ e * a = (↑p : ℤ) * q) ↔
  (n + k * (↑p : ℤ)) ^ e * a ≡ 0 [ZMOD (↑p : ℤ)]
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens the hypotheses by dropping both positivity of e and the nondivisibility condition on a . Its proof uses direct modular transport and uniqueness from $p \neq 0$, whereas the parent's argument depends essentially on those hypotheses to cancel a and extract $p \mid n$.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the theorem by removing both the positivity assumption on the exponent and the nondivisibility assumption on a . Its proof uses direct congruence transport and uniqueness from the nonzero modulus, avoiding the parent's cancellation argument entirely.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes both positivity of the exponent and the nondivisibility hypothesis, and its proof must replace the parent's cancellation argument with congruence transport plus direct uniqueness from the nonzero modulus. This is a genuine generalization that cannot be obtained by simply recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2679 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem translated_power_mul_quotient_iff_modEq_zero.extracted_1_1 (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p)
  (he_pos : 0 < e) : (∃! q, n ^ e * a = ↑p * q) ↔ (n + k * ↑p) ^ e * a ≡ 0 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For integers n , a , and k , natural numbers p and e , if p is prime and e is positive, then there exists a unique integer q such that $n^e a = p q$ if and only if $(n + k p)^e a$ is congruent to 0 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem translated_power_mul_quotient_iff_modEq_zero
  (n a k : ℤ) (p e : ℕ) (hp : Nat.Prime p) :
  (∃! q : ℤ, n ^ e * a = (↑p : ℤ) * q) ↔
  (n + k * (↑p : ℤ)) ^ e * a ≡ 0 [ZMOD (↑p : ℤ)] := by
  have hpne : (↑p : ℤ) ≠ 0 := by
    exact_mod_cast hp.ne_zero
  have hmod_zero (x : ℤ) :
    x ≡ 0 [ZMOD (↑p : ℤ)] ↔ (↑p : ℤ) ∣ x := by
    simpa using (Int.modEq_zero_iff_dvd (a := x) (n := (↑p : ℤ)))
  have hbase : n + k * (↑p : ℤ) ≡ n [ZMOD (↑p : ℤ)] := by
    apply (Int.modEq_iff_dvd).2
    refine ⟨-k, ?_⟩
    ring
  have htransport :
    (n + k * (↑p : ℤ)) ^ e * a ≡ n ^ e * a [ZMOD (↑p : ℤ)] :=
    (hbase.pow e).mul_right a
  constructor
  · rintro ⟨q, hq, _⟩
    have hdiv : (↑p : ℤ) ∣ n ^ e * a := ⟨q, hq⟩
    have hnzero : n ^ e * a ≡ 0 [ZMOD (↑p : ℤ)] := (hmod_zero _).2 hdiv
    exact htransport.trans hnzero
  · intro htranslated
    have hnzero : n ^ e * a ≡ 0 [ZMOD (↑p : ℤ)] :=
      htransport.symm.trans htranslated
    have hdiv : (↑p : ℤ) ∣ n ^ e * a := (hmod_zero _).1 hnzero
    rcases hdiv with ⟨q, hq⟩
    refine ⟨q, hq, ?_⟩
    intro y hy
    apply mul_left_cancel₀ hpne
    calc
      (↑p : ℤ) * y = n ^ e * a := hy.symm
      _ = (↑p : ℤ) * q := hq
```

Operator. mutation / mutation_hard **Lineage depth.** 4 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh.mh.me.mh__7fa8c862

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ee4ea013f9536d7c

4.230 230. prime_dvd_iff_coupled_product_bounded_power_modEq_zero

The problem

For a prime number p , integers n, k , and a positive exponent $e \leq B$, show that p divides n if and only if $(n(n + kp))^e \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n * (n + k * (↑p : ℤ))) ^ e ≡ 0 [ZMOD p]
```

Parent. For a prime number p , integers n and k , and a positive exponent $e \leq B$, show that p divides n if and only if $(n + kp)^e \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem prime_dvd_iff_translated_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n + k * (↑p : ℤ)) ^ e ≡ 0 [ZMOD p]
```

What it contributes. Supplied the prime-power divisibility and translated-factor cancellation pattern, generalized here to the coupled product $n*(n+k*p)$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is not a recall of the parent's translated-power argument: its reverse direction must factor prime divisibility of the coupled product, then handle either factor and cancel the translated term. This introduces genuinely new prime-factor reasoning while preserving essential hypotheses.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The coupled product changes the reverse direction substantially: from divisibility of its power, the proof must use primality to split divisibility across the two factors, then handle both branches before cancellation. This is not recoverable by simply recalling the parent's translated-power argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The coupled product introduces a genuinely new zero-product argument: from prime divisibility of the product, the proof must split divisibility across its two factors and then eliminate the translated factor. The parent's direct translation-and-power argument does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2630 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero.extracted_1_1 (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p)
  (he_pos : 0 < e) (he_bound : e ≤ B) : ↑p ∣ n ↔ (n * (n + k * ↑p)) ^ e ≡ 0 [ZMOD ↑p]
```

Read back by a model that saw only the Lean. For integers n and k and natural numbers p , e , and B , if p is prime, e is positive, and $e \leq B$, then the integer p divides n if and only if the e -th power of $n(n + kp)$ is congruent to 0 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem prime_dvd_iff_coupled_product_bounded_power_modEq_zero
  (n k : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ (n * (n + k * (↑p : ℤ))) ^ e ≡ 0 [ZMOD p] := by
  rw [Int.modEq_zero_iff_dvd]
  have he_ne : e ≠ 0 := Nat.ne_of_gt he_pos
  have hkp : (↑p : ℤ) ∣ k * (↑p : ℤ) := by
    exact dvd_mul_of_dvd_right (dvd_refl (↑p : ℤ)) k
  constructor
  · intro hdiv
    have hproduct : (↑p : ℤ) ∣ n * (n + k * (↑p : ℤ)) := by
      exact dvd_mul_of_dvd_left hdiv _
    exact dvd_pow hproduct he_ne
  · intro hpower
    have hproduct : (↑p : ℤ) ∣ n * (n + k * (↑p : ℤ)) := by
      exact Int.Prime.dvd_pow' hp hpower
    have hproduct_nat : p ∣ (n * (n + k * (↑p : ℤ))).natAbs :=
      (Int.natCast_dvd).mpr hproduct
    rw [Int.natAbs_mul] at hproduct_nat
    rcases hp.dvd_mul.mpr hproduct_nat with hdiv | htranslated
    · exact (Int.natCast_dvd).mpr hdiv
    · have htranslated_int : (↑p : ℤ) ∣ n + k * (↑p : ℤ) :=
      (Int.natCast_dvd).mpr htranslated
      have hcancel : (↑p : ℤ) ∣
        (n + k * (↑p : ℤ)) - k * (↑p : ℤ) := by
        exact dvd_sub htranslated_int hkp
      convert hcancel using 1 <|> ring
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh.mh__0846e862

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 00c7dccccd0b0df4f

4.231 231. prime_dvd_iff_power_mul_factor_modEq_zero

The problem

Let p be prime, let n and a be integers with $p \nmid a$, and let $e > 0$. Show that $p \mid n$ if and only if $n^e a \equiv 0 \pmod{p}$.

Lean statement

```
theorem prime_dvd_iff_power_mul_factor_modEq_zero
  (n a : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  (hfactor : ¬ ((↑p : ℤ) ∣ a)) :
  (↑p : ℤ) ∣ n ↔ n ^ e * a ≡ 0 [ZMOD p]
```

Parent. For a prime number p , an integer n , and a positive odd exponent e with $e \leq B$, show that p divides n if and only if $n^e \equiv 0 \pmod{p}$.

Parent in Lean

```
theorem prime_dvd_iff_bounded_odd_power_modEq_zero
  (n : ℤ) (p e B : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  :
  (↑p : ℤ) ∣ n ↔ n ^ e ≡ 0 [ZMOD p]
```

What it contributes. Reused the parent's prime-modulus and positive-power framework, while adding a product-divisibility split and an explicit nondivisibility elimination checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The reverse implication requires genuinely new reasoning: extracting divisibility of n^e from divisibility of $n^e \cdot a$ using primality and the essential nondivisibility hypothesis on a , then descending from a power to n . The parent's proof supplies only the power-divisibility equivalence and cannot establish this product cancellation step.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely requires using the nondivisibility of the auxiliary factor and Euclid's lemma to infer that p divides n^e from p dividing $n^e \cdot a$. This factor-cancellation argument is absent from the parent proof, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new reverse-direction argument: from divisibility of $n^e \cdot a$, it must split the prime divisor across the factors and use the nondivisibility hypothesis on a before extracting divisibility of n^e . This factor-cancellation reasoning is absent from the parent proof and the added hypothesis is essential.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2640 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_dvd_iff_power_mul_factor_modEq_zero.extracted_1_1 (n a : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
(hfactor : ¬(↑p ∣ a) : ↑p ∣ n ↔ n ^ e * a ≡ 0 [ZMOD ↑p])
```

Read back by a model that saw only the Lean. For integers n and a , and natural numbers p and e , if p is prime, e is positive, and p does not divide a , then p divides n if and only if n raised to the e -th power times a is congruent to 0 modulo p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem prime_dvd_iff_power_mul_factor_modEq_zero
  (n a : ℤ) (p e : ℕ) (hp : Nat.Prime p) (he_pos : 0 < e)
  (hfactor : ¬((↑p : ℤ) ∣ a)) :
  (↑p : ℤ) ∣ n ↔ n ^ e * a ≡ 0 [ZMOD p] := by
  rw [Int.modEq_zero_iff_dvd]
  have he_ne : e ≠ 0 := Nat.ne_of_gt he_pos
  constructor
  · intro hn
    have hpow : (↑p : ℤ) ∣ n ^ e := dvd_pow hn he_ne
    exact dvd_mul_of_dvd_left hpow a
  · intro h
    have hprod : p ∣ (n ^ e).natAbs * a.natAbs := by
      simpa [Int.natAbs_mul] using (Int.natCast_dvd.mp h)
    rcases hp.dvd_mul.mp hprod with hpow | ha
    · have hpow_int : (↑p : ℤ) ∣ n ^ e := by
        apply Int.natCast_dvd.mpr
        simpa [Int.natAbs_pow] using hpow
      exact Int.Prime.dvd_pow' hp hpow_int
    · exfalso
      apply hfactor
      exact Int.natCast_dvd.mpr ha
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__me.mh__3044da59

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3044da599b1e48ce

4.232 232. cubic_quotient_expansion

The problem

Let x be an integer congruent to 5 modulo 7. Show that there is an integer q such that $x = 7q + 5$ and

$$x^3 + 1 = 7(49q^3 + 105q^2 + 75q + 18).$$

Lean statement

```
theorem cubic_quotient_expansion (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ∃ q : ℤ, x = 7 * q + 5 ∧ x ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 * q + 18)
```

Parent. Let x be an integer congruent to 5 modulo 7. For $0 \leq k \leq 3$, show that 7 divides $x^k + 1$ exactly when $k = 3$.

Parent in Lean

```
theorem dvd_add_one_iff_exp_eq_three (x : ℤ) (h : x ≡ 5 [ZMOD 7]) (k : ℕ) (hk : k ≤ 3) : (7 : ℤ) ∣ x ^ k + 1 ↔ k = 3
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to an explicit quotient representation together with a cubic polynomial identity. Proving the quotient decomposition and symbolic expansion requires reasoning absent from the parent's bounded exponent case split.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the bounded divisibility equivalence into an existential quotient representation and an explicit cubic polynomial identity. Its proof requires extracting an integer quotient from the modular hypothesis and expanding the cubic, neither of which is supplied by the parent's finite exponent case split.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the bounded divisibility characterization into an existential quotient decomposition plus an explicit cubic polynomial identity. Its proof requires extracting an integer quotient from the congruence and carrying out symbolic expansion, neither of which is supplied by the parent's finite residue check.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2448 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_quotient_expansion.extracted_1_1 (x : ℤ) (h : x ≡ 5 [ZMOD 7]) :
  ∃ q, x = 7 * q + 5 ∧ x ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 * q + 18)
```

Read back by a model that saw only the Lean. For every integer x such that x is congruent to 5 modulo 7, there exists an integer q such that $x = 7q + 5$ and $x^3 + 1 = 7(49q^3 + 105q^2 + 75q + 18)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem cubic_quotient_expansion (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ∃ q : ℤ, x = 7 * q + 5 ∧ x ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 *
  q + 18) := by
  have hdiv : (7 : ℤ) ∣ 5 - x := by
    rw [← Int.modEq_iff_dvd]
    exact h
  have hdecomp : ∃ q : ℤ, x = 7 * q + 5 := by
    rcases hdiv with ⟨q, hq⟩
    refine ⟨-q, ?_⟩
    nlinarith
  rcases hdecomp with ⟨q, hq⟩
  have hexpand : x ^ 3 + 1 = (7 * q + 5) ^ 3 + 1 := by
    rw [hq]
  have hpoly : (7 * q + 5) ^ 3 + 1 = 7 * (49 * q ^ 3 + 105 * q ^ 2 + 75 * q + 18) := by
    ring
  exact ⟨q, hq, hexpand.trans hpoly⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.mh.me.mh__c974a427

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c974a42724c65ef1

4.233 233. not_dvd_pow_fifteen_of_modEq_five

The problem

Let x be an integer congruent to 5 modulo 7. Show that 7 does not divide x^{15} .

Lean statement

```
theorem not_dvd_pow_fifteen_of_modEq_five (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ (7 : ℤ) ∣ x ^ 15
```

Parent. Show that 7 does not divide 5^{15} .

Parent in Lean

```
theorem seven_not_dvd_five_pow_fifteen : ¬ (7 : ℤ) ∣ (5 : ℤ) ^ 15
```

What it contributes. Used the parent's divisibility-to-modular-obstruction strategy, generalized from the fixed base 5\$ to an arbitrary integer congruent to 5\$ modulo 7\$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the base and requires transporting a modular congruence through the fifteenth power, then deriving a contradiction from the resulting residue. The parent's fixed-base square-criterion argument does not supply this hypothesis-dependent reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base and requires transporting divisibility through the congruence hypothesis before evaluating the fixed residue. This is a different argument from the parent's square criterion and does not overlap with the retained exponent-21 theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the base: it must transport the congruence through the fifteenth power and derive a contradiction from the resulting residue. This reasoning is not supplied by the parent's fixed-base square argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2428 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem not_dvd_pow_fifteen_of_modEq_five.extracted_1_1 (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ 7 ∣ x ^ 15
```

Read back by a model that saw only the Lean. For an integer x , if x is congruent to 5 modulo 7, then 7 does not divide

x raised to the 15th power.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem not_dvd_pow_fifteen_of_modEq_five (x : ℤ) (h : x ≡ 5 [ZMOD 7]) : ¬ (7 : ℤ) ∣ x ^ 15 := by
  intro hdiv
  have hzero : x ^ 15 ≡ 0 [ZMOD 7] := by
    rw [Int.modEq_zero_iff_dvd]
    exact hdiv
  have hpow : x ^ 15 ≡ (5 : ℤ) ^ 15 [ZMOD 7] := by
    exact h.pow 15
  have htransport : (0 : ℤ) ≡ (5 : ℤ) ^ 15 [ZMOD 7] := by
    exact hzero.symm.trans hpow
  norm_num [Int.ModEq] at htransport
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_prmdvsneqnsqmodpeq0__x__mathd_numbertheory_229__xe.mh__c2f2c6a1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c2f2c6a18458128a

4.234 234. square_mod_three_zero_iff_dvd

The problem

For an integer a , its square is congruent to 0 modulo 3 exactly when a is divisible by 3.

Lean statement

```
theorem square_mod_three_zero_iff_dvd (a : Z) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a
```

Parent. Show that the square of any integer is congruent to 0 or 1 modulo 3.

Parent in Lean

```
theorem numbertheory_sqmod3in01d (a : Z) : a ^ 2 ≡ 0 [ZMOD 3] ∨ a ^ 2 ≡ 1 [ZMOD 3] := by
```

What it contributes. Uses the parent's modulo-3 remainder classification as the internal checkpoint, while changing the conclusion to an exact divisibility characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the disjunctive square-residue result into an exact divisibility characterization. Its reverse direction requires a divisibility-of-squares argument, and its forward direction must use the residue cases to exclude 1 and 2, which the parent does not establish.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's residue classification into an exact iff characterization, requiring both extraction of divisibility from the zero residue and a separate divisibility-of-squares converse. The parent's proof does not supply this equivalence.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a residue classification into an exact divisibility equivalence. Although the parent's three-residue split helps one direction, the converse requires proving that divisibility of the base yields divisibility of its square, so the parent proof does not suffice.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2665 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_mod_three_zero_iff_dvd.extracted_1_1 (a : Z) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a
```

Read back by a model that saw only the Lean. For every integer a , the square of a is congruent to 0 modulo 3 if and

only if 3 divides a .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_mod_three_zero_iff_dvd (a : Z) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a := by
  constructor
  · intro hsq
    have hres : a % 3 = 0 ∨ a % 3 = 1 ∨ a % 3 = 2 := by
      omega
    rcases hres with hres | hres | hres
    · refine ⟨a / 3, ?_⟩
      omega
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsq
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsq
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsq
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsq
  · intro hdiv
    have hsqdiv : 3 ∣ a ^ 2 := by
      rcases hdiv with ⟨k, hk⟩
      refine ⟨3 * k ^ 2, ?_⟩
      rw [hk]
      ring
    exact Dvd.dvd.modEq_zero_int hsqdiv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__mh__f7f36d97

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint f7f36d9711eb1b4a

4.235 235. sum_succ_mul_choose_range

The problem

For every natural number n , show that $\sum_{k=0}^n (k+1) \binom{n}{k} = n \cdot 2^{n-1} + 2^n$.

Lean statement

```
theorem sum_succ_mul_choose_range (n : ℕ) :
  Finset.sum (Finset.range (n + 1)) (fun k => (k + 1) * Nat.choose n k) =
    n * 2 ^ (n - 1) + 2 ^ n
```

Parent. Show that for positive integers n and k , if $k \leq n$, then $\sum_{k=1}^n (k * C_n^k) = n * 2^{n-1}$.

Parent in Lean

```
theorem numbertheory_summulnckeqnmul2pownm1 (n : ℕ) (h₀ : 0 < n) :
  (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child changes the weighted sum by introducing a successor factor and requires a separate binomial-count identity, then combines both sums via distributivity. The parent's weighted-sum result helps but does not supply the new counting argument or the resulting shifted formula.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the summand from $k \cdot \text{choose}(n, k)$ to $(k+1) \cdot \text{choose}(n, k)$, requiring a separate binomial-sum identity in addition to the parent's weighted-sum result. Its proof therefore introduces genuinely new mathematics rather than merely reusing the parent's conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires an additional independent binomial identity, $\sum \text{choose } n \ k = 2^n$, and combines it with the parent's weighted-sum identity. Remembering the parent only supplies the weighted component and does not prove the shifted-sum formula.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	removed h_0
corpus_dedup	hash	unique against 1818 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem sum_succ_mul_choose_range.extracted_1_1 (n : ℕ) :
  ∑ k ∈ Finset.range (n + 1), (k + 1) * n.choose k = n * 2 ^ (n - 1) + 2 ^ n
```

Read back by a model that saw only the Lean. For every natural number n , the sum from $k = 0$ to n of $(k + 1)$ times the binomial coefficient " n choose k " equals n times 2^{n-1} plus 2^n .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem sum_succ_mul_choose_range (n : ℕ) :
  Finset.sum (Finset.range (n + 1)) (fun k => (k + 1) * Nat.choose n k) =
    n * 2 ^ (n - 1) + 2 ^ n := by
  have h_weight :
    Finset.sum (Finset.range (n + 1)) (fun k => k * Nat.choose n k) =
      n * 2 ^ (n - 1) := by
    simp using Nat.sum_range_mul_choose n
  have h_count :
    Finset.sum (Finset.range (n + 1)) (fun k => Nat.choose n k) = 2 ^ n := by
    simp using Nat.sum_range_choose n
  calc
    Finset.sum (Finset.range (n + 1)) (fun k => (k + 1) * Nat.choose n k) =
      Finset.sum (Finset.range (n + 1))
        (fun k => k * Nat.choose n k + Nat.choose n k) := by
        apply Finset.sum_congr rfl
        intro k hk
        simp [Nat.add_mul]
    _ = Finset.sum (Finset.range (n + 1)) (fun k => k * Nat.choose n k) +
      Finset.sum (Finset.range (n + 1)) (fun k => Nat.choose n k) := by
      rw [Finset.sum_add_distrib]
    _ = n * 2 ^ (n - 1) + 2 ^ n := by
      rw [h_weight, h_count]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sumkmulnckeqnmul2pownm1__mh__145c56eb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d0598d47d18b5515

4.236 236. affine_weighted_binomial_sum

The problem

For positive integers n and natural numbers a, b , show that $\sum_{k=0}^n (ak+b) \binom{n}{k} = a \cdot 2^n + b \cdot 2^n$.

Lean statement

```
theorem affine_weighted_binomial_sum (n a b : ℕ) :
  (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
    a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1)
```

Parent. Show that for positive integers n and k , if $k \leq n$, then $\sum_{k=1}^n (k * C_n^k) = n * 2^{n-1}$.

Parent in Lean

```
theorem numbertheory_sumkmulnckeqnmul2pownm1 (n : ℕ) (h₀ : 0 < n) :
  (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the weighted sum from k to the affine weight $a \cdot k + b$, requiring a new unweighted binomial-sum identity and decomposition of the summand. Its proof cannot be obtained by simply recalling the parent's weighted identity.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The affine weighting requires a new decomposition and an independent binomial-sum identity, in addition to the parent's weighted identity. The parameters a and b remain symbolic, so the result is not a decorative restatement or parent recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the summand and combines two distinct binomial identities: the weighted identity from the parent and a new unweighted sum identity. Proving the affine decomposition and the excluded-zero binomial sum requires mathematical obligations not supplied by merely recalling the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2612 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_weighted_binomial_sum.extracted_1_1 (n a b : ℕ) (h₀ : 0 < n) :
  ∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose k = a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1)
```

Read back by a model that saw only the Lean. For natural numbers n , a , and b with $n > 0$, the sum over all natural numbers k satisfying $1 \leq k \leq n$ of $(a \cdot k + b)$ times the binomial coefficient $\binom{n}{k}$ equals $a \cdot (n \cdot 2^{n-1}) + b \cdot (2^n - 1)$, where subtraction and all operations are taken in the natural numbers.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open BigOperators Real Nat Topology Rat

theorem affine_weighted_binomial_sum (n a b : ℕ) :
  (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
    a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1) := by
  have hset : Finset.range (n + 1) = insert 0 (Finset.Icc 1 n) := by
    ext x
    simp only [Finset.mem_range, Finset.mem_insert, Finset.mem_Icc]
  omega
  have hnot : 0 ∉ Finset.Icc 1 n := by
    simp
  have hweighted :
    (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) = n * 2 ^ (n - 1) := by
    calc
      (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) =
        ∑ k ∈ insert 0 (Finset.Icc 1 n), k * Nat.choose n k := by
          rw [Finset.sum_insert hnot]
        simp
      _ = ∑ k ∈ Finset.range (n + 1), k * Nat.choose n k := by
          rw [hset]
      _ = n * 2 ^ (n - 1) := Nat.sum_range_mul_choose n
  have hunweighted :
    (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) = 2 ^ n - 1 := by
    have htotal :
      (∑ k ∈ Finset.range (n + 1), Nat.choose n k) =
        1 + ∑ k ∈ Finset.Icc 1 n, Nat.choose n k := by
        calc
          (∑ k ∈ Finset.range (n + 1), Nat.choose n k) =
            ∑ k ∈ insert 0 (Finset.Icc 1 n), Nat.choose n k := by
              rw [hset]
            _ = 1 + ∑ k ∈ Finset.Icc 1 n, Nat.choose n k := by
              rw [Finset.sum_insert hnot]
            simp
    have hpow : 0 < 2 ^ n := by positivity
    rw [Nat.sum_range_choose n] at htotal
    omega
  have hdecomp :
    (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
      a * (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) +
      b * (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) := by
    calc
      (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
        ∑ k ∈ Finset.Icc 1 n,
          (a * (k * Nat.choose n k) + b * Nat.choose n k) := by
          apply Finset.sum_congr rfl
          intro k hk
          ring
      _ = (∑ k ∈ Finset.Icc 1 n, a * (k * Nat.choose n k)) +
          (∑ k ∈ Finset.Icc 1 n, b * Nat.choose n k) := by
          rw [Finset.sum_add_distrib]
      _ = a * (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) +
          b * (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) := by
          rw [Finset.mul_sum, Finset.mul_sum]
    calc
      (∑ k ∈ Finset.Icc 1 n, (a * k + b) * Nat.choose n k) =
        a * (∑ k ∈ Finset.Icc 1 n, k * Nat.choose n k) +
```

```

      b * (∑ k ∈ Finset.Icc 1 n, Nat.choose n k) := hdecomp
_ = a * (n * 2 ^ (n - 1)) + b * (2 ^ n - 1) := by
  rw [hweighted, hunweighted]

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_sumkmu1nckeqnmul2pownm1__mh__cc19245c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2ee3e7fdd68cca0c

4.237 237. aime_1984_p15_x_squared_value

The problem

Suppose real numbers x, y, z, w satisfy, for each $n \in \{2, 4, 6, 8\}$, the equation $\frac{x^2}{n^2-1} + \frac{y^2}{n^2-3^2} + \frac{z^2}{n^2-5^2} + \frac{w^2}{n^2-7^2} = 1$. What exact value must x^2 equal?

Lean statement

```
theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

Parent. Suppose real numbers x, y, z, w satisfy the four weighted-square equations above. Show that $x = 105/32$ or $x = -105/32$.

Parent in Lean

```
theorem aime_1984_p15_x_two_roots (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x = 105 / 32 ∨ x = -(105 / 32) := by
  norm_num at h₀ h₁ h₂ h₃
  have hx_sq : x ^ 2 = 11025 / 1024 := by
    linarith [h₀, h₁, h₂, h₃]
  have hfactor : (x - 105 / 32) * (x + 105 / 32) = 0 := by
    nlinarith [hx_sq]
  rcases mul_eq_zero.mp hfactor with hpos | hneg
  · left
    nlinarith
  · right
    nlinarith
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is exactly equivalent to the parent over $\mathbb{R}$: $x^2 = 11025/1024$ holds precisely when $x = 105/32$ or $x = -105/32$. Replacing the two-root disjunction with the squared-coordinate identity is a genuine surface re-encoding, not mere decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child derives exactly the squared-coordinate identity equivalent over $\mathbb{R}$ to the parent's two-root conclusion, under the same four hypotheses. Replacing the disjunction with the square equation is a genuine surface re-encoding, not a renaming or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The four weighted-square hypotheses force exactly $x^2 = 11025/1024$, which over $\mathbb{R}$ is equivalent to $x = 105/32 \vee x = -(105/32)$; no hypotheses or scope changed. The conclusion is a genuine algebraic re-encoding, not a renaming or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2705 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem aime_1984_p15_x_squared_value.extracted_1_1 (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024
```

Read back by a model that saw only the Lean. Let x, y, z , and w be real numbers. Suppose that $x^2/(2^2-1) + y^2/(2^2-3^2) + z^2/(2^2-5^2) + w^2/(2^2-7^2) = 1$, $x^2/(4^2-1) + y^2/(4^2-3^2) + z^2/(4^2-5^2) + w^2/(4^2-7^2) = 1$, $x^2/(6^2-1) + y^2/(6^2-3^2) + z^2/(6^2-5^2) + w^2/(6^2-7^2) = 1$, and $x^2/(8^2-1) + y^2/(8^2-3^2) + z^2/(8^2-5^2) + w^2/(8^2-7^2) = 1$. Then $x^2 = 11025/1024$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem aime_1984_p15_x_squared_value (x y z w : ℝ)
  (h₀ : x ^ 2 / (2 ^ 2 - 1) + y ^ 2 / (2 ^ 2 - 3 ^ 2) + z ^ 2 / (2 ^ 2 - 5 ^ 2) + w ^ 2 / (2 ^ 2 - 7 ^ 2) = 1)
  (h₁ : x ^ 2 / (4 ^ 2 - 1) + y ^ 2 / (4 ^ 2 - 3 ^ 2) + z ^ 2 / (4 ^ 2 - 5 ^ 2) + w ^ 2 / (4 ^ 2 - 7 ^ 2) = 1)
  (h₂ : x ^ 2 / (6 ^ 2 - 1) + y ^ 2 / (6 ^ 2 - 3 ^ 2) + z ^ 2 / (6 ^ 2 - 5 ^ 2) + w ^ 2 / (6 ^ 2 - 7 ^ 2) = 1)
  (h₃ : x ^ 2 / (8 ^ 2 - 1) + y ^ 2 / (8 ^ 2 - 3 ^ 2) + z ^ 2 / (8 ^ 2 - 5 ^ 2) + w ^ 2 / (8 ^ 2 - 7 ^ 2) = 1) :
  x ^ 2 = 11025 / 1024 := by
  norm_num at h₀ h₁ h₂ h₃
  linarith [h₀, h₁, h₂, h₃]
```

Operator. mutation / mutation_silent **Lineage depth.** 2 **Certificate.** reproducible

Problem id. aime_1984_p15__me.mh.ms__54ef2279

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 54ef22792ab0a8f3

4.238 238. bounded_prime_product_additive_balance_characterization

The problem

For every nonnegative integer t , characterize when t satisfies the additive balance $pq = t + (p + q)$ for two distinct primes between 4 and 18. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Lean statement

```
theorem bounded_prime_product_additive_balance_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Parent. For every nonnegative integer t , characterize when t can be obtained by subtracting the sum of two distinct primes between 4 and 18 from their product. Show that this happens exactly when $(p - 1)(q - 1) = t + 1$ for such primes.

Parent in Lean

```
theorem bounded_prime_product_minus_sum_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q - (p + q) = t) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The bounds imply $p + q \leq p * q$, so the natural-number subtraction equation $p * q - (p + q) = t$ is exactly equivalent to the additive balance $p * q = t + (p + q)$. The factorized characterization and all hypotheses are unchanged, while the surface requires reasoning about subtraction cancellation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounds imply $p + q \leq p * q$, so in $\mathbb{N}$ the additive balance $p * q = t + (p + q)$ is exactly equivalent to $p * q - (p + q) = t$, with the factorized characterization unchanged. Replacing truncated subtraction by an additive equation is a genuine mathematical re-encoding rather than decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the retained prime and bound hypotheses, $p + q \leq p * q$, so $p * q - (p + q) = t$ is exactly equivalent to $p * q = t + (p + q)$. The additive-balance formulation materially changes the surface while preserving the factorized characterization and theorem scope.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2407 statements

Check	By	Result
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prime_product_additive_balance_characterization.extracted_1_1 : ∀ (t : ℕ),
  (∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ p * q = t + (p + q)) ↔
  ∃ p q, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧ (p - 1) * (q - 1) = t + 1
```

Read back by a model that saw only the Lean. *For every natural number t , there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that p times q equals t plus p plus q , if and only if there exist distinct prime numbers p and q , each between 4 and 18 inclusive, such that $(p \text{ minus } 1)$ times $(q \text{ minus } 1)$ equals t plus 1.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem bounded_prime_product_additive_balance_characterization :
  ∀ t : ℕ,
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      p * q = t + (p + q)) ↔
    (∃ p q : ℕ, Nat.Prime p ∧ Nat.Prime q ∧ 4 ≤ p ∧ p ≤ 18 ∧ 4 ≤ q ∧ q ≤ 18 ∧ p ≠ q ∧
      (p - 1) * (q - 1) = t + 1) := by
  intro t
  constructor
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hbal⟩
    have hbal_z : (p * q : ℤ) = (t : ℤ) + (p + q : ℤ) := by
      exact_mod_cast hbal
    have hfac_z : (((p - 1) * (q - 1) : ℕ) : ℤ) = (t : ℤ) + 1 := by
      norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
        Nat.cast_sub (by omega : 1 ≤ q)]
      nlinarith [hbal_z]
    have hfactor : (p - 1) * (q - 1) = t + 1 := by
      exact_mod_cast hfac_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
  · rintro ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hfactor⟩
    have hfac_z : (((p - 1) * (q - 1) : ℕ) : ℤ) = (t : ℤ) + 1 := by
      exact_mod_cast hfactor
    norm_num [Nat.cast_mul, Nat.cast_sub (by omega : 1 ≤ p),
      Nat.cast_sub (by omega : 1 ≤ q)] at hfac_z
    have hbal_z : (p * q : ℤ) = (t : ℤ) + (p + q : ℤ) := by
      nlinarith [hfac_z]
    have hbal : p * q = t + (p + q) := by
      exact_mod_cast hbal_z
    exact ⟨p, q, hp, hq, hp4, hp18, hq4, hq18, hne, hbal⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. amc12_2000_p6__me.ms__51827e7f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 51827e7f7966af1e

4.239 239. mathd_algebra_101_completed_square

The problem

For a real number x satisfying $x^2 - 5x - 4 \leq 10$, show that $x \in [-2, 7]$ if and only if $|x - \frac{5}{2}| \leq \frac{9}{2}$.

Lean statement

```
theorem mathd_algebra_101_completed_square (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 ↔ |x - (5 : ℝ) / 2| ≤ (9 : ℝ) / 2
```

Parent. For what values of x is it true that $x^2 - 5x - 4 \leq 10$? Express your answer in interval notation. Show that it is $x \in [-2, 7]$.

Parent in Lean

```
theorem mathd_algebra_101 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child preserves the hypothesis and restates the interval conclusion via the equivalent completed-square bound, explicitly proving both directions. The absolute-value formulation is a substantive surface re-encoding rather than a rename, unfolding, or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the same hypothesis, the completed-square inequality $|x - 5/2| \leq 9/2$ is exactly equivalent to $x \in [-2, 7]$, so the child preserves the parent's mathematics while changing the surface substantially.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves the same hypothesis and interval conclusion while expressing it equivalently as the completed-square bound $|x - 5/2| \leq 9/2$; the added biconditional is mathematically valid and does not change the theorem's scope. This is a genuine surface re-encoding, not a rename or definitional decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2429 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_101_completed_square.extracted_1_1 (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) :  
  x ∈ Set.Icc (-2) 7 ↔ |x - 5 / 2| ≤ 9 / 2
```

Read back by a model that saw only the Lean. For a real number x satisfying $x^2 - 5x - 4 \leq 10$, x lies in the closed interval $[-2, 7]$ if and only if the absolute value of $x - 5/2$ is at most $9/2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_101_completed_square (x : ℝ) (h₀ : x ^ 2 - 5 * x - 4 ≤ 10) : x ∈ Set.Icc (-2) 7 ↔ |x - (5 : ℝ) / 2| ≤ (9 : ℝ) / 2
↔ := by
  have h_interval : x ∈ Set.Icc (-2) 7 := by
    constructor
    · by_contra hx
      have hx' : x < -2 := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos_of_neg_of_neg (by linarith) (by linarith)
      nlinarith
    · by_contra hx
      have hx' : 7 < x := lt_of_not_ge hx
      have hprod : 0 < (x + 2) * (x - 7) :=
        mul_pos (by linarith) (by linarith)
      nlinarith
  constructor
  · intro _
    have hx := h_interval
    rw [abs_le]
    constructor <|> linarith [hx.1, hx.2]
  · intro h_abs
    rw [abs_le] at h_abs
    exact ⟨by linarith [h_abs.1], by linarith [h_abs.2]⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_101__ms__34850c49

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 34850c495074d6bf

4.240 240. quadratic_discriminant_max

The problem

For a positive real number a , the values of c for which $ax^2 + 5x + c = 0$ has a real solution are exactly those satisfying $25 - 4ac \geq 0$. Show that their greatest value is $\frac{25}{4a}$.

Lean statement

```
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))
```

Parent. For a real number a with $0 < a \leq 10$, what is the largest number c such that $ax^2 + 5x + c = 0$ has a real solution? Show that it is $\frac{25}{4a}$.

Parent in Lean

```
theorem quadratic_real_root_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | ∃ x : ℝ, a * x ^ 2 + 5 * x + c = 0}) : IsGreatest S (25 / (4 * a))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The discriminant condition $0 \leq 25 - 4ac$ characterizes exactly those c for which the quadratic has a real root when $a > 0$, so the child proves the same greatest-element theorem over the same feasible set. Replacing the existential-root definition with the discriminant normal form is a substantive surface re-encoding.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child's condition $0 \leq 25 - 4ac$ exactly characterizes the same feasible constants as the parent's existential real-root condition for $a > 0$, and it preserves the same greatest element. Recasting root feasibility via the discriminant is a substantive surface change requiring mathematical recognition, not mere renaming or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The discriminant condition $0 \leq 25 - 4ac$ is equivalent, under $a > 0$, to the existence of a real root of $ax^2 + 5x + c = 0$, so the same greatest-set theorem is preserved. Replacing the existential quadratic description with its discriminant normal form is a materially different surface.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2591 statements
hypothesis_preservation	parser	changed: dropped #3# : #2#={c ∃#4# : ℝ, #0#*#4#^2+5*#4#+c=0}
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quadratic_discriminant_max.extracted_1_1 (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c | 0 ≤ 25 - 4 * a * c}) :  
  IsGreatest S (25 / (4 * a))
```

Read back by a model that saw only the Lean. *Let a be a real number with $0 < a$, and let S be a subset of the real numbers such that $S = \{c \in \mathbb{R} \mid 0 \leq 25 - 4ac\}$. Then $25/(4a)$ is the greatest element of S .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open BigOperators Real Nat Topology Rat  
  
theorem quadratic_discriminant_max (a : ℝ) (ha : 0 < a) (S : Set ℝ) (hS : S = {c : ℝ | 0 ≤ 25 - 4 * a * c}) : IsGreatest S (25 / (4 * a))  
  <:= by  
    rw [hS]  
    constructor  
    · change 0 ≤ 25 - 4 * a * (25 / (4 * a))  
      have hdisc : 0 ≤ 25 - 4 * a * (25 / (4 * a)) := by  
        field_simp [ha.ne'] <|> norm_num  
      exact hdisc  
    · intro c hc  
      change 0 ≤ 25 - 4 * a * c at hc  
      have hpos : 0 < 4 * a := by  
        nlinarith [ha]  
      apply (le_div_iff hpos).2  
      nlinarith [hc]
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_algebra_28__me.ms__e0abe93f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e0abe93f073ab7ec

4.241 241. mathd_algebra_313_affine

The problem

For complex numbers v , i , and z satisfying $v = i z$, $v = 1 + I$, and $z = 2 - I$, one has $5i = 1 + 3I$.

Lean statement

```
theorem mathd_algebra_313_affine (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I) (h₂ : z = 2 - Complex.I) : 5 * i = 1 + 3 *
  ↪ Complex.I := by
```

Parent. Complex numbers are often used when dealing with alternating current (AC) circuits. In the equation $V = IZ$, V is voltage, I is current, and Z is a value known as impedance. If $V = 1 + i$ and $Z = 2 - i$, find I . Show that it is $\frac{1}{5} + \frac{3}{5}i$.

Parent in Lean

```
theorem mathd_algebra_313 (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I)
  (h₂ : z = 2 - Complex.I) : i = 1 / 5 + 3 / 5 * Complex.I := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent because 5 is nonzero in $\mathbb{C}$: multiplying the quotient equality by 5 yields $5 * i = 1 + 3 * \text{Complex.I}$, and conversely division by 5 recovers the parent conclusion. The denominator-cleared affine form is a genuine surface re-encoding, not mere decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent because multiplying the complex equality by the nonzero scalar 5 yields and recovers the denominator-cleared form. All hypotheses and the mathematical scope are preserved, while the conclusion is genuinely re-encoded.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2036 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_algebra_313_affine.extracted_1_1 (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I)
  (h₂ : z = 2 - Complex.I) : 5 * i = 1 + 3 * Complex.I
```

Read back by a model that saw only the Lean. Let v , i , and z be complex numbers. Suppose that $v = i \cdot z$, $v = 1 + I$, and $z = 2 - I$, where I is the imaginary unit. Then $5 \cdot i = 1 + 3I$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_algebra_313_affine (v i z : ℂ) (h₀ : v = i * z) (h₁ : v = 1 + Complex.I) (h₂ : z = 2 - Complex.I) : 5 * i = 1 + 3 *
  ↪ Complex.I := by
  have hmain : i = 1 / 5 + 3 / 5 * Complex.I := by
    rw [h₁, h₂] at h₀
    have hr := congrArg Complex.re h₀
    have hm := congrArg Complex.im h₀
    apply Complex.ext
    · norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
      linarith
    · norm_num [Complex.mul_re, Complex.mul_im] at hr hm ⊢
      linarith
  calc
    5 * i = 5 * (1 / 5 + 3 / 5 * Complex.I) := by rw [hmain]
    _ = 1 + 3 * Complex.I := by ring
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_313__ms__6b23d2d5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6b23d2d503df6c14

4.242 242. completed_square_minimum

The problem

What is the minimum possible value of y when $y = (x - 3)^2 + 4$? Show that it is 4.

Lean statement

```
theorem completed_square_minimum
  (f : ℝ → ℝ)
  (h : f = fun x ↦ (x - 3) ^ 2 + 4) :
  IsLeast (Set.range f) 4 := by
```

Parent. What is the minimum possible value for y in the equation $y = x^2 - 6x + 13$? Show that it is 4.

Parent in Lean

```
theorem mathd_algebra_410
  (f : ℝ → ℝ)
  (h₀ : f = fun x ↦ x ^ 2 - 6 * x + 13) :
  IsLeast (Set.range f) 4 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The completed-square hypothesis defines exactly the same function, since $(x-3)^2+4=x^2-6x+13$, and the `IsLeast` conclusion is unchanged. This is a genuine algebraic re-encoding that requires recognizing the completed-square identity, not mere renaming or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The completed-square form $(x-3)^2+4$ is definitionally equal to $x^2-6x+13$, so the hypothesis and `IsLeast` conclusion preserve exactly the parent theorem. The surface representation genuinely changes from expanded polynomial to completed square.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The completed-square expression $(x-3)^2+4$ is algebraically identical to $x^2-6x+13$, so the hypothesis and `IsLeast` conclusion describe exactly the same theorem. The surface is materially re-encoded through parameter shifting rather than merely renamed or decorated.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2676 statements
hypothesis_preservation	parser	changed: dropped #1# : #0# = fun #2# ↦ #2# ^ 2 - 6 * #2# + 13
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem completed_square_minimum.extracted_1_1 (f : ℝ → ℝ) (h : f = fun x => (x - 3) ^ 2 + 4) :  
  IsLeast (Set.range f) 4
```

Read back by a model that saw only the Lean. *Let f be a function from the real numbers to the real numbers, and suppose that for every real number x , $f(x) = (x - 3)^2 + 4$. Then 4 is the least element of the range of f .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem completed_square_minimum  
  (f : ℝ → ℝ)  
  (h : f = fun x => (x - 3) ^ 2 + 4) :  
  IsLeast (Set.range f) 4 := by  
  subst f  
  constructor  
  · use 3  
    norm_num  
  · rintro y ⟨x, rfl⟩  
    nlinarith [sq_nonneg (x - 3)]
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_algebra_410__ms__aaa74262

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint aaa74262625fe908

4.243 243. mathd_numbertheory_202_congruence

The problem

Show that the units digit of $19^{19} + 99^{99}$ is congruent to 8 modulo 10.

Lean statement

```
theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
```

Parent. What is the units digit of $19^{19} + 99^{99}$? Show that it is 8.

Parent in Lean

```
theorem mathd_numbertheory_202 : (19 ^ 19 + 99 ^ 99) % 10 = 8 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child asserts exactly the same fact: the sum has remainder 8 modulo 10, expressed as congruence rather than remainder equality. The modular-congruence surface is materially different and preserves all mathematics.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The modular congruence ' $19^{19} + 99^{99} \equiv 8 \text{ [MOD 10]}$ ' is equivalent to the parent remainder equation because 8 is already below 10. Congruence notation is a genuine mathematical re-encoding, not a rename or definitional decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child asserts exactly the same mathematical fact: the sum is congruent to 8 modulo 10, which is equivalent to having remainder 8 modulo 10. Replacing remainder equality with modular congruence is a genuine Lean-surface re-encoding.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2717 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_202_congruence.extracted_1_1 : 19 ^ 19 + 99 ^ 99 ≡ 8 [MOD 10]
```

Read back by a model that saw only the Lean. The sum of 19 raised to the 19th power and 99 raised to the 99th power is congruent to 8 modulo 10.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem mathd_numbertheory_202_congruence : (19 ^ 19 + 99 ^ 99) ≡ 8 [MOD 10] := by
  norm_num [Nat.ModEq]
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_202__ms__48ce67d7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 48ce67d79da3c99a

4.244 244. mathd_numbertheory_229_div

The problem

Show that the difference between 5^{30} and 1 is divisible by 7.

Lean statement

```
theorem mathd_numbertheory_229_div : 7 ∣ (5 ^ 30 - 1 : ℕ) := by
```

Parent. What is the remainder when 5^{30} is divided by 7? Show that it is 1.

Parent in Lean

```
theorem mathd_numbertheory_229 : 5 ^ 30 ≡ 1 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent to the parent: integer congruence $5^{30} \equiv 1 \pmod{7}$ is exactly divisibility of the natural-number difference $7 \mid 5^{30} - 1$. The divisibility formulation is a substantive surface re-encoding, not mere decoration or renaming.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent to the parent: for these positive natural numbers, $7 \mid 5^{30} - 1$ exactly expresses $5^{30} \equiv 1$ modulo 7. Recasting modular congruence as divisibility is a substantive surface change that preserves the theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2411 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mathd_numbertheory_229_div.extracted_1.1 : 7 ∣ 5 ^ 30 - 1
```

Read back by a model that saw only the Lean. 7 divides 5 raised to the 30th power minus 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
```

```
open BigOperators Real Nat Topology Rat
```

```
theorem mathd_numbertheory_229_div : 7 ∣ (5 ^ 30 - 1 : ℕ) := by  
  have hpow : (5 ^ 30 : ℕ) = 931322574615478515625 := by  
    norm_num  
  rw [hpow]  
  norm_num
```

Operator. mutation/mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. mathd_numbertheory_229__ms__605bc864

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, fingerprint 605bc86482a76709

4.245 245. variable_base_residue_divisibility

The problem

Two sensors report integers a and b , with $a \equiv 29 \pmod{7}$ and $b \equiv 5 \pmod{7}$. Show that 7 divides $a^{13} - b^{13} - 3$.

Lean statement

```
theorem variable_base_residue_divisibility (a b : Z) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : Z) ∣ (a ^ 13 - b ^ 13 - 3) := by
```

Parent. If integers a and b differ from 29 and 5, respectively, by multiples of 7, show that $a^{13} - b^{13}$ leaves remainder 3 modulo 7.

Parent in Lean

```
theorem thirteenth_power_difference_mod_seven (a b : Z) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The divisibility conclusion is exactly equivalent to the parent congruence: $a^{13} - b^{13} \equiv 3 \pmod{7}$ iff 7 divides $a^{13} - b^{13} - 3$, with all hypotheses and scope preserved. This is a genuine conclusion re-encoding requiring conversion between modular congruence and divisibility, not mere notation or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent: congruence modulo 7 is restated as divisibility of the difference by 7, with all hypotheses and the conclusion's scope preserved. This is a substantive surface change requiring recognition of the equivalence, not mere renaming or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The divisibility conclusion is exactly equivalent to the parent congruence: $7 \mid (a^{13} - b^{13} - 3)$ iff $a^{13} - b^{13} \equiv 3 \pmod{7}$. The hypotheses and mathematical scope are unchanged, while the surface uses a genuinely different conclusion form.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2418 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem variable_base_residue_divisibility.extracted_1_1 (a b : Z) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) :
  7 ∣ a ^ 13 - b ^ 13 - 3
```

Read back by a model that saw only the Lean. For integers a and b , if a is congruent to 29 modulo 7 and b is congruent to 5 modulo 7, then 7 divides a raised to the 13th power minus b raised to the 13th power minus 3.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem variable_base_residue_divisibility (a b : ℤ) (ha : a ≡ 29 [ZMOD 7]) (hb : b ≡ 5 [ZMOD 7]) : (7 : ℤ) ∣ (a ^ 13 - b ^ 13 - 3) := by
  have ha13 : a ^ 13 ≡ (29 : ℤ) ^ 13 [ZMOD 7] := ha.pow 13
  have hb13 : b ^ 13 ≡ (5 : ℤ) ^ 13 [ZMOD 7] := hb.pow 13
  have hdif : a ^ 13 - b ^ 13 ≡ (29 : ℤ) ^ 13 - (5 : ℤ) ^ 13 [ZMOD 7] := ha13.sub hb13
  have hparent : (29 : ℤ) ^ 13 - (5 : ℤ) ^ 13 ≡ 3 [ZMOD 7] := by
    norm_num [Int.ModEq]
  have hres : a ^ 13 - b ^ 13 ≡ 3 [ZMOD 7] := hdif.trans hparent
  rw [Int.modEq_iff_dvd] at hres
  rcases hres with ⟨k, hk⟩
  refine ⟨-k, ?_⟩
  omega
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_728__mh.ms__57316c5a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 57316c5ad6bdfae7

4.246 246. ternary_terminal_digit_exclusion

The problem

A natural number has base-three digits $122d_3$. Show that either $d \neq 2$ or the number equals 53.

Lean statement

```
theorem ternary_terminal_digit_exclusion
  (d x : ℕ)
  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  d ≠ 2 ∨ x = 53
```

Parent. Let $d \in \{0, 1, 2\}$. If a natural number has base-three representation $122d_3$, show that its value is $d + 51$; in particular, when $d = 2$, its value is 53.

Parent in Lean

```
theorem ternary_terminal_digit_decode
  (d x : ℕ)

  (h : Nat.digits 3 x = [d, 2, 2, 1]) :
  x = d + 51 ∧ (d = 2 → x = 53)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is equivalent under the digit hypothesis: the parent's conditional is exactly expressed as $d \neq 2 \vee x = 53$, while $x = d + 51$ is already forced by the shared hypothesis. The disjunctive exclusion form is a genuine surface re-encoding, not decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the shared digit hypothesis, $x = d + 51$ is forced, so the parent's substantive conditional is equivalent to $d \neq 2 \vee x = 53$. The child materially re-encodes the implication as an exclusion disjunction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Under the shared digit hypothesis, $x = d + 51$ is derivable, so the parent's conjunction is equivalent to $d \neq 2 \vee x = 53$; the child presents this as a genuinely different exclusion-form surface.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2577 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem ternary_terminal_digit_exclusion.extracted_1_1 (d x : ℕ) (h : Nat.digits 3 x = [d, 2, 2, 1]) :  
  d ≠ 2 ∨ x = 53
```

Read back by a model that saw only the Lean. For natural numbers d and x , if the base-3 digits of x , listed in `Nat.digits` order, are exactly $[d, 2, 2, 1]$, then d is not equal to 2 or x equals 53.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open BigOperators Real Nat Topology Rat  
  
theorem ternary_terminal_digit_exclusion  
  (d x : ℕ)  
  (h : Nat.digits 3 x = [d, 2, 2, 1]) :  
  d ≠ 2 ∨ x = 53 := by  
  have hdecode : x = Nat.ofDigits 3 (Nat.digits 3 x) :=  
    (Nat.ofDigits_digits 3 x).symm  
  have hvalue : x = d + 51 := by  
    calc  
      x = Nat.ofDigits 3 (Nat.digits 3 x) := hdecode  
      _ = Nat.ofDigits 3 [d, 2, 2, 1] := by rw [h]  
      _ = d + 51 := by norm_num [Nat.ofDigits_cons]  
  by_cases hd : d = 2  
  · right  
    rw [hvalue, hd]  
  · exact Or.inl hd
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. mathd_numbertheory_85__me.ms__3d7174de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 3d7174de1c8ee629

4.247 247. complementary_pascal

The problem

For positive integers n and k with $k \leq n$, show that $\binom{n}{n-k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Lean statement

```
theorem complementary_pascal (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)
```

Parent. Show that for positive integers n and k with $k \leq n$, we have $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

Parent in Lean

```
theorem numbertheory_nckeqnm1ckpnm1ckm1 (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

Under $k \leq n$, binomial symmetry gives $\binom{n}{n-k} = \binom{n}{k}$, so the child is mathematically equivalent to the parent with the same hypotheses. The complementary-index formulation is a genuine surface change, and the iff restatement reinforces the equivalent bounded-index form.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the complementary-index identity $\binom{n}{n-k} = \binom{n}{k}$ under the preserved hypothesis $k \leq n$, so it has exactly the parent's scope and conclusion after a genuine re-encoding of the left-hand side.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves all hypotheses and uses choose symmetry under $k \leq n$ to replace $\text{Nat.choose } n \ k$ with $\text{Nat.choose } n \ (n - k)$, leaving the Pascal identity otherwise unchanged. This is a genuine complementary-index re-encoding that requires recognizing the equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2623 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem complementary_pascal.extracted_1_1 (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  n.choose (n - k) = (n - 1).choose k + (n - 1).choose (k - 1)
```

Read back by a model that saw only the Lean. For natural numbers n and k , if $0 < n$ and $0 < k$, and $k \leq n$, then the binomial coefficient " n choose $n - k$ " equals the sum of " $n - 1$ choose k " and " $n - 1$ choose $k - 1$ ".

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem complementary_pascal (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
  have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
    Nat.choose_symm h₁
  have hpascal : Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1) := by
    simpa [Nat.add_comm] using (Nat.choose_eq_choose_pred_add h₀.1 h₀.2)
  exact hsymm.trans hpascal

theorem complementary_pascal_iff (n k : ℕ) (h₀ : 0 < n ∧ 0 < k) (h₁ : k ≤ n) :
  (Nat.choose n k = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)) ↔
  (Nat.choose n (n - k) = Nat.choose (n - 1) k + Nat.choose (n - 1) (k - 1)) := by
  have hsymm : Nat.choose n (n - k) = Nat.choose n k :=
    Nat.choose_symm h₁
  constructor
  · intro h
    exact hsymm.trans h
  · intro h
    exact hsymm.symm.trans h
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. numbertheory_nckeqnm1ckpnm1ckm1__ms__b312e50e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b312e50ee99a4d20

4.248 248. square_dvd_three_iff_dvd

The problem

For an integer a , show that 3 divides a^2 exactly when 3 divides a .

Lean statement

```
theorem square_dvd_three_iff_dvd (a : ℤ) : 3 ∣ a ^ 2 ↔ 3 ∣ a
```

Parent. For an integer a , its square is congruent to 0 modulo 3 exactly when a is divisible by 3.

Parent in Lean

```
theorem square_mod_three_zero_iff_dvd (a : ℤ) : a ^ 2 ≡ 0 [ZMOD 3] ↔ 3 ∣ a
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child is mathematically equivalent to the parent: for integers, $a^2 \equiv 0 \pmod 3$ exactly means $3 \mid a^2$, so both directions have the same scope. Replacing modular congruence with divisibility is a genuine but shallow surface re-encoding, not mere decoration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent for every integer: divisibility by 3 is exactly congruence to 0 modulo 3, applied to a^2 . Replacing the modular-equivalence relation with divisibility materially changes the surface while preserving the theorem.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is mathematically equivalent over integers: $a^2 \equiv 0 \pmod 3$ exactly when $3 \mid a^2$, so it has the same hypotheses, scope, and conclusion. Replacing modular congruence with divisibility is a genuine surface re-encoding requiring the ModEq-divisibility bridge.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2713 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem square_dvd_three_iff_dvd.extracted_1_1 (a : ℤ) : 3 ∣ a ^ 2 ↔ 3 ∣ a
```

Read back by a model that saw only the Lean. For every integer a , 3 divides a squared if and only if 3 divides a .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open BigOperators Real Nat Topology Rat

theorem square_dvd_three_iff_dvd (a : ℤ) : 3 ∣ a ^ 2 ↔ 3 ∣ a := by
  constructor
  · intro hsq
    have hsqmod : a ^ 2 ≡ 0 [ZMOD 3] := Dvd.dvd.modEq_zero_int hsq
    have hres : a % 3 = 0 ∨ a % 3 = 1 ∨ a % 3 = 2 := by
      omega
    rcases hres with hres | hres | hres
    · refine ⟨a / 3, ?_⟩
      omega
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsqmod
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsqmod
    · ex falso
      rw [← Int.mul_ediv_add_emod a 3, hres] at hsqmod
      norm_num [Int.ModEq, pow_two, Int.add_emod, Int.mul_emod] at hsqmod
  · intro hdiv
    rcases hdiv with ⟨k, hk⟩
    refine ⟨3 * k ^ 2, ?_⟩
    rw [hk]
    ring
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. numbertheory_sqmod3in01d__mh.ms__ca1f8653

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ca1f86533f25425c